



Contents

- [Inicio](#)
 - [📁 Unidades](#)
 - [📄 Boletines](#)
 - [📄 Licencia](#)
- [U01 — Introducción a Java](#)
 - [🎯 Objetivo de la unidad](#)
 - [📖 Mapa de la unidad](#)
 - [📄 Boletines de la unidad](#)
 - [✅ Criterios de evaluación cubiertos \(RA1\)](#)
 - [📖 ¿Por dónde empiezo?](#)
- [01 — ¿Qué es Java?](#)
 - [💡 La idea en una frase](#)
 - [🔍 ¿De dónde sale este lenguaje?](#)
 - [🏠 ¿Qué es eso del bytecode?](#)
 - [☕ La Trilogía del Café: JVM, JRE y JDK](#)
 - [🌐 ¿Por qué sigue vivo Java tantos años después?](#)
 - [🏠 Ejemplo guiado: recorrer tu primer programa](#)
 - [🎯 Mini-chequeo](#)
 - [✅ Resumen en 3 frases](#)
- [02 — Instalación del JDK](#)
 - [💡 La idea en una frase](#)
 - [🛒 ¿Qué JDK instalo?](#)
 - [🚀 Los primeros pasos](#)
 - [🔧 El IDE: tu navaja suiza](#)
 - [🏠 Ejemplo guiado: tu primer proyecto](#)
 - [🎯 Mini-chequeo](#)
 - [✅ Resumen en 3 frases](#)
- [03 — Tu primer programa: Hola Mundo](#)
 - [💡 La idea en una frase](#)
 - [👋 Hola Mundo, el ritual de iniciación](#)
 - [🔬 Disección de Hola Mundo](#)
 - [🔑 ¿Por qué public static void main\(String\[\] args\)?](#)

-  El método que no se llama
-  Sé el Código: tú eres la JVM
-  Mini-chequeo
-  Resumen en 3 frases
- 04 — El depurador: detective de código
 -  La idea en una frase
 -  ¿Qué es depurar?
 -  Las cuatro herramientas del detective
 -  Ejemplo guiado: el caso del sospechoso
 -  La regla de oro del depurador
 -  Mini-chequeo
 -  Resumen en 3 frases
- 05 — Comentarios y documentación
 -  La idea en una frase
 -  Los tres tipos de comentarios
 -  El consejo que te ahorrará horas: comenta el por qué, no el qué
 -  Javadoc: documentación que se genera sola
 -  Ejemplo guiado: documentar un programa completo
 -  Mini-chequeo
 -  Resumen en 3 frases
- 06 — Argumentos de línea de comandos
 -  La idea en una frase
 -  ¿Qué es args?
 -  Cómo se indexan las palabras
 -  ¿Para qué sirve pasar argumentos?
 -  Ejemplo guiado: el programa que te saluda
 -  Mini-chequeo
 -  Resumen en 3 frases
- 07 — El compilador y sus errores
 -  La idea en una frase
 -  Compilar ≠ ejecutar
 -  El Error del Novato: 4 errores en un programa
 -  Cómo leer un mensaje de error de javac

-  [Errores típicos del novato \(y su remedio\)](#)
-  [EL LÍO: el código revuelto](#)
-  [Mini-chequeo](#)
-  [Resumen en 3 frases](#)
- [08 — El IDE y tu flujo de trabajo](#)
 -  [La idea en una frase](#)
 -  [Anatomía de un proyecto](#)
 -  [El ciclo de trabajo \(tu nuevo bucle de vida\)](#)
 -  [Los atajos que te harán parecer un pro](#)
 -  [Autocompletado: tu compañero silencioso](#)
 -  [Ejemplo guiado: de cero a ejecutar en 60 segundos](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
- [09 — Repaso interactivo: consolida lo aprendido](#)
 -  [La idea en una frase](#)
 -  [Sé el Código, my friend...](#)
 -  [Fireside Chat: JDK vs JRE](#)
 -  [¿Quién Soy?](#)
 -  [CONRAD VS EL MUNDO: “El compilador me odia”](#)
 -  [Laboratorio de Tortura: el programa que no arranca](#)
 -  [Logros de esta unidad](#)
 -  [Atrévete a Pensar](#)
 -  [Crucigrama de Bits](#)
 -  [Preguntas de Entrevista de Trabajo](#)
 -  [No hay preguntas tontas](#)
 -  [Post-Créditos](#)
- [12 3 4 Unidad 2: Variables, Tipos de Datos y Operadores](#)
 - [Variables: Las Cajas Donde Viven Tus Datos](#)
 - [Operadores: El Gimnasio de los Datos](#)
 - [¡No Hay Preguntas Tontas!](#)
 - [Más Ejemplos de Código](#)
 - [Resumen \(lo que importa de verdad\)](#)
 - [Ejercicios Propuestos](#)

-  Unidad 3: Estructuras de Control y Excepciones
 - if, else y switch: El Arte de Decidir
 - Bucles: Cómo Hacer Que el Ordenador Se Repita
 - Excepciones: Cuando Tu Programa Tropezaba
 - Ejercicios Propuestos
 - Resumen
-  Unidad 4: Algorítmica I — Fundamentos
 - ¿Qué es un Algoritmo? (El Arte de Dar Instrucciones Precisas)
 - Búsqueda Lineal: El Buscador de Zapatos Perdidos
 - Búsqueda Binaria: El Buscador Jedi
 - Ordenación Burbuja: Simple pero Torpe
 - Ordenación por Inserción: Como Ordenar Cartas en la Mano
 - Complejidad Algorítmica: Big O para Mortales
 -  BE THE CODE, MY FRIEND: Implementa Búsqueda Binaria desde Cero
 - ¡No Hay Preguntas Tontas!
 - Resumen de la Unidad
-  Unidad 5: Algorítmica II — Técnicas Avanzadas
 - 1. Recursividad: una función que se llama a sí misma (sin volverse loca)
 - 2. Ejemplos clásicos (los clásicos por algo son)
 - 3. Divide y vencerás (Divide & Conquer)
 - 4. Quicksort — el rápido (cuando le sale bien)
 - 5. Mergesort — el fiable (siempre cumple lo que promete)
 - 6. Comparación: ¿cuándo uso cada uno?
 - 7.  BE THE CODE, MY FRIEND: implementa Quicksort desde cero
 - 8. ¡No Hay Preguntas Tontas!
 - Resumen de la unidad
-  Unidad 4: POO - Clases y Objetos
 - POO: Tus Objetos Cobran Vida
 - Clases y Objetos: Cortapastas vs Galletas
 - Resumen
 - Ejercicios Propuestos
-  Unidad 5: Visibilidad, Encapsulación y Static
 - Visibilidad: El Arte de No Enseñar Todo

- [Encapsulación: El Pilar que Sostiene la POO](#)
- [Static: Lo Que Pertenece a la Clase \(No al Objeto\)](#)
- [Resumen](#)
- [Ejercicios Propuestos](#)
-  [Unidad 8: Herencia, Polimorfismo e Interfaces](#)
 - [Herencia: Cuando Tus Hijos Siguen Tus Pasos \(Pero Mejor\)](#)
 - [Polimorfismo: El Camaleón de la POO](#)
 - [La Clase Object: El Tatarabuelo de Todo](#)
 - [Clases Abstractas: El Boceto Que No Puedes Usar Directamente](#)
 - [Interfaces: El Contrato Que Tu Código Firma](#)
 - [Interfaces Funcionales: Una Sola Misión](#)
 - [Jerarquía de Clases y Composición](#)
 - [Resumen Global](#)
-  [Unidad 8: Arrays y Colecciones](#)
 - [Arrays: El Aparcamiento de Datos](#)
 - [Colecciones: Cuando un Array Se Queda Pequeño](#)
 - [Ejercicios Propuestos](#)
-  [Unidad 9: Genéricos y Mapas](#)
 - [Genéricos: El Que Lo Cambió Todo](#)
 - [Mapas: La Guía Telefónica](#)
 - [Ejercicios Propuestos](#)
-  [Unidad 11: Consola, Ficheros y Expresiones Regulares](#)
 - [Comunicación por Consola](#)
 - [Ficheros](#)
 - [Expresiones Regulares](#)
 - [Ejercicios Propuestos](#)
-  [Unidad 12: Conexión a Bases de Datos con JDBC](#)
 - [¿Qué es JDBC?](#)
 - [Conexión a SQLite](#)
 - [CRUD con JDBC](#)
 - [PreparedStatement y SQL Injection](#)
 - [Patrón DAO](#)
 - [Transacciones](#)

- [!\[\]\(7e19807c61da14f515588e95cd49886c_img.jpg\) BE THE CODE, MY FRIEND: Implementa un DAO Completo Desde Cero](#)
- [¡No Hay Preguntas Tontas!](#)
- [Ejercicios Propuestos](#)
- [Buenas Prácticas: El Decálogo del JDBC](#)
- [Resumen Exprés](#)
- [!\[\]\(8ff9e60a4b0560d7ec99179ef4779d9e_img.jpg\) Unidad 13: Servir y Consumir APIs con Web](#)
 - [Hola, Mundo Web](#)
 - [1. El Protocolo HTTP en 30 Segundos](#)
 - [2. Servidor Web Mínimo con HttpServer](#)
 - [3. Sirviendo HTML](#)
 - [4. Parámetros GET: El Cliente Pregunta](#)
 - [5. Formularios POST: El Cliente Envía Datos](#)
 - [6. Devolviendo JSON \(Como una API de Verdad\)](#)
 - [7. Mini Proyecto: Gestor de Tareas \(API REST\)](#)
 - [8. Consumir APIs Externas con HttpClient](#)
 - [9. Frameworks Modernos](#)
 - [? ¡No Hay Preguntas Tontas!](#)
 - [Resumen](#)
 - [Ejercicios Propuestos](#)
- [Boletín U01 — Inicial Resuelto](#)
 - [Ejercicio 1: Desordena esto](#)
 - [Ejercicio 2: ¿Qué imprime?](#)
 - [Ejercicio 3: Cazador de errores](#)
 - [Ejercicio 4: Tu ficha personal](#)
 - [Ejercicio 5: Completa el programa](#)
 - [Ejercicio 6: Empareja conceptos](#)
 - [Ejercicio 7: CodeWars — Square\(n\) Sum](#)
 - [Ejercicio 8: El detective de errores](#)
 - [Ejercicio 9: Tu biografía](#)
- [Boletín U01 — Inicial](#)
 - [Ejercicio 1: Desordena esto](#)
 - [Ejercicio 2: ¿Qué imprime?](#)
 - [Ejercicio 3: Cazador de errores](#)

- [Ejercicio 4: Tu ficha personal](#)
- [Ejercicio 5: Completa el programa](#)
- [Ejercicio 6: Empareja conceptos](#)
- [Ejercicio 7: CodeWars — Square\(n\) Sum](#)
- [Ejercicio 8: El detective de errores](#)
- [Ejercicio 9: Tu biografía](#)
- [Boletín U01 — Avanzado Resuelto](#)
 - [★ Ejercicio 1: ASCII art con prints](#)
 - [★ Ejercicio 2: Sin ejecutar — secuencias de escape](#)
 - [★ ★ Ejercicio 3: El reloj de milisegundos](#)
 - [★ ★ Ejercicio 4: Contador de argumentos](#)
 - [★ ★ ★ Ejercicio 5: La edad cósmica](#)
 - [★ ★ ★ Ejercicio 6: CodeWars — Grasshopper - Summation](#)
 - [★ ★ ★ Ejercicio 7: AceptaElReto — 119 Futbolistas](#)
 - [★ ★ ★ Ejercicio 8: Javadoc de campeonato](#)
 - [!\[\]\(effbd7993c63c039a58fd3395789cf3f_img.jpg\) Referencias](#)
- [Boletín U01 — Avanzado](#)
 - [★ Ejercicio 1: ASCII art con prints](#)
 - [★ Ejercicio 2: Sin ejecutar — secuencias de escape](#)
 - [★ ★ Ejercicio 3: El reloj de milisegundos](#)
 - [★ ★ Ejercicio 4: Contador de argumentos](#)
 - [★ ★ ★ Ejercicio 5: La edad cósmica](#)
 - [★ ★ ★ Ejercicio 6: CodeWars — Grasshopper - Summation](#)
 - [★ ★ ★ Ejercicio 7: AceptaElReto — 119 Futbolistas](#)
 - [★ ★ ★ Ejercicio 8: Javadoc de campeonato](#)
 - [!\[\]\(144980d038f2541d7b588a8a9132bd70_img.jpg\) Referencias](#)
- [Boletín U01 — Extras](#)
- [Boletín 2 - Inicial Resuelto: Variables y Operadores](#)
 - [Ejercicio 1: Declara el tipo correcto](#)
 - [Ejercicio 2: ¿Qué imprime?](#)
 - [Ejercicio 3: Encuentra el error](#)
 - [Ejercicio 4: El intercambio](#)
 - [Ejercicio 5: Escribe este programa](#)

- [Ejercicio 6: AceptaElReto 149 — San Fermín](#)
- [Ejercicio 7: CodeWars — Even or Odd](#)
- [Boletín 2 - Inicial: Variables y Operadores](#)
 - [Ejercicio 1: Conversor de temperaturas](#)
 - [Ejercicio 2: ¿Qué imprime?](#)
 - [Ejercicio 3: Calculadora de descuentos](#)
 - [Ejercicio 4: Precedencia de pesadilla](#)
 - [Ejercicio 5: El tipo perfecto](#)
 - [Ejercicio 6: El intercambio mágico](#)
 - [Ejercicio 7: CodeWars — Will you make it?](#)
- [Boletín 2 - Resuelto: Variables y Operadores](#)
 -  [Ejercicio 1: ¿Par o impar?](#)
 -  [Ejercicio 2: Constante del mal](#)
 -   [Ejercicio 3: Dado trucado](#)
 -   [Ejercicio 4: Año bisiesto](#)
 -   [Ejercicio 5: Nombre al revés](#)
 -    [Ejercicio 6: AceptaElReto 152 — Números de pares](#)
 -    [Ejercicio 7: El acertijo del ++](#)
 -    [Ejercicio 8: AceptaElReto 140 — Suma de dígitos](#)
 -  [Referencias](#)
- [Boletín 2 - Intermedio: Variables y Operadores](#)
 -  [Ejercicio 1: Calculadora de propinas](#)
 -  [Ejercicio 2: Conversor dólar-euro](#)
 -   [Ejercicio 3: ¿Qué imprime? — el casting traidor](#)
 -   [Ejercicio 4: Interés compuesto \(sin bucle\)](#)
 -    [Ejercicio 5: El enigma del post-incremento](#)
 -    [Ejercicio 6: CodeWars — Convert boolean values to strings 'Yes' or 'No'](#)
 -    [Ejercicio 7: AceptaElReto — 114 Último dígito del factorial](#)
 -  [Referencias](#)
- [Boletín 02 — Extras: Variables, Tipos de Datos y Operadores](#)
- [Boletín 3 - Inicial Resuelto: Estructuras de Control](#)
 - [Ejercicio 1: ¿Qué imprime este if-else?](#)
 - [Ejercicio 2: Completa el switch](#)

- [Ejercicio 3: Encuentra el error \(bucle infinito\)](#)
- [Ejercicio 4: Escribe este programa \(while\)](#)
- [Ejercicio 5: El for de toda la vida](#)
- [Ejercicio 6: AceptaElReto 200 — Aburrimiento en las aulas](#)
- [Ejercicio 7: CodeWars — Century From Year](#)
- [Boletín 3 - Inicial: Estructuras de Control](#)
 - [Ejercicio 1: ¿Qué imprime este if?](#)
 - [Ejercicio 2: Encuentra el error en el for](#)
 - [Ejercicio 3: Completa el programa \(do-while\)](#)
 - [Ejercicio 4: Cuenta atrás](#)
 - [Ejercicio 5: Menú con switch](#)
 - [Ejercicio 6: Tabla de multiplicar del 7](#)
 - [Ejercicio 7: CodeWars — Grasshopper - Grade book](#)
- [Boletín 3 - Resuelto: Estructuras de Control](#)
 - [★ Ejercicio 1: El clasificador de notas sarcástico](#)
 - [★ Ejercicio 2: Tabla de multiplicar](#)
 - [★ ★ Ejercicio 3: División segura con try-catch](#)
 - [★ ★ Ejercicio 4: El triángulo constructor](#)
 - [★ ★ Ejercicio 5: Fibonacci con for](#)
 - [★ ★ ★ Ejercicio 6: AceptaElReto 340 — Juegos de naipes](#)
 - [★ ★ ★ Ejercicio 7: Excepción personalizada](#)
 - [★ ★ ★ Ejercicio 8: AceptaElReto 100 — Kaprekar](#)
 - [📖 Referencias](#)
- [Boletín 3 - Intermedio: Estructuras de Control](#)
 - [★ Ejercicio 1: Números pares e impares](#)
 - [★ Ejercicio 2: Suma acumulativa](#)
 - [★ ★ Ejercicio 3: Número perfecto](#)
 - [★ ★ Ejercicio 4: Menú con validación](#)
 - [★ ★ ★ Ejercicio 5: Criba de Eratóstenes \(números primos\)](#)
 - [★ ★ ★ Ejercicio 6: CodeWars — Vowel Count](#)
 - [★ ★ ★ Ejercicio 7: AceptaElReto — 151 ¿Es matriz identidad?](#)
 - [📖 Referencias](#)
- [Boletín 03 — Extras: Estructuras de Control y Excepciones](#)

- Boletín 4 - Inicial Resuelto: Algorítmica I
 - ★ Ejercicio 1: Búsqueda lineal
 - ★ Ejercicio 2: Búsqueda binaria
 - ★ ★ Ejercicio 3: Burbuja
 - ★ ★ Ejercicio 4: Inserción
 - ★ ★ Ejercicio 5: Contar pasos burbuja
 - ★ ★ ★ Ejercicio 6: Buscar en matriz ordenada
- Boletín 4 - Inicial: Algorítmica I
 - ★ Ejercicio 1: El más grande y el más pequeño
 - ★ Ejercicio 2: Suma total
 - ★ ★ Ejercicio 3: Buscador elemental
 - ★ ★ Ejercicio 4: Invertir array
 - ★ ★ Ejercicio 5: Media de doubles
 - ★ ★ ★ Ejercicio 6: CodeWars — Sum without highest and lowest number
 - ★ ★ ★ Ejercicio 7: CodeWars — You're a square!
- Boletín 4 - Intermedio Resuelto: Algorítmica I
 - ★ ★ Ejercicio 1: Pares con suma objetivo
 - ★ ★ Ejercicio 2: Eliminar duplicados (in-place)
 - ★ ★ ★ Ejercicio 3: Mayor suma de subarray (Kadane)
 - ★ ★ ★ Ejercicio 4: Rotar array k veces
 - ★ ★ ★ Ejercicio 5: Contar ocurrencias (binaria optimizada)
- Boletín 4 - Intermedio: Algorítmica I
 - ★ ★ Ejercicio 1: Mover ceros al final
 - ★ ★ Ejercicio 2: El número que falta
 - ★ ★ Ejercicio 3: Primer carácter no repetido
 - ★ ★ ★ Ejercicio 4: Anagramas
 - ★ ★ ★ Ejercicio 5: Ordenar 0s, 1s y 2s (Dutch National Flag)
 - ★ ★ ★ Ejercicio 6: CodeWars — Disemvowel Trolls
 - ★ ★ ★ Ejercicio 7: AceptaElReto — 219 La lotería
 -  Referencias
- Boletín 04 — Extras: Algorítmica I: Fundamentos
- Boletín 5 - Inicial Resuelto: Algorítmica II
 - ★ Ejercicio 1: Factorial recursivo

- [!\[\]\(eae20f1adff742df783f6f7e3bbe72d1_img.jpg\) Ejercicio 2: Fibonacci recursivo](#)
- [!\[\]\(43c6e08c5a1618d745b54da5c843274e_img.jpg\) !\[\]\(f5ee48910650695cea680b2433c1d60d_img.jpg\) Ejercicio 3: Suma recursiva de array](#)
- [!\[\]\(da0f02caffeb5a74776a1d5d1892b059_img.jpg\) !\[\]\(edb096eed27f3ac1241ba8d18d05acad_img.jpg\) Ejercicio 4: Invertir cadena](#)
- [!\[\]\(554d866cfdb5a2c8f73998019542d337_img.jpg\) !\[\]\(c1170582320733ace24db86bc6d97423_img.jpg\) !\[\]\(51c4b897e692428305845816e97ca71e_img.jpg\) Ejercicio 5: Fibonacci con memoización](#)
- [!\[\]\(810c0da19263e18e2f95623517bed1dc_img.jpg\) !\[\]\(8982e3c27257070d79f8096c6d667915_img.jpg\) !\[\]\(cf565e63df317467714cf7de070ecedc_img.jpg\) Ejercicio 6: Quicksort](#)
- [Boletín 5 - Inicial: Algorítmica II](#)
 - [Ejercicio 1: Potencia recursiva](#)
 - [Ejercicio 2: Contar dígitos recursivo](#)
 - [Ejercicio 3: ¿Qué imprime?](#)
 - [Ejercicio 4: Palíndromo recursivo](#)
 - [Ejercicio 5: Encuentra el error](#)
 - [Ejercicio 6: Sumatorio recursivo](#)
 - [Ejercicio 7: CodeWars — Reversed Strings](#)
 - [Ejercicio 8: AceptaElReto — 165 Número hyperpar](#)
- [Boletín 5 - Intermedio Resuelto: Algorítmica II](#)
 - [!\[\]\(76264d27c3f3933123acb961f2a449e7_img.jpg\) !\[\]\(8996ef01d8e0ad40113c18b518eacb7a_img.jpg\) Ejercicio 1: Torres de Hanoi](#)
 - [!\[\]\(8c5548d81cda50e4e278860b2cb9a70a_img.jpg\) !\[\]\(968380ddaf0d76ade5d8f4f1e1863930_img.jpg\) Ejercicio 2: Mergesort](#)
 - [!\[\]\(66c2875dd6880e6275ef3a346ad549a5_img.jpg\) !\[\]\(2ada657f1fdc0f1bc4a1fdfa02cd2c71_img.jpg\) !\[\]\(76f4e6395be77cbccf207fb6a912d80b_img.jpg\) Ejercicio 3: Búsqueda binaria recursiva](#)
 - [!\[\]\(f4f98dd477adfbaaa41172b7a4eb1b6f_img.jpg\) !\[\]\(b6e96929eea52c91e9f8575ba0ef04db_img.jpg\) !\[\]\(43e386126f14977d38517af086ca8272_img.jpg\) Ejercicio 4: Potencia rápida recursiva \(\$O\(\log n\)\$ \)](#)
 - [!\[\]\(24770a065ef897ee478ad8281a77157c_img.jpg\) !\[\]\(2b184a7e5ac47275077002c60f746e93_img.jpg\) !\[\]\(4f257034d4c8bf0f38c1b25f64531c6d_img.jpg\) Ejercicio 5: Subconjuntos \(backtracking\)](#)
- [Boletín 5 - Intermedio: Algorítmica II: Técnicas Avanzadas](#)
 - [!\[\]\(87dea8aae0859a12622bcc9b0f91d405_img.jpg\) Ejercicio 1: MCD por Euclides recursivo](#)
 - [!\[\]\(53681b0617bf7a5a2ca7e57adc711cdf_img.jpg\) Ejercicio 2: Contar ocurrencias recursivo](#)
 - [!\[\]\(486dac1986de10b6ebf546ce6d49a686_img.jpg\) !\[\]\(829cd040f98e850170614cd7b7a83afb_img.jpg\) Ejercicio 3: Invertir número recursivo](#)
 - [!\[\]\(d017b17ca9469c125b9fc39ccae8b424_img.jpg\) !\[\]\(e96995675751e0aaf26814ed33089d88_img.jpg\) Ejercicio 4: Combinaciones de un array \(backtracking\)](#)
 - [!\[\]\(d980fb959df077de6ec8adf76ca4415d_img.jpg\) !\[\]\(8a3ab67bb52e8721499298ae3f2b6a27_img.jpg\) !\[\]\(a2db61d04bd335432ce11d9cb9749faa_img.jpg\) Ejercicio 5: N-Reinas \(backtracking\)](#)
 - [!\[\]\(b331937a30dd0b320294f59f5755afba_img.jpg\) !\[\]\(27e83dcb0f111d5cd7466fae9d78591c_img.jpg\) !\[\]\(15c0601ce017a51578fe67d004260d62_img.jpg\) Ejercicio 6: CodeWars — Tribonacci Sequence](#)
 - [!\[\]\(4bede35f70f4535ef59852b293baa901_img.jpg\) !\[\]\(7efe6da23ff606518ea1e7fa153ab90c_img.jpg\) !\[\]\(2317ff60d74d28a52e58057884c2c1a8_img.jpg\) Ejercicio 7: AceptaElReto — 140 Suma de dígitos](#)
 - [!\[\]\(d7ce026f31a9811a89cfcaf9dc600322_img.jpg\) !\[\]\(6ae8b39a7b6251fafd9eb420017db3f1_img.jpg\) !\[\]\(0ee58d86e501a393865f2f3d6ca5fadc_img.jpg\) Ejercicio 8: AceptaElReto — 162 Suma de dígitos \(otra vez no\)](#)
 - [!\[\]\(ab33b6683a0a7c656563d9548d884625_img.jpg\) Referencias](#)
- [Boletín 05 — Extras: Algorítmica II: Técnicas Avanzadas](#)
- [Boletín 4 - Inicial Resuelto: POO - Clases y Objetos](#)
 - [Ejercicio 1: Completa la clase Perro](#)

- [Ejercicio 2: ¿Qué imprime?](#)
- [Ejercicio 3: Encuentra el error](#)
- [Ejercicio 4: Escribe la clase Persona](#)
- [Ejercicio 5: ¿Constructor por defecto?](#)
- [Ejercicio 6: AceptaElReto 291 — Números afortunados](#)
- [Ejercicio 7: CodeWars — Grasshopper - Summation](#)
- [Boletín 4 - Inicial: POO: Clases y Objetos](#)
 - [Ejercicio 1: Completa la clase Alumno](#)
 - [Ejercicio 2: ¿Qué imprime?](#)
 - [Ejercicio 3: Encuentra el error](#)
 - [Ejercicio 4: Escribe la clase Película](#)
 - [Ejercicio 5: Constructor vacío y setters](#)
 - [Ejercicio 6: AceptaElReto — 417 Números binomiales](#)
 - [Ejercicio 7: CodeWars — Return the day](#)
- [Boletín 4 - Resuelto: POO - Clases y Objetos](#)
 -  [Ejercicio 1: Clase Rectángulo](#)
 -  [Ejercicio 2: Clase CuentaBancaria](#)
 -   [Ejercicio 3: Clase Hora con validación](#)
 -   [Ejercicio 4: equals\(\) en Persona](#)
 -   [Ejercicio 5: Clase Punto y distancia euclídea](#)
 -    [Ejercicio 6: AceptaElReto 417 — Números binomiales](#)
 -    [Ejercicio 7: AceptaElReto 458 — El espejo](#)
 -    [Ejercicio 8: Simulación de batalla Pokémon \(lite\)](#)
 -  [Referencias](#)
- [Boletín 6 - Intermedio: POO: Clases y Objetos](#)
 -  [Ejercicio 1: Clase Libro](#)
 -  [Ejercicio 2: Clase Termómetro](#)
 -   [Ejercicio 3: Clase Fecha con validación](#)
 -   [Ejercicio 4: Clase Reloj con adelanto](#)
 -    [Ejercicio 5: Clase Matriz 2D](#)
 -    [Ejercicio 6: CodeWars — Build Tower](#)
 -    [Ejercicio 7: AceptaElReto — 246 Buscando el pin](#)
 -  [Referencias](#)

- [Boletín 06 — Extras: POO: Clases y Objetos](#)
- [Boletín 5 - Inicial Resuelto: Visibilidad, Encapsulación y Static](#)
 - [Ejercicio 1: Encuentra el error de visibilidad](#)
 - [Ejercicio 2: Completa los getters y setters](#)
 - [Ejercicio 3: ¿Qué imprime? Static vs instancia](#)
 - [Ejercicio 4: Escribe la clase utilitaria](#)
 - [Ejercicio 5: Escribe la clase Config](#)
 - [Ejercicio 6: AceptaElReto 106 — Código de barras](#)
 - [Ejercicio 7: CodeWars — Convert boolean to string](#)
- [Boletín 5 - Inicial: Visibilidad, Encapsulación y Static](#)
 - [Ejercicio 1: Encuentra el error de visibilidad \(la nevera\)](#)
 - [Ejercicio 2: Completa los getters y setters de la clase Producto](#)
 - [Ejercicio 3: ¿Qué imprime? Static vs instancia II](#)
 - [Ejercicio 4: Escribe la clase utilitaria MathUtils](#)
 - [Ejercicio 5: Escribe la clase AppConfig](#)
 - [Ejercicio 6: AceptaElReto — 117 Encontrar el mayor](#)
 - [Ejercicio 7: CodeWars — Opposite number](#)
- [Boletín 5 - Resuelto: Visibilidad, Encapsulación y Static](#)
 - [★ Ejercicio 1: Clase Empleado con validación](#)
 - [★ Ejercicio 2: Círculo encapsulado](#)
 - [★ ★ Ejercicio 3: JavaBean Alumno](#)
 - [★ ★ Ejercicio 4: Contador de objetos con static](#)
 - [★ ★ Ejercicio 5: Conversor de unidades \(estáticos\)](#)
 - [★ ★ ★ Ejercicio 6: AceptaElReto 364 — Spiderman](#)
 - [★ ★ ★ Ejercicio 7: AceptaElReto 462 — Tres dedos](#)
 - [★ ★ ★ Ejercicio 8: Simulación estática](#)
 - [!\[\]\(694fcb4611893e9db5249daba48abfc1_img.jpg\) Referencias](#)
- [Boletín 5 - Intermedio: Visibilidad, Encapsulación y Static](#)
 - [★ Ejercicio 1: Clase CuentaAhorros encapsulada](#)
 - [★ Ejercicio 2: Clase Persona con validación estricta](#)
 - [★ ★ Ejercicio 3: Clase TicketCompra con ID autoincremental](#)
 - [★ ★ Ejercicio 4: Clase Biblioteca con contador estático](#)
 - [★ ★ ★ Ejercicio 5: Clase CalculadoraEstadística utilitaria](#)

- [!\[\]\(7e19807c61da14f515588e95cd49886c_img.jpg\) Ejercicio 6: CodeWars — Find the odd int](#)
- [!\[\]\(8ff9e60a4b0560d7ec99179ef4779d9e_img.jpg\) Ejercicio 7: AceptaElReto — 120 Número de pares y nones](#)
- [!\[\]\(ab9b69bf5753a01c76b30af859454360_img.jpg\) Referencias](#)
- [Boletín 07 — Extras: Visibilidad, Encapsulación y Static](#)
- [Boletín 6 - Inicial Resuelto: Herencia y Polimorfismo](#)
 - [Ejercicio 1: ¿Qué imprime? — La llamada a la familia](#)
 - [Ejercicio 2: Encuentra el error — extends mal usado](#)
 - [Ejercicio 3: Completa el código — the instanceof](#)
 - [Ejercicio 4: Escribe este programa — la jerarquía de animales](#)
 - [Ejercicio 5: ¿Qué imprime? — Referencias polimórficas](#)
 - [Ejercicio 6: Encuentra el error — polimorfismo y casting](#)
 - [Ejercicio 7: Escribe este programa — el zoo polimórfico](#)
 - [!\[\]\(c5af66b13c724ca428497900cdbbc9b3_img.jpg\) Referencias para seguir practicando](#)
- [Boletín 6 - Inicial: Herencia y Polimorfismo](#)
 - [Ejercicio 1: ¿Qué imprime? — La familia musical](#)
 - [Ejercicio 2: Encuentra el error — extends mal usado II](#)
 - [Ejercicio 3: Completa el código — instanceof con downcasting](#)
 - [Ejercicio 4: Escribe este programa — la herencia de vehículos](#)
 - [Ejercicio 5: ¿Qué imprime? — Polimorfismo con referencias](#)
 - [Ejercicio 6: Encuentra el error — ClassCastException](#)
 - [Ejercicio 7: Escribe este programa — la granja polimórfica](#)
 - [!\[\]\(1fde827780c8f912fd3ae9174d52d155_img.jpg\) Referencias para seguir practicando](#)
- [Boletín 6 - Resuelto: Herencia y Polimorfismo](#)
 - [!\[\]\(49ab9fdb6ddb6816bcb8ccc012d5cebd_img.jpg\) Ejercicio 1: La orquesta polimórfica](#)
 - [!\[\]\(a10cf212d457430b842f8ac59c63db70_img.jpg\) Ejercicio 2: Herencia de constructores — la cadena alimenticia](#)
 - [!\[\]\(e8a826213cf8b53a8c13f5432344afc9_img.jpg\) Ejercicio 3: El problema del jarrón \(Fragile Base Class\)](#)
 - [!\[\]\(7ffe3c6e7552aa3eb962276cd7a9a979_img.jpg\) Ejercicio 4: Batalla de personajes](#)
 - [!\[\]\(28e94a65fe1d8cf887928bbaaa2c7303_img.jpg\) Ejercicio 5: La calculadora de figuras](#)
 - [!\[\]\(7db790dc622e1ac5f1c44afb7a5212a6_img.jpg\) Ejercicio 6: Downcasting seguro](#)
 - [!\[\]\(86147531a4f05b1215989ff8ab43fe6d_img.jpg\) Ejercicio 7 \(ProgramaMe\): El simulador de ecosistema](#)
 - [!\[\]\(c3492017d65b370ec6b463430fff1ce7_img.jpg\) Ejercicio 8 \(ProgramaMe\): Vehículos con combustible](#)
 - [!\[\]\(eadeaa5506f71c8d915378340dd044f1_img.jpg\) Referencias para seguir practicando](#)
- [Boletín 6 - Intermedio: Herencia, Polimorfismo e Interfaces](#)

- [!\[\]\(effbd7993c63c039a58fd3395789cf3f_img.jpg\) Ejercicio 1: Interfaz FiguraGeometrica](#)
- [!\[\]\(144980d038f2541d7b588a8a9132bd70_img.jpg\) Ejercicio 2: Jerarquía de Empleados](#)
- [!\[\]\(c4ce2d477989700c971cf3d240ad9283_img.jpg\) !\[\]\(5013555a72072875cb154b597e002a46_img.jpg\) Ejercicio 3: Sistema de pagos con interfaz](#)
- [!\[\]\(bf2038c114ec21ea58ad011774351c98_img.jpg\) !\[\]\(1ad0c3425edfa4762c2f20e33e3e5bbf_img.jpg\) Ejercicio 4: Interfaces múltiples: Volador y Nadador](#)
- [!\[\]\(74d2fc5645add84f8511beb934060048_img.jpg\) !\[\]\(ddc5533caa0187e636e3d3234e0983a3_img.jpg\) !\[\]\(7c207f8f59385c6dd11f9d9bdc7a0d1d_img.jpg\) Ejercicio 5: Sistema de notificaciones polimórfico](#)
- [!\[\]\(57a1bf910af99362b80b3ac4f2eecbac_img.jpg\) !\[\]\(6b114000ab07dda576e2920e2dc838fa_img.jpg\) !\[\]\(9129a6a4a4b11facb5cf665660eef788_img.jpg\) Ejercicio 6: CodeWars — Is this a triangle?](#)
- [!\[\]\(7cbd4924d31aa4c46df484d5c5bf4696_img.jpg\) !\[\]\(b3461332979d340882fda628798720af_img.jpg\) !\[\]\(5dc380853f1779b9d7a8d6fbbcdbb357_img.jpg\) Ejercicio 7: AceptaElReto — 154 El ascensor](#)
- [!\[\]\(2894a5ead456ff0145f35b01b4a6c7df_img.jpg\) Referencias](#)
- [Boletín 08 — Extras: Herencia, Polimorfismo e Interfaces](#)
- [Boletín 8 - Inicial Resuelto: Arrays y Colecciones](#)
 - [Ejercicio 1: ¿Qué imprime? — Array básico](#)
 - [Ejercicio 2: Encuentra el error — ArrayIndexOutOfBoundsException](#)
 - [Ejercicio 3: Completa el código — for-each](#)
 - [Ejercicio 4: Escribe este programa — Array de 10 enteros al revés](#)
 - [Ejercicio 5: ¿Qué imprime? — ArrayList misterioso](#)
 - [Ejercicio 6: Encuentra el error — colección sin genérico](#)
 - [Ejercicio 7: Escribe este programa — HashSet sin duplicados](#)
 - [!\[\]\(6b9cc5709aff55a0cbbb7dd78fcf3ba1_img.jpg\) Referencias para seguir practicando](#)
- [Boletín 8 - Inicial: Arrays y Colecciones](#)
 - [Ejercicio 1: ¿Qué imprime? — Array de booleanos](#)
 - [Ejercicio 2: Encuentra el error — NullPointerException](#)
 - [Ejercicio 3: Completa el código — for básico para buscar el mayor](#)
 - [Ejercicio 4: Escribe este programa — contar números pares](#)
 - [Ejercicio 5: ¿Qué imprime? — ArrayList remove por índice vs valor](#)
 - [Ejercicio 6: Encuentra el error — length vs length\(\)](#)
 - [Ejercicio 7: Escribe este programa — búsqueda lineal](#)
 - [!\[\]\(cd4202444435d9e729cdf801e3403cf4_img.jpg\) Referencias para seguir practicando](#)
- [Boletín 8 - Resuelto: Arrays y Colecciones](#)
 - [!\[\]\(a595a22b25819b91e6cd221ac336245d_img.jpg\) Ejercicio 1: El inversor de arrays](#)
 - [!\[\]\(912cec22f21865fe086f1aaa377b4c97_img.jpg\) Ejercicio 2: ArrayList de números pares](#)
 - [!\[\]\(d66d6cd7d0eca6e4b13ae9e87c7f7f49_img.jpg\) Ejercicio 3: Estadísticas con array](#)
 - [!\[\]\(2e58555536753c4dae915c3689ae8744_img.jpg\) !\[\]\(704be90457dfbf7435f703bdc5293a9e_img.jpg\) Ejercicio 4: Buscaminas simplificado](#)
 - [!\[\]\(9a8314206d1a19a24da8a71532ac6fea_img.jpg\) !\[\]\(d0819331239cfe31193900cc06944bf3_img.jpg\) Ejercicio 5: Lista de la compra con ArrayList](#)

- [!\[\]\(83eb2aa26b610eb6a9dca7cf4702d681_img.jpg\) Ejercicio 6: HashSet y TreeSet — eliminando duplicados](#)
- [!\[\]\(94dfacbf937cdd7da4837a6fcd8fc785_img.jpg\) Ejercicio 7 \(ProgramaMe\): El juego de la memoria](#)
- [!\[\]\(dae8c3c5fa7c80febd6526a5e8a853bf_img.jpg\) Ejercicio 8 \(ProgramaMe\): Estadísticas de texto con colecciones](#)
- [!\[\]\(8f38ab9775d1331a4e1fd6648d0a83f1_img.jpg\) Referencias para seguir practicando](#)
- [Boletín 8 - Intermedio: Arrays y Colecciones](#)
 - [!\[\]\(5e48b3241d711ef916255d822ab3415f_img.jpg\) Ejercicio 1: La fusión de arrays ordenados](#)
 - [!\[\]\(c4c3604751fde0df44855086d7798e30_img.jpg\) Ejercicio 2: Rotación circular a la derecha](#)
 - [!\[\]\(a16ee0329f478adb49fd7876490e96fe_img.jpg\) Ejercicio 3: Suma de diagonales \(matriz cuadrada\)](#)
 - [!\[\]\(a8a494e883c31c6f4a74584cc935a790_img.jpg\) Ejercicio 4: Cola del supermercado con LinkedList](#)
 - [!\[\]\(8a50a875cd94cd57f6a7348a4d34b45f_img.jpg\) Ejercicio 5: Intersección y unión de conjuntos](#)
 - [!\[\]\(7ddc0498d10972f0f157498fccf65370_img.jpg\) Ejercicio 6: Eliminar duplicados manteniendo orden](#)
 - [!\[\]\(1481f3a2b8d8d64dbbf1eb2242b57620_img.jpg\) Ejercicio 7 \(CodeWars\): Find the odd int](#)
 - [!\[\]\(1d12b5523dd7503f9f6b056602d4b0bb_img.jpg\) Ejercicio 8 \(AceptaElReto\): La lotería de la peña](#)
 - [!\[\]\(a3cf9896eece75d50b7dee194989c9bc_img.jpg\) Referencias](#)
- [Boletín 09 — Extras: Arrays y Colecciones](#)
- [Boletín 9 - Inicial Resuelto: Genéricos y Mapas](#)
 - [Ejercicio 1: Completa el código — clase genérica](#)
 - [Ejercicio 2: ¿Qué imprime? — HashMap básico](#)
 - [Ejercicio 3: Encuentra el error — tipo primitivo en genérico](#)
 - [Ejercicio 4: Escribe este programa — mini agenda HashMap](#)
 - [Ejercicio 5: Completa el código — getOrDefault](#)
 - [Ejercicio 6: ¿Qué imprime? — método genérico](#)
 - [Ejercicio 7: Encuentra el error — la clave mutable](#)
 - [!\[\]\(26147612c3694ee09d020cac0faae7d9_img.jpg\) Referencias para seguir practicando](#)
- [Boletín 9 - Inicial: Genéricos y Mapas](#)
 - [Ejercicio 1: Completa el código — clase con dos tipos genéricos](#)
 - [Ejercicio 2: ¿Qué imprime? — HashMap con merge](#)
 - [Ejercicio 3: Encuentra el error — TreeSet sin Comparable](#)
 - [Ejercicio 4: Escribe este programa — contador de palabras con HashMap](#)
 - [Ejercicio 5: ¿Qué imprime? — método genérico con límite](#)
 - [Ejercicio 6: Encuentra el error — clave duplicada el primero se pierde](#)
 - [Ejercicio 7: Escribe este programa — TreeMap con valores por defecto](#)
 - [!\[\]\(46af714d66844046465576855b12c56a_img.jpg\) Referencias para seguir practicando](#)

- [Boletín 9 - Resuelto: Genéricos y Mapas](#)
 - [★ Ejercicio 1: Clase genérica Pareja](#)
 - [★ Ejercicio 2: Contador de palabras con HashMap](#)
 - [★ Ejercicio 3: Método genérico maximo](#)
 - [★ ★ Ejercicio 4: Agenda completa con menú](#)
 - [★ ★ Ejercicio 5: TreeMap — ordenando por clave](#)
 - [★ ★ ★ Ejercicio 6 \(ProgramaMe\): Wildcards — el método de comparación](#)
 - [★ ★ ★ Ejercicio 7 \(ProgramaMe\): Sistema de votaciones](#)
 - [★ ★ ★ Ejercicio 8 \(ProgramaMe\): Cache simple con genéricos](#)
 - [🔗 Referencias para seguir practicando](#)
- [Boletín 9 - Intermedio: Genéricos y Mapas](#)
 - [★ Ejercicio 1: Pila genérica](#)
 - [★ Ejercicio 2: HashMap inverso](#)
 - [★ Ejercicio 3: Método genérico — filtrar por condición](#)
 - [★ ★ Ejercicio 4: Caché LRU con LinkedHashMap](#)
 - [★ ★ Ejercicio 5: TreeMap — frecuencia de letras](#)
 - [★ ★ Ejercicio 6 \(Wildcards\): Suma de números genérica](#)
 - [★ ★ ★ Ejercicio 7 \(ProgramaMe\): Sistema de inventario genérico](#)
 - [★ ★ ★ Ejercicio 8 \(CodeWars + AceptaElReto\)](#)
 - [📚 Referencias](#)
- [Boletín 10 — Extras: Genéricos y Mapas](#)
- [Boletín 10 - Inicial Resuelto: Consola y Ficheros](#)
 - [Ejercicio 1: ¿Qué imprime? — printf\(\)](#)
 - [Ejercicio 2: Encuentra el error — nextInt\(\) + nextLine\(\)](#)
 - [Ejercicio 3: Completa el código — leer archivo](#)
 - [Ejercicio 4: Escribe este programa — Hola mundo archivo](#)
 - [Ejercicio 5: ¿Qué imprime? — el contador de líneas](#)
 - [Ejercicio 6: Encuentra el error — archivo no cerrado](#)
 - [Ejercicio 7: Escribe este programa — Scanner que suma números](#)
 - [🔗 Referencias para seguir practicando](#)
- [Boletín 10 - Inicial: Consola, Ficheros y Regex](#)
 - [Ejercicio 1: ¿Qué imprime? — printf con conversiones](#)
 - [Ejercicio 2: Encuentra el error — IOException sin capturar](#)

- [Ejercicio 3: Completa el código — try-with-resources](#)
- [Ejercicio 4: Escribe este programa — guardar array en archivo](#)
- [Ejercicio 5: ¿Qué imprime? — Scanner next vs nextLine](#)
- [Ejercicio 6: Encuentra el error — File.createNewFile sin comprobar](#)
- [Ejercicio 7: Escribe este programa — menú con Scanner y switch](#)
- [🔗 Referencias para seguir practicando](#)
- [Boletín 10 - Resuelto: Consola y Ficheros](#)
 - [★ Ejercicio 1: La entrevista de trabajo](#)
 - [★ Ejercicio 2: Calculadora de propinas](#)
 - [★ Ejercicio 3: El diario personal \(append\)](#)
 - [★ ★ Ejercicio 4: El contador de líneas, palabras y caracteres](#)
 - [★ ★ Ejercicio 5: El gestor de contactos \(archivo\)](#)
 - [★ ★ Ejercicio 6: Lectura con NIO \(Files y Paths\)](#)
 - [★ ★ ★ Ejercicio 7 \(ProgramaMe\): Cifrado César con archivos](#)
 - [★ ★ ★ Ejercicio 8 \(ProgramaMe\): Serialización de objetos](#)
 - [🔗 Referencias para seguir practicando](#)
- [Boletín 10 - Intermedio: Consola, Ficheros y Regex](#)
 - [★ Ejercicio 1: Formateador de ticket de compra](#)
 - [★ Ejercicio 2: Buscador de archivos por extensión](#)
 - [★ Ejercicio 3: Lector de CSV con Scanner](#)
 - [★ ★ Ejercicio 4: Filtro de líneas por palabra clave](#)
 - [★ ★ Ejercicio 5: Separador de líneas pares e impares](#)
 - [★ ★ Ejercicio 6: Split con regex — analizador de frases](#)
 - [★ ★ ★ Ejercicio 7 \(ProgramaMe\): Validador de datos con regex](#)
 - [★ ★ ★ Ejercicio 8 \(CodeWars + AceptaElReto\)](#)
 - [📚 Referencias](#)
- [Boletín 11 — Extras: Consola, Ficheros y Expresiones Regulares](#)
- [Boletín 14 - Inicial Resuelto: JDBC - Conexión y Consultas](#)
 - [Ejercicio 1: Los 5 pasos](#)
 - [Ejercicio 2: Completa la conexión](#)
 - [Ejercicio 3: ¿Qué imprime?](#)
 - [Ejercicio 4: Encuentra el error](#)
 - [Ejercicio 5: INSERT con PreparedStatement](#)

- [Ejercicio 6: ¿Qué hace executeUpdate\(\)?](#)
- [Ejercicio 7: Diferencia entre executeQuery y executeUpdate](#)
- [Boletín 14 - Inicial: Conexión a BD con JDBC](#)
 - [Ejercicio 1: ¿Qué necesitas para usar JDBC?](#)
 - [Ejercicio 2: Completa el código — try-with-resources](#)
 - [Ejercicio 3: ¿Qué imprime? — ResultSet vacío](#)
 - [Ejercicio 4: Encuentra el error — SQLException sin manejo](#)
 - [Ejercicio 5: Escribe este programa — mostrar nombres de columnas](#)
 - [Ejercicio 6: ¿Qué imprime? — executeQuery en UPDATE](#)
 - [Ejercicio 7: Encuentra el error — PreparedStatement con índices incorrectos](#)
 - [Ejercicio 8: Completa el código — cerrar recursos](#)
 - [🔗 Referencias para seguir practicando](#)
- [Boletín 14 - Resuelto: JDBC - Conexión y Consultas](#)
 - [★ Ejercicio 2: INSERT con PreparedStatement](#)
 - [★ Ejercicio 3: UPDATE con PreparedStatement](#)
 - [★ Ejercicio 4: DELETE con PreparedStatement](#)
 - [★ ★ Ejercicio 5: Búsqueda por nombre con LIKE](#)
 - [★ ★ Ejercicio 6: Contar alumnos por curso](#)
 - [★ ★ Ejercicio 7: Transacciones básicas](#)
 - [★ ★ ★ Ejercicio 8: CRUD completo con menú](#)
 - [★ ★ ★ Ejercicio 9: Paginación con LIMIT y OFFSET](#)
 - [★ ★ ★ Ejercicio 10: Exportar a CSV con metadatos](#)
- [Boletín 14 - Intermedio: Conexión a BD con JDBC](#)
 - [★ Ejercicio 1: Conexión desde archivo de propiedades](#)
 - [★ Ejercicio 2: INSERT con clave autogenerada](#)
 - [★ Ejercicio 3: UPDATE condicional](#)
 - [★ ★ Ejercicio 4: INNER JOIN con PreparedStatement](#)
 - [★ ★ Ejercicio 5: Fechas en JDBC](#)
 - [★ ★ Ejercicio 6: Batch INSERT — 100 alumnos de prueba](#)
 - [★ ★ ★ Ejercicio 7 \(ProgramaMe\): Patrón DAO](#)
 - [★ ★ ★ Ejercicio 8 \(CodeWars + AceptaElReto\)](#)
 - [📚 Referencias](#)
- [Boletín 12 — Extras: Conexión a Bases de Datos con JDBC](#)

- Boletín 11 - Inicial resuelto: Interfaz Web
 - Ejercicio 1: Hola Mundo Web
 - Ejercicio 2: Hora del servidor
 - Ejercicio 3: HTML bonito
 - Ejercicio 4: Saludo personalizado
 - Ejercicio 5: Contador de visitas
 - Ejercicio 6: Tabla HTML
- Boletín 13 - Inicial: Servir y Consumir APIs con Web
 - Ejercicio 1: Servidor de la hora
 - Ejercicio 2: Página que dice tu nombre
 - Ejercicio 3: Contador de visitas global
 - Ejercicio 4: Generador de excusas para entregas tarde
 - Ejercicio 5: Tabla de multiplicar personalizada
 - Ejercicio 6: Página de estado del servidor
 - Ejercicio 7: Conversor de euros a pesetas (sí, pesetas)
 -  Referencias
- Boletín 11 - Resuelto: Interfaz Web
 -  Ejercicio 1: Servidor de archivos estáticos
 -  Ejercicio 2: Formulario de contacto
 -   Ejercicio 3: API JSON de productos
 -   Ejercicio 4: Chat en memoria
 -    Ejercicio 5: Mini framework MVC
 -    Ejercicio 6: Subida de archivos
- Boletín 13 - Intermedio: Servir y Consumir APIs con Web
 -  Ejercicio 1: API de frases motivacionales
 -  Ejercicio 2: Traductor chungo (pero funcional)
 -   Ejercicio 3: API REST de tareas con prioridad
 -   Ejercicio 4: El tiempo que NO hace
 -   Ejercicio 5: Catálogo de películas con filtros
 -    Ejercicio 6: Middleware de logging
 -    Ejercicio 7: Server-Sent Events — El reloj del servidor
 -  Ejercicio 8: Piedra, papel, tijera online
 -  Referencias

- [Boletín 13 — Extras: Servir y Consumir APIs con Web](#)

APUNTES

Programación Java

— DAM/DAW —

```
1 public class HolaMundo {  
2     public static void main(String[] args) {  
3         // Paso de entrada del programa  
4         System.out.println("¡Hola, Java!");  
5     }  
6 }
```

clase

método principal

imprime
en consola

7
8
9
10
11
12
13
14



010101



Por

Sergi García Barea

licencia
CC BY SA



[🚀 Empezar por la Unidad 1](#)[🐙 Ver en GitHub](#)

Sergi Garcia Barea — CC BY-SA 4.0

[↓ ¿Prefieres estudiar sin conexión? Descarga el curso completo](#)[📄 PDF \(Castellano\)](#)[📄 PDF \(Valencià\)](#)[📖 EPUB \(Castellano\)](#)[📖 EPUB \(Valencià\)](#)

Unidades

UNIDAD 1

RA1

Introducción a Java

Primer contacto con Java: instala el JDK, escribe tu primer programa, conoce el método `main`, los comentarios y los argumentos de línea de comandos.

[👉 Ver unidad 👈](#)

UNIDAD 2

RA2

Variables, Tipos y Operadores

Declara variables, usa tipos primitivos, operadores aritméticos y lógicos, conversiones de tipo y lee datos por teclado con `Scanner`.

[👉 Ver unidad 👈](#)

UNIDAD 3

RA3

Estructuras de Control y Excepciones

Domina `if / else`, `switch`, bucles `while` y `for`, y maneja excepciones con `try / catch` para que tu programa nunca se rompa.

[👉 Ver unidad 👈](#)

UNIDAD 4

RA2, RA6

Algorítmica I: Fundamentos

Aprende a pensar como un programador: divide problemas en partes, usa pseudocódigo, diagramas de flujo y crea funciones reutilizables.

[👉 Ver unidad 👈](#)

UNIDAD 5

RA2, RA6

Algorítmica II: Técnicas Avanzadas

Algoritmos de ordenación, búsqueda binaria, recursividad y técnicas divide y vencerás para resolver problemas más complejos.

[👉 Ver unidad 👈](#)

UNIDAD 6

RA2, RA4

POO: Clases y Objetos

Programación Orientada a Objetos: crea clases, instancia objetos, define atributos y métodos, y entiende la magia de los constructores.

[👉 Ver unidad 👈](#)

UNIDAD 7

RA4

Visibilidad, Encapsulación y Static

Controla quién ve qué: modificadores de acceso (`public`, `private`, `protected`), encapsulación con getters/setters, miembros `static` y constantes.

[← Ver unidad →](#)**UNIDAD 8**

RA4, RA7

Herencia, Polimorfismo e Interfaces

Herencia, polimorfismo, clases abstractas e interfaces: la base del diseño flexible y reutilizable en Java. Aprende a sobrescribir métodos y a usar `super`.

[← Ver unidad →](#)**UNIDAD 9**

RA6

Arrays y Colecciones

Arrays unidimensionales y multidimensionales, la clase `Arrays`, `ArrayList`, `LinkedList` y cómo elegir la colección adecuada para cada problema.

[← Ver unidad →](#)**UNIDAD 10**

RA6

Genéricos y Mapas

Genéricos para clases y métodos seguros de tipos, la interfaz `Map` y sus implementaciones `HashMap`, `TreeMap`, y cómo iterar sobre ellas.

[← Ver unidad →](#)**UNIDAD 11**

RA5

Consola, Ficheros y Regex

Entrada/salida por consola, lectura y escritura de ficheros de texto y binarios, serialización de objetos y expresiones regulares para buscar patrones.

[← Ver unidad →](#)**UNIDAD 12**

RA9

Conexión a BD con JDBC

Conecta Java con bases de datos relacionales usando JDBC: `Connection`, `Statement`, consultas, inserciones, actualizaciones y transacciones seguras.

[← Ver unidad →](#)**UNIDAD 13**

RA5

Servir y Consumir APIs con Web

Crea un servidor HTTP con Java `HttpServer`, sirve páginas HTML/JS, maneja peticiones JSON, implementa una API REST completa desde cero.

[← Ver unidad →](#)**Boletines****UNIDAD 1**

Boletines de la unidad

-  Inicial resuelto
-  Inicial
-  Avanzado resuelto
-  Avanzado
-  Extras

UNIDAD 2

Boletines de la unidad

-  Inicial resuelto
-  Inicial
-  Intermedio resuelto
-  Intermedio
-  Extras

UNIDAD 3

Boletines de la unidad

-  Inicial resuelto
-  Inicial
-  Intermedio resuelto
-  Intermedio
-  Extras

UNIDAD 4

Boletines de la unidad

-  Inicial resuelto
-  Inicial
-  Intermedio resuelto
-  Intermedio
-  Extras

UNIDAD 5

Boletines de la unidad

-  Inicial resuelto
-  Inicial
-  Intermedio resuelto
-  Intermedio
-  Extras

UNIDAD 6

Boletines de la unidad

-  Inicial resuelto
-  Inicial
-  Intermedio resuelto
-  Intermedio
-  Extras

UNIDAD 7

Boletines de la unidad

-  Inicial resuelto
-  Inicial
-  Intermedio resuelto
-  Intermedio
-  Extras

UNIDAD 8

Boletines de la unidad

-  Inicial resuelto
-  Inicial
-  Intermedio resuelto
-  Intermedio
-  Extras

UNIDAD 9

Boletines de la unidad

-  Inicial resuelto
-  Inicial
-  Intermedio resuelto
-  Intermedio
-  Extras

UNIDAD 10

Boletines de la unidad

-  Inicial resuelto
-  Inicial
-  Intermedio resuelto
-  Intermedio
-  Extras

UNIDAD 11

UNIDAD 12

Boletines de la unidad

-  Inicial resuelto
-  Inicial
-  Intermedio resuelto
-  Intermedio
-  Extras

Boletines de la unidad

-  Inicial resuelto
-  Inicial
-  Intermedio resuelto
-  Intermedio
-  Extras

UNIDAD 13

Boletines de la unidad

-  Inicial resuelto
-  Inicial
-  Intermedio resuelto
-  Intermedio
-  Extras

Licencia



CC BY-SA 4.0 — *Sergi Garcia Barea*

[Licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional](https://creativecommons.org/licenses/by-sa/4.0/)

Puedes compartir y adaptar el material para cualquier propósito, incluso comercial, siempre que reconozcas la autoría y compartas las modificaciones bajo la misma licencia.

U01 — Introducción a Java

Tu primer café con Java ☕

 **El mapa del paquete:**  Bienvenida →  **AQUÍ ESTÁS (U01)** →  Variables → 
Control →  Algorítmica →  Técnicas →  POO →  Visibilidad →  Herencia → 
Colecciones →  Genéricos →  Ficheros →  JDBC →  APIs

Bienvenido al curso de Programación en Java. ¿Sabías que tu ordenador es básicamente un cachorro muy listo pero con cero iniciativa? No hace nada hasta que le das órdenes precisas. Y para eso necesitamos un **entorno de desarrollo**, un lenguaje y muchas ganas de compilar.

Esta primera unidad te deja en la puerta del castillo: vas a instalar el JDK, a escribir tu primer programa, a conocer la trilogía del café (JVM, JRE y JDK), a usar el depurador como un detective y a hablar con la máquina mediante comentarios y argumentos. No necesitas experiencia previa: solo un editor de texto y curiosidad.

Esta unidad se lee como un **libro de 9 capítulos**: cada punto desarrolla una idea completa y enlaza con el siguiente.

Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar la diferencia entre **JDK**, **JRE** y **JVM** con la analogía de la cafetería.
- Instalar y configurar un JDK y comprobar tu instalación con `java -version`.
- Escribir, compilar y ejecutar tu primer programa Java.
- Diseccionar el método `main` y entender cada pieza del esqueleto de un programa.
- Usar el **depurador** (breakpoints, step over/into, watch) como un detective.
- Escribir comentarios de una línea, de bloque y **Javadoc**.
- Manejar los **argumentos de línea de comandos** con `args`.
- Reconocer y corregir los errores de compilación más típicos del novato.
- Trabajar con un **IDE** y conocer tu flujo de trabajo diario.

Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · ¿Qué es Java?	Historia, bytecode, la trilogía JVM/JRE/JDK	Todos
02 · Instalación del JDK	Eclipse Temurin, java -version, PATH, IDE	Todos
03 · Tu primer programa	Hola Mundo, estructura, el método main	Todos
04 · El depurador	Breakpoints, step over/into, watch	Todos
05 · Comentarios y documentación	//, /* */, /** */ y Javadoc	Todos
06 · Argumentos de línea de comandos	El array args, cómo leerlo y usarlo	Todos
07 · El compilador y sus errores	Compilar vs ejecutar, errores típicos del novato	Todos
08 · El IDE y tu flujo de trabajo	Proyectos, atajos, crear/ejecutar/depurar	Todos
09 · Repaso interactivo	Sé el Código, Fireside, Quién Soy, Laboratorio, Crucigrama...	Todos

 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

Boletines de la unidad

Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

☒ Inicial resuelto
 ☐ Inicial por resolver
  Avanzado resuelto
 ☐ Avanzado por resolver
  Extras

☒ Criterios de evaluación cubiertos (RA1)

RA1: Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.

CE	Criterio	Dónde se cubre
a)	Bloques que componen la estructura de un programa	✓ Puntos 3 y 7
b)	Proyectos de desarrollo de aplicaciones	✓ Punto 8
c)	Entornos integrados de desarrollo	✓ Puntos 2 y 8
i)	Comentarios en el código	✓ Punto 5

✦ El depurador que se ve en el punto 4 sienta las bases del RA3 (probado y depurado de programas), que se completa en la U03.

¿Por dónde empiezo?

- ¿Cero conocimientos? → Arranca en el [punto 1](#), no hay ningún requisito previo.
- ¿Ya sabes qué es un IDE? → Ve directo al [punto 3](#) y vuelve atrás solo si te pierdes.
- ¿Vienes a repasar? → Haz el [Repaso interactivo](#) y los [boletines](#).

📌 Primer punto: [01 · ¿Qué es Java?](#)

▶▶ Al acabar la unidad, continúa en [U02 · Variables, Tipos y Operadores](#).

01 — ¿Qué es Java?

La trilogía del café y el secreto de la taza humeante ☕

📖 Estás en: 🚀 U01 · Introducción a Java → 01 · ¿Qué es Java?

📌 La idea en una frase

Java es un lenguaje de programación que se ejecuta dentro de una máquina virtual (la JVM), lo que le permite correr igual en Windows, Linux o macOS: “escribe una vez, corre en cualquier sitio”.

Si tu ordenador es un cachorro muy listo pero con cero iniciativa, Java es uno de los idiomas que puedes usar para darle órdenes. Y la JVM es el intérprete que traduce tus órdenes al idioma concreto de cada cachorro.

🔍 ¿De dónde sale este lenguaje?

Java nació en **1995** en **Sun Microsystems**, y su historia tiene una de las curiosidades más divertidas del sector: se inspiró en la **máquina de café** de la oficina. Por eso el logo es una taza humeante. No me lo invento.

La idea de sus creadores era simple pero ambiciosa: un lenguaje que funcionara en **cualquier dispositivo**, sin importar el sistema operativo ni el hardware. Antes de Java, cada plataforma necesitaba su propia versión del programa. Java propuso la solución: no programar para el ordenador, sino para una **máquina virtual** que el ordenador simula.

💡 **Dato freak:** el lenguaje se llamó primero *Oak* (roble), por un árbol que se veía desde la oficina. Tuvieron que cambiarlo por motivos de marca registrada y, según la leyenda, el nombre “Java” se eligió por el café que tomaban durante las reuniones.

📦 ¿Qué es eso del bytecode?

Cuando escribes Java no escribes instrucciones de la CPU de tu ordenador: escribes instrucciones para la **JVM**. El proceso es así:

1. Tú escribes código fuente en un archivo `.java`.

2. El **compilador** (javac) lo traduce a *bytecode*: un idioma intermedio que entiende la JVM. El resultado es un archivo `.class`.
3. La **JVM** lee el bytecode y lo ejecuta en tu máquina concreta.

Tu código (`.java`) --javac--> bytecode (`.class`) --JVM--> ¡se ejecuta!

La gracia es que ese `.class` es el mismo para todas las plataformas: la JVM de Windows, la de Linux y la de macOS saben leerlo. Solo cambia la JVM, no tu programa.



La Trilogía del Café: JVM, JRE y JDK

Aquí está el momento estrella de la unidad. Java funciona como una **cafetería de especialidad**, y tienes que memorizar quién es quién:

Concepto	Es...	Analogía
JVM	La máquina que ejecuta el bytecode	La máquina de café: tiene su receta y funciona igual en cualquier sitio
JRE	Todo lo necesario para <i>ejecutar</i> Java	La cafetería entera: máquina, vasos, azúcar...
JDK	Todo lo necesario para <i>crear</i> programas	El kit completo para montar tu cafetería: máquina, granos, molinillo y manual de barista

- **JVM (Java Virtual Machine)**: la máquina de café. Ejecuta el bytecode y viaja con tu programa a todas partes.
- **JRE (Java Runtime Environment)**: la cafetería entera. Tiene la JVM y las librerías necesarias para *ejecutar* café ya hecho. Si solo quieres ejecutar programas, te basta.
- **JDK (Java Development Kit)**: el kit para montar tu propia cafetería. Incluye el JRE más el compilador javac, el generador de documentación javadoc y otras herramientas de desarrollo. Si quieres *crear* programas, lo necesitas.

```
// Imagina que esto es un grano de café verde:
public class Cafe {
    public static void main(String[] args) {
        System.out.println("☕ ¡Café listo!");
    }
}
```

El JDK compila esto a bytecode (café molido), el JRE lo pasa por la JVM y... ¡tachán! café en tu pantalla.

⚠ **Advertencia:** no confundas JDK con JRE. El JDK es el *cuchillo del chef*; el JRE es el *plato servido*. El JDK incluye el JRE, así que instalando el JDK tienes las dos cosas. Instalar solo el JRE te permite ejecutar programas, pero no crearlos.

¿Por qué sigue vivo Java tantos años después?

No es nostalgia. Java sigue siendo uno de los lenguajes más demandados del mundo porque:

- **Es multiplataforma:** el mismo programa corre en móviles, servidores, cajeros y hasta en la lavadora inteligente.
- **Domina el mundo empresarial:** las grandes empresas (banca, seguros, logística) llevan décadas construyendo sus sistemas en Java.
- **Es el lenguaje oficial de Android** (en su variante Kotlin convive, pero Java sigue omnipresente).
- **Tiene una comunidad enorme:** cualquier error que tengas, probablemente alguien ya lo resolvió hace diez años en un foro.
- **Es exigente, y eso te hace mejor:** Java te obliga a ser ordenado. Los programadores que aprenden con Java suelen escribir código más limpio.



Ejemplo guiado: recorrer tu primer programa

Ya has visto Cafe. Vamos a fijar las piezas de cualquier programa Java antes de que lo diseñes tú en el punto 3:

```
public class MiPrimerPrograma {  
    public static void main(String[] args) {  
        System.out.println("¡Holaaaa, mundo!");  
        System.out.println("Estoy aprendiendo Java");  
        System.out.println("Y me está gustando (de momento)");  
    }  
}
```

- `public class MiPrimerPrograma`: declaras una clase llamada `MiPrimerPrograma`. La clase es el contenedor de tu programa.
- `public static void main(String[] args)`: el “botón de inicio”. Java busca esta línea al arrancar.
- `System.out.println(...)`: la voz del programa; grita texto por la consola.

💡 **Detalle práctico:** la clase pública debe llamarse **exactamente igual que el archivo**. Si el archivo se llama `MiPrimerPrograma.java`, la clase debe ser `MiPrimerPrograma`. Esto es una regla de Java, no un capricho.

Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Cuál es la diferencia entre JDK y JRE en una frase?
2. ¿Qué hace el compilador `javac` con tu código `.java`?
3. ¿Por qué el logo de Java es una taza humeante?

>  Respuestas

Resumen en 3 frases

1. Java ejecuta su código dentro de una **máquina virtual (JVM)**, lo que le da portabilidad total.
2. **JDK** crea (compila a bytecode), **JRE** ejecuta, **JVM** transporta: como Amazon pero con café.
3. Java sigue siendo rey en el mundo empresarial y aprenderlo te forma como programador ordenado.



Vocabulario rápido

Término	Idea general
JVM	La máquina virtual que ejecuta el bytecode
JRE	Entorno de ejecución (JVM + librerías)
JDK	Kit de desarrollo (compilador + JRE + herramientas)
Bytecode	Código intermedio que entiende la JVM
Compilador	Traductor de código fuente a bytecode

 [Volver al índice de la unidad](#) · Anterior: [Índice de la unidad](#) · Siguiente: [02 · Instalación del JDK](#)

02 — Instalación del JDK

Montando el chiringuito sin que sobren tornillos 🛠️

📖 Estás en: 🚀 U01 · Introducción a Java → 02 · Instalación del JDK

📌 La idea en una frase

Para crear programas Java necesitas el JDK. Instalarlo es más fácil que montar un mueble de Ikea, y no te sobrarán tornillos.

En el punto anterior viste que el JDK es el kit completo para *crear* programas. Ahora toca ponerlo en tu máquina y comprobar que todo funciona.

🛒 ¿Qué JDK instalo?

El Java original de Sun/Oracle ha tenido una vida movida con las licencias. Hoy la opción más sensata para estudiar es:

- **Eclipse Temurin** (antes AdoptOpenJDK): un JDK **libre, gratuito y mantenido por la comunidad**. Es el heredero de confianza del OpenJDK.
- **Oracle JDK**: la versión oficial de Oracle. Perfecta para empresas, pero con licencia más restrictiva.
- **OpenJDK**: el proyecto de referencia, del que salen los demás.

💡 **Consejo:** si usas [Eclipse Temurin](#) te ahorrarás dolores de cabeza. Es como el JDK oficial pero sin humos raros. Descarga el instalador según tu sistema (Windows x64, macOS, Linux) y déjalo con los valores por defecto.

La versión a elegir: la **última LTS** (soporte a largo plazo). A día de hoy, cualquier 17, 21 o superior te sirve para todo el curso. No te obsesiones con las versiones: los apuntes funcionan con todas.

🚀 Los primeros pasos

Paso 1: instalar

Ejecuta el instalador de Temurin y acepta los valores por defecto. En Windows, marca la opción de añadir el JDK al PATH si te la ofrece (así podrás usar java desde cualquier terminal).

Paso 2: verificar

Abre una terminal (PowerShell en Windows, la terminal que prefieras en Linux/macOS) y escribe:

```
> java -version
openjdk version "21" 2026-01-01
OpenJDK Runtime Environment (build 21+35)
OpenJDK 64-Bit Server VM (build 21+35, mixed mode)

> javac -version
javac 21
```

Si ves algo parecido, ¡enhorabuena! Tienes poderes de compilación. Si en cambio te dice 'java' no se reconoce como un comando interno o externo, significa que el PATH no está bien configurado: busca en el menú de Windows “Editar las variables de entorno del sistema” y añade la carpeta bin del JDK a la variable Path.

💡 **¿Qué es el PATH?** Es la lista de carpetas donde Windows busca los comandos que escribes. Si la carpeta `...\jdk-21\bin` está en el PATH, al escribir java Windows la encuentra. Sin ella, Windows se encoge de hombros.

Paso 3: ¿por qué hay dos comandos?

- **java:** **ejecuta** programas (arranca la JVM con tu bytecode).
- **javac:** **compila** código fuente .java a bytecode .class.

Se necesitan los dos: javac convierte tu código y java lo pone en marcha. Los verás juntos todo el curso.

El IDE: tu navaja suiza

El JDK es el motor, pero la mayor parte del tiempo no escribirás código en un bloc de notas: usarás un **IDE** (Entorno de Desarrollo Integrado). El IDE reúne en una sola aplicación:

- **Editor de código** con colores, autocompletado y resaltado de errores mientras escribes.

- **Compilador y ejecutor** con un botón: ya no necesitas teclear `javac` y `java` a mano (aunque es bueno saberlo).
- **Depurador** integrado, el arma secreta que usarás en el punto 4.
- **Gestión de proyectos**: tus programas no son solo archivos sueltos, sino *proyectos* con estructura.

IDE	Puntos fuertes	
IntelliJ (Community)	IDEA	El favorito del sector; autocompletado bestial. Algo más pesado al arrancar
NetBeans		Simple, oficial de Oracle, perfecto para empezar
Eclipse		Clásico, muy usado en empresas, un pelín más denso

💡 **Recomendación para el curso:** IntelliJ IDEA Community Edition si tu ordenador lo soporta; NetBeans si quieres algo más ligero. Los tres valen: el IDE es una herramienta, no el objetivo.



Ejemplo guiado: tu primer proyecto

Vamos a dejar todo montado antes de escribir código:

1. Abre tu IDE y elige **New Project**.
2. Selecciona **Java** y asegúrate de que apunta al JDK que instalaste (el IDE lo detecta solo).
3. Dale un nombre al proyecto (por ejemplo, `MiPrimerCurso`).
4. El IDE te crea una estructura de carpetas (la verás en el punto 8).
5. Crea una clase nueva llamada `HolaMundo` y escribe dentro:

```
public class HolaMundo {
    public static void main(String[] args) {
        System.out.println("¡Hola, Mundo! Llevo años esperando a que me crearas.");
    }
}
```

6. Pulsa el botón **Run** (el triángulo verde ►) y mira la consola.

Si ves el mensaje en la consola, tu chiringuito está montado: JDK + IDE + primer programa. Estás oficialmente dentro.

⚠ **Advertencia:** no confundas la consola del IDE con la terminal del sistema. La consola del IDE es donde se imprimen los `System.out.println` al ejecutar. Si no ves salida, busca la pestaña “Console” / “Salida”.

Mini-chequeo

1. ¿Qué comando comprueba que tu instalación funciona?
2. ¿Para qué sirve el PATH y qué pasa si el JDK no está en él?
3. ¿Cuál es la diferencia entre java y javac?

>  Respuestas

Resumen en 3 frases

1. Instala **Eclipse Temurin** (un JDK libre) y verifica con `java -version` y `javac -version`.
2. `javac` compila, `java` ejecuta, y el PATH es la dirección postal que el sistema necesita para encontrarlos.
3. Un **IDE** (IntelliJ, NetBeans o Eclipse) reúne editor, compilador y depurador: es tu navaja suiza.

Vocabulario rápido

Término	Idea general
PATH	Lista de carpetas donde el sistema busca comandos
IDE	Entorno integrado: editor + compilador + depurador
LTS	Versión con soporte a largo plazo (la recomendada)
Terminal	La ventana donde escribes comandos como <code>java</code>
Proyecto	La carpeta donde viven tus clases y configuraciones

 [Volver al índice de la unidad](#) · Anterior: [01 · ¿Qué es Java?](#) · Siguiente: [03 · Tu primer programa](#)

03 — Tu primer programa: Hola Mundo

Cómo hablarle a un extraterrestre muy literal 🙄

📖 Estás en: 🚀 U01 · Introducción a Java → 03 · Tu primer programa

La idea en una frase

Programar es hablarle a un extraterrestre muy literal: si le dices “saluda”, no lo hace. Tienes que decirle *cómo*, *cuándo* y *por qué*.

El ordenador es tonto pero preciso: no interpreta, **ejecuta**. Cada línea, en orden, sin saltarse ninguna. Tu primer programa va a gritar “¡Hola!” por la consola, y a partir de ahí todo es añadir más órdenes.

Hola Mundo, el ritual de iniciación

Todo programador empieza por aquí. Es como el primer café de la mañana: no es opcional.

```
public class HolaMundo {  
    public static void main(String[] args) {  
        System.out.println("¡Hola, Mundo! Llevo años esperando a que me crearas.");  
    }  
}
```

Compíllalo (`javac HolaMundo.java`) y ejecútalo (`java HolaMundo`), o pulsa el botón ► de tu IDE. La consola dirá:

```
¡Hola, Mundo! Llevo años esperando a que me crearas.
```

Disección de Hola Mundo

Vamos a diseccionar esto como si fuera una rana en biología:

- `public class HolaMundo`: declaras una clase. Piensa en ello como decirle a Java: “Oye, voy a crear una cosa que se llama `HolaMundo`”. `public` significa que es accesible desde

fuera y la clase debe llamarse igual que el archivo.

- `public static void main(String[] args)`: este es el **botón de inicio**. Cuando ejecutas el programa, Java busca esta línea y dice “¡por aquí se empieza!”.
- `System.out.println(...)`: es **la voz** del programa. Le dices que grite algo por la consola. `print` imprime sin salto de línea; `println` imprime y salta de línea.

```
public class MiPrimerPrograma {  
    public static void main(String[] args) {  
        // Esto es mi primer programa  
        System.out.println("¡Holaaaa, mundo!");  
        System.out.println("Estoy aprendiendo Java");  
        System.out.println("Y me está gustando (de momento)");  
    }  
}
```

💡 **Detalle práctico:** cada instrucción termina con `;`. Es el punto final de cada frase. Sin él, el compilador piensa que la frase continúa y se lío. Los `{}` delimitan los bloques: los de la clase contienen la clase, los del `main` contienen las órdenes.

🔑 ¿Por qué `public static void main(String[] args)`?

Parece un conjuro de Harry Potter. Desmenucemos cada palabra:

Palabra	Qué significa
<code>public</code>	Java puede encontrarlo desde fuera: el botón es visible
<code>static</code>	Puede llamarse sin necesidad de crear un objeto (lo verás en la U06)
<code>void</code>	No devuelve ningún valor: hace su trabajo y se calla
<code>main</code>	El nombre exacto que Java busca al arrancar. No vale otro
<code>String[] args</code>	Un bolsillo donde puedes meter argumentos al ejecutar (punto 6)

La firma es **obligatoria tal cual**. Si cambias `main` por `inicio`, Java no encuentra la puerta y el programa no hace nada.



El método que no se llama

Aquí va una de las trampas favoritas en los exámenes. Observa:

```
public class Saludos {  
    public static void main(String[] args) {  
        System.out.println("¡Hola desde el método main!");  
    }  
  
    public static void saludo() {  
        System.out.println("Esto nunca se ejecuta...");  
    }  
}
```

¿Se ejecutará correctamente? **Sí**, pero solo imprime la primera línea. El método `saludo()` existe, pero como nunca lo llamas desde `main`, se queda ahí haciendo el vago. Java solo ejecuta lo que está dentro del `main` (a no ser que explícitamente llames a otros métodos). El método `saludo()` es como un actor que tiene el guion aprendido pero nunca sale al escenario.



Truco de memoria: `main` es la puerta de entrada de la casa. Puede haber muchas habitaciones (métodos), pero nadie entra por la ventana. Si no llamas a la puerta, las habitaciones se quedan vacías.



Sé el Código: tú eres la JVM

Vas a ser Java por un momento. Coge papel y boli (o mentalmente). Te dan este código:

```
public class Computadora {  
    public static void main(String[] args) {  
        int x = 5;  
        int y = 10;  
        int z = x + y;  
        System.out.println("El resultado es: " + z);  
    }  
}
```

Sigue los pasos como si fueras la JVM:

1. Encuentras la clase `Computadora`.

2. Buscas el método main — ahí está.
3. Creas un espacio llamado x y metes un 5.
4. Creas y y metes un 10.
5. Creas z, sumas x e y (15), lo guardas.
6. Gritas por pantalla: “El resultado es: 15”.

>  Solución

Mini-chequeo

1. ¿Qué pasa si renombro main a inicio?
2. ¿Imprime lo mismo `System.out.print("Hola")` que `System.out.println("Hola")`?
3. ¿Por qué falta un `;` en el código de abajo? Señala el error:

```
public class Erroneo {  
    public static void main(String[] args) {  
        System.out.println("Me falta algo")  
    }  
}
```

>  Respuestas

Resumen en 3 frases

1. Todo programa tiene una **clase** (contenedor) y un **método main** (puerta de entrada).
2. `System.out.println()` es la voz del programa; `;` es el punto final de cada frase.
3. La JVM ejecuta **línea a línea, en orden**: tú decides qué entra por la puerta.

Vocabulario rápido

Término	Idea general
Clase	El contenedor del código (una “cosa” en Java)
Método	Un bloque de órdenes con nombre
main	El método que Java ejecuta al arrancar

Término	Idea general
println	Imprime texto y salta de línea
Consola	La ventana de texto donde se imprime la salida

 [Volver al índice de la unidad](#) · **Anterior:** [02 · Instalación del JDK](#) · **Siguiente:** [04 · El depurador](#)

04 — El depurador: detective de código

Visión de rayos X para tu código 🕵️

📖 Estás en: 🚀 U01 · Introducción a Java → 04 · El depurador

📌 La idea en una frase

El depurador (debugger) es como tener visión de rayos X para tu código: para el programa donde tú quieres y te enseña el valor de cada variable en cada instante.

Tu primer programa funciona. Pero pronto tendrás un programa raro: una variable edad que sale 25 cuando debería salir 18. ¿Qué haces? ¿Le pegas al ordenador? No. Usas el **depurador**.

🔍 ¿Qué es depurar?

Depurar (debug) es **quitarle los bichos** (bugs) al programa. Y el secreto de los programadores experimentados no es adivinar: es **ver**. El depurador te permite:

- **Parar** el programa en una línea concreta.
- **Avanzar** instrucción a instrucción.
- **Inspeccionar** el valor de las variables en cada momento.
- **Modificar** valores sobre la marcha en algunos entornos.

Es como ver una serie de crímenes en cámara lenta: puedes pausar, rebobinar (no del todo), y observar quién hace qué.

💡 **Detalle práctico:** los errores se dividen en dos familias. Los de *compilación* los atrapa javac y te dice la línea (punto 7). Los de *lógica* compilan perfecto pero hacen lo que no deben: para esos no hay mejor arma que el depurador.

🔧 Las cuatro herramientas del detective

Herramienta	Atajo (IntelliJ/VS Code)	Qué hace
Breakpoint (punto de ruptura)	clic en el margen	Le dices a Java “para AQUÍ, quiero ver qué pasa”
Step Over	F8	“Ejecuta esta línea pero no me cuentes los detalles internos”
Step Into	F7	“Ejecuta esta línea Y llévame dentro de esa llamada, quiero espiar”
Watch / Variables	panel lateral	“Enséñame el valor de la variable AHORA MISMO”

- **Breakpoint:** marca una línea. El programa se detiene justo antes de ejecutarla y espera tus órdenes.
- **Step Over (F8):** ejecuta la línea completa de golpe, sin entrar en los métodos que llame.
- **Step Into (F7):** entra *dentro* del método llamado para ver sus líneas una a una.
- **Watch:** el panel que muestra las variables vivas y sus valores actuales. También puedes escribir una expresión y ver qué devuelve.

⚠ **Advertencia:** si te pierdes dentro de un método ajeno con Step Into, usa **Step Out** (Shift+F8 en IntelliJ): sale del método y vuelve al punto donde lo llamaste. Es el botón “he entrado sin querer, sacadme de aquí”.



Ejemplo guiado: el caso del sospechoso

Aquí tienes un programa perfecto para estrenar el depurador:

```
public class DetectivesDeCodigo {  
    public static void main(String[] args) {  
        int sospechoso = 0;  
        for (int i = 0; i < 10; i++) {  
            sospechoso += i; // Pon un breakpoint aquí  
        }  
        System.out.println("El culpable es: " + sospechoso);  
    }  
}
```

El bucle for lo verás a fondo en la U03, pero puedes intuirlo: repite la línea de dentro mientras *i* sea menor que 10, con *i* valiendo 0, 1, 2... La línea clave es `sospechoso += i` (que es lo mismo que `sospechoso = sospechoso + i`).

El ejercicio de detective:

1. Pon un **breakpoint** en la línea de `sospechoso += i` (clic en el margen, aparece un punto rojo).
2. Ejecuta en **modo depuración** (el botón del bichito 🐛, no el de ▶).
3. El programa se detiene. Mira el panel **Variables**: `sospechoso` vale 0 y `i` vale 0.
4. Pulsa **Step Over (F8)** varias veces y observa cómo cambian `sospechoso` e `i` en cada vuelta.

>  Qué deberías ver

La regla de oro del depurador

Cuando algo falla, **no adivines: observa**. El proceso es siempre el mismo:

1. **Reproduce** el fallo (que ocurra siempre que puedas).
2. Pon un breakpoint **antes** de la zona sospechosa.
3. **Avanza** con F8 y **observa** variables hasta localizar la línea donde el valor se tuerce.
4. En cuanto veas dónde se desvía el valor, habrás encontrado el bug.

💡 **Consejo:** si el programa se ejecuta hasta el final sin detenerse, es que el breakpoint está en una línea que **nunca se alcanza** (por ejemplo, dentro de un método que nadie llama, como el `saludo()` del punto 3). El breakpoint no se dispara: otra pista de detective.

Mini-chequeo

1. ¿Qué diferencia hay entre **Step Over** y **Step Into**?
2. Si pones un breakpoint y el programa no se detiene, ¿qué sospechas?
3. ¿En qué modo hay que ejecutar el programa para que los breakpoints funcionen?

>  Respuestas

Resumen en 3 frases

1. El depurador para el programa donde tú marcas (**breakpoint**) y te muestra las variables en vivo.
2. **Step Over** no entra en los métodos; **Step Into** sí; **Watch** te muestra los valores.
3. Ante un bug, no adivines: **reproduce, para, avanza y observa** hasta encontrar dónde se tuerce el valor.

Vocabulario rápido

Término	Idea general
Bug	Un error en el programa
Debug	Proceso de encontrar y arreglar bugs
Breakpoint	Marca donde el programa se detiene
Step Over / Into	Avanzar sin entrar / entrando en métodos
Watch	Panel que muestra variables y expresiones

 [Volver al índice de la unidad](#) · Anterior: [03 · Tu primer programa](#) · Siguiente: [05 · Comentarios y documentación](#)

05 — Comentarios y documentación

Notas pegajosas digitales para tu código 📝

📖 Estás en: 🚀 U01 · Introducción a Java → 05 · Comentarios y documentación

La idea en una frase

Un comentario es un mensaje que te dejas a ti mismo (o a otros). El ordenador lo ignora por completo: es para los humanos, no para las máquinas.

El código explica *qué* hace la máquina; los comentarios explican *por qué* lo hace. Y el *por qué* es oro puro: seis meses después, ese comentario te evitará horas de “¿qué estaba pensando cuando escribí esto?”.

Los tres tipos de comentarios

Java tiene tres formas de escribir comentarios, cada una con su uso:

Tipo	Sintaxis	Uso
De una línea	<code>// texto</code>	Notas rápidas al lado del código
De bloque	<code>/* texto */</code>	Explicaciones largas, varias líneas
Javadoc	<code>/** texto */</code>	Documentación automática de clases y métodos

```
// Comentario de una línea: "Aquí va la magia"

/*
Comentario de varias líneas:
"Si esto funciona, no lo toques.
 Si no funciona, no lo toques tampoco.
 Ya llamaremos a alguien."
*/

/**
 * Comentario Javadoc (el elegante):
 * Sirve para generar documentación automática.
 * @param argumentos la lista de argumentos de la línea de comandos
 * @return nada, esto es void, ¿no te enteras?
 */
```



Detalle práctico: los comentarios pueden ir en medio de una línea sin problema. `System.out.println(/* "Cuatro" */ "Cinco")` imprime Cinco: el comentario se ignora y el resto de la línea sigue viva.



El consejo que te ahorrará horas: comenta el *por qué*, no el *qué*

Este es el consejo más rentable de toda la unidad:

```
int i = 0; // Declaro i con valor 0    ← MAL: el código ya lo dice
int i = 0; // Empezamos desde 0 porque el usuario no ha pulsado nada    ← BIEN
```

Escribir `// Declaro i con valor 0` es como poner “Abro la puerta” en una puerta. El código ya dice eso. El comentario útil añade información que **el código no puede expresar**: el contexto, la intención, la decisión de diseño.

- ✗ `int temperatura = 30; // temperatura vale 30`
- ✓ `int temperatura = 30; // Refresca por debajo de 25 según el jefe`



Truco de memoria: si el comentario describe la misma acción que ves en el código, bórralo. El buen comentario responde a *¿por qué?*, nunca a *¿qué?*.



Javadoc: documentación que se genera sola

Javadoc es un comentario especial (`/** ... */`) que colocas **justo antes** de una clase o de un método. La herramienta javadoc (incluida en el JDK) lo convierte en páginas de documentación automáticas, iguales a las oficiales de Java.

```
/**
 * Clase que representa a un alumno del curso de programación.
 *
 * @author Sergi Garcia
 * @version 1.0
 */
public class SobreMi {

    /**
     * Punto de entrada del programa.
     *
     * @param args argumentos recibidos desde la línea de comandos
     */
    public static void main(String[] args) {
        System.out.println("Me llamo Sergi y me gusta la programación");
    }
}
```

Etiquetas que debes conocer:

Etiqueta	Dónde	Qué documenta
@author	Clase	Quién escribió el código
@version	Clase	Versión del código
@param	Método	Qué significa cada parámetro
@return	Método	Qué devuelve el método (si no es void)

Para generarla: `javadoc SobreMi.java` en la terminal y se crearán los `.html` con la documentación. Pruébalo: es como tener un manual de instrucciones hecho por ti mismo.



Consejo: en los exámenes y en la vida real, los comentarios Javadoc en los métodos suelen “caer” en las rúbricas. Además, en el trabajo, los que documentan duermen mejor... y los que no, vuelven al código a las 3 de la mañana con cara de pánico.



Ejemplo guiado: documentar un programa completo

Vamos a aplicar todo a un programa real:

```
/**
 * Clase que saluda de distintas formas.
 *
 * @author Alumno DAM
 * @version 1.0
 */
public class Saludo {

    /**
     * Método principal: imprime los dos mensajes de la clase.
     *
     * @param args argumentos de línea de comandos (no usados aquí)
     */
    public static void main(String[] args) {
        // Primer mensaje: bienvenida al curso
        System.out.println("Bienvenidos al curso de Programación");

        /* Segundo mensaje:
         * usamos println para saltar de línea y dejar la consola limpia */
        System.out.println("Este curso va a molar");
    }
}
```

Fíjate en que cada comentario aporta algo: el Javadoc documenta la API, el `//` explica la intención y el `/* */` justifica una decisión de formato. Ninguno repite lo que el código ya dice.



Mini-chequeo

1. ¿Cuál es la diferencia entre `//`, `/* */` y `/** */`?
2. ¿Qué imprime este programa?


```
public class Comentarios {  
    public static void main(String[] args) {  
        // System.out.println("Uno");  
        System.out.println("Dos");  
        /* System.out.println("Tres"); */  
        System.out.println(/* "Cuatro" */ "Cinco");  
    }  
}
```

3. ¿Es buen comentario // x = 10?

>  Respuestas

Resumen en 3 frases

1. Los comentarios son para humanos: //, /* */ y /** */, según lo que necesites.
2. Comenta el **por qué**, no el qué: el código ya dice qué hace.
3. **Javadoc** genera documentación automática con @author, @version, @param y @return.

Vocabulario rápido

Término	Idea general
Comentario	Texto que el compilador ignora
Javadoc	Comentario que genera documentación automática
Etiqueta	@param, @return... metadatos del Javadoc
Documentación	Explicación escrita del código
Compilador	Traductor que ignora los comentarios

 [Volver al índice de la unidad](#) · Anterior: [04 · El depurador](#) · Siguiente: [06 · Argumentos de línea de comandos](#)

06 — Argumentos de línea de comandos

El bolsillo secreto del main 📁

📖 Estás en: 🚀 U01 · Introducción a Java → 06 · Argumentos de línea de comandos

📌 La idea en una frase

El parámetro `args` del `main` es un bolsillo donde Java mete todo lo que escribas después del nombre de tu programa al ejecutarlo.

Cuando escribes `java MiPrograma hola mundo`, esas palabras sueltas (`hola`, `mundo`) no son casuales: Java las guarda en el array `args` para que tu programa las pueda usar.

📁 ¿Qué es `args`?

Recuerda la firma del `main`:

```
public static void main(String[] args)
```

`String[] args` significa: “un array de cadenas de texto llamado `args`”. Un array es una lista de cosas (los verás a fondo en la U09). Aquí, la lista contiene **cada palabra que escribiste después del nombre de la clase al ejecutar**.

```
public class UsoDeArgumentos {
    public static void main(String[] args) {
        System.out.println("Has escrito " + args.length + " palabras:");
        for (int i = 0; i < args.length; i++) {
            System.out.println("Palabra " + (i + 1) + ": " + args[i]);
        }
    }
}
```

Si ejecutas:

```
> java UsoDeArgumentos Java mola mucho
```

Verás:

Has escrito 3 palabras:

Palabra 1: Java

Palabra 2: mola

Palabra 3: mucho

Cómo se indexan las palabras

En Java, las listas empiezan a contar en 0, como casi todo en programación:

Índice	Valor
args[0]	"Java"
args[1]	"mola"
args[2]	"mucho"
args.length	3 (cuántos hay)

- `args.length` te dice cuántos argumentos hay (aquí, 3).
- `args[0]` es el primero, `args[1]` el segundo... y así.
- Si ejecutas **sin argumentos**, `args.length` vale 0 y no hay nada dentro. Intentar leer `args[0]` reventaría con una `ArrayIndexOutOfBoundsException` (lo veremos en la U03).

 **Advertencia:** cuidado con el error del novato: `args[0]` es el **primer** argumento, no “el argumento número 0”. El primero cuenta como índice 0. Es como las plantas de un edificio: la baja es la 0.

¿Para qué sirve pasar argumentos?

Pasar argumentos por línea de comandos es la forma más antigua y directa de “configurar” un programa sin tocar el código:

- **Datos de entrada:** `java Calculadora 5 3` → tu programa recibe 5 y 3.
- **Modos de ejecución:** `java App --verbose 0` o `java App --silencioso`.
- **Archivos:** `java Convertidor entrada.txt salida.txt`.

- **Programas reales:** los comandos de la terminal (como `git status` o `ls -la`) son exactamente eso: programas que reciben argumentos.

En la práctica, el array `args` aparece cada vez que quieras que tu programa hable con “el mundo exterior” antes de arrancar. Más adelante usarás el Scanner (U02) para pedir datos durante la ejecución; los `args` son la vía de entrada **antes** de ejecutarse.



Ejemplo guiado: el programa que te saluda

Vamos a hacer que tu programa sepa tu nombre sin preguntarlo:

```
public class SaludoPersonal {
    public static void main(String[] args) {
        if (args.length > 0) {
            System.out.println("Hola, " + args[0] + ". ¡Bienvenido al curso!");
        } else {
            System.out.println("Hola, desconocido. ¿Olvidaste tu nombre?");
        }
    }
}
```

Si ejecutas `java SaludoPersonal Sergi`, saluda a Sergi. Si ejecutas `java SaludoPersonal`, te recuerda que no pasaste argumentos. El `if` aquí es un aperitivo de las estructuras de control de la U03.



Detalle práctico: cuando ejecutas desde el IDE (botón ►), puedes configurar los argumentos en el menú *Run → Edit Configurations → Program arguments*. Escribe ahí `Sergi 20` y pulsa Run: tu programa los recibirá igual que si los teclearas en la terminal.



Mini-chequeo

1. Si ejecutas `java MiPrograma uno dos tres`, ¿cuánto vale `args.length` y qué contiene `args[2]`?
2. ¿Qué pasa si accedes a `args[5]` cuando solo hay 3 argumentos?
3. ¿Cómo saludas a la primera palabra que reciba tu programa?

>  **Respuestas**

✓ Resumen en 3 frases

1. args es el array de cadenas que Java crea con todo lo escrito tras el nombre de tu clase.
2. Los índices empiezan en **0**: args[0] es el primer argumento y args.length dice cuántos hay.
3. Los argumentos son la forma de dar **datos de entrada** a un programa antes de que arranque.

Vocabulario rápido

Término	Idea general
Argumento	Una palabra que pasas al programa al ejecutarlo
args	El array donde viven los argumentos
Array	Una lista de elementos numerados desde 0
args.length	Cuántos argumentos hay
Línea de comandos	La terminal donde escribes java ...

 [Volver al índice de la unidad](#) · Anterior: [05 · Comentarios y documentación](#) · Siguiente: [07 · El compilador y sus errores](#)

07 — El compilador y sus errores

Tu nueva relación tóxica con javac 🤖

📖 Estás en: 🚀 U01 · Introducción a Java → 07 · El compilador y sus errores

📌 La idea en una frase

El compilador no te odia: es un profesor de lengua puntilloso que marca cada coma mal puesta. Aprender a leer sus mensajes es aprender a programar.

Toda tu carrera va a estar salpicada de mensajes de error. Los novatos los temen; los profesionales los leen como quien lee una carta. En este punto vas a aprender a entender qué te está diciendo javac (y por qué Java es tan puntilloso).

🔍 Compilar ≠ ejecutar

Dos momentos distintos, dos familias de errores:

Momento	Comando	Qué ocurre
Compilar	<code>javac MiPrograma.java</code>	El compilador traduce tu código a bytecode. Si hay errores de sintaxis, se queja aquí
Ejecutar	<code>java MiPrograma</code>	La JVM ejecuta el bytecode. Aquí pueden aparecer errores de <i>runtime</i> (en tiempo de ejecución)

```
Tu código (.java) --javac--> bytecode (.class) --java--> salida
(errores de compilación aquí)                (errores de ejecución aquí)
```

- **Errores de compilación:** el compilador detecta que has escrito mal algo. Te dice la **línea exacta** y el motivo. No se genera el `.class`.
- **Errores de ejecución (runtime):** el programa compila, pero revienta al correr (por ejemplo, `ArrayIndexOutOfBoundsException` del punto 6).
- **Errores de lógica:** todo “funciona”, pero el resultado es incorrecto. El más peligroso: ni el compilador ni el runtime te avisan. Ahí entra el depurador del punto 4.



El Error del Novato: 4 errores en un programa

Este código está fatal. Encuentra los 4 errores antes de mirar la solución:

```
Public class Calculadora
{
    public static void main(string[] args) {
        System.out.println("Suma: " + 5 + 3)
        SYSTEM.OUT.PRINTLN("Resta: " + (5 - 3));
    }
}
```

>  Los 4 errores



Cómo leer un mensaje de error de javac

Cuando algo no compila, javac te muestra algo como:

```
Calculadora.java:3: error: ';' expected
    System.out.println("Suma: " + 5 + 3)
                                ^
1 error
```

Tres datos clave:

1. **Calculadora.java:3** → el archivo y la **línea** donde está el problema (mira la ^).
2. **error: ';' expected** → el *qué*: esperaba un punto y coma.
3. **El ^** → el punto exacto donde se ha atascado.



Consejo: el error se suele encontrar en la línea marcada, pero a veces es la **anterior**: si falta un { o un ;, javac a veces se da cuenta una línea más tarde. Empieza a mirar en la línea del ^ y, si no, sube una.



Errores típicos del novato (y su remedio)

Error	Mensaje típico	Remedio
Public en vez de public	class, interface, or enum expected	Los nombres de Java son exactos: public, class, String, System
Falta ;	';' expected	Cada instrucción termina en ;
Falta { o }	reached end of file while parsing o class, interface...	Cuenta las llaves de apertura y cierre
string en vez de String	cannot find symbol	String es una clase con S mayúscula
Clase no coincide con archivo	class X is public, should be declared in a file named X.java	La clase public se llama igual que el archivo
System.out.println sin las S mayúsculas	cannot find symbol: variable system	System, out en minúscula y println

 **Truco de memoria:** los nombres de las *clases* (String, System, Scanner...) empiezan en mayúscula; los nombres de *variables y métodos* (out, println, main) en minúscula. Cuando dudes, mira cómo lo escribió Java.

EL LÍO: el código revuelto

Tu jefe ha dejado este código hecho un desastre. Las líneas están mezcladas. ¿Eres capaz de ordenarlas para que sea un programa Java válido que imprima “La suma es: 8”?

```
System.out.println("La suma es: " + (a + b));
int a = 5;
public class CalculoLioso {
    System.out.println("Calculando...");
    public static void main(String[] args) {
        int b = 3;
```


Pista: busca primero dónde empieza la clase y el método main.

>  Solución

Mini-chequeo

1. ¿Qué significa `Calculadora.java:3: error: ';' expected?`
2. ¿Por qué `Public` (con P mayúscula) da error?
3. ¿Cuál es la diferencia entre un error de compilación y uno de lógica?

>  Respuestas

Resumen en 3 frases

1. Compilar (`javac`) y ejecutar (`java`) son momentos distintos con errores distintos.
2. El mensaje de error te dice **archivo, línea y motivo**: aprende a leerlo en vez de temerlo.
3. Los fallos clásicos (mayúsculas, `;`, `{}`, nombre de clase) se corrigen con un vistazo... una vez sabes qué mirar.

Vocabulario rápido

Término	Idea general
Compilar	Traducir código a bytecode
Error de compilación	El compilador rechaza tu sintaxis
Error de runtime	El programa revienta al ejecutarse
Error de lógica	Compila, pero el resultado es incorrecto
Sensible a mayúsculas	<code>Public</code> $\neq$ <code>public</code>

 [Volver al índice de la unidad](#) · Anterior: [06 · Argumentos de línea de comandos](#) · Siguiente: [08 · El IDE y tu flujo de trabajo](#)

08 — El IDE y tu flujo de trabajo

Tu navaja suiza para no vivir en la terminal 🗡️

📖 Estás en: 🚀 U01 · Introducción a Java → 08 · El IDE y tu flujo de trabajo

📌 La idea en una frase

El IDE (Entorno de Desarrollo Integrado) es tu navaja suiza: editor, compilador, ejecutor y depurador, todo en una sola ventana.

En el punto 2 instalaste el JDK y le diste un primer vistazo al IDE. Ahora vas a montar el flujo de trabajo diario: cómo se organiza un proyecto, cómo crear clases, ejecutarlas, depurarlas y moverte con atajos sin tocar el ratón.

📁 Anatomía de un proyecto

Cuando creas un proyecto Java, el IDE te genera una estructura de carpetas. Las importantes son:

```
MiPrimerCurso/  
├─ src/                ← TU código (.java) vive aquí  
│   └─ HolaMundo.java  
└─ out/ (o target)    ← el bytecode (.class) compilado  
    └─ HolaMundo.class
```

- **src** (source): tus archivos `.java`. Es lo que tú escribes y lo que se sube a Git.
- **out / target**: lo que genera el compilador. No se toca a mano y normalmente ni se sube a Git.

💡 **Detalle práctico:** no edites nunca los `.class`. Si borras `src` y editas `out`, el IDE lo regenerará al compilar. Tu código fuente es la única fuente de verdad.

🔄 El ciclo de trabajo (tu nuevo bucle de vida)

Programar es un bucle de cuatro pasos que repetirás millones de veces:

1. **Editar:** escribes o cambias código en `src`.
2. **Compilar:** el IDE compila al guardar (o al pulsar Build). Aquí mueren los errores de sintaxis.
3. **Ejecutar:** pulsas ▶ (Run). Aquí mueren los errores de runtime.
4. **Depurar:** si la lógica falla, pulsas 🐛 (Debug) y empiezas el detective del punto 4.

Editar → Compilar → Ejecutar → Depurar → (volver a Editar)

⚠ **Advertencia:** el botón ▶ (Run) y el 🐛 (Debug) NO son lo mismo. Run ejecuta sin breakpoints; Debug los respeta. Si pones un breakpoint y pulsas ▶, el programa no se detendrá y pensarás que el depurador está roto.

Los atajos que te harán parecer un pro

Deja el ratón en paz. Estos son los atajos esenciales (valen para IntelliJ y similares):

Atajo (Windows)	Acción
psvm + Tab	Escribe el esqueleto <code>public static void main(String[] args) {}</code>
sout + Tab	Escribe <code>System.out.println()</code>
Shift + F10	Ejecutar (Run)
Shift + F9	Ejecutar en modo depuración (Debug)
F8 / F7	Step Over / Step Into (depurador)
Ctrl + /	Comentar / descomentar la línea
Ctrl + D	Duplicar la línea
Ctrl + B	Ir a la definición (navegar)

💡 **Truco de memoria:** `psvm` = `public static void main` y `sout` = `System.out`. Son los dos fragmentos que más escribirás en el curso. Aprende los atajos y el IDE te los rellena solo.

Autocompletado: tu compañero silencioso

El IDE mientras escribes te sugiere, resalta errores en rojo antes de compilar y puede **arreglar** problemas con un clic (la bombilla 💡):

- Escribes `Syste` y el IDE te ofrece `System` (con la `S` mayúscula que tanto cuesta).
- Escribes una línea sin `;` y la bombilla te ofrece “Insert semicolon”.
- Renombras una variable y el IDE actualiza todas sus apariciones: es *refactorizar*.

El autocompletado no es trampa: es la razón por la que la gente usa IDE en lugar de un bloc de notas. Tu código sale con menos errores tontos porque el IDE te corrige mientras piensas.

Ejemplo guiado: de cero a ejecutar en 60 segundos

1. **Nuevo proyecto:** File → New → Project → Java. Nómbralo `MiPrimerCurso`.
2. **Nueva clase:** clic derecho en `src` → New → Java Class. Nómbrala `HolaMundo`.
3. Escribe `psvm` y pulsa `Tab`: aparece el `main`. Dentro, escribe `sout` + `Tab` y escribe tu mensaje:

```
public class HolaMundo {  
    public static void main(String[] args) {  
        System.out.println("¡Hola desde mi IDE!");  
    }  
}
```

4. Pulsa **Shift + F10** (Run). Mira la consola del IDE.
5. Pon un breakpoint en el `println`, pulsa **Shift + F9** (Debug) y observa el panel de variables.

Si has llegado al paso 5 sin pánico, tu flujo de trabajo está montado. A partir de aquí, el curso entero es llenar este esqueleto de contenido.

Mini-chequeo

1. ¿Dónde viven tus archivos `.java` y dónde los `.class`?
2. ¿Qué diferencia hay entre los botones ▶ (Run) y 🐛 (Debug)?
3. ¿Qué escribe `psvm` + `Tab`?

>  Respuestas



Resumen en 3 frases

1. Un proyecto tiene src (tu código) y out (bytecode): el IDE compila por ti.
2. Tu bucle diario es **editar** → **compilar** → **ejecutar** → **depurar**, y Run y Debug no son lo mismo.
3. Los atajos (psvm, sout, F8/F7) y el autocompletado te hacen más rápido y con menos errores tontos.



Vocabulario rápido

Término	Idea general
IDE	Editor + compilador + ejecutor + depurador en una app
Proyecto	La carpeta que agrupa tu código y configuraciones
src	Carpeta de código fuente
Refactorizar	Cambiar código manteniendo su comportamiento
Consola del IDE	Donde se imprime la salida de tus programas

 [Volver al índice de la unidad](#) · Anterior: [07 · El compilador y sus errores](#) · Siguiente: [09](#) · [Repaso interactivo](#)

09 — Repaso interactivo: consolida lo aprendido

El cierre práctico de la unidad, con chicha y un bug que no va a funcionar 🐛

📖 Estás en: 🚀 U01 · Introducción a Java → 09 · Repaso interactivo

📌 La idea en una frase

En este punto no aprendemos nada nuevo: lo convertimos todo en práctica. Y, como siempre, algo no va a funcionar. 🐛

★ Sé el Código, my friend...

Eres la JVM. Acaban de darte este programa para ejecutar:

```
public class Misterio {  
    public static void main(String[] args) {  
        System.out.println("Café " + 1 + 2);  
        System.out.println("Café " + (1 + 2));  
    }  
}
```

¿Qué imprimes por pantalla? Elige sabiamente:

1. Café 3 y Café 3 → La JVM siempre suma los números, ocurra lo que ocurra. ❌
2. Café 12 y Café 3 → ✅ ¡Correcto! En la primera línea, al ver texto antes del +, Java concatena: "Café " + 1 es "Café 1" y luego + 2 da "Café 12". En la segunda, los paréntesis fuerzan la suma primero: Café 3.
3. Café 1 2 y Café 12 → Los paréntesis no existen para la JVM. ❌

> 🔄 Solución

🔥 Fireside Chat: JDK vs JRE

Dos veteranos del café discuten junto a la taza humeante.

JRE: — Mira, yo soy el entorno de ejecución. Traigo la JVM, las librerías, todo lo necesario para *ejecutar* Java. Sin mí, tu `.class` no corre ni de coña.

JDK: — Ejecutar, ejecutar... ¿Y quién crea el `.class`? Yo. Traigo el compilador `javac`, el `javadoc`, el `jar`... Yo soy el kit completo. Tú eres solo la mitad servida.

JRE: — Oye, que sin mí tú eres un catálogo de herramientas sin enchufar. Compilas un programa y... ¿dónde lo ejecutas? En mi casa.

JDK: — *sarcástico* Ah, claro, como si yo no te llevara dentro. Cuando instalo yo, vienes tú incluido. Eres como el regalo que se incluye en el pack grande.

JRE: — Vale, pero para el usuario final que solo quiere *usar* programas Java, ¿a quién llaman? ¡A mí! Soy más ligero, menos complejo.

JDK: — Y sin compilador. O sea, un cocinero sin receta. Hablaremos cuando quieras *crear* algo.

JRE: — *suspiro* Siempre el mismo argumento.

La lección: el **JDK incluye al JRE**. Para crear programas necesitas el JDK. Para ejecutarlos ya instalados, te basta el JRE. Nunca te pongas del lado de uno solo en un examen.



¿Quién Soy?

Adivina qué concepto de la unidad soy:

1. Traduzco tu código `.java` a bytecode. Soy un profesor de lengua muy puntilloso: cualquier coma mal puesta me ofende.
2. Soy la máquina de café: ejecuto el bytecode y funciono igual en cualquier sistema operativo.
3. Soy la puerta de entrada de tu programa. Si cambio mi nombre, Java no encuentra la entrada y todo se queda a oscuras.
4. Soy el detective del IDE: te dejo parar el programa donde quieras y espiar las variables.

>  Respuestas



CONRAD VS EL MUNDO: “El compilador me odia”

CONRAD, nuestro compilador cascarrabias, opina sobre el clásico del novato.

CONRAD: — ¡OTRA VEZ! Viene un alumno y me dice: *CONRAD, no compila*. Y yo: vale, ¿qué dice el error? *Pues no sé, no lo he leído*. ¡Ay, madre mía! Te doy la línea exacta, el motivo y hasta la flechita ^ señalando el sitio, ¿y no lo lees? Es como quejarse de que la sopa está fría sin mirarla.

Y luego está el clásico: Public con mayúscula. ¿POR QUÉ? La palabra es public. Con minúscula. Así, pequeñita. Llevo 25 años compilando y sigo viendo Public, String sin la S... ¡Y las clases que no coinciden con el nombre del archivo! ¡Si lo pide el propio archivo!

La lección: antes de llorar sobre el teclado, **lee el mensaje de error**. Archivo, línea, motivo. El 90% de los errores del novato se arreglan solos con esa información. El compilador no te odia: te está pasando las respuestas del examen.

Laboratorio de Tortura: el programa que no arranca

Duración estimada: 30 minutos **Herramienta:** tu IDE y un archivo nuevo

El escenario: copia este programa en tu IDE y haz que funcione. Tiene **3 errores** que impiden que compile y 1 error de lógica que hace que el resultado sea incorrecto cuando lo arregles.

```
public class Tortura
{
    public static void main(string[] args) {
        int a = 3;
        int b = 4;
        System.out.println("La suma es: " + a + b)
        System.out.println("El producto es: " + (a * b));
    }
}
```

Fallo intencionado: uno de los errores parece correcto a simple vista porque “se ve bien”, pero cambia por completo la salida del programa.

Tu tarea: conseguir que compile, que ejecute y que **toda** la salida sea correcta. Si el resultado no cuadra, diagnostica.

Pistas para cuando te frustres (no antes):

1. ¿Hay algún ; que falte? *no → sigue buscando.*

‣ ¿Y si sigo atascado?

2. ¿Compila ya? *no → mira el mensaje de error y las mayúsculas.*

‣ ¿Y si sigo atascado?

3. ¿Ejecuta pero la suma sale rara? *Es el error de lógica: el + sin paréntesis concatena en vez de sumar.*

‣ **Solución final**

```
public class Tortura {  
    public static void main(String[] args) {  
        int a = 3;  
        int b = 4;  
        System.out.println("La suma es: " + (a + b));  
        System.out.println("El producto es: " + (a * b));  
    }  
}
```

Salida correcta: La suma es: 7 y El producto es: 12.



Logros de esta unidad

Logro

Cómo conseguirlo

El Café de la Mañana	Explicar JVM, JRE y JDK con la analogía de la cafetería sin mirar
Primer Grito	Ejecutar tu Hola Mundo sin ayuda en el IDE
Detective de Bugs	Encontrar los 3+1 errores del Laboratorio sin usar las pistas
Atajo Pro	Escribir un programa usando solo psvm + sout y atajos
Traductor	Generar la documentación Javadoc de una clase tuya y abrir los HTML



Atrévete a Pensar

1. Sin ejecutar: ¿qué imprime este programa?

```
public class Misterio2 {  
    public static void main(String[] args) {  
        System.out.print("Uno ");  
        System.out.println("Dos");  
        System.out.print("Tres");  
    }  
}
```

2. **El saludo que no saluda:** en el ejemplo Saludos del punto 3, ¿cómo harías para que saludo() también se ejecutara, sin cambiar el método en sí?
3. **El detective del IDE:** tu programa “funciona” pero imprime La suma es: 34 en vez de 7. ¿Qué herramienta usas y qué líneas examinas primero?
4. **Verdadero o falso:** “javac y java hacen lo mismo: el primero es para Windows y el segundo para Linux.”

> 💡 Soluciones

🧩 Crucigrama de Bits

Horizontal:

1. Entorno de Desarrollo Integrado (3 letras)
3. El método que Java busca al arrancar (4 letras)
5. Kit completo para crear programas (3 letras)
6. Código intermedio que entiende la JVM (8 letras)

Vertical:

2. Máquina virtual que ejecuta el bytecode (3 letras)
4. Imprime texto sin saltar de línea (5 letras)

> 📄 Soluciones

💬 Preguntas de Entrevista de Trabajo

Preguntas reales que te harían para programador Java junior.

1. “Explícame, como si yo fuera tu abuela, la diferencia entre JDK, JRE y JVM.”
2. “¿Qué es el método `main` y por qué tiene esa firma exacta?”
3. “Un programa compila pero hace lo que no debe. ¿Cuál es tu proceso para arreglarlo?”
4. “¿Qué es un array y qué pasa si accedes a un índice que no existe?”
5. “¿Cómo le pasas datos a un programa Java sin que te los pida por teclado?”



No hay preguntas tontas

? ¿Puedo escribir Java en un bloc de notas o necesito sí o sí un IDE?

Puedes, y de hecho es un buen ejercicio: `notepad MiPrograma.java`, `javac MiPrograma.java`, `java MiPrograma`. Pero el IDE te ahorra errores tontos con el autocompletado y te da el depurador. Los pros usan IDE; los masoquistas, bloc de notas.

? ¿Por qué el primer `println` de Hola Mundo tenía “args” si no lo uso?

`String[] args` es obligatorio en la firma del `main` aunque no lo uses. Java lo espera sí o sí. Es como la firma en un contrato: está aunque no se ejecute esa cláusula. En el punto 6 viste cómo usarlo.

? ¿Java y JavaScript son primos?

No, ni siquiera son del mismo planeta. Java es a JavaScript como un perro es a un perrito caliente. El nombre fue una estrategia de marketing de Netscape para montarse en el boom de Java.



Post-Créditos

El programador novato termina su primer programa, lo compila y lo ejecuta. Todo perfecto. Entonces se acerca CONRAD, el compilador cascarrabias, con cara de pocos amigos y una taza humeante en la mano.

CONRAD: — Muy bonito el Hola Mundo. ¿Y ahora qué? ¿Vas a imprimir texto toda tu vida? Los programas de verdad *guardan* datos, *calculan* y *deciden*. Sin variables, sin tipos, sin operadores... esto es un triler de cartas.

Novato: — ¿Y eso cuándo se aprende?

CONRAD: — *toma un sorbo* La próxima parada. Ya te veo declarando tu primer int... y cometiendo tu primer error de tipos. Espérame, que yo estaré ahí para reírme.

El novato guarda su proyecto, cierra el IDE y siente que esto solo acaba de empezar.

PRÓXIMAMENTE EN U02: Variables, tipos de datos y operadores. El momento en el que tus programas dejan de decir “Hola” y empiezan a *calcular*. 

 [Volver al índice de la unidad](#) · Anterior: [08 · El IDE y tu flujo de trabajo](#) · Siguiente: [U02 · Variables, Tipos y Operadores](#)

Unidad 2: Variables, Tipos de Datos y Operadores

Objetivos de aprendizaje

- Declarar y usar variables de tipos primitivos
- Comprender la diferencia entre tipos primitivos y String
- Utilizar operadores aritméticos, relacionales y lógicos
- Aplicar casting y conversiones entre tipos
- Generar números aleatorios con Math.random()

Variables: Las Cajas Donde Viven Tus Datos

Imagina que la memoria de tu ordenador es un almacén gigante lleno de estanterías. Cada estantería tiene cajas. Las **variables** son esas cajas, y cada caja tiene una etiqueta para que sepas qué hay dentro.

Las Cajas (Declaración de Variables)

Para crear una caja le dices a Java:

```
tipo nombreDeLaCaja = valorQueMetoDentro;
```

Ejemplos reales:

```
int edad = 25;           // Caja etiquetada "edad" con un 25 dentro
double precio = 19.99;   // Caja con decimales
String nombre = "María"; // Caja mágica que guarda texto
boolean hambre = true;   // Caja de verdadero/falso (ahora mismo: true)
```

Consejo:

Las variables se llaman así porque... ¡varían! Puedes cambiar su contenido. `int edad = 25;` y luego `edad = 26;` al día de tu cumple. La etiqueta es la misma, el contenido cambia.

Las Reglas de Nomenclatura (o cómo no meter la pata)

- Pueden tener letras, números, _ y \$. *No* espacios ni ñ's ni cosas raras.
- No pueden empezar con número. 1numero es ilegal. numero1 es legal. Así de tiquismiquis es Java.
- Mayúsculas importan: edad, Edad y EDAD son tres cajas distintas. Como si etiquetaras “Zapatos”, “zapatos” y “ZAPATOS”.
- No uses palabras reservadas: int, class, if, while... son de Java, no tuyas.
- Usa **camelCase**: miVariableEjemplo. Como un camello, con joroba en medio.

Los 8 Primitivos: Cajas de Tamaños Distintos

Java tiene 8 tipos primitivos. Piensa en ellos como cajas de distintos tamaños en tu almacén:

Tipo	Tamaño	Lo que cabe	Analogía
byte	8 bits	-128 a 127	Caja de cerillas
short	16 bits	-32.768 a 32.767	Caja de zapatos
int	32 bits	-2.147M a 2.147M	Caja de mudanza (la que más usarás)
long	64 bits	-9 cuatrillones a +9 cuatrillones	Contenedor de barco
float	32 bits	Decimales de precisión simple	Vaso de agua
double	64 bits	Decimales de precisión doble	Cubo de agua
char	16 bits	Un solo carácter Unicode	Una letra en una caja de zapatos
boolean	1 bit	true o false	Interruptor de luz

```
byte nivel = 100;
short poblacion = 30000;
int habitantes = 150000;           // El más usado
long distancia = 384400000L;      // La L al final es obligatoria
float precio = 12.99f;            // La f al final es obligatoria
double pi = 3.14159265359;
char letra = 'A';                 // Comillas SIMPLES para char
boolean esJavaDivertido = true;   // Esto es opinable
```

Nota:

Usa `int` para casi todo lo numérico entero. Solo usa `long` si vas a contar estrellas. Usa `double` para decimales a menos que ahorrar memoria sea tu fetiche.

String: La Caja Mágica (No es primitivo, pero parece)

`String` no es primitivo, es una **clase**. Pero se comporta tan natural que parece primitivo. Es como un amigo que encaja tan bien en tu grupo que jurarías que es familia.

```
String saludo = "Hola, DAM";  
String nombre = new String("Ana"); // También se puede crear así
```

⚠ Advertencia:

Los `String` son **inmutables**. Una vez creados, no se pueden cambiar. Cuando haces `texto = texto + " más"`, en realidad estás tirando el viejo y creando uno nuevo. Es como si cada vez que quisieras poner un cartel nuevo quemaras el anterior.

Constantes: Cajas con Superglue

Las constantes se declaran con `final`. Una vez que metes algo ahí, no sales ni con palanca.

```
final double IVA = 0.21;  
final int MAXIMO_INTENTOS = 3;  
final String NOMBRE_APP = "Gestión DAM";  
  
IVA = 0.10; // ERROR: ¡Has roto Java! (de compilación, no te preocupes)
```

Por convención, las constantes se escriben `EN_MAYÚSCULAS_CON_GUIONES_BAJOS`. Como si estuvieran gritando "¡SOY INMUTABLE!".

Casting: Aprieta que Cabe

Hay dos tipos de conversiones:

- **Implícita (Widening)**: De caja pequeña a grande. Java lo hace solo. `int` → `long` → `double`. Como cambiar de un piso a una mansión. No preguntas, simplemente te mudas.
- **Explícita (Narrowing)**: De grande a pequeña. Java te obliga a poner (tipo) delante. Como meter una maleta XXL en el maletero de un Smart: tienes que empujar (tipo) y rezar.

```
// Implícita: ensanchando
int num = 100;
long numLong = num;           // Cabe, no problema
double numDouble = num;       // También

// Explícita: estrechando
double precio = 19.99;
int entero = (int) precio;     // Pierdes los .99 → sale 19
System.out.println(entero);    // 19 – los céntimos desaparecen en el olvido
```

Math.random(): El Casino de Java

`Math.random()` devuelve un número aleatorio entre 0.0 y 1.0 (el 1.0 no está incluido, como cuando te toca la lotería pero no).

```
double aleatorio = Math.random();           // Entre 0.0 y 0.999999...
int dado = (int) (Math.random() * 10);      // Entre 0 y 9
int dadoReal = (int) (Math.random() * 6) + 1; // Entre 1 y 6 (como un dado)
```

💡 Consejo:

Para un número entre min y max: `(int)(Math.random() * (max - min + 1)) + min`. Ejemplo del 5 al 10: `(int)(Math.random() * 6) + 5`.

★ BE THE CODE, MY FRIEND

💡 **Don Tip:** El casting explícito con `(tipo)` puede truncar valores. Siempre comprueba si el valor cabe antes de forzarlo.

Ejercicio 1: El Guardia de Almacén

Eres el guardia de un almacén de datos. Te dan estas instrucciones:

```
int a = 10;
double b = a;
int c = (int) b;
byte d = (byte) c;
System.out.println(d);
```

Sigue el proceso paso a paso:

1. `int a = 10;` — Metes un 10 en una caja `int`.
2. `double b = a;` — Coges el 10 y lo metes en una caja `double`. Conversión implícita.
3. `int c = (int) b;` — Coges el 10.0 del `double`. Necesitas `(int)` porque pasar de `double` a `int` requiere apretar.
4. `byte d = (byte) c;` — Coges el 10 del `int` y lo metes en un `byte`. Cabe, pero fuerzas con `(byte)`.
5. Imprime: **10**.

Ahora prueba este:

```
int grande = 300;
byte pequeno = (byte) grande;
System.out.println(pequeno);
```

¿Qué sale? (Pista: en un `byte` solo caben -128 a 127. Sobran 172. En binario, se truncan los bits sobrantes). Resultado: **44**. Es como intentar meter un elefante en un Mini Cooper y que salga un perro salchicha.

★ BE THE CODE, MY FRIEND

👁 **Don Tip:** `==` compara referencias (¿son el mismo objeto?), `.equals()` compara contenido (¿tienen el mismo texto?).

Ejercicio 2: ¿Qué Imprime Este Lío de Strings?

Sin ejecutar, di QUÉ imprime exactamente este código:

```
String a = "Hello";
String b = "Hello";
String c = new String("Hello");
System.out.println(a == b);
System.out.println(a == c);
```

Respuesta: `true` y `false`. `a` y `b` apuntan al mismo objeto en el “pool de Strings” (Java reutiliza literales iguales). Pero `c` se creó con `new String(...)`, así que es un objeto nuevo. `==` compara referencias, no contenido. Para comparar contenido usa `.equals()`: `a.equals(c)` devolvería `true`. ¡Trampa típica de examen!

★ BE THE CODE, MY FRIEND

☛ **Don Tip:** Los operadores ++ pre y post tienen prioridades distintas. Pre: primero cambia, luego usa. Post: primero usa, luego cambia.

Ejercicio 3: Incremento Misterioso

¿Qué imprime este código?

```
int a = 5;
int b = a * 2 + ++a;
System.out.println("a = " + a);
System.out.println("b = " + b);
```

Respuesta: a = 6, b = 16. ++a se evalúa primero (unario), a pasa a 6. Luego $a * 2 + 6 \rightarrow 5 * 2 + 6 \rightarrow 16$.

Si hubiera sido $a * 2 + a++$, sería distinto: $5 * 2 + 5 = 15$ (primero usa a=5, luego incrementa a 6).

Métodos Útiles de String (porque los necesitarás)

```
String texto = "  Programación DAM  ";
texto.length();           // 18
texto.trim();              // "Programación DAM" (sin espacios)
texto.toUpperCase();       // "  PROGRAMACIÓN DAM  "
texto.toLowerCase();       // "  programación dam  "
texto.contains("DAM");     // true
texto.startsWith(" ");    // true
texto.endsWith("AM ");    // true
texto.indexOf("DAM");      // 14 ¿dónde empieza "DAM"?
texto.substring(2, 13);    // "Programación"
texto.replace("DAM", "DAW"); // "  Programación DAW  "
```

Operadores: El Gimnasio de los Datos

Las variables están muy bien, pero no sirven de nada si no haces cosas con ellas. Los **operadores** son las máquinas de pesas de tu gimnasio de datos: suman, restan, comparan y transforman.

Operadores Aritméticos: El Día en el Gym

Operador	Ejercicio	Ejemplo
+	Press de banca	$5 + 3 = 8$
-	Curl de bíceps	$5 - 3 = 2$
*	Sentadilla	$5 * 3 = 15$
/	Peso muerto	$10 / 3 = 3$ (enteros) o $10.0 / 3 = 3.333...$
%	El odiado abdominal	$10 \% 3 = 1$ (el resto de 10/3)

```
int a = 10;
int b = 3;
double c = 10.0;

System.out.println(a / b);           // 3 (división entera)
System.out.println(a % b);           // 1 (el resto)
System.out.println(c / b);           // 3.333... (división real)
System.out.println((double) a / b); // 3.333... (obligas decimal)
```

División entera mata. Si tienes `int alumnos = 17;` `int grupos = 5;` y haces `alumnos / grupos`, Java dice que cada grupo tiene **3** alumnos. Para Java, 17 dividido entre 5 son 3. Punto.

Precedencia: ¿Quién Va Primero?

```
int resultado = 2 + 3 * 4;           // 14 – la multiplicación se cuele
int conParentesis = (2 + 3) * 4;    // 20 – los paréntesis tienen pase VIP
```

La ley del comedor:

1. **Paréntesis ()** — Pase VIP, van los primeros.
2. **Multiplicación, división y módulo * / %** — Los populares.
3. **Suma y resta + -** — Los normales, los últimos.

Operadores de Asignación Compuesta: El Atajo Perezoso

```
int x = 10;
x += 5;    // x = 15 (x = x + 5, pero más cool)
x -= 3;    // x = 12
x *= 2;    // x = 24
x /= 4;    // x = 6
x %= 3;    // x = 0
```

Es como si en lugar de ir a la cocina a por un vaso de agua, tuvieras un grifo en el sofá.

++ y --: Flexiones para Variables

```
int a = 5;
int b = a++; // b = 5, a = 6 (POST: "usa y luego sube")
int c = ++a; // a = 7, c = 7 (PRE: "sube y luego usa")
```

💡 Consejo:

Regla de oro: Si usas ++ o -- *dentro* de una expresión complicada, estarás escribiendo código que ni tú entenderás en una semana. Úsalos solos, en su propia línea.

★ BE THE CODE, MY FRIEND

👁️ **Don Tip:** Desglosa la expresión paso a paso. ¿Qué valor tiene x en cada momento?

Ejercicio 4: El Acróbata de las Variables

Sin ejecutar, calcula qué vale todo aquí:

```
int x = 3;
int y = x++ + ++x;
System.out.println("x = " + x + ", y = " + y);
```

Paso a paso:

1. $x = 3$
2. $x++$ — POST: usa x (3), luego incrementa x a 4. El valor de $x++$ es 3.
3. $++x$ — PRE: x vale 4 ahora. Incrementa x a 5, luego vale 5.
4. $y = 3 + 5 = 8$
5. Resultado: $x = 5$, $y = 8$.

A los programadores profesionales también les cuesta. Por eso casi nadie escribe código así en producción. Pero en los exámenes... ¡ay, aparece!

Operadores Relacionales: El Juez de la Discusión

```
int edad = 18;
boolean puedeVotar = edad >= 18;           // true
boolean tieneDescuento = edad < 12 || edad > 65; // false
boolean noEsEl = edad != 18;              // false
```

Operadores Lógicos: El Club Nocturno

- **&& (AND):** ¿Tienes más de 18 Y tienes entrada? Las dos deben cumplirse.
- **|| (OR):** ¿Tienes más de 18 O eres el dueño? Basta una.
- **! (NOT):** ¿NO tienes menos de 18?

```
boolean mayorEdad = true;
boolean tieneEntrada = false;

boolean entra = mayorEdad && tieneEntrada; // false
boolean entraVip = mayorEdad || tieneEntrada; // true

int x = 5;
boolean resultado = (x > 10) && (++x > 0); // false, y x sigue siendo 5
```

¡Cortocircuito! Con **&&**, si lo primero es false, Java ni se molesta en mirar lo segundo. Con **||**, si lo primero es true, igual.

El Operador Ternario: El Bouncer del Club

```
String mensaje = (edad >= 18) ? "Pasa, joven" : "Vuelve cuando crezcas";

int nota = 7;
String resultado = nota >= 5 ? "Aprobado" : "Suspenso";
```

La estructura es: condición ? valorSiTrue : valorSiFalse.

El corrector automático del instituto ha escupido este código lleno de errores. Identifica y corrige los 5 errores que tiene:

```
public class LioVariables {  
    public static void main(String[] args) {  
        int a = 10.5;  
        double b = "Hola";  
        String c = true;  
        int d = a + c;  
        System.out.println("Resultado: " + d)  
    }  
}
```

Pista: cada variable debe tener el tipo correcto para su valor.

👁️ **Don Tip:** Repasa los tipos primitivos: `int` solo admite enteros, `double` admite decimales, `String` va con comillas dobles.

¡No Hay Preguntas Tontas!

? ¡No Hay Preguntas Tontas!

Q: ¿Por qué `long` lleva L y `float` lleva f al final?

A: Porque si pones `long x = 3000000000;` sin la L, Java piensa que es un `int` y se queja. El `float` necesita f porque por defecto los decimales son `double`.

Q: ¿Qué pasa si divido un `int` entre otro `int` y espero decimales?

A: Java te dará un entero. $5 / 2 = 2$, no 2.5 . Para decimales, al menos uno debe ser `double`: $5 / 2.0 = 2.5$ o `(double)5 / 2 = 2.5`.

Q: ¿Cuál es la diferencia entre `=` y `==`? Siempre me lío.

A: `=` es *asignar*: “coge este valor y mételo en esta caja”. `==` es *comparar*: “¿son iguales?”. Confundirlos es el error más clásico. Es como confundir “pon la mesa” con “¿está puesta la mesa?”.

Q: ¿Y el `%` para qué sirve en la vida real?

A: Para saber si un número es par (`numero % 2 == 0`), para ciclos, para juegos. Sin él no tendrías horas en un reloj ni nada cíclico.

Q: Precedencia, asociatividad... ¿tengo que memorizarlo todo?

A: No. La regla de oro: *cuando tengas dudas, pon paréntesis*. `resultado = (a + b) * (c - d)` es mucho más legible. Los paréntesis no duelen.

Más Ejemplos de Código

```
public class CalculadoraBasica {
    public static void main(String[] args) {
        int a = 15;
        int b = 4;

        System.out.println("a + b = " + (a + b));
        System.out.println("a - b = " + (a - b));
        System.out.println("a * b = " + (a * b));
        System.out.println("a / b = " + (a / b) + " (división entera)");
        System.out.println("a / b real = " + ((double) a / b));
        System.out.println("a % b = " + (a % b) + " (resto)");
    }
}
```

```
public class JuegoDeDados {
    public static void main(String[] args) {
        int dado1 = (int) (Math.random() * 6) + 1;
        int dado2 = (int) (Math.random() * 6) + 1;
        int suma = dado1 + dado2;

        System.out.println("Dado 1: " + dado1);
        System.out.println("Dado 2: " + dado2);
        System.out.println("Suma: " + suma);

        boolean esPar = suma % 2 == 0;
        String mensaje = esPar ? "Suma par - ganas" : "Suma impar - pierdes";
        System.out.println(mensaje);
    }
}
```

Resumen (lo que importa de verdad)

- Las variables son cajas etiquetadas en la memoria.
- 8 tipos primitivos: `byte < short < int < long < float < double < char < boolean`.
- `final` es superglue: constante que no cambia.
- Casting implícito = meter caja pequeña en grande. Explícito = a la inversa con pérdidas.
- `String` no es primitivo, es una clase. Pero se comporta.
- `Math.random()` para jugar a la lotería.
- `+` `-` `*` `/` `%` son los básicos. El `%` te da el resto.
- `++` y `--` son flexiones. Pre (primero sube) vs Post (primero usa).
- `&&` y `||` tienen cortocircuito: si la primera ya decide, no miran la segunda.
- El ternario `? :` es un if-else en una línea.
- La precedencia se soluciona con paréntesis. Siempre.

Ejercicios Propuestos

1. **Cajas variadas** Declara una variable de cada tipo primitivo, asígnale un valor coherente e imprime el resultado.
2. **El casting asesino** Declara un `double` con valor 9.99. Conviértelo a `int` explícitamente. ¿Qué valor se pierde?
3. **Nombre al revés** Pide al usuario su nombre y muestra: longitud, mayúsculas, primera y última letra.
4. **Dado trucado** Genera 10 números aleatorios entre 1 y 6. Cuenta cuántos 6 han salido.
5. **Constante del mal** Declara `final double PRECIO_BASE = 100;` e `final double IVA = 0.21;` Calcula el precio final e intenta modificar IVA después.
6. **¿Cuánto mide?** Calcula el área de un círculo con `Math.PI`. Radio = 7.5.
7. **Adivina el número** La máquina elige un número al azar del 1 al 100. El usuario introduce un número y el programa dice si es mayor, menor o igual.
8. **El intercambio** Declara `int a = 5;` `int b = 10;`. Intercambia sus valores usando una tercera variable temporal.
9. **Área y perímetro** Calcula el área y el perímetro de un rectángulo con base 7.5 y altura 3.2.
10. **¿Par o impar?** Pide un número y determina si es par o impar usando `%`.
11. **Año bisiesto** Pide un año. Determina si es bisiesto: divisible entre 4 Y (no entre 100 O sí entre 400).
12. **Ternario en acción** Pregunta la edad al usuario. Usa el ternario para mostrar “Mayor de edad” o “Menor de edad”.

13. **El acertijo del ++** Sin ejecutar, determina el resultado de: `int a = 2; int b = a++ * 3 + -a;`
14. **Cortocircuito** Declara `int x = 0;`. Haz `boolean test = (5 < 3) && (++x == 1);`. Imprime `x`. ¿Se incrementó?
15. **Conversión Celsius ⇔ Fahrenheit** Pide grados Celsius. Convierte a Fahrenheit ($^{\circ}\text{F} = ^{\circ}\text{C} \times 9/5 + 32$).
16. **Los desbordados** Declara `int max = Integer.MAX_VALUE;` y súmale 1. ¿Qué pasa?
-

RAs trabajados en esta unidad:

- **RA2** - Escribe y prueba programas sencillos



Sergi Garcia Barea — CC BY-SA 4.0

Unidad 3: Estructuras de Control y Excepciones

Objetivos de aprendizaje

- Utilizar estructuras de selección: if/else if/else y switch
- Emplear bucles while, do-while y for
- Comprender el flujo con break y continue
- Manejar excepciones con try-catch-finally
- Lanzar excepciones propias con throw

if, else y switch: El Arte de Decidir

Tu programa es como un robot torpe pero obediente. Sin estructuras de selección, haría TODO lo que le dices, en orden, siempre. Necesitamos que TOME DECISIONES.

if: El Portero de la Discoteca

Imagina que if es un portero de discoteca. Evalúa tu cara (la condición) y decide si pasas o no.

```
if (tieneEdad >= 18) {  
    System.out.println("Puedes pasar, disfruta de la música!");  
}
```

Si la condición es true, entras. Si es false, te quedas en la calle.

if-else: El Portero con Plan B

A veces el portero tiene una alternativa: “No pasas, pero vete al bar de al lado”.

```
if (tieneEdad >= 18) {  
    System.out.println("Adelante, el techno te espera");  
} else {  
    System.out.println("Lo siento, vuelve cuando crezcas");  
}
```

if-else if-else: El Bouncer con Múltiples Listas

```

int nota = 85;

if (nota >= 90) {
    System.out.println("Sobresaliente – eres el orgullo de la familia");
} else if (nota >= 70) {
    System.out.println("Notable – no está mal");
} else if (nota >= 50) {
    System.out.println("Aprobado – por los pelos");
} else {
    System.out.println("Suspenso – tus padres quieren hablar contigo");
}

```

⚠ **Advertencia: Error mortal:** poner ; después de la condición. `if (edad >= 18);` ← el ; vacío hace que el bloque de después se ejecute SIEMPRE.

★ BE THE CODE, MY FRIEND: El Detective de la Edad

💡 **Don Tip:** Recorre las condiciones de arriba a abajo. La primera que sea true ejecuta su bloque y se salta el resto.

El Detective de la Edad

¿Qué imprime esto?

```

int edad = 17;
boolean conPadres = true;

if (edad >= 18) {
    System.out.println("Entrada libre");
} else if (conPadres) {
    System.out.println("Pasas con tus viejos");
} else {
    System.out.println("A casa, campeón");
}

```

Solución: `edad >= 18` → false. `conPadres` → true. Imprime “Pasas con tus viejos”. Esto se llama “seguimiento de código” y es tu superpoder.

? ¡No Hay Preguntas Tontas!

Q: ¿Puedo tener un `if` sin llaves?

A: Técnicamente sí, pero ejecutará SOLO la primera línea después del `if`. Siempre pon llaves. Siempre.

Q: ¿Qué pasa si pongo `=` en lugar de `==`?

A: En Java no compila porque `if (edad = 18)` devuelve un `int`, no un `boolean`. ¡El compilador te salva!

Q: ¿El orden de los `else if` importa?

A: ¡Y tanto! Java evalúa de arriba a abajo y se queda con el PRIMERO que cumple. Evalúa siempre de más específico a menos específico.

Q: ¿Puedo poner cien `else if` seguidos?

A: Puedes, pero si tienes más de 3-4 condiciones, plantéate usar `switch`.

El Operador Ternario: El Ninja de una Sola Línea

```
int edad = 20;  
String mensaje = (edad >= 18) ? "Mayor de edad" : "Menor de edad";
```

💡 **Consejo:** El ternario anidado es como las muñecas rusas: parece chulo hasta que tienes que depurarlo a las 3 de la mañana.

`switch`: La Máquina Expendedora de Código

`if-else` es un portero. `switch` es una máquina expendedora: metes un número, obtienes tu producto.

```

int dia = 3;
String nombreDia;

switch (dia) {
    case 1:
        nombreDia = "Lunes – la resaca del finde";
        break;
    case 2:
        nombreDia = "Martes – todavía duele";
        break;
    case 3:
        nombreDia = "Miércoles – mitad de semana!";
        break;
    case 4:
        nombreDia = "Jueves – ya casi";
        break;
    case 5:
        nombreDia = "Viernes – se acerca la gloria";
        break;
    case 6:
        nombreDia = "Sábado – libertad!";
        break;
    case 7:
        nombreDia = "Domingo – la depresión pre-lunes";
        break;
    default:
        nombreDia = "Eso no es un día, inventado";
}

```

⚠ **Advertencia:** Olvidar el break causa *fall-through*: el código “se cae” al siguiente case. El 99% de las veces es un error.

💡 **Consejo:** Desde Java 14 puedes usar switch como expresión con flechitas ->. No necesita break:

```

String tipo = switch (dia) {
    case 1, 2, 3, 4, 5 -> "Laborable – a currar";
    case 6, 7 -> "Festivo – a dormir";
    default -> "No válido";
};

```

★ BE THE CODE, MY FRIEND: Fall-Through

👁️ **Don Tip:** Sin break, el código 'se cae' al siguiente case. Síguelo sin saltar nada hasta que encuentres un break.

Fall-Through

¿Qué imprime este código?

```
int x = 2;
switch (x) {
    case 1:
        System.out.println("Uno");
    case 2:
        System.out.println("Dos");
    case 3:
        System.out.println("Tres");
        break;
    default:
        System.out.println("Otro");
}
```

Solución: Como no hay break en case 2, se ejecuta “Dos” y se cae a “Tres” (ahí el break frena). Nunca llega a default. Si x = 1, imprime “Uno”, “Dos” y “Tres”.

✂ EL LÍO

El asistente de programación ha mezclado las líneas de este programa. Ordena las líneas para que el programa funcione correctamente y muestre si un número es positivo, negativo o cero:

```
} else if (numero < 0) {
public class Clasificador {
System.out.println("El número es positivo");
System.out.println("El número es cero");
int numero = -7;
System.out.println("El número es negativo");
if (numero > 0) {
} else {
public static void main(String[] args) {
}
```

Pista: no olvides las llaves de apertura y cierre de la clase y el main.

💡 **Don Tip:** Primero estructura la clase y el main, luego coloca el if-else if-else dentro del main.

Bucles: Cómo Hacer Que el Ordenador Se Repita

Los humanos nos cansamos de repetir cosas. Los ordenadores, no. Los bucles son la forma de decirle “haz esto 500 veces” sin escribirlo 500 veces.

while: El Bucle “¿Ya Llegamos?”

Imagina a un niño en un viaje en coche que pregunta “¿ya llegamos?” una y otra vez mientras la condición no se cumpla.

```
int kmRecorridos = 0;

while (kmRecorridos < 100) {
    System.out.println("¿Ya llegamos? Llevamos " + kmRecorridos + " km");
    kmRecorridos++;
}

System.out.println("¡Por fin hemos llegado!");
```

📄 **Nota:** while primero comprueba la condición. Si es false desde el principio, NO ejecuta el bloque ni una vez.

⚠️ **Advertencia: Bucle infinito:** olvidar actualizar la variable de control es como tener al niño en el coche para siempre.

```
int i = 1;
while (i <= 5) {
    System.out.println("¡Atrapado en el tiempo!");
    // Falta i++ → ESTO NUNCA TERMINA
}
```

do-while: “Al Menos Inténtalo”

Primero hace, luego pregunta. Garantiza que el bloque se ejecute al menos una vez.

```
int opcion;
do {
    System.out.println("=== MENÚ ===");
    System.out.println("1. Comer");
    System.out.println("2. Dormir");
    System.out.println("3. Programar");
    System.out.println("0. Salir");
    opcion = sc.nextInt();
} while (opcion != 0);
```



Consejo: Usa do-while cuando necesites que algo pase al menos una vez: menús, confirmaciones, preguntas existenciales.

for: “Lo Haré Exactamente N Veces”

El bucle for es el alemán de los bucles: disciplinado, sabe exactamente cuántas veces va a repetir.

```
for (int i = 1; i <= 5; i++) {
    System.out.println("Vuelta " + i + " de 5");
}
```

El for mete tres cosas separadas por ;:

1. **Inicialización:** `int i = 1` — “empieza aquí”
2. **Condición:** `i <= 5` — “sigue mientras sea cierto”
3. **Actualización:** `i++` — “cómo avanzas”

```
// Recorrer un array
int[] numeros = {10, 20, 30, 40, 50};

for (int i = 0; i < numeros.length; i++) {
    System.out.println("Elemento " + i + ": " + numeros[i]);
}
```



Advertencia: Los arrays empiezan en 0. Si pones `i <= numeros.length`, explota con `ArrayIndexOutOfBoundsException`. Es el clásico *off-by-one error*.

for-each: “Quiero Ver Todos los Caramelos”


```
String[] nombres = {"Ana", "Luis", "Eva"};

for (String nombre : nombres) {
    System.out.println("Hola " + nombre + ", te he puesto en la lista");
}
```

 **Nota:** El for-each es de SOLO LECTURA. No puedes modificar el array mientras lo recorres.

★ BE THE CODE, MY FRIEND: El Puzzle de los Bucles Anidados

 **Don Tip:** El bucle externo controla las filas, el interno las columnas. ¿Cuántas iteraciones hace cada uno?

El Puzzle de los Bucles Anidados

¿Qué imprime esto?

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

Solución:

```
*
* *
* * *
```

¡Has dibujado un triángulo con bucles! Eres básicamente un artista digital.

break y continue: El Mando a Distancia de los Bucles

```
// break: "En cuanto vea un 5, me piro"
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        break; // ¡ZAS, fuera!
    }
    System.out.println(i);
}
// Salida: 1, 2, 3, 4

// continue: "Los pares no me molan, siguiente"
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) {
        continue; // "paso de este"
    }
    System.out.println(i);
}
// Salida: 1, 3, 5, 7, 9 (solo impares)
```

? ¡No Hay Preguntas Tontas!

Q: ¿Y si me olvido de poner `i++` en un `for`?

A: Bucle infinito. En `for` es más difícil olvidarlo porque está en la tercera casilla, pero si lo borras... enhorabuena, has creado el primer bucle sin fin.

Q: ¿Cuándo uso `while` y cuándo `for`?

A: Si sabes cuántas veces (contar, recorrer array), usa `for`. Si dependes de una condición (seguir pidiendo hasta que el usuario se canse), usa `while`.

Q: ¿El `for-each` es más lento?

A: En arrays y colecciones como `ArrayList`, no. Y siempre es más LEGIBLE.

Tabla de Supervivencia: ¿Qué Bucle Usar?

Bucle	Cuándo usarlo
<code>for</code>	Sabes el número exacto de vueltas
<code>while</code>	No sabes cuántas, solo cuándo parar

Bucle	Cuándo usarlo
do-while	Necesitas que pase al menos una vez
for-each	Quieres ver todos los elementos sin índices

EL RING: if/else vs switch

Dos estructuras de control se enfrentan en el ring. ¿Quién gana?

if/else: «Yo soy el todoterreno. Puedo evaluar cualquier condición: rangos, combinaciones lógicas, objetos... ¡No tengo límites!»

switch: «Sí, pero para comparar un mismo valor contra muchas opciones, soy más rápido y más legible. Mírame: un solo switch (día) y 7 case. Con tus if encadenados pareces una escalera de caracol.»

if/else: «¿Rápido? En rendimiento moderno da igual. Y además, ¿qué pasa con los rangos? Intenta hacer case > 18: en switch. No puedes.»

switch: «Para eso están los if. Cada uno a lo suyo. Yo para menús, días de la semana, estados de una máquina. Tú para decisiones complejas. ¿Por qué peleamos?»

if/else: «Tienes razón. Al final, nos necesitamos mutuamente.»

💡 **Don Tip:** Usa switch cuando compares una variable contra valores fijos concretos. Usa if cuando tengas rangos, condiciones booleanas compuestas o lógica más compleja.

Excepciones: Cuando Tu Programa Tropieza

Los programas fallan. Es un hecho de la vida como la muerte, los impuestos y que el café se enfríe. Las excepciones son la forma que tiene Java de manejar los fallos con dignidad.

¿Qué es una Excepción?

Una excepción es un evento anómalo. Algo que no debería pasar, pero pasa.

```
int resultado = 10 / 0;    // ArithmeticException
int[] array = {1, 2, 3};
int valor = array[5];     // ArrayIndexOutOfBoundsException
int num = sc.nextInt();   // InputMismatchException si escribes "hola"
```

 **Nota:** El mensaje rojo gigante que escupe Java se llama *stack trace* y es la caja negra de tu avión: dice exactamente dónde y cómo se estrelló todo.

La Jerarquía del Caos

Las excepciones se dividen en **checked** (el compilador te obliga a capturarlas, como `IOException`) y **unchecked** (`RuntimeException`, como `NullPointerException`). Los **Error** son cosas graves (`OutOfMemoryError`) — no intentes capturarlos. Es como tapar una presa con un chicle.

try-catch: Caer con Red

```
try {
    System.out.print("Dime un número: ");
    int numero = sc.nextInt();
    System.out.println("Tu número: " + numero);
} catch (InputMismatchException e) {
    System.out.println("¡Te dije un NÚMERO!");
}
```

Puedes tener varios catch para diferentes desastres:

```
try {
    int[] numeros = new int[3];
    numeros[0] = 10;
    numeros[1] = 0;
    int division = numeros[0] / numeros[2];
    int valor = numeros[5];
} catch (ArithmeticException e) {
    System.out.println("Error matemático: " + e.getMessage());
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Te pasaste de índice: " + e.getMessage());
} catch (Exception e) {
    System.out.println("Algo raro pasó: " + e.getMessage());
}
```



Consejo: Pon los catch más específicos PRIMERO. Si pones catch (Exception e) al principio, los demás nunca se ejecutan.

finally: “No Importa Qué, Lávate los Dientes”

El bloque finally se ejecuta SIEMPRE, haya o no excepción.

```
Scanner sc = null;
try {
    sc = new Scanner(System.in);
    int num = sc.nextInt();
} catch (InputMismatchException e) {
    System.out.println("Error de entrada");
} finally {
    System.out.println("Cerrando recursos... (como un adulto responsable)");
    if (sc != null) sc.close();
}
```



Nota: finally se ejecuta incluso si hay un return dentro de try. Desde Java 7, try-with-resources cierra automáticamente.

throw: Lanzar Excepciones a Propósito

```

public static void validarEdad(int edad) {
    if (edad < 0) {
        throw new IllegalArgumentException(
            "Edad negativa? Te has colado");
    }
    if (edad > 150) {
        throw new IllegalArgumentException(
            edad + " años? O eres inmortal o me tomas el pelo");
    }
    System.out.println("Edad válida: " + edad);
}

```

★ BE THE CODE, MY FRIEND: El Detector de Problemas

💡 **Don Tip:** Cuando salta una excepción, el flujo salta directamente al catch. Lo que va después en el try no se ejecuta.

El Detector de Problemas

¿Qué imprime este programa?

```

public class PruebaExcepciones {
    public static void main(String[] args) {
        try {
            System.out.println("1. Entrando en peligro");
            int[] datos = {10, 20};
            System.out.println("2. Array creado");
            System.out.println("3. " + datos[2]);
            System.out.println("4. Esto no se imprime");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("5. ¡Capturado! Índice fuera de rango");
        } finally {
            System.out.println("6. FINALLY: Siempre");
        }
        System.out.println("7. El programa sigue como si nada");
    }
}

```

Solución: 1 → 2 → datos[2] (índice 2 no existe en array de 2) → 5 → 6 → 7. La línea 4 nunca se imprime. Es como el primo que promete venir a la cena de Navidad.

? ¡No Hay Preguntas Tontas!

Q: ¿Y si no pongo try-catch? ¿El programa explota?

A: Sí. Java escupe un *stack trace* rojo. En aplicaciones de verdad es inaceptable. En tus ejercicios, pasa más de lo que crees.

Q: ¿Cuándo creo mi propia excepción?

A: Cuando la que necesitas no existe. Por ejemplo, `SaldoInsuficienteException`. Crea una clase que herede de `Exception` o `RuntimeException`.

Q: ¿finally se ejecuta incluso si hay `System.exit()`?

A: ¡No! Es el botón nuclear. Ni finally puede evitarlo.

Excepciones Propias

```
class SaldoInsuficienteException extends Exception {  
    public SaldoInsuficienteException(String mensaje) { super(mensaje); }  
}
```

Úsala con `throw` y `throws` como cualquier excepción checked.

Ejercicios Propuestos

Selección

1. **¿Eres mayor?** — Pide la edad al usuario y dile si puede votar, conducir o debería estar durmiendo la siesta.
2. **La calculadora de insultos** — Pide dos números y un operador con `switch`. Captura división por cero.
3. **El clasificador de notas sarcástico** — Pide una nota (0-100) y clasifícala con `if`.
4. **Días del mes** — Pide un mes (1-12) y muestra los días con `switch`.
5. **Par o impar con ternario** — Pide un número y usa el ternario en una línea.
6. **El validador ruinas** — Pide tres números y determina el mayor con `if` anidados.

Bucles

1. **El contador vengativo** — while del 1 al 100. Que se queje cada 10 números.
2. **Sumatorio con trauma** — Pide números hasta 0. Suma y dile algo a los negativos.
3. **Factorial** — Calcula el factorial con for.
4. **Adivina el número** — El programa elige uno del 1 al 100. Usa do-while.
5. **El triángulo constructor** — Dibuja un triángulo de asteriscos con bucles anidados.
6. **Fibonacci** — Muestra los primeros N números de Fibonacci.

Excepciones

1. **División segura** — Divide dos números. Captura `ArithmeticException` e `InputMismatchException`.
2. **El indexador imprudente** — Array de 5 enteros. Pide índice. Captura `ArrayIndexOutOfBoundsException`.
3. **Conversión kamikaze** — Convierte cadena a entero. Captura `NumberFormatException`.
4. **try-with-resources** — Lee un archivo que no existe. Captura `FileNotFoundException`.
5. **Excepción personalizada** — Crea `EdadInvalidaException`. Lánzala si edad < 0 o > 150.
6. **finally: la prueba definitiva** — Con return dentro de try, demuestra que finally se ejecuta antes.

Resumen

Concepto	Cuándo usarlo
if/else	Decisiones con 2-3 caminos posibles
switch	Muchos caminos basados en un mismo valor
while	Repetir mientras se cumpla una condición (0+ veces)
do-while	Repetir al menos una vez, luego comprobar
for	Repetir un número conocido de veces
try/catch	Capturar y manejar errores sin romper el programa
finally	Código que se ejecuta siempre, haya o no error

Buenas prácticas:

- Prefiere switch sobre múltiples if-else encadenados
 - Los bucles for son más legibles que while cuando sabes el número de iteraciones
 - Captura excepciones específicas, no Exception genérica
 - Usa try-with-resources para recursos que hay que cerrar
 - No uses excepciones para control de flujo normal
-

RAs trabajados en esta unidad:

- **RA3** - Estructuras de control
-



Sergi Garcia Barea — CC BY-SA 4.0



Unidad 4: Algorítmica I — Fundamentos

Objetivos de aprendizaje

- Entender qué es un algoritmo y cómo diseñarlo
- Implementar búsqueda lineal y binaria en Java
- Implementar ordenación burbuja e inserción
- Analizar la complejidad algorítmica con notación Big O
- Elegir el algoritmo adecuado según el contexto

¿Qué es un Algoritmo? (El Arte de Dar Instrucciones Precisas)

Un **algoritmo** es una secuencia de pasos finita, ordenada y sin ambigüedades que resuelve un problema. Básicamente, es una receta de cocina, pero para tu ordenador.

Cuando cocinas una tortilla de patatas, sigues un algoritmo mental:

1. Pelar las patatas
2. Cortarlas en rodajas finas
3. Freírlas en aceite abundante
4. Batir los huevos
5. Mezclar todo y cuajar

Si un paso es ambiguo (“echa sal al gusto”), cada persona lo interpreta diferente. Un algoritmo de verdad no deja espacio a la interpretación: cada paso debe ser **preciso** y **determinista**.

Nota:

La palabra “algoritmo” viene del matemático persa **Al-Juarismi** (siglo IX). Escribió un libro sobre cómo hacer cálculos con números indios. Siglos después, los informáticos le robamos la palabra.

Algoritmos en Programación

En informática, los algoritmos más clásicos se dividen en dos grandes familias:

- **Búsqueda:** encontrar un elemento dentro de un conjunto de datos.

- **Ordenación:** poner un conjunto de datos en un orden determinado (numérico, alfabético, etc.).

Y de eso va esta unidad: de buscar y ordenar como un profesional.

```
// El algoritmo más simple que existe: sumar dos números
public class AlgoritmoSimple {
    public static void main(String[] args) {
        int a = 5;
        int b = 3;
        int resultado = a + b; // Esto ES un algoritmo: pasos precisos que producen un
resultado
        System.out.println("5 + 3 = " + resultado);
    }
}
```

⚠ Advertencia:

No todo código es un algoritmo. Un algoritmo es la *idea*. El código es su *materialización*. Puedes implementar el mismo algoritmo en Java, Python o ensamblador. La esencia es la misma.

Propiedades de un Algoritmo

Para que un algoritmo sea considerado como tal, debe cumplir:

1. **Finito:** debe terminar en algún momento. Si se ejecuta para siempre, no es un algoritmo, es una pesadilla.
2. **Preciso:** cada paso debe estar definido sin ambigüedad.
3. **Entrada:** debe recibir cero o más valores de entrada.
4. **Salida:** debe producir al menos un valor de salida.
5. **Eficaz:** debe resolver el problema en tiempo finito.

Búsqueda Lineal: El Buscador de Zapatos Perdidos

Imagina que pierdes una zapatilla en tu habitación. ¿Qué haces? Miras debajo de la cama, detrás de la puerta, en el armario... básicamente **revisas cada sitio hasta encontrarla**.

Pues eso es la **búsqueda lineal**: recorres un array elemento por elemento hasta encontrar lo que buscas. Es simple, directa, y funciona incluso si los datos están desordenados.

```
public class BusquedaLineal {

    public static int buscar(int[] array, int objetivo) {
        for (int i = 0; i < array.length; i++) {
            if (array[i] == objetivo) {
                return i; // ¡Encontrado! Devuelve la posición
            }
        }
        return -1; // No está en el array
    }

    public static void main(String[] args) {
        int[] numeros = {34, 12, 56, 78, 23, 9, 45, 67};

        int resultado = buscar(numeros, 23);
        if (resultado != -1) {
            System.out.println("¡Encontrado el 23 en la posición " + resultado + "!");
        } else {
            System.out.println("El 23 no está en el array.");
        }

        resultado = buscar(numeros, 99);
        if (resultado == -1) {
            System.out.println("El 99 no está. Como unas zapatillas que nunca aparecen.");
        }
    }
}
```

Análisis: ¿Cómo de rápido es?

En el **mejor caso**, el elemento está en la primera posición → encuentras en 1 paso.

En el **peor caso**, el elemento está al final o no existe → recorres los n elementos enteros.

Decimos que su **complejidad es $O(n)$** — lineal. Si el array tiene 10 elementos, tardas ~10 pasos; si tiene 10.000, tardas ~10.000 pasos. Crece al mismo ritmo que los datos.

 **Consejo:**

La búsqueda lineal es como buscar en tu nevera: si es pequeña, da igual el método. Pero si tienes un almacén de 10.000 items, necesitas algo mejor.

Búsqueda Binaria: El Buscador Jedi

La **búsqueda binaria** es como buscar una palabra en el diccionario. No abres por la página 1 y pasas de una en una. Abres por la mitad, ves si la palabra está antes o después, y descartas media tonelada de papel en cada intento.

Requisito imprescindible: el array debe estar ordenado. Si no, este método no funciona.

```

public class BusquedaBinaria {

    public static int buscar(int[] array, int objetivo) {
        int izquierda = 0;
        int derecha = array.length - 1;

        while (izquierda <= derecha) {
            int medio = izquierda + (derecha - izquierda) / 2; // Mitad del segmento

            if (array[medio] == objetivo) {
                return medio; // ¡Bingo!
            }

            if (array[medio] < objetivo) {
                izquierda = medio + 1; // Descartamos la mitad izquierda
            } else {
                derecha = medio - 1; // Descartamos la mitad derecha
            }
        }
        return -1; // No encontrado
    }

    public static void main(String[] args) {
        // OJO: tiene que estar ORDENADO
        int[] numeros = {2, 5, 8, 12, 19, 24, 31, 37, 42, 50, 58, 63};

        int resultado = buscar(numeros, 31);
        System.out.println("31 encontrado en posición: " + resultado); // 6

        resultado = buscar(numeros, 3);
        System.out.println("3 encontrado en: " + resultado); // -1
    }
}

```

¿Por qué $\text{izquierda} + (\text{derecha} - \text{izquierda}) / 2$ en lugar de $(\text{izquierda} + \text{derecha}) / 2$?

Porque si el array es muy grande (cerca de `Integer.MAX_VALUE` elementos), `izquierda + derecha` puede desbordarse. La fórmula alternativa evita ese problema. Es un clásico bug que apareció incluso en la biblioteca de Java original.

⚠ Advertencia:

Si ejecutas búsqueda binaria sobre un array **no ordenado**, los resultados son basura. No hay error de compilación, no hay excepción. Simplemente obtienes la respuesta equivocada. Como buscar “merienda” en un diccionario que tiene las palabras al azar.

Análisis: $O(\log n)$

En cada paso, la búsqueda binaria **descarta la mitad** del array restante.

- Array de 16 elementos → 4 pasos máximos
- Array de 32 elementos → 5 pasos
- Array de 1024 elementos → 10 pasos
- Array de 1.000.000 elementos → 20 pasos

Eso es $O(\log n)$, o complejidad logarítmica. Crece muy despacio incluso con datos enormes. Es la diferencia entre preguntar a 1000 personas una por una vs. preguntar “¿está a la izquierda o a la derecha?” y descartar a 500 de golpe.

```
public class DemoComplejidad {  
    public static void main(String[] args) {  
        // Simulación visual de pasos necesarios  
        System.out.println("n\tLineal\tBinaria");  
        System.out.println("-----");  
        for (int n = 10; n <= 1000000; n *= 10) {  
            int pasosLineal = n;  
            int pasosBinaria = (int)(Math.log(n) / Math.log(2));  
            System.out.println(n + "\t" + pasosLineal + "\t\t" + pasosBinaria);  
        }  
    }  
}
```

Salida esperada:

n	Lineal	Binaria
10	10	4
100	100	7
1000	1000	10
10000	10000	14
100000	100000	17
1000000	1000000	20

Con un millón de elementos, la búsqueda lineal necesita un millón de pasos. La binaria solo **20**. Lee eso otra vez. **20**.

Nota:

$\log_2(n)$ en informática se asume siempre en base 2. No confundas con el logaritmo decimal de toda la vida. Cuando un informático dice “log n”, piensa en “mitad, mitad, mitad...”.

Ordenación Burbuja: Simple pero Torpe

La **ordenación burbuja** (bubble sort) es el algoritmo de ordenación más sencillo de entender... y el más lento de los que merecen la pena. Funciona así:

Ve recorriendo el array. Si el elemento actual es mayor que el siguiente, intercámbialos. Repite hasta que no haya intercambios.

Los elementos grandes “suben como burbujas” hacia el final del array. De ahí el nombre.


```

public class Burbuja {

    public static void ordenar(int[] array) {
        int n = array.length;
        boolean huboIntercambio;

        for (int i = 0; i < n - 1; i++) {
            huboIntercambio = false;

            for (int j = 0; j < n - 1 - i; j++) {
                if (array[j] > array[j + 1]) {
                    // Intercambio
                    int temp = array[j];
                    array[j] = array[j + 1];
                    array[j + 1] = temp;
                    huboIntercambio = true;
                }
            }

            // Si no hubo intercambio, el array ya está ordenado
            if (!huboIntercambio) break;
        }
    }

    public static void main(String[] args) {
        int[] datos = {64, 34, 25, 12, 22, 11, 90};

        System.out.print("Antes: ");
        for (int n : datos) System.out.print(n + " ");

        ordenar(datos);

        System.out.print("\nDespués: ");
        for (int n : datos) System.out.print(n + " ");
        // 11 12 22 25 34 64 90
    }
}

```

¿Por qué es tan lento?

Dos bucles anidados. Para un array de n elementos:

- Primer bucle: n veces
- Segundo bucle: $\sim n$ veces (en realidad $n-i-1$, pero a grandes rasgos n)

Total: $\sim n \times n = n^2$ operaciones. Complejidad $O(n^2)$.

Para 10 elementos $\rightarrow$ 100 operaciones (bien). Para 1000 elementos $\rightarrow$ 1.000.000 de operaciones (empieza a doler). Para 1.000.000 elementos $\rightarrow$ 1.000.000.000.000 operaciones (tu ordenador pide la jubilación).

Consejo:

Burbuja solo se usa en dos situaciones: (1) estás aprendiendo, (2) sabes que el array tendrá < 50 elementos. Para todo lo demás, hay mejores alternativas.

La Optimización del Flag

Fíjate en la variable `huboIntercambio`. Si en una pasada completa no intercambiamos nada, es que el array ya está ordenado y podemos parar. Esta optimización no mejora el peor caso (array invertido), pero ayuda en arrays casi ordenados.

Ordenación por Inserción: Como Ordenar Cartas en la Mano

La **ordenación por inserción** (insertion sort) es como ordenar las cartas que te reparten en el póker. Tomas una carta nueva y la colocas en su sitio dentro de las que ya tienes ordenadas en la mano.

```

public class Insercion {

    public static void ordenar(int[] array) {
        for (int i = 1; i < array.length; i++) {
            int clave = array[i]; // La carta que vamos a colocar
            int j = i - 1;

            // Desplazar elementos mayores hacia la derecha
            while (j >= 0 && array[j] > clave) {
                array[j + 1] = array[j];
                j--;
            }
            array[j + 1] = clave; // Colocar la carta en su sitio
        }
    }

    public static void main(String[] args) {
        int[] datos = {9, 5, 1, 4, 3};

        System.out.print("Antes: ");
        for (int n : datos) System.out.print(n + " ");

        ordenar(datos);

        System.out.print("\nDespués: ");
        for (int n : datos) System.out.print(n + " ");
        // 1 3 4 5 9
    }
}

```

¿Cómo funciona paso a paso?

Dado {9, 5, 1, 4, 3}:

```

Paso 0: [9] | 5 1 4 3 → la "mano" empieza con el 9
Paso 1: [5 9] | 1 4 3 → 5 se coloca a la izquierda del 9
Paso 2: [1 5 9] | 4 3 → 1 se coloca al principio
Paso 3: [1 4 5 9] | 3 → 4 se cuela entre el 1 y el 5
Paso 4: [1 3 4 5 9] → 3 se coloca entre el 1 y el 4

```

Análisis y ¿Cuándo es buena?

También es $O(n^2)$ en el peor caso (array invertido). Pero:

- **Mejor caso (array casi ordenado):** $O(n)$. Solo hace una pasada. Es rapidísimo.
- Es **estable**: mantiene el orden relativo de elementos iguales.
- **No necesita memoria extra** (ordena in-place).
- Es **más rápido que burbuja** en la práctica, aunque ambos sean $O(n^2)$.

💡 Consejo:

La ordenación por inserción es la reina de los datos **casi ordenados**. Si sabes que tu array tiene 100 elementos y ya está “casi bien” (solo unos pocos fuera de sitio), inserción te va a sorprender. Se usa como paso final en algoritmos avanzados (TimSort, usado por Java y Python).

```
public class Comparacion {
    public static void main(String[] args) {
        // Demostración: inserción vs burbuja con array casi ordenado
        int[] casiOrdenado = {1, 2, 3, 4, 6, 5, 7, 8, 9, 10};
        // Solo el 6 y el 5 están intercambiados

        int[] burbuja = casiOrdenado.clone();
        int[] insercion = casiOrdenado.clone();

        // Con burbuja da varias pasadas aunque casi esté ordenado
        // Con inserción resuelve en un suspiro
        System.out.println("Inserción es ideal cuando tienes que añadir");
        System.out.println("un elemento a un array ya ordenado.");
    }
}
```

Complejidad Algorítmica: Big O para Mortales

La **notación Big O** describe cómo crece el tiempo de ejecución de un algoritmo cuando crece la cantidad de datos de entrada. No te da el tiempo exacto en segundos (eso depende del hardware), te da la **tendencia**.

Las Complejidades Más Comunes

Notación	Nombre	Ejemplo	Para n=1000
$O(1)$	Constante	Acceder a un array por índice	1 operación
$O(\log n)$	Logarítmica	Búsqueda binaria	~10 ops
$O(n)$	Lineal	Búsqueda lineal	1000 ops
$O(n \log n)$	Casi lineal	Ordenación rápida (lo verás más adelante)	~10.000 ops
$O(n^2)$	Cuadrática	Burbuja, inserción	1.000.000 ops
$O(2^n)$	Exponencial	Fibonacci recursivo sin optimizar	¡inviabile!

```

public class EjemplosComplejidad {

    // O(1) – CONSTANTE: siempre igual, no importa el tamaño
    public static int obtenerPrimero(int[] array) {
        return array[0]; // Un solo paso, siempre
    }

    // O(n) – LINEAL: crece proporcional a n
    public static int sumar(int[] array) {
        int suma = 0;
        for (int num : array) {
            suma += num; // n pasos
        }
        return suma;
    }

    // O(n²) – CUADRÁTICA: dos bucles anidados
    public static void imprimirPares(int[] array) {
        for (int i = 0; i < array.length; i++) {
            for (int j = 0; j < array.length; j++) {
                System.out.println(array[i] + ", " + array[j]); // n × n pasos
            }
        }
    }

    public static void main(String[] args) {
        int[] datos = {10, 20, 30, 40, 50};

        System.out.println("O(1): " + obtenerPrimero(datos));
        System.out.println("O(n): " + sumar(datos));
        System.out.println("O(n²): mira la consola llenándose de pares...");
        imprimirPares(datos);
    }
}

```

Reglas Prácticas de Big O

1. **Ignora constantes:** $O(2n)$ es lo mismo que $O(n)$. El 2 no importa cuando n tiende a infinito.
2. **Quédate con el término dominante:** $O(n^2 + n) \rightarrow O(n^2)$. El n^2 come al n cuando n crece.
3. **Los bucles anidados multiplican:** un bucle dentro de otro $\rightarrow n \times n \rightarrow O(n^2)$.

4. Los bucles secuenciales suman: un bucle y luego otro $\rightarrow O(n + n) \rightarrow O(2n) \rightarrow O(n)$.

 Nota:

“Big O” describe el **peor caso** (o cota superior). Si dices que búsqueda lineal es $O(n)$, estás diciendo “como mucho, tardará lo mismo que recorrer todo el array”. Existen también Big Omega Ω (mejor caso) y Big Theta Θ (caso promedio), pero con Big O tienes suficiente para empezar.

```
public class ReglasBigO {
    public static void main(String[] args) {
        // REGLA 1: Las constantes no importan
        //  $O(2n) \rightarrow O(n)$ 
        //  $O(100n) \rightarrow O(n)$ 

        // REGLA 2: El término dominante se queda
        //  $O(n^2 + 5n + 1) \rightarrow O(n^2)$ 
        //  $O(n + \log n) \rightarrow O(n)$ 
        //  $O(n! + n^2) \rightarrow O(n!)$ 

        // REGLA 3: Bucles anidados  $\rightarrow$  multiplica
        //  $\text{for}(i...) \{ \text{for}(j...) \} \rightarrow O(n * m) \rightarrow O(n^2)$ 

        // REGLA 4: Bucles secuenciales  $\rightarrow$  suma
        //  $\text{for}(i...) \{ \} \text{for}(i...) \{ \}$ 
        //  $O(n + n) \rightarrow O(2n) \rightarrow O(n)$ 

        System.out.println("Big O no es magia, es simplificar.");
        System.out.println("Pregúntate: ¿qué pasa cuando n se hace MUY grande?");
    }
}
```

BE THE CODE, MY FRIEND: Implementa Búsqueda Binaria desde Cero

💡 **Don Tip:** La búsqueda binaria reduce el problema a la mitad en cada paso. Céntrate en los límites izquierda y derecha — el error más común es no actualizarlos bien.

Este ejercicio es un clásico en entrevistas técnicas de Google, Amazon y compañía. Si algún día quieres trabajar en una FAANG, la búsqueda binaria la tienes que saber escribir dormido,

borracho y con una mano atada a la espalda.

Instrucciones:

Sin mirar el código de arriba, implementa el método `busquedaBinaria` que recibe un array ordenado de enteros y un objetivo, y devuelve la posición del objetivo o -1 si no existe.

```
/**
 * Implementa este método:
 *
 * public static int busquedaBinaria(int[] array, int objetivo)
 *
 * Requisitos:
 * - No uses Arrays.binarySearch() – el sentido es programarlo
 * - No mires el código de la unidad – fuerza tu cerebro a recordar
 * - Piénsalo así: coge el medio, compara, descarta la mitad, repite
 */
public class RetoBinaria {

    public static int busquedaBinaria(int[] array, int objetivo) {
        // 🧠 TU CÓDIGO AQUÍ
        return -1;
    }

    public static void main(String[] args) {
        int[] pruebas = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19};

        System.out.println("Probando búsqueda binaria...");
        System.out.println("El 7 está en posición: " + busquedaBinaria(pruebas, 7));
        // 3
        System.out.println("El 13 está en posición: " + busquedaBinaria(pruebas, 13));
        // 6
        System.out.println("El 8 está en posición: " + busquedaBinaria(pruebas, 8));
        // -1
        System.out.println("El 19 está en posición: " + busquedaBinaria(pruebas, 19));
        // 9
        System.out.println("El 1 está en posición: " + busquedaBinaria(pruebas, 1));
        // 0
        System.out.println("Primer elemento fuera: " + busquedaBinaria(new int[]{},
5)); // -1
    }
}
```


Pistas (si te atascas):

- Necesitas dos punteros: izquierda y derecha.
- Calcula el medio como $\text{izquierda} + (\text{derecha} - \text{izquierda}) / 2$.
- Si `array[medio] == objetivo`, devuelve medio.
- Si `array[medio] < objetivo`, mueve izquierda a `medio + 1`.
- Si `array[medio] > objetivo`, mueve derecha a `medio - 1`.
- El bucle termina cuando `izquierda > derecha`.
- Array vacío → directamente -1.

💡 Consejo:

El error más común en búsqueda binaria (incluso en programadores con 10 años de experiencia) es el **off-by-one**: ¿`<=` o `<`? ¿`medio + 1` o `medio`? Dibuja el array en un papel con 3 elementos y simula los casos. El papel y boli son tus mejores herramientas de debugging.

Solución (inténtalo antes de mirar)

```
public static int busquedaBinaria(int[] array, int objetivo) {
    if (array == null || array.length == 0) return -1;

    int izquierda = 0;
    int derecha = array.length - 1;

    while (izquierda <= derecha) {
        int medio = izquierda + (derecha - izquierda) / 2;

        if (array[medio] == objetivo) {
            return medio;
        } else if (array[medio] < objetivo) {
            izquierda = medio + 1;
        } else {
            derecha = medio - 1;
        }
    }
    return -1;
}
```

El departamento de calidad ha recibido este algoritmo de ordenación. Algo huele mal. Identifica los errores y explica por qué no funciona:

```
public class BurbujaLiosa {  
    public static void ordenar(int[] arr) {  
        for (int i = 0; i < arr.length; i++) {  
            for (int j = 0; j < arr.length; j++) {  
                if (arr[j] > arr[j + 1]) {  
                    int temp = arr[j];  
                    arr[j] = arr[j + 1];  
                    arr[j + 1] = temp;  
                }  
            }  
        }  
    }  
}
```

Pista: hay un error de índices y otro de rendimiento.

💡 **Don Tip:** Cuando un bucle accede a `arr[j + 1]`, asegúrate de que `j + 1` no se salga del array. También piensa: ¿recorro menos elementos en cada pasada?

¡No Hay Preguntas Tontas!

Q: ¿Por qué no usamos siempre búsqueda binaria si es tan rápida?

A: Porque requiere que los datos estén **ordenados**. Ordenar también cuesta tiempo. Si tienes que ordenar cada vez que buscas, pierdes la ventaja. La búsqueda binaria es ideal cuando ordenas una vez y buscas muchas veces.

Q: ¿Big O sirve para algo en el mundo real o es solo teoría de examen?

A: Sirve y mucho. Cuando tu app web empieza a ir lenta con 1000 usuarios, la diferencia entre $O(n)$ y $O(n^2)$ es la diferencia entre “echar un café mientras carga” y “jubilarte antes de que termine”. Las empresas tech (Google, Amazon, Netflix) matan por eficiencia. Un algoritmo malo en producción cuesta dinero real en servidores.

Q: ¿Hay algoritmos de ordenación más rápidos que $O(n^2)$?

A: Sí, y los verás en la siguiente unidad. QuickSort, MergeSort y HeapSort son del orden $O(n \log n)$, que es mucho más rápido que $O(n^2)$. Por ejemplo, para 1.000.000 de elementos, $O(n$

$\log n$) son ~20 millones de operaciones; $O(n^2)$ son 1 billón.

Q: ¿Puedo mezclar tipos de búsqueda y ordenación?

A: Claro. Es muy común ordenar con QuickSort y luego buscar con binaria. O si el array es muy pequeño (< 50), usar búsqueda lineal aunque esté ordenado, porque la sobrecarga de la binaria no compensa.

Q: Vale, pero... ¿cuándo uso cada algoritmo de ordenación?

A: Regla práctica:

- Array pequeño (< 50 elementos) → cualquiera vale, usa inserción por simplicidad.
- Array casi ordenado → **inserción** arrasa.
- Array grande → no uses burbuja ni inserción. Espera a la próxima unidad.
- Array enorme con memoria limitada → **HeapSort**.
- Array enorme y necesitas estabilidad → **MergeSort**.
- Array enorme y velocidad bruta → **QuickSort**.

Q: ¿Qué significa que un algoritmo sea “estable”?

A: Un algoritmo de ordenación es estable si mantiene el orden relativo de elementos con el mismo valor. Por ejemplo, si tienes dos usuarios con edad 25 (“Ana” antes que “Luis”), un algoritmo estable los deja en ese orden. Inserción y MergeSort son estables; Burbuja se puede hacer estable o no según la implementación; QuickSort no lo es por defecto.

Q: ¿Y todo esto sirve para aprobar el módulo?

A: Sirve para aprobar el módulo, para las entrevistas técnicas, para escribir código que no de pena, y para que cuando alguien diga “es $O(n^2)$ ” tú sepas exactamente por qué es malo. Así que sí, pon atención.

EL ACERTIJO

Tienes 9 monedas aparentemente idénticas. Una de ellas es falsa y pesa menos que las demás. Dispones de una balanza de dos platillos. ¿Cuál es el número mínimo de pesadas que necesitas para encontrar la moneda falsa?

Pista: no las peses una por una. Usa divide y vencerás.

👁️ **Don Tip:** Si divides 9 monedas en 3 grupos de 3, con una sola pesada puedes descartar 6 monedas.

Resumen de la Unidad

Algoritmo	Complejidad	¿Cuándo usarlo?
Búsqueda lineal	$O(n)$	Arrays pequeños o desordenados
Búsqueda binaria	$O(\log n)$	Arrays grandes ORDENADOS
Burbuja	$O(n^2)$	Solo para aprender o arrays muy pequeños
Inserción	$O(n^2) / O(n)$	Arrays pequeños o casi ordenados

Conceptos clave:

- Un algoritmo es una secuencia finita, precisa y no ambigua de pasos.
- La notación Big O describe la tasa de crecimiento del tiempo de ejecución.
- $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$
- Ordenar antes de buscar puede merecer la pena si haces muchas búsquedas.
- El mejor algoritmo depende del contexto: tamaño, orden inicial, requisitos de memoria y estabilidad.

⚠️ Advertencia:

Esto es solo el principio. La siguiente unidad sube el nivel: QuickSort, MergeSort, búsqueda con hash, y complejidad en casos reales. Pero si dominas estos fundamentos, lo demás es cuesta abajo. Algorítmica es como montar en bici: al principio parece imposible, luego no se te olvida nunca.



Sergi Garcia Barea — CC BY-SA 4.0



Unidad 5: Algorítmica II — Técnicas Avanzadas

Bienvenido, valiente explorador del stack de llamadas. Hasta ahora has sobrevivido a arrays, bucles, objetos y condiciones. Has dominado el arte de hacer que el ordenador haga lo que le dices. Pero el bosque se oscurece. Llegó la hora de enfrentarte a los algoritmos que **cambian tu forma de pensar** (y de paso, te hacen quedar genial en las entrevistas técnicas de Google, que nunca se sabe).

En esta unidad vamos a dejar atrás los bucles obvios y nos adentramos en la **recursividad** y **divide y vencerás**. No tengas miedo. Respira hondo. Y recuerda: para entender la recursividad, primero tienes que entender la recursividad.

Vamos allá.

1. Recursividad: una función que se llama a sí misma (sin volverse loca)

La recursividad es cuando una función se invoca a sí misma. Sí, suena a bucle infinito y a dolor de cabeza. Pero con una **condición de parada** bien puesta, es una de las herramientas más elegantes y potentes que existen en programación.

Piénsalo así: en lugar de resolver el problema entero de golpe, resuelves una pequeña parte y le pasas el resto a... ti mismo. Como cuando limpias tu habitación: coges una cosa del suelo y luego vuelves a llamar a la misma función “limpiarHabitacion” con lo que queda. Eventualmente no queda nada, y has terminado.

Las dos caras de la moneda recursiva

[!NOTE] Toda función recursiva necesita **dos partes** imprescindibles. Si te falta una, estás muerto:

- **Caso base:** la condición que detiene la recursión. Sin esto, tu programa se ejecuta hasta que la JVM se cansa y te lanza un `StackOverflowError`.
- **Caso recursivo:** la llamada a sí misma, normalmente con una versión más pequeña del mismo problema.

La estructura general es siempre la misma:

```

public static tipo funcion(parametros) {
    if (/* condicion de parada */) {
        return /* valor base */;
    } else {
        // hacer algo
        return funcion(/* version mas pequena */);
    }
}

```

La pila de llamadas (call stack) — el origen de toda la diversión

Cada vez que una función se llama, Java reserva un trocito de memoria en el **stack** (la pila). Ese trocito se llama **stack frame** y guarda: los parámetros de la función, las variables locales, y la dirección de vuelta para cuando la función termine.

Si llamas a una función 5 veces, tienes 5 frames en el stack. Si la llamas 10.000 veces... bum. `StackOverflowError`.

```

public class StackExplorer {

    static int prof = 0;

    static void recursivo() {
        prof++;
        System.out.println("Llamada numero: " + prof);
        recursivo(); // ¡no hay caso base!
    }

    public static void main(String[] args) {
        try {
            recursivo();
        } catch (StackOverflowError e) {
            System.out.println("Stack explotó en la llamada: " + prof);
        }
    }
}

```

Pruébalo. Verás que el número varía según tu máquina y la JVM. Normalmente entre 10.000 y 20.000 llamadas. No es infinito. Nada lo es.

[!WARNING] Sin caso base, no hay piedad. El stack tiene un límite y Java no te va a salvar. Cada frame ocupa espacio, y cuando el vaso se desborda, la JVM dice “hasta aquí llegamos”.

Recursividad vs iteración: el duelo

Aspecto	Recursividad	Iteración (bucles)
Legibilidad	Muy elegante para problemas jerárquicos	Más verbosa pero clara
Memoria	Gasta stack (cada llamada = frame nuevo)	Solo variables locales
Velocidad	Más lenta (overhead de llamadas)	Más rápida
Stack overflow	Riesgo real si profundidad es alta	No aplica
Casos ideales	Árboles, grafos, backtracking, divide y vencerás	Recorridos lineales, procesamiento simple

La regla de oro: **usa recursividad cuando el problema sea inherentemente recursivo** (árboles, expresiones anidadas, algoritmos de ordenación avanzados). Para lo demás, un for de toda la vida.

2. Ejemplos clásicos (los clásicos por algo son)

Vamos a ver tres ejemplos que te perseguirán el resto de tu carrera. Acostúmbrate a ellos. Aparecen en exámenes, entrevistas, y conversaciones de ascensor con otros programadores.

2.1. Factorial — el “Hello World” de la recursividad

```

public class Factorial {

    static int fact(int n) {
        if (n <= 1) return 1;           // caso base
        return n * fact(n - 1);        // caso recursivo
    }

    public static void main(String[] args) {
        System.out.println("fact(5) = " + fact(5)); // 120
        System.out.println("fact(0) = " + fact(0)); // 1
        System.out.println("fact(7) = " + fact(7)); // 5040
    }
}

```

¿Qué pasa cuando llamamos a `fact(4)`? Vamos paso a paso:

```

fact(4) → 4 * fact(3)
        → 4 * (3 * fact(2))
        → 4 * (3 * (2 * fact(1)))
        → 4 * (3 * (2 * 1))
        → 4 * (3 * 2)
        → 4 * 6
        → 24

```

Primero “baja” hasta el caso base, luego “sube” haciendo las multiplicaciones. Como un ascensor que baja al sótano y vuelve a subiteando puertas.

2.2. Fibonacci (versión ingenua — no hagas esto en casa)


```

public class FiboNaive {

    static long fibo(int n) {
        if (n <= 1) return n;
        return fibo(n - 1) + fibo(n - 2);
    }

    public static void main(String[] args) {
        long inicio = System.currentTimeMillis();
        System.out.println("fibo(40) = " + fibo(40));
        long fin = System.currentTimeMillis();
        System.out.println("Tiempo: " + (fin - inicio) + " ms");
    }
}

```

Ejecuta esto. Serán unos segundos de infierno. ¿El problema? `fibo(40)` genera un **árbol de llamadas monstruoso**. Cada llamada se bifurca en dos, que se bifurcan en dos, etc. El número total de llamadas es aproximadamente $O(2^n)$.

```

          fibo(5)
        /      \
      fibo(4)   fibo(3)
     /  \    /  \
  fibo(3) fibo(2) fibo(2) fibo(1)
 /  \  /  \  /  \
fibo(2) fibo(1) ... ..
/    \
fibo(1) fibo(0)

```

Cuenta las veces que se llama a `fib(1)`. Exacto, demasiadas. Es como preguntarle a tu compañero una y otra vez la misma pregunta esperando una respuesta diferente. Eso, señor mío, es locura.

[!TIP] Este es el ejemplo perfecto para entender por qué **la eficiencia importa**. Un algoritmo que se ve bonito en código puede ser un desastre en tiempo de ejecución. Fibonacci naive es el rey de los algoritmos bonitos pero inútiles para números grandes.

2.3. Fibonacci optimizado (con memoización — ahora sí)

```

public class FiboOptimizado {

    static long[] memo;

    static long fibo(int n) {
        if (n <= 1) return n;
        if (memo[n] != 0) return memo[n];          // ya lo calculamos antes
        memo[n] = fibo(n - 1) + fibo(n - 2);
        return memo[n];
    }

    public static void main(String[] args) {
        int n = 92;
        memo = new long[n + 1];
        long inicio = System.currentTimeMillis();
        System.out.println("fibo(" + n + ") = " + fibo(n)); // instantáneo
        long fin = System.currentTimeMillis();
        System.out.println("Tiempo: " + (fin - inicio) + " ms");
    }
}

```

¿Qué cambia? Guardamos los resultados en un array (memo[]) para no recalcular. Si ya hemos calculado fib(10), lo devolvemos directamente. Cada fibonacci se calcula **una sola vez**.

La mejora es brutal: de **$O(2^n)$** a **$O(n)$** . Esa es la mayor subida de nivel desde que pasaste de piedra a pokéball.

[!NOTE] Esto se llama **memoización** (sí, sin la “r”). Es la base de la programación dinámica, que verás en unidades más avanzadas. Consiste en: “si ya lo calculé, no lo vuelvo a calcular, lo guardo en un mapa o array y lo reutilizo”.

2.4. Suma de un array

```

public class SumArray {

    static int sumar(int[] arr, int i) {
        if (i == arr.length) return 0;           // caso base: no hay más elementos
        return arr[i] + sumar(arr, i + 1);       // caso recursivo
    }

    public static void main(String[] args) {
        int[] nums = {3, 8, 2, 10, 5};
        System.out.println("Suma: " + sumar(nums, 0)); // 28

        int[] vacio = {};
        System.out.println("Suma vacia: " + sumar(vacio, 0)); // 0
    }
}

```

Este ejemplo es casi idéntico al factorial, pero recorriendo un array. El índice `i` avanza hasta llegar a `arr.length`. En ese momento, devolvemos 0 y empezamos a sumar hacia arriba.

[!TIP] | Piensa en la recursividad como si pelaras una cebolla. Quitas una capa (un elemento), y lo que queda es el mismo problema pero más pequeño. Eventualmente no queda cebolla.

3. Divide y vencerás (Divide & Conquer)

La estrategia es tan antigua como Julio César, pero aplicada a algoritmos sigue siendo igual de efectiva. El principio es sencillo:

1. **Dividir** el problema en subproblemas más pequeños y manejables.
2. **Conquistar** cada subproblema recursivamente (llamadas recursivas).
3. **Combinar** las soluciones de los subproblemas para obtener la solución del problema original.

```

Input grande
|
├─Dividir→ Subproblema A   → Conquistar (recursivo) → Combinar → Output
└─Dividir→ Subproblema B   → Conquistar (recursivo) ─

```

Los dos reyes indiscutibles de esta técnica son los algoritmos de ordenación más famosos del mundo: **Quicksort** y **Mergesort**. Merecen sección propia. Y corbata.

4. Quicksort — el rápido (cuando le sale bien)

Creado por Tony Hoare en 1959. Sí, tiene más años que tus padres. Y sigue siendo el algoritmo de ordenación más usado del mundo. Por algo será.

Cómo funciona

1. Elegimos un **pivote** (un elemento del array).
2. Colocamos todos los elementos **menores** que el pivote a su izquierda, y los **mayores** a su derecha. Esto se llama **particionar**.
3. Aplicamos el mismo proceso recursivamente a las dos mitades (izquierda y derecha del pivote).

Cuando el array tiene 0 o 1 elementos... ya está ordenado. Caso base.

Implementación

```

public class Quicksort {

    static void qs(int[] arr, int izq, int der) {
        if (izq >= der) return;    // caso base: 0 o 1 elementos

        int pivote = arr[(izq + der) / 2];    // elegimos el del medio
        int i = izq, j = der;

        // partición
        while (i <= j) {
            while (arr[i] < pivote) i++;
            while (arr[j] > pivote) j--;
            if (i <= j) {
                int tmp = arr[i];
                arr[i] = arr[j];
                arr[j] = tmp;
                i++;
                j--;
            }
        }

        // llamadas recursivas a cada mitad
        qs(arr, izq, j);
        qs(arr, i, der);
    }

    public static void main(String[] args) {
        int[] datos = {9, 3, 7, 1, 8, 2, 6, 4, 5};
        qs(datos, 0, datos.length - 1);
        System.out.println(java.util.Arrays.toString(datos));
    }
}

```

Ejecución paso a paso con {3, 1, 4, 1, 5, 9, 2, 6}:

Array inicial: [3, 1, 4, 1, 5, 9, 2, 6]

Pivote = 5 (posición media)

Posiciones $i=0$, $j=7$

1. i avanza hasta encontrar 9 (≥ 5), j retrocede hasta encontrar 2 (≤ 5)
2. Intercambiamos 9 y 2 $\rightarrow$ [3, 1, 4, 1, 5, 2, 9, 6]
3. $i=5$, $j=4 \rightarrow i > j$, terminamos partición
4. Llamada recursiva izquierda: [3, 1, 4, 1, 5, 2]
5. Llamada recursiva derecha: [9, 6]
- ...

[!TIP] El secreto de Quicksort está en la partición. Si consigues que los elementos se repartan más o menos equilibradamente, el algoritmo vuela. Si no... prepara los $O(n^2)$.

Elección del pivote

Estrategia	Ventaja	Desventaja
Primer elemento	Simple	Pesimo si array ya ordenado
Último elemento	Simple	Pesimo si array ya ordenado
Elemento central	Mejor equilibrio	Sigue teniendo casos malos
Mediana de tres (primero, medio, último)	Muy robusto	Un poco más de cálculo
Aleatorio	Evita el caso peor en la práctica	Aleatoriedad no determinista

Complejidad

- **Caso promedio:** $O(n \log n)$ — casi siempre.
- **Mejor caso:** $O(n \log n)$ — cuando el pivote divide siempre en mitades iguales.
- **Peor caso:** $O(n^2)$ — cuando el array ya está ordenado y eliges el primer o último pivote.

[!WARNING] Si eliges siempre el primer elemento como pivote y el array ya está ordenado, Quicksort se vuelve más lento que una tortuga con resaca. $O(n^2)$. Literalmente peor que un for anidado cutre.

¿Es estable?

No. Durante la partición, dos elementos iguales pueden intercambiarse de posición. Si necesitas estabilidad (orden original de elementos iguales), mejor usa Mergesort.

5. Mergesort — el fiable (siempre cumple lo que promete)

Creado por John von Neumann en 1945. Sí, el mismo de la arquitectura de ordenadores. El tío no paraba.

Cómo funciona

1. **Dividir** el array en dos mitades (por la mitad exactamente, no hay que elegir pivote).
2. **Ordenar** cada mitad recursivamente.
3. **Fusionar** (merge) las dos mitades ordenadas en un único array ordenado.

[7, 3, 9, 1, 8, 2, 6, 4]

[7, 3, 9, 1] [8, 2, 6, 4]

[7, 3] [9, 1] [8, 2] [6, 4]

[7] [3] [9] [1] [8] [2] [6] [4] ← caso base

[3, 7] [1, 9] [2, 8] [4, 6] ← fusionar

[1, 3, 7, 9] [2, 4, 6, 8] ← fusionar

[1, 2, 3, 4, 6, 7, 8, 9] ← fusionar

Implementación


```

public class Mergesort {

    static void ms(int[] arr, int izq, int der) {
        if (izq >= der) return; // caso base

        int mid = (izq + der) / 2;
        ms(arr, izq, mid);           // ordenar mitad izquierda
        ms(arr, mid + 1, der);       // ordenar mitad derecha
        fusionar(arr, izq, mid, der); // combinar las dos mitades
    }

    static void fusionar(int[] arr, int izq, int mid, int der) {
        int[] tmp = new int[der - izq + 1];
        int i = izq, j = mid + 1, k = 0;

        // Comparar elementos de las dos mitades
        while (i <= mid && j <= der)
            tmp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];

        // Copiar lo que quede de la mitad izquierda
        while (i <= mid) tmp[k++] = arr[i++];

        // Copiar lo que quede de la mitad derecha
        while (j <= der) tmp[k++] = arr[j++];

        // Copiar el array temporal al original
        System.arraycopy(tmp, 0, arr, izq, tmp.length);
    }

    public static void main(String[] args) {
        int[] datos = {9, 3, 7, 1, 8, 2, 6, 4, 5};
        ms(datos, 0, datos.length - 1);
        System.out.println(java.util.Arrays.toString(datos));
    }
}

```

Complejidad

- **Siempre:** $O(n \log n)$. No importa cómo esté el array. Mergesort no tiene caso malo.
- **Memoria:** $O(n)$ extra por el array temporal tmp. Ese es su punto débil.

¿Es estable?

Sí. Cuando dos elementos son iguales, el de la izquierda va primero en la fusión. Esto mantiene el orden original de elementos iguales, algo que Quicksort no puede garantizar.

[INOTE] | Estabilidad no es un concepto teórico aburrido. Es importante cuando ordenas por múltiples criterios. Por ejemplo, si ordenas una lista de alumnos por nota y luego por nombre, quieres que los que tienen la misma nota mantengan el orden alfabético. Mergesort te lo da. Quicksort te hace llorar.

6. Comparación: ¿cuándo uso cada uno?

No hay un “mejor algoritmo” universal. Todo depende del contexto. Aquí tienes una guía práctica para tomar decisiones:

Situación	Elige
Array pequeño (< 50 elementos)	Da igual, usa <code>Arrays.sort()</code>
Array grande con datos al azar	Quicksort (irá genial en promedio)
Array grande, casi ordenado o con muchos duplicados	Mergesort o Quicksort con mediana de tres
Necesitas estabilidad (orden relativo)	Mergesort
La memoria es justa (sistema embebido, móvil)	Quicksort ($O(\log n)$ vs $O(n)$)
No te pagan por pensar	<code>Arrays.sort()</code> y a seguir con tu vida
El array es enorme y los datos están en disco	Mergesort externo (se usa en bases de datos)

En Java, `Arrays.sort()` para tipos primitivos usa **Dual-Pivot Quicksort** (una versión mejorada con dos pivotes). Para objetos usa **TimSort** (una mezcla de Mergesort e Insertion Sort). Por algo Will Smith no canta sobre algoritmos de ordenación.

Rendimiento en la práctica

Algoritmo	n=10	n=100	n=1.000	n=10.000	n=100.000	n=1.000.000
Quicksort	~0 ms	~0 ms	~0 ms	~1 ms	~15 ms	~120 ms
Mergesort	~0 ms	~0 ms	~0 ms	~2 ms	~20 ms	~150 ms
Burbuja (para llorar)	~0 ms	~1 ms	~100 ms	~10.000 ms	no esperes	

[!WARNING] | Nunca, bajo ningún concepto, uses Burbuja (Bubble Sort) en producción. Es $O(n^2)$, lento, y tus compañeros te odiarán. Es como usar un caracol para repartir pizzas. Existe, pero no debería.

7. ★ BE THE CODE, MY FRIEND: implementa Quicksort desde cero

💡 **Don Tip:** Divide y vencerás: elige un pivote, parte el array en menores y mayores, y repite recursivamente. Si dominas ese patrón, Quicksort es tuyo.

Cierra esta página. Abre un editor de texto en blanco. No mires ni una línea de lo que has leído hasta ahora.

Escribe Quicksort desde cero. La firma del método debe ser:

```
static void quicksort(int[] arr, int inicio, int fin)
```

Cuando termines, pégale un vistazo a tu código y compáralo con el de esta unidad.

Niveles de logro:

- ★ Lo tienes, pero has tenido que mirar el código una vez. Aprobado raspado.
- ★★ Te ha salido a la primera y funciona. Eres una máquina.
- ★★★ Te ha salido a la primera, sin errores de off-by-one, y además has elegido mediana de tres como pivote. No necesitas este curso. Vete a dar una charla TED.

[!TIP] Pista mental gratuita: necesitas tres cosas — elegir un pivote, partir el array en dos zonas (menores y mayores), y llamar recursivamente a cada lado. Si memorizas ese patrón,

8. ¡No Hay Preguntas Tontas!

¿La recursividad es más lenta que un bucle?

Casi siempre, sí. Cada llamada recursiva añade overhead: crear un stack frame, guardar variables, calcular dirección de retorno, etc. Para operaciones simples como sumar un array, un for siempre gana. **Pero** hay problemas que se expresan de forma mucho más natural con recursividad: árboles, expresión de paréntesis anidados, recorrido de laberintos, backtracking. La regla es: usa bucles para lo simple, recursividad para lo complejo.

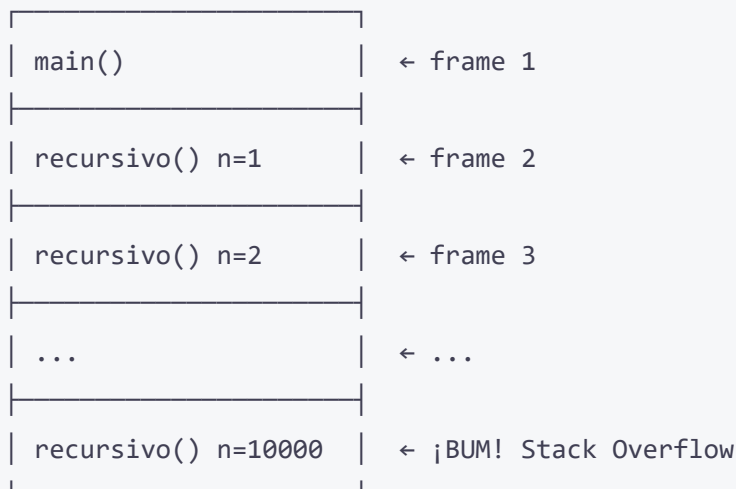
¿Qué es exactamente un StackOverflowError?

El **call stack** es una región de memoria donde la JVM guarda información de cada llamada a función. Cada llamada ocupa un espacio (stack frame) que contiene: parámetros, variables locales, y la dirección de retorno.

Cuando haces demasiadas llamadas recursivas (o una recursión infinita), el stack se llena. La siguiente llamada ya no cabe. La JVM dice “no puedo más” y te lanza un `StackOverflowError`.

Es como llenar un vaso de agua: cada gota es una llamada. Llegas a un punto en que una gota más lo desborda todo.

Stack memoria:



¿Qué es tail recursion? ¿Java la optimiza?

Tail recursion (recursión de cola) es cuando la llamada recursiva es la **última operación** que realiza la función. No hay ninguna operación pendiente después de la llamada.

```
// Sin tail recursion – hay una multiplicación pendiente
static int factNormal(int n) {
    if (n <= 1) return 1;
    return n * factNormal(n - 1); // la multiplicación espera a que vuelva
}

// Con tail recursion – la llamada es lo último
static int factTail(int n, int acum) {
    if (n <= 1) return acum;
    return factTail(n - 1, n * acum); // nada pendiente después
}
```

En lenguajes funcionales como Scala, Haskell o Scheme, el compilador detecta tail recursion y la convierte en un bucle internamente (no gasta stack). **Java no hace esto**. Ni ahora ni en el futuro previsible. Si llamas a `factTail(10000, 1)` en Java, explota igual.

[!NOTE] Aunque Java no optimice tail recursion, escribir funciones con tail recursion es un buen hábito: clarifica tu intención, es más fácil de leer, y si algún día usas un lenguaje que sí lo haga, ya lo dominas.

¿Puedo usar recursividad sin saberlo?

Sí, constantemente. Cada vez que llamas a `Arrays.sort()`, estás usando Quicksort o Mergesort recursivo por debajo. Cuando recorres un sistema de archivos con `File.listFiles()`, la JVM puede usar recursividad internamente. La recursividad no es una característica exótica: es el pan de cada día de la computación.

¿Y si mi recursividad explota el stack? ¿Qué hago?

Tienes varias opciones:

1. **Aumentar el tamaño del stack** de la JVM: `java -Xss2m MiPrograma` (a 2 MB).
2. **Convertir a iterativo** (usa una pila explícita con un `Stack<T>` o `Deque<T>`).
3. **Usar una técnica diferente** que no requiera tanta profundidad.
4. **Aceptar la derrota** y buscar otro trabajo. (No, es broma. Haz la opción 1 o 2).

¿Divide y vencerás solo sirve para ordenar arrays?

Para nada. Es uno de los paradigmas más universales de la computación. Aquí tienes aplicaciones famosas:

- **Búsqueda binaria:** parte un array ordenado por la mitad y busca en el lado correcto. $O(\log n)$.
- **Multiplicación de matrices (Strassen):** divide matrices grandes en submatrices. $O(n^{2.807})$ en lugar de $O(n^3)$.
- **Transformada Rápida de Fourier (FFT):** divide señales digitales en frecuencias. Base del JPEG, MP3, WiFi.
- **Convex Hull:** encuentra el polígono convexo mínimo que contiene un conjunto de puntos.
- **Algoritmo de Karatsuba:** multiplica números grandes más rápido que el método tradicional.

Una vez entiendes Divide y Vencerás, empiezas a ver patrones por todas partes. Es como cuando aprendes una palabra nueva y de repente la ves en todas partes.



EL ACERTIJO

Estás en un laberinto con 3 interruptores. Cada uno controla una bombilla en una habitación cerrada. Solo puedes entrar a la habitación UNA vez. ¿Cómo averiguas qué interruptor enciende qué bombilla?

Pista: las bombillas no solo se encienden, también se calientan.

💡 **Don Tip:** Si una bombilla ha estado encendida mucho tiempo, estará caliente aunque la apagues. Úsalo a tu favor.

Resumen de la unidad

Concepto	Clave
Recursividad	Caso base + caso recursivo. Sin caso base = <code>StackOverflowError</code>
Factorial	$O(n)$. El ejemplo canónico de recursividad lineal
Fibonacci naive	$O(2^n)$. No lo uses para nada serio

Concepto	Clave
Fibonacci optimizado	$O(n)$ con memoización. La diferencia entre lento y rápido
Divide y vencerás	Dividir, conquistar, combinar. Paradigma universal
Quicksort	$O(n \log n)$ promedio, $O(n^2)$ peor, inestable. El más rápido en la práctica
Mergesort	$O(n \log n)$ siempre, estable, $O(n)$ memoria extra. El más fiable
Partición	El corazón de Quicksort. Cómo repartir elementos alrededor del pivote
Fusión	El corazón de Mergesort. Cómo combinar dos arrays ordenados
Estabilidad	Mergesort la mantiene, Quicksort no. Importante para ordenaciones multi-criterio
Tail recursion	Cuando la llamada recursiva es lo último. Java no la optimiza
Stack overflow	El límite del call stack. Cada llamada cuesta memoria

[!NOTE] Esta unidad cubre los RA2 (uso de estructuras de control avanzadas incluyendo recursividad) y RA6 (aplicación de algoritmos de ordenación, análisis de complejidad y elección justificada del algoritmo según el contexto). Efectivamente, la programación no es solo escribir código que funcione: es pensar en **cómo** hacerlo eficiente, elegir la herramienta adecuada, y entender qué pasa bajo el capó.

En la siguiente unidad exploraremos **estructuras de datos avanzadas**: árboles, grafos y tablas hash. Prepárate para dibujar flechas y cuadraditos. Muchos cuadraditos.



Sergi Garcia Barea — CC BY-SA 4.0



Unidad 4: POO – Clases y Objetos

Objetivos de aprendizaje

- Crear clases, objetos y constructores
- Diferenciar entre clase y objeto
- Usar Scanner para entrada por consola
- Entender paquetes e imports
- Sobrescribir toString() y equals()
- Usar this y sobrecarga de constructores

POO: Tus Objetos Cobran Vida

Hasta ahora hemos escrito programas lineales: esto, luego esto, luego esto. Pero el mundo real no funciona así. En el mundo real tienes *cosas*: un perro, un coche, un profesor de programación con gafas de pasta. Cada cosa tiene **atributos** (color, edad, número de ganas de corregir exámenes) y **comportamientos** (ladrar, acelerar, poner faltas de ortografía).

La **Programación Orientada a Objetos (POO)** es simplemente eso: escribir código como funciona el mundo real.

Clases y Objetos: Cortapastas vs Galletas

Una **clase** es un *cortapastas* (un molde). Un **objeto** es la *galleta* que haces con ese molde.

Puedes tener un solo cortapastas con forma de estrella y hacer millones de galletas estrella. Todas tendrán la misma forma, pero cada una puede tener diferente cantidad de pepitas de chocolate.

```
// Esta es la clase (el cortapastas)
public class Galleta {
    String sabor;
    boolean tieneChocolate;

    public void comer() {
        System.out.println("¡Nam, galleta sabor " + sabor);
    }
}
```


Para crear galletas (objetos) a partir del molde:

```
Galleta g1 = new Galleta();    // Primera galleta
g1.sabor = "Chocolate";
g1.tieneChocolate = true;

Galleta g2 = new Galleta();    // Segunda galleta (mismo molde)
g2.sabor = "Vainilla";
g2.tieneChocolate = false;

g1.comer(); // "Ñam, galleta sabor Chocolate"
g2.comer(); // "Ñam, galleta sabor Vainilla"
```

Dos galletas distintas (objetos distintos) con el mismo cortapastas (clase). Cada una con sus propios valores.

Scanner: El Intérprete Amigable

Hasta ahora los datos estaban *escritos en el código*. Pero... ¿y si queremos que el usuario meta datos? Ahí aparece Scanner, nuestro intérprete personal.

```
import java.util.Scanner;

public class CharlaConElUsuario {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in); // Creamos un intérprete

        System.out.print("¿Cómo te llamas? ");
        String nombre = sc.nextLine();        // Lee una línea entera

        System.out.print("¿Cuántos años tienes? ");
        int edad = sc.nextInt();              // Lee un número entero

        System.out.print("¿Cuánto mides (en metros)? ");
        double altura = sc.nextDouble();     // Lee un decimal

        System.out.println("Hola " + nombre + ", tienes " + edad + " años y mides " +
            altura + " m.");

        sc.close(); // Cierra el intérprete (buena educación)
    }
}
```

💡 **Consejo:** Scanner es de java.util. Si no pones import java.util.Scanner; arriba del todo, Java te mirará con cara de “no sé quién es ese tal Scanner”. Los imports son como presentar a tus amigos antes de hablar de ellos.

⚠️ **Advertencia:** Cuando mezclas nextInt() y nextLine() pueden pasar cosas raras. nextInt() lee el número pero *deja el salto de línea sin leer*. Luego nextLine() lee ese salto de línea y parece que se salta la pregunta. Solución: pon un sc.nextLine(); extra después de nextInt() para consumir esa basurilla.

Constructores: La Fiesta de Bienvenida

Cuando creas un objeto, se ejecuta un **constructor**, que es un método especial que prepara el objeto. Como la fiesta de bienvenida a un nuevo empleado.

```

public class Persona {
    String nombre;
    int edad;

    // Constructor sin parámetros ("vale, te pongo valores por defecto")
    public Persona() {
        this.nombre = "Desconocido";
        this.edad = 0;
    }

    // Constructor con parámetros ("te paso los datos, tú inicialízalos")
    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    public void presentarse() {
        System.out.println("Hola, soy " + this.nombre + " y tengo " + this.edad + "
años.");
    }
}

```

```

Persona p1 = new Persona();           // "Desconocido", 0
Persona p2 = new Persona("Ana", 25);  // "Ana", 25
p2.presentarse();                     // "Hola, soy Ana y tengo 25 años."

```

★ BE THE CODE, MY FRIEND: El Creador de Personas

👁 Don Tip: Sigue cada new como si crearas un objeto en la memoria. Después sigue las llamadas a métodos paso a paso.

Vas a ser la JVM. Te dan:

```

public class Coche {
    String marca;
    int velocidad;

    public Coche(String marca) {
        this.marca = marca;
        this.velocidad = 0;
    }

    public void acelerar(int incremento) {
        this.velocidad += incremento;
    }

    public void frenar(int decremento) {
        this.velocidad -= decremento;
        if (this.velocidad < 0) this.velocidad = 0;
    }

    public static void main(String[] args) {
        Coche c = new Coche("Seat");
        c.acelerar(50);
        c.acelerar(30);
        c.frenar(20);
        System.out.println(c.marca + " va a " + c.velocidad + " km/h");
    }
}

```

Traza mental:

1. Se crea un objeto Coche con marca "Seat". Su velocidad empieza en 0.
2. `c.acelerar(50)` → `this.velocidad += 50` → velocidad = 50.
3. `c.acelerar(30)` → `this.velocidad += 30` → velocidad = 80.
4. `c.frenar(20)` → `this.velocidad -= 20` → velocidad = 60.
5. Imprime: **"Seat va a 60 km/h"**.

Variante: ¿Y si frenamos con 100 en lugar de 20? `c.frenar(100)`: velocidad pasaría a -20, pero el `if` la pone a 0. Es como si el coche tuviera frenos de verdad: no puedes ir a velocidad negativa.

Packages: Los Barrios de la Ciudad Java

Java tiene MILES de clases. Para no volverse loco, las organiza en **paquetes** (packages). Piensa en ellos como barrios de una ciudad:

- `java.lang` — El centro histórico. `String`, `Math`, `System`. Se importa solo.
- `java.util` — El barrio de las herramientas. `Scanner`, `ArrayList`, `Date`.
- `java.io` — La zona portuaria. Para leer y escribir archivos.
- `java.net` — El aeropuerto. Para comunicaciones en red.
- `javax.swing` — El barrio de los arquitectos. Interfaces gráficas.

Para usar una clase de otro barrio, tienes que *importarla*:

```
import java.util.Scanner;           // "Quiero usar el Scanner del barrio util"
import java.util.*;                 // "Tráeme TODO lo que haya en java.util"
```

Las clases de `java.lang` no necesitan import. Es como tu casa: no necesitas pedir permiso para entrar en tu propia cocina.

? ¡No Hay Preguntas Tontas!

Q: Vale, `new` crea objetos. Pero, ¿qué pinta `new`? ¿Es un operador?

A: `new` es el *constructor de objetos* de Java. Reserva memoria, llama al constructor y devuelve la dirección del objeto. Sin `new`, no hay objeto. Es como la llave que enciende el coche: sin ella, no arrancas.

Q: ¿Y por qué `String nombre = "Ana";` no lleva `new`?

A: Porque Java es un amor con `String`. Es tan común que te deja crear strings con comillas directamente (literales). Es un *atajo*. Detrás del telón, Java lo trata casi como si tuviera `new`. Pero `Scanner sc = "System.in";` no funciona. `Scanner` no es especial, `String` sí.

Q: ¿Cuál es la diferencia entre `equals()` y `==` para comparar objetos?

A: `==` compara si dos variables apuntan al *mismo objeto* (misma dirección). `equals()` compara si el *contenido* es el mismo. Con `String`:

```
String a = "Hola";
String b = "Hola";
String c = new String("Hola");

System.out.println(a == b);    // true (Java reutiliza literales iguales)
System.out.println(a == c);    // false (c es otro objeto)
System.out.println(a.equals(c)); // true (el contenido es el mismo)
```

Q: ¿Puedo tener un método main en cualquier clase?

A: Sí. Cada clase puede tener su propio main. Es como tener varias puertas de entrada a una casa. Cuando ejecutas `java NombreClase`, Java busca el main de esa clase concreta. Los demás main de otras clases... ahí están, durmiendo.

Q: ¿Qué pasa si no pongo constructor y luego hago `new Persona("Ana")`?

A: Java te dirá: “Oye, no existe un constructor `Persona(String)`. Te voy a dejar tirado con un error de compilación.” Si no pones NINGÚN constructor, Java te da uno vacío sin parámetros. Pero si pones ALGÚN constructor, el vacío NO se crea automáticamente.

Constructores: Las Instrucciones del Horno

Un constructor es un método especial con el MISMO nombre que la clase y que NO devuelve nada (ni void). Es como programar el horno: “temperature: 180, mode: turbo”.

Constructor por Defecto (El Horno por Defecto)

Si no escribes ningún constructor, Java te regala uno gratis. Inicializa todo a null, 0 o false. Como un horno frío.

Constructor Parametrizado (El Horno con Programa)

Tú le dices cómo quieres la galleta.

```

public class Galleta {
    String forma;
    boolean tieneChocolate;
    int temperatura;

    public Galleta(String forma, boolean tieneChocolate, int temperatura) {
        this.forma = forma;
        this.tieneChocolate = tieneChocolate;
        this.temperatura = temperatura;
    }
}

```

Sobrecarga de Constructores: Varios Hornos, Una Cocina

Puedes tener varios constructores con distintos parámetros. Como una lavadora: programa corto, programa largo, programa de “esto es una camisa de seda, que no se me encoja”.

```

public class Galleta {
    String forma;
    boolean tieneChocolate;

    public Galleta() {
        this("redonda", false); // Llama al otro constructor
    }

    public Galleta(String forma, boolean tieneChocolate) {
        this.forma = forma;
        this.tieneChocolate = tieneChocolate;
    }
}

```

La Palabra Clave this: Yo, Mí, Me, Conmigo

this es el objeto diciendo “OYE, HABLO DE MÍ, NO DE OTRO”. Sirve para:

1. Desambiguar: cuando el parámetro se llama igual que el atributo.

```
public class Persona {
    String nombre;

    public Persona(String nombre) {
        this.nombre = nombre; // "this.nombre" es el de arriba, "nombre" es el
        parámetro
    }
}
```

2. Llamar a otro constructor: `this(...)`.

3. Pasarte a ti mismo como argumento.

```
public class Coche {
    String marca;
    int velocidad;

    public Coche(String marca) {
        this.marca = marca;
    }

    public void acelerar(int inc) {
        this.velocidad += inc;
        System.out.println(this.marca + " va a " + this.velocidad + " km/h");
    }
}
```



Nota: Si no pones `this` cuando hay ambigüedad, Java se confunde. Es como si en una conversación dices “nombre” y hay dos personas llamadas “nombre”. Te van a mirar raro.

★ BE THE CODE, MY FRIEND: La Caja Misteriosa



Don Tip: Fíjate en qué objeto se llama cada método. La variable de referencia determina qué métodos puedes invocar.

Tú eres Java. Te dan este código:


```

public class Caja {
    int ancho;
    int alto;
    int profundo;

    public Caja(int ancho, int alto, int profundo) {
        this.ancho = ancho;
        this.alto = alto;
        this.profundo = profundo;
    }

    public int volumen() {
        return ancho * alto * profundo;
    }
}

public class Main {
    public static void main(String[] args) {
        Caja c1 = new Caja(2, 3, 4);
        Caja c2 = new Caja(5, 1, 2);
        System.out.println(c1.volumen());
    }
}

```

- ¿Cuántos objetos hay en memoria al final de main?
- ¿Cuánto imprime el System.out.println?
- ¿Qué pasa si a c1 le cambias ancho = 10? ¿c2 se ve afectado?

Solución: 2 objetos, 24, NO (cada objeto tiene su propia copia). Las variables de instancia son independientes para cada objeto.

★ BE THE CODE, MY FRIEND: ¿Qué Constructor Se Llama?

💡 **Don Tip:** Comprueba el número y tipo de argumentos. Java elige el constructor que mejor coincide.

Sin ejecutar, ¿qué imprime este código?

```

public class Pedido {
    String producto;
    int cantidad;

    public Pedido() {
        this("Sin producto", 0);
        System.out.println("Constructor vacío");
    }

    public Pedido(String producto, int cantidad) {
        this.producto = producto;
        this.cantidad = cantidad;
        System.out.println("Constructor con parámetros");
    }

    public static void main(String[] args) {
        Pedido p = new Pedido();
        System.out.println(p.producto + " x" + p.cantidad);
    }
}

```

Solución: Se llama a Pedido() que hace this("Sin producto", 0), lo que primero ejecuta Pedido(String, int) (imprime "Constructor con parámetros"), luego vuelve y termina Pedido() (imprime "Constructor vacío"). Después imprime "Sin producto x0". El this(...) siempre va primero.

? ¡No Hay Preguntas Tontas!

Q: Vale, ¿y por qué no puedo hacer simplemente `int galleta1 = 42;` en vez de todo este rollo de clases?

A: Porque un `int` solo guarda un número. Una clase guarda NÚMEROS + TEXTO + MÉTODOS + todo lo que quieras. Es como comparar un posavasos con un dron. El dron puede hacer mil cosas, el posavasos... posa vasos.

Q: ¿Cuántos objetos puedo crear? ¿Hay límite?

A: Hasta que te quedes sin memoria (y entonces Java lanza `OutOfMemoryError` y tu programa muere). Pero vamos, con 8 GB de RAM puedes crear millones de objetos pequeños. No te preocupes.

Q: `this` es una palabra reservada, ¿no? ¿Puedo usarla fuera de una clase?

A: No. `this` fuera de una clase es como pedir una pizza en una ferretería. No tiene sentido. Solo existe dentro del contexto de un objeto.

Q: ¿Por qué hace falta escribir `new`? ¿No podría Java crear el objeto solito?

A: No, porque `new` es el “permiso de construcción”. Sin `new`, solo declaras una variable (como `Coche c;`), pero no hay coche en el garaje, solo una plaza de parking vacía. Hasta que no hagas `new`, el objeto no existe.

Métodos `toString()` y `equals()`: La Carta de Presentación

`toString()` es cómo tu objeto se presenta en público. Por defecto Java imprime algo como `Persona@3e3abc` (la clase y una dirección de memoria). No muy útil.

```
public class Persona {
    private String nombre;
    private int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    @Override
    public String toString() {
        return nombre + " (" + edad + " años)";
    }
}
```

`equals()` es para preguntar: “¿este objeto ES IGUAL a este otro?” (por su contenido, no porque sean el mismo sitio en memoria).

```
public class Persona {  
    private String dni;  
  
    @Override  
    public boolean equals(Object o) {  
        if (this == o) return true;  
        if (o == null || getClass() != o.getClass()) return false;  
        Persona otra = (Persona) o;  
        return dni != null && dni.equals(otra.dni);  
    }  
  
    @Override  
    public int hashCode() {  
        return Objects.hash(dni);  
    }  
}
```

💡 **Consejo:** Sobrescribe toString() siempre. Tu yo del futuro (y tu profesor) te lo agradecerá. equals() y hashCode() van juntos como el jamón y el queso. Si sobrescribes uno, sobrescribe el otro.

Ejemplo Completo: Coche (Con Acelerador de Verdad)

```

public class Coche {
    private String marca;
    private String modelo;
    private int velocidad;

    public Coche(String marca, String modelo) {
        this.marca = marca;
        this.modelo = modelo;
        this.velocidad = 0;
    }

    public void acelerar(int kmh) {
        this.velocidad += kmh;
        System.out.println(marca + " " + modelo + ": " + velocidad + " km/h");
    }

    public void frenar(int kmh) {
        this.velocidad = Math.max(0, this.velocidad - kmh);
    }

    @Override
    public String toString() {
        return marca + " " + modelo + " - " + velocidad + " km/h";
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Coche c = (Coche) o;
        return marca.equals(c.marca) && modelo.equals(c.modelo);
    }

    @Override
    public int hashCode() {
        return Objects.hash(marca, modelo);
    }
}

```

Buenas Prácticas (O Cómo No Meter la Pata)

- Atributos **privados siempre** (ya veremos por qué en el siguiente tema. Spoiler: no queréis que nadie toque vuestras partes privadas).
- `this` cuando haya ambigüedad. Si no la hay, puedes omitirlo (pero ponerlo no duele).
- `toString()` siempre. Es tu amigo.
- `equals()` y `hashCode()` si comparas objetos por valor.
- Inicializa todo en el constructor. No dejes atributos bailando.

EL RING: Clase vs Objeto

Dos conceptos discuten acaloradamente. ¿Quién tiene razón?

Clase: «Yo soy el molde, el plano, la idea platónica. Sin mí no existirías. Yo defino qué atributos y métodos tienen los objetos. ¡Soy la creadora!»

Objeto: «Sí, pero yo soy quien realmente hace cosas. Tú eres solo un archivo `.java` en el disco. Yo ocupo memoria, tengo estado, puedo cambiar mis atributos. Sin mí tu código no sirve para nada.»

Clase: «¿Ah sí? ¿Y cuántos de ti existen? Puedes tener miles de objetos creados a partir de mí. Yo soy única, tú eres una copia. ¡Soy original, eres reproducible!»

Objeto: «Exacto. Porque tú eres el plano, pero yo soy el edificio construido. Nadie vive en un plano. Cuando ejecutas el programa, el que trabaja soy yo.»

Clase: «Vale, nos necesitamos. Sin clase no hay objeto. Sin objeto, la clase es solo teoría.»

Objeto: «Trato hecho.»

💡 **Don Tip:** La clase define el QUÉ (atributos) y el CÓMO (métodos). El objeto es el QUIÉN (la instancia concreta que ejecuta y tiene valores propios).

Resumen

- Clase = molde / cortapastas. Objeto = galleta / resultado.
- `new` crea objetos. Llama al constructor.
- Scanner lee del teclado como un intérprete amigable.
- Paquetes = barrios. `import` = pedir permiso para entrar.
- `String` es especial: se puede crear con comillas sin `new`.

- `this` desambigua y permite llamar a otros constructores.
 - Sobrecarga: varios constructores con distintos parámetros.
 - `toString()` y `equals()` se sobrescriben siempre.
-

Ejercicios Propuestos

Introducción a POO

1. **Saludo personalizado** Usa Scanner para preguntar nombre, edad y ciudad. Muestra: “Hola, soy [nombre], tengo [edad] años y vivo en [ciudad].”
2. **La frase mutante** Pide una frase al usuario y muestra: número de caracteres, en mayúsculas, en minúsculas, primera palabra (antes del espacio), y reemplaza todas las vocales por ‘e’.
3. **Suma de strings** Pide dos números como texto (String). Conviértelos a enteros con `Integer.parseInt()` y muestra su suma. Si no son números válidos, ¿qué error sale?
4. **Dados virtuales** Genera 5 números aleatorios entre 1 y 100. Muestra el mayor y el menor (puedes usar `Math.max()` y `Math.min()` o hacerlo a mano).
5. **¿Letra, número o símbolo?** Pide un carácter al usuario. Usa métodos de `Character` (`isDigit()`, `isLetter()`...) para decirle qué es.
6. **Clase Perro** Crea una clase `Perro` con atributos `nombre` (String) y `edad` (int). Un método `ladrar()` que imprima “Guau, soy [nombre]”. Crea dos perros distintos y haz que cada uno ladre.
7. **Calculadora IMC** Pide peso (kg) y altura (m). Calcula el $IMC = peso / (altura * altura)$. Muestra el resultado con 2 decimales. Usa `Math.round()` o imprime con formato.
8. **Contador de objetos** Crea una clase `Contador` con un atributo `static int totalObjetos`. En el constructor, incrementa `totalObjetos`. En el main, crea 5 objetos `Contador` y al final imprime `Contador.totalObjetos`. ¿Sale 5?

Clases y Objetos

1. **Clase Rectángulo:** Crea `Rectangulo` con `ancho` y `alto` (double). Constructor parametrizado, getters/setters, `calcularArea()` y `calcularPerimetro()`. Sobrescribe `toString()`.

2. **Clase CuentaBancaria:** CuentaBancaria con titular, saldo y numeroCuenta. Dos constructores: solo titular (saldo 0) o titular + saldo. Métodos ingresar(double) y retirar(double) (valida saldo suficiente). toString().
3. **Clase Hora:** Hora con hora (0-23), minuto (0-59), segundo (0-59). Constructor con validación. incrementarSegundo() que maneje desbordamientos.
4. **Sobrecarga de constructores:** Clase Email con destinatario, asunto, cuerpo. Tres constructores: todo, solo destinatario+asunto (cuerpo vacío), solo destinatario (asunto y cuerpo por defecto).
5. **equals() en Persona:** Añade fechaNacimiento (LocalDate) a Persona. Sobrescribe equals() usando nombre + fechaNacimiento.
6. **Clase Punto:** Punto con x e y (int). Constructor, getters, distancia(Punto otro) (distancia euclídea), toString(). Calcula perímetro de un triángulo dados 3 puntos.
7. **Clase Fracción:** Fraccion con numerador y denominador (int). Métodos: sumar, restar, multiplicar, dividir, simplificar(). toString() como "numerador/denominador".
8. **Clase Juego:** Juego con nombre, genero, precio, edadMinima. Método esAptoPara(int edad). Crea varios juegos y muestra cuáles son aptos para alguien de 12 años.

RAs trabajados en esta unidad:

- **RA2** - Programas sencillos
- **RA4** - Clases



Sergi Garcia Barea — CC BY-SA 4.0

Unidad 5: Visibilidad, Encapsulación y Static

Objetivos de aprendizaje

- Aplicar los 4 niveles de visibilidad
- Encapsular atributos con getters y setters
- Validar datos en setters
- Entender el patrón JavaBeans
- Usar miembros estáticos (variables, métodos, constantes)
- Crear clases utilitarias con constructor privado

Visibilidad: El Arte de No Enseñar Todo

El Gran Problema (O Cómo la Gente Toca Tus Cosas)

Imagina que vives en una casa de cristal. Cualquiera puede verlo todo: tu ropa interior, tus colección de cromos de Pokémon, esa caja de galletas vacía que guardas por si acaso. Incómodo, ¿verdad?

Pues lo mismo pasa con tus objetos. Si todo es público, cualquiera desde cualquier sitio puede hacer:

```
Persona p = new Persona();  
p.edad = -666; // ¡Edad negativa! Esto no tiene sentido.  
p.saldo = 999999; // Multiplicate por 0 el dinero.
```

Y tu objeto queda hecho unos zorros. Necesitamos **control de acceso**. Necesitamos... **VISIBILIDAD**.

Los 4 Niveles de Visibilidad: De la Valla a la Caja Fuerte

Modificador	Se ve desde		Es como...
public	Todes,	absolutamente	Una valla publicitaria en Times Square
	todes		

Modificador	Se ve desde			Es como...
protected	Mismo paquete	+	subclases (hijos)	Los secretos de familia: lo saben tus primos y tus hijos
<i>package-private</i> (default)	Mismo (vecindario)		paquete	El cotilleo del barrio
private	Solo la clase			Tu diario secreto con candado

“El private es como tu cajón de los calcetines desaparejados: existe, pero no hace falta que nadie más lo vea.”

public: La Valla Publicitaria

Todo el mundo lo ve. Desde cualquier clase, cualquier paquete. Es como poner tu número de teléfono en una pancarta.

```
public class VallaPublicitaria {
    public String mensaje; // "CÓMPRAME, SOY UNA CLASE"

    public void mostrar() {
        System.out.println(mensaje);
    }
}
```

Úsalo para lo que QUIERAS que otros usen. No para tus atributos (a menos que te guste el caos).

private: El Diario con Candado

Solo la clase ve sus propios private. Ni su madre, ni su mejor amigo, ni el perro.

```
public class DiarioSecreto {
    private String contenido; // Nadie fuera de esta clase puede leerlo

    public void escribir(String mensaje) {
        this.contenido = "Querido diario: " + mensaje;
    }

    public String leer() {
        return contenido; // Solo yo puedo decidir qué mostrar
    }
}
```

⚠ **Advertencia:** Nunca, NUNCA, hagas un atributo public. Es como dejar la puerta de tu casa abierta con un cartel: “Pasen y toquen todo”.

protected: Los Secretos de Familia

Es como las historias vergonzosas de la familia. Tus primos (mismo paquete) y tus hijos (subclases) pueden acceder. Pero un desconocido de otro paquete... no.

package-private (default): El Cotilleo del Barrio

Si NO pones ningún modificador, Java asume “package-private”. Lo ven las clases del mismo paquete. Como el grupo de WhatsApp del vecindario.

Tabla Comparativa: ¿Quién Ve Qué?

```
package barrio;

public class Casa {
    public String direccion;    // Lo sabe todo el mundo
    protected String telefono; // Lo sabe la familia
    int numeroHabitaciones;    // Lo saben los vecinos (package-private)
    private String contrasenaWifi; // SOLO YO
}
```

Desde el mismo paquete (barrio):

```

public class Vecino {
    public void espiar() {
        Casa c = new Casa();
        System.out.println(c.direccion);           // OK: public
        System.out.println(c.telefono);           // OK: protected (mismo paquete)
        System.out.println(c.numeroHabitaciones); // OK: package-private
        // System.out.println(c.contrasenaWifi);  // ERROR: private
    }
}

```

Desde otro paquete, siendo subclase:

```

package otraCiudad;
import barrio.Casa;

public class CasaHeredada extends Casa {
    public void espiar() {
        System.out.println(direccion);           // OK: public
        System.out.println(telefono);           // OK: protected (soy subclase)
        // System.out.println(numeroHabitaciones); // ERROR: package-private
        // System.out.println(contrasenaWifi);    // ERROR: private
    }
}

```

★ BE THE CODE, MY FRIEND: El Banco

💡 **Don Tip:** Los getters exponen datos, los setters los validan. Si un setter permite saldos negativos, el banco quiebra.

Eres una clase Banco. Tienes estos miembros:

```

public class Banco {
    public String nombreBanco;
    protected String direccionSucursal;
    String listaClientes;
    private double saldoCaja;

    public void mostrarInfo() {
        System.out.println(nombreBanco);
        System.out.println(direccionSucursal);
        System.out.println(listaClientes);
        System.out.println(saldoCaja);
    }
}

```

Pregunta: ¿Puede una clase Sucursal en otro paquete ver listaClientes? ¿Y direccionSucursal? ¿Puede main() de una clase en el mismo paquete ver saldoCaja?

Solución: no (package-private, otro paquete no lo ve), sí si es subclase (protected), no (private). La propia clase siempre puede ver todo (por eso el método mostrarInfo() funciona).

? ¡No Hay Preguntas Tontas!

Q: ¿Y si no pongo nada? ¿Package-private es lo mismo que “default”?

A: Sí, se llaman “default” o “package-private”. Es el nivel que Java asume cuando no escribes public, private o protected. No es que exista una palabra clave default para visibilidad (esa palabra es para otra cosa).

Q: ¿Por qué debería hacer private un atributo y luego crear getters y setters públicos? ¡Es más trabajo!

A: Porque así CONTROLAS lo que entra y sale. Puedes validar: “Edad no puede ser negativa”. Puedes cambiar la implementación interna sin que nadie se entere. Es como tener un portero en tu discoteca: dejas entrar a quien quieres y echas a los que van borrachos.

Q: Mi profesor dijo que protected es “para que las subclases lo vean”. ¿Qué más da?

A: Mucho. Piensa en una clase Vehiculo con protected int velocidadMaxima. La subclase Coche puede usarlo. Pero una clase Taller del mismo paquete también puede. Si quieres que

SOLO las subclases lo vean y NO los vecinos de paquete... malas noticias: `protected` no discrimina entre “subclase de otro paquete” y “mismo paquete”. A las dos les deja.

Q: ¿Y los métodos? ¿También tienen visibilidad?

A: ¡Claro! Todo tiene visibilidad. Puedes tener un método `private` que solo se usa internamente, como `private void calcularImpuesto()`. Nadie fuera de la clase necesita saber cómo calculas los impuestos (ni tú mismo quieres saberlo).

Q: Si hago todo `public` total es más rápido de escribir, ¿no?

A: Es más rápido de escribir y más lento de depurar. Cuando alguien (o tú) meta un valor imposible en un atributo público, te pasarás horas buscando quién lo cambió. Con encapsulación, el error se detecta al instante en el setter.

Encapsulación: El Pilar que Sostiene la POO

Encapsulación = **privacidad + control**. Es la idea de que:

1. Tus atributos son `private`.
2. Controlas el acceso con getters y setters `public`.
3. Dentro de los setters, VALIDAS.

Ejemplo de vida (o muerte):

```

public class CuentaBancaria {
    private double saldo; // Nadie toca el saldo directamente

    public void ingresar(double cantidad) {
        if (cantidad > 0) {
            this.saldo += cantidad;
        }
    }

    public void retirar(double cantidad) {
        if (cantidad > 0 && cantidad <= saldo) {
            this.saldo -= cantidad;
        } else {
            System.out.println("No tienes tanto dinero, amigo");
        }
    }

    public double getSaldo() {
        return saldo; // Solo lectura
    }
}

```

⚠ **Advertencia:** Si haces los atributos public, estás renunciando a la encapsulación. Es como llevar la cartera abierta en el metro. Tarde o temprano alguien meterá mano.

Ventajas de la Encapsulación (O Por Qué No Dormirás Peor)

- **Control:** Validas y filtras. Nada de edades negativas.
- **Mantenibilidad:** Cambias internamente y el código cliente ni se entera.
- **Seguridad:** Nadie deja tu objeto en un estado inconsistente.
- **Bajo acoplamiento:** Cada clase va a lo suyo. No se meten unas en los asuntos de otras.

La Convención JavaBeans: El Protocolo

JavaBeans es una convención (no obligatoria, pero sí sensata) que dice:

1. Clase pública.
2. Constructor sin argumentos.
3. Atributos privados.

4. Getters y setters públicos.
5. Implementa `Serializable` (opcional).

Convención de nombres:

Tipo	Getter	Setter
String nombre	getNombre()	setNombre(String n)
boolean activo	isActive()	setActivo(boolean a)
int cantidad	getCantidad()	setCantidad(int c)

Buenas Prácticas (El Decálogo del Programador Paranoico)

- `private` para atributos. Siempre. Por defecto. Sin discusión.
- Getter solo si hace falta (atributos inmutables no necesitan setter).
- Valida en los setters. No confíes en nadie.
- `protected` para métodos que las subclasses necesiten. No abuses.
- Empieza con el acceso más restrictivo y ábrelo solo si es necesario.

Static: Lo Que Pertenece a la Clase (No al Objeto)

La Gran Diferencia: El Grupo de WhatsApp vs Los Mensajes Privados

Imagina que eres parte de una clase de 30 alumnos. Tienes:

- **El grupo de WhatsApp de la clase** (static): todos ven el mismo mensaje. Si alguien escribe “mañana hay examen”, los 30 lo ven. Es compartido.
- **Tus mensajes privados** (instancia): solo tú los ves. Cada alumno tiene los suyos. No se mezclan.

Pues en Java es igual:

- **Variables de clase** (static): una sola copia para todos los objetos. Todos comparten el mismo valor.
- **Variables de instancia** (sin static): cada objeto tiene su propia copia. Cada una independiente.

“Static es el grupo de WhatsApp de la clase. Instancia son los DMs que nadie más ve.”

Los métodos **estáticos** (con `static`) pertenecen a la *clase*, no a los objetos:

```
public class UtilidadesMatematicas {  
    public static int sumar(int a, int b) {  
        return a + b;  
    }  
  
    public static double media(double a, double b) {  
        return (a + b) / 2;  
    }  
}  
  
// Llamada: ni necesito crear un objeto, llamo a la clase directamente  
int resultado = UtilidadesMatematicas.sumar(5, 3);  
double med = UtilidadesMatematicas.media(10, 20);
```

La diferencia práctica: los métodos de instancia (sin `static`) necesitan un objeto:

```
String texto = "Hola";  
int longitud = texto.length(); // length() NO es static. Necesito el objeto texto.
```



Nota: `Math.random()`, `Math.sqrt()`, `Integer.parseInt()`... todos son estáticos. No necesitas crear un `Math m = new Math();`. Sería como comprar un nevera para tener un imán. Usa `Math.random()` directamente.

Atributos Estáticos: El Cartel del Colegio

Un atributo `static` es como el cartel de “Quedan 10 minutos para el recreo” en el pasillo. Está ahí, lo ve todo el mundo, y solo hay uno.

```

public class Estudiante {
    private static int totalEstudiantes = 0; // Variable de clase
    private String nombre;                  // Variable de instancia
    private int id;

    public Estudiante(String nombre) {
        this.nombre = nombre;
        this.id = ++totalEstudiantes; // Incrementa el contador global
    }

    public static int getTotalEstudiantes() {
        return totalEstudiantes;
    }

    public int getId() {
        return id;
    }
}

```

En memoria se ve así:

CLASE: Estudiante

totalEstudiantes = 3	← static: UNO para toda la clase
----------------------	----------------------------------

OBJETO e1

nombre
id = 1

OBJETO e2

nombre
id = 2

OBJETO e3

nombre
id = 3

Cada objeto tiene su nombre e id (su DNI), pero todos comparten totalEstudiantes.



Consejo: Usa `NombreClase.miembroEstatico` para acceder. `Estudiante.getTotalEstudiantes()`. No uses `objeto.getTotalEstudiantes()`, aunque funcione, es confuso.

Métodos Estáticos: El Teléfono de la Clase

Los métodos `static` se llaman usando la clase, no un objeto. Es como el número de teléfono de atención al cliente de una empresa: NO necesitas hablar con un empleado concreto, llamas al número general.

Características de los métodos estáticos:

- **NO pueden acceder a atributos de instancia** (no saben de qué objeto hablan).
- **NO tienen `this`** (no hay “yo” porque no hay objeto).
- Solo pueden llamar a otros métodos estáticos (directamente).

```
public class Calculadora {  
    public static int sumar(int a, int b) {  
        return a + b;  
    }  
  
    public static double dividir(int a, int b) {  
        if (b == 0) throw new ArithmeticException("¡No dividirás por cero!");  
        return (double) a / b;  
    }  
}  
  
// Uso: SIN crear ningún objeto  
public class Main {  
    public static void main(String[] args) {  
        int resultado = Calculadora.sumar(10, 5);  
        System.out.println(resultado); // 15  
    }  
}
```

El Ejemplo Definitivo: La Clase Math

`java.lang.Math` es la clase utilitaria por excelencia. TODOS sus métodos son estáticos. No puedes (ni quieres) hacer `new Math()`.

```
double max = Math.max(10, 20);           // 20
double min = Math.min(10, 20);           // 10
double raiz = Math.sqrt(25);              // 5.0
double potencia = Math.pow(2, 10);        // 1024.0
double absoluto = Math.abs(-7);           // 7
double random = Math.random();            // Aleatorio [0.0, 1.0)
double pi = Math.PI;                      // 3.141592653589793
```



Nota: Math tiene el constructor **privado**. Nadie puede instanciarla. Es como una estatua: para admirarla, no para hacerle clones.

Clases Utilitarias: El “No Necesito Pareja” de Java

Una clase utilitaria es una clase que SOLO tiene miembros estáticos. Son como el amigo que está soltero y feliz: no necesita instanciarse para ser útil.

Para evitar que alguien intente crear un objeto, le ponemos el constructor private:

```
public class StringUtils {
    private StringUtils() {} // Nadie puede hacer new StringUtils()

    public static boolean esVacio(String str) {
        return str == null || str.trim().isEmpty();
    }

    public static String invertir(String str) {
        return str == null ? null : new StringBuilder(str).reverse().toString();
    }

    public static String capitalizar(String str) {
        if (esVacio(str)) return str;
        return str.substring(0, 1).toUpperCase() + str.substring(1).toLowerCase();
    }
}
```

Constantes: Lo que Nunca Cambia (Como el Amor de tu Madre)

`static final` es “una constante de clase”. Por convención en MAYÚSCULAS.

```
public class Config {
    public static final String NOMBRE_APP = "Gestión DAM";
    public static final String VERSION = "2.1.0";
    public static final int MAX_USUARIOS = 100;
    public static final double IVA = 0.21;
}
```

Úsalas así: Config.IVA, Config.MAX_USUARIOS. Nunca cambian. Son más firmes que tus propósitos de Año Nuevo.

★ BE THE CODE, MY FRIEND: Los Gatos Estáticos

💡 **Don Tip:** Lo static pertenece a la clase, no al objeto. Todos los objetos comparten el mismo valor.

Mira este código y responde SIN EJECUTAR:

```
public class Gato {
    public static int totalGatos = 0;
    public String nombre;

    public Gato(String nombre) {
        this.nombre = nombre;
        totalGatos++;
    }

    public static void decirTotal() {
        System.out.println("Hay " + totalGatos + " gatos");
        // System.out.println(nombre); // ¿Esto funciona?
    }
}
```

- ¿Funciona System.out.println(nombre); dentro del método decirTotal()?
- Si creas 3 gatos y luego haces Gato.decirTotal(), ¿qué imprime?
- ¿Y si creas 5 gatos más? ¿Qué imprime ahora?

Solución: NO funciona (es una variable de instancia), imprime 3, imprime 8. El método decirTotal() no sabe qué “nombre” pedirle al objeto.

★ BE THE CODE, MY FRIEND: Static vs Instancia

💡 **Don Tip:** Un método static no puede acceder a variables de instancia porque no tiene this. Piensa: ¿pertenece al objeto o a la clase?

¿Qué imprime este código?

```
public class PruebaStatic {
    int x = 1;
    static int y = 1;

    public void incrementarX() { x++; }
    public static void incrementarY() { y++; }

    public static void main(String[] args) {
        PruebaStatic a = new PruebaStatic();
        PruebaStatic b = new PruebaStatic();

        a.incrementarX();
        b.incrementarX();
        a.incrementarX();

        PruebaStatic.incrementarY();
        PruebaStatic.incrementarY();
        b.incrementarY(); // también vale aunque sea confuso

        System.out.println("a.x = " + a.x);
        System.out.println("b.x = " + b.x);
        System.out.println("y = " + PruebaStatic.y);
    }
}
```

Solución: a.x = 3 (se incrementó 2 veces en a y 1 en b, no, a.x se incrementó 2 veces → a.incrementarX(), a.incrementarX()), b.x = 2 (b.incrementarX() una vez), y = 3 (se incrementó 3 veces: dos desde clase, una desde objeto b). Cada objeto tiene su propio x, pero y es compartido.

? ¡No Hay Preguntas Tontas!

Q: ¿Puedo llamar a un método estático desde un objeto? ¿Como miObjeto.metodoEstatico()?

A: Técnicamente Sí. Java te lo deja. Pero es como llamar a tu madre por el apellido. Funciona, pero queda raro. La convención es usar la clase: `Clase.metodoEstatico()`. Algunos IDEs te marcan una warning.

Q: Entonces main es estático porque...

A: Porque cuando empieza el programa, NO hay ningún objeto todavía. Alguien tiene que arrancar la fiesta. main es el primero en llegar. Tiene que ser estático para poder ejecutarse sin que nadie haya creado un objeto antes.

Q: ¿Los métodos estáticos son más rápidos?

A: Ligeramente. No necesitas la referencia al objeto. Pero la diferencia es tan pequeña que en el 99.9% de los casos no lo notarás. No te obsesiones con la velocidad de los estáticos. Preocúpate de que tu código tenga sentido.

Q: ¿Puedo tener un atributo estático que sea un objeto de su propia clase?

A: ¡Sí! Es el patrón **Singleton**. Tienes un `private static MiClase instancia = new MiClase();` y un método `getInstance()`. Es como tener una única piedra filosofal. Pero eso es otro tema...

Q: ¿Puedo ponerle static a todo y ahorrarme crear objetos?

A: Puedes, pero entonces no estás haciendo POO, estás haciendo “Programación Estática a lo bruto”. Java te deja, pero es como usar un destornillador para clavar un clavo: puedes, pero para eso existe el martillo. Usa static para lo que es de la clase, no para todo.

Diferencia Rápida (Para que no te lées)

Variable de instancia	Variable estática
Pertenece al objeto	Pertenece a la clase
Necesitas new	Usas <code>NombreClase.variable</code>
Cada objeto tiene la suya	Una copia para todos
this disponible	No hay this
Lee/escribe en el objeto	Lee/escribe en la clase

Buenas Prácticas

- Usa `static` para métodos que NO dependan del estado del objeto (como `Math.sqrt()`).
- Usa `static` para constantes (`static final`).
- Usa `static` para contadores compartidos.
- NO abuses de `static`. No conviertas todo en estático “porque es más fácil”. Perderás los beneficios de la POO.
- Constructor privado en clases utilitarias (para que nadie las instancie).

EL LÍO

El siguiente código debería encapsular correctamente una clase `CuentaBancaria`, pero tiene varios errores de visibilidad y lógica. Encuéntralos:

```
public class CuentaBancaria {  
    public double saldo;  
    private String titular;  
  
    public CuentaBancaria(String titular, double saldoInicial) {  
        titular = titular;  
        saldo = saldoInicial;  
    }  
  
    public double getSaldo() {  
        return saldo;  
    }  
  
    private void setSaldo(double saldo) {  
        if (saldo >= 0) {  
            this.saldo = saldo;  
        }  
    }  
}
```

Preguntas:

1. ¿El atributo `saldo` está bien encapsulado?
2. ¿Qué error hay en el constructor con `titular = titular`?
3. ¿Por qué `setSaldo` es `private`? ¿Cómo retira dinero el usuario entonces?

💡 **Don Tip:** `this` resuelve ambigüedades. Si parámetro y atributo se llaman igual, sin `this` te asignas el parámetro a sí mismo.

Resumen

- Visibilidad: `public` > `protected` > `package-private` > `private`.
 - Encapsulación: atributos `private`, getters/setters `public` con validación.
 - JavaBeans: clase pública, constructor sin args, atributos privados, getters/setters.
 - `static` = del grupo (clase). Sin `static` = de cada uno (objeto).
 - Métodos estáticos no acceden a atributos de instancia ni tienen `this`.
 - `static final` = constante de clase.
 - Clases utilitarias tienen constructor privado y solo miembros estáticos.
-

Ejercicios Propuestos

Visibilidad

1. **Clase Empleado con validación:** Empleado con nombre, salarioBase (double) y departamento privados. El salario no puede ser negativo. El setter debe lanzar `IllegalArgumentException` si es inválido. Método `calcularSalarioAnual()`.
2. **Clase Círculo encapsulado:** Círculo con radio privado. Setter valida que el radio sea positivo. `getArea()` y `getPerimetro()`. Intenta acceder a radio desde otra clase. ¿Qué pasa?
3. **JavaBean Alumno:** Alumno con nombre, edad, curso, notaMedia (double) y matriculado (boolean). Constructor sin args: nombre vacío, matriculado false. Nota media entre 0 y 10.
4. **Clase Inmutable Hora:** Hora sin setters. Solo constructor con validación. Getters y `toHoraString()`. Nadie puede cambiar la hora después de crearla.
5. **Protected y herencia:** Crea `Animal` en paquete `zoologico` con `protected String nombre`. Crea `Perro` en otro paquete. ¿Quién ve nombre?
6. **Cuenta Bancaria encapsulada:** `CuentaBancaria` con saldo privado. Solo `ingresar(double)` y `retirar(double)` modifican el saldo. Validaciones.

7. **Getter sin Setter:** Clase Configuración con `static final String VERSION = "1.0"` (público). Atributos privados `maxIntentos` y `timeout`. Getter para ambos, setter solo para `maxIntentos`. ¿Por qué?
8. **Refactorización:** Te dan Coche con atributos públicos `marca`, `modelo`, `año`. Refactoriza: hazlos privados, añade getters/setters validando que el año esté entre 1886 y año actual + 1.

Static

1. **Contador de objetos:** Clase Usuario con contador estático de objetos creados, id autoincremental, `static final String DOMINIO_EMAIL = "@dam.com"` y método `generarEmail()` que devuelva nombre + `DOMINIO_EMAIL`.
2. **Clase utilitaria OperacionesArray:** Clase `OperacionesArray` con constructor privado y métodos: `sumar(int[])`, `media(double[])`, `maximo(int[])`, `minimo(int[])`, `estaOrdenado(int[])`, `buscar(int[], int)`. Usa sin instanciar.
3. **Conversor de unidades:** Clase `Conversor` con constantes `KM_A_MILLAS = 0.621371`, `LIBRA_A_KG = 0.453592`, etc. Métodos: `kmAMillas`, `millasAKm`, `celsiusAFahrenheit`, `fahrenheitACelsius`, `librasAKg`, `kgALibras`.
4. **Simulación de aleatorios:** Clase `Aleatorio` con métodos: `entero(int min, int max)`, `decimal(double min, double max)`, `booleano()`, `colorHex()`, `elemento(String[])`.
5. **Clase Config con constantes:** Clase `Config` con `MAX_INTENTOS_LOGIN = 3`, `TIMEOUT_SEGUNDOS = 300`, `RUTA_LOG = "./logs/app.log"`. Atributo privado `contadorAccesos` con `incrementarAcceso()` y `getAccesos()`.
6. **Validador de datos:** Clase utilitaria `Validador` con: `esEmailValido(String)`, `esTelefonoValido(String)` (9 dígitos), `esDNIValido(String)` (8 dígitos + letra), `esFechaValida(int, int, int)`.
7. **Estadísticas de notas:** Programa que dadas notas `double[]` calcule con métodos estáticos: media, máxima, mínima, número de aprobados (≥ 5) y desviación típica. Clase `Estadisticas`.
8. **Juego de dados:** Simula lanzar dos dados 1000 veces con `Math.random()`. Clase `Dado` con método `lanzar()` (1-6). Cuenta cuántas veces sale cada suma (2-12). Clase `Simulacion` con estructura estática.

RAs trabajados en esta unidad:

- **RA4** - Clases (visibilidad, encapsulación)
- **RA7** - Estaticidad



Sergi Garcia Barea — CC BY-SA 4.0



Unidad 8: Herencia, Polimorfismo e Interfaces

Objetivos de aprendizaje

- Heredar de una clase con `extends`
- Usar `super` para acceder a miembros de la superclase
- Sobrescribir métodos con `@Override`
- Diferenciar IS-A vs HAS-A
- Entender el polimorfismo dinámico (dynamic binding)
- Usar `instanceof` y downcasting correctamente
- Aplicar polimorfismo con colecciones y parámetros
- Diseñar clases abstractas con métodos abstractos y concretos
- Implementar interfaces con `implements`
- Diferenciar cuándo usar abstract class vs interface
- Usar métodos default en interfaces
- Conocer las interfaces funcionales (`Consumer`, `Supplier`, `Predicate`, `Function`)
- Sobreescribir `toString()`, `equals()` y `hashCode()` de la clase `Object`
- Construir jerarquías de clases completas

Herencia: Cuando Tus Hijos Siguen Tus Pasos (Pero Mejor)

¿Recuerdas cuando heredaste la nariz de tu abuela o el genio para enfadar a tus profesores? Pues en Java pasa lo mismo, pero con menos drama y más código reutilizable.

La herencia es el mecanismo por el cual una clase *hija* (subclase) obtiene todos los miembros de una clase *padre* (superclase). Y puede añadir los suyos propios o *mejorar* los existentes.

extends: “Soy como tú, pero con mejoras”

Usamos `extends` para decirle a Java que una clase es hija de otra:

```

public class Animal {
    protected String nombre;
    protected int edad;
    public void hacerSonido() {
        System.out.println("Algún sonido genérico...");
    }
}

public class Perro extends Animal {
    public void hacerSonido() {
        System.out.println("¡Guau guau!");
    }
    public void moverCola() {
        System.out.println("*mueve la cola felizmente*");
    }
}

```

Perro ahora tiene nombre, edad, hacerSonido() (mejorado) y moverCola(). Cortesía de la herencia.

💡 **Consejo:** Código reutilizado, neuronas ahorradas. La herencia existe para que NO tengas que copiar y pegar el mismo código en 15 clases.

super: Llamando a mamá/papá para que te ayuden

A veces queremos *extender* el método del padre, no reemplazarlo. Ahí entra super:

```

public class Gato extends Animal {
    @Override
    public void hacerSonido() {
        super.hacerSonido();
        System.out.println("¡MIAU!");
    }
}

```

super es como gritar “¡MAAAAMÁ!” en el supermercado. Le dices a Java: “ejecuta la versión de mi padre, y luego yo hago lo mío”.

⚠️ **Advertencia:** super solo sirve para invocar constructores y métodos de la superclase. No puedes pasarlo como parámetro ni asignarlo a una variable.

@Override: “Papá, yo lo hago mejor”

@Override le dice al compilador: “asegúrate de que realmente estoy sobrescribiendo un método del padre”:

```
public class Pez extends Animal {  
    @Override  
    public void hacerSonido() { } // ✓ existe en Animal  
    @Override  
    public void nadar() { }      // X ERROR: no existe en Animal  
}
```

 **Nota:** Siempre pon @Override. No es obligatorio, pero es como el cinturón de seguridad: no pasa nada si no lo haces... hasta que pasa.

La regla de oro: IS-A vs HAS-A

- **IS-A** (es-un): Relación de herencia. Perro IS-A Animal.
- **HAS-A** (tiene-un): Relación de composición. Coche HAS-A Motor.

```
public class Coche extends Vehiculo { } // IS-A ✓  
public class Coche { private Motor m; } // HAS-A ✓
```

¿Who Wants to Be a Millionaire? — Edición Java: ¿Cuál es la relación correcta? a) Cliente extends Persona b) Cliente has-a Persona c) Coche extends Rueda **Respuesta:** La a. Cliente IS-A Persona. Coche NO es una Rueda, tiene ruedas (HAS-A).

Jerarquía de clases: El árbol genealógico

```
public class Animal { }  
public class Mamifero extends Animal { }  
public class Canino extends Mamifero { }  
public class Perro extends Canino { }
```

Perro hereda de Canino, que hereda de Mamifero, que hereda de Animal.



protected: El miembro que solo la familia ve

```
public class Animal {  
    private String secreto;    // Solo Animal  
    protected String familia; // Animal y subclases  
    public String nombre;     // Todos  
}
```

★ BE THE CODE, MY FRIEND:

👁️ **Don Tip:** super siempre llama al método del padre. Si no lo usas, estás sobrescribiendo completamente.

```
public class Animal {  
    private String idSecreto = "X-123";  
    protected String nombre = "Animal";  
    public int edad = 5;  
}  
  
public class Perro extends Animal {  
    public void mostrar() {  
        System.out.println(idSecreto); // ¿compila?  
        System.out.println(nombre);    // ¿compila?  
        System.out.println(edad);      // ¿compila?  
    }  
}
```

Respuesta: idSecreto NO (private), nombre sí (protected), edad sí (public).

¿Qué imprime?

```

class Abuelo { void decir() { System.out.println("Abuelo"); } }
class Padre extends Abuelo { void decir() { System.out.println("Padre"); } }
class Hijo extends Padre { void decir() { System.out.println("Hijo"); } }
public class Test {
    public static void main(String[] args) { new Hijo().decir(); }
}

```

Respuesta: “Hijo”. Java busca el método desde la clase más específica hacia arriba.

¿Qué imprime con herencia encadenada y super?

```

class Vehiculo {
    void describir() { System.out.println("Soy un vehículo"); }
}
class Coche extends Vehiculo {
    void describir() { System.out.println("Soy un coche"); }
    void describirCompleto() { super.describir(); this.describir(); }
}
class Deportivo extends Coche {
    void describir() { System.out.println("Soy un coche deportivo"); }
}
public class Test {
    public static void main(String[] args) {
        Deportivo d = new Deportivo();
        d.describirCompleto();
        d.describir();
    }
}

```

Respuesta: “Soy un vehículo”, “Soy un coche deportivo”, “Soy un coche deportivo”.
 super.describir() va a Vehiculo, this.describir() se resuelve en runtime como Deportivo.

? ¡No Hay Preguntas Tontas!

Q: ¿Puedo heredar de varias clases a la vez? **A:** No. Java no permite herencia múltiple (el *problema del diamante*). Pero existen las interfaces para eso.

Q: ¿Clase final? ¿Método final? **A:** Clase final no puede tener hijas (String). Método final no puede sobreescribirse.

Q: ¿Atributo con mismo nombre en subclase y superclase? **A:** El de la subclase *oculta* al de la superclase. Pero los atributos NO son polimórficos como los métodos. Con referencia de la superclase, ves el de la superclase.

Q: ¿Los constructores se heredan? **A:** No. Pero la subclase llama al constructor del padre con `super()`. Si no lo pones, Java pone `super()` automáticamente.

El problema de la clase base frágil

Tienes Jarrón con `romper()`. JarrónChino lo sobrescribe. Alguien añade `caerAlSuelo()` a Jarrón que llama a `romper()`. De repente JarrónChino se comporta inesperadamente. Es el *fragile base class problem*.

Q: ¿Entonces la herencia es mala? **A:** ¡No! Es una herramienta. Bien usada es perfecta para relaciones IS-A claras. El problema es usarla cuando una simple composición bastaría.

EL LÍO

El siguiente código de herencia está mal. Encuentra los errores:

```

public class Vehiculo {
    private String marca;

    public Vehiculo(String marca) {
        this.marca = marca;
    }

    public void arrancar() {
        System.out.println("Vehículo arrancado");
    }
}

public class Coche extends Vehiculo {
    private int puertas;

    public Coche(String marca, int puertas) {
        this.puertas = puertas;
    }

    @Override
    public void arrancar() {
        System.out.println("Coche arrancado con " + puertas + " puertas");
    }
}

```

¿Qué error impide que Coche compile? ¿Por qué el constructor de Coche falla?

💡 **Don Tip:** Si el padre tiene un constructor con parámetros, el hijo debe llamarlo con `super()`. Si no, el compilador intenta llamar al constructor vacío del padre... que no existe.

Resumen visual de la herencia

Concepto	Traducción Java
Tu madre te da sus genes	<code>class Hija extends Madre</code>
Le pides ayuda a tu padre	<code>super.metodo()</code>
“Yo lo hago mejor”	<code>@Override</code>
El secreto que solo sabe la familia	<code>protected</code>

Ejercicios Propuestos – Herencia

1. **La familia Simpson:** Clase base `IntegranteFamilia` con nombre y edad. Homero, Marge, Bart, Lisa que hereden. Cada una con un método único (`comerRosquilla()`, `decirAyeCaramba()`).
2. **Vehículos terrestres:** `Vehiculo` → `Coche`, `Moto`, `Bicicleta`. Todos con `mover()`.
3. **El problema del jarrón:** Base con `a()` y `b()` (`b` llama a `a`). Derivada sobrescribe `a()`. Llama a `b()`. ¿Qué ocurre?
4. **Biblioteca de medios:** `Medio` → `Libro`, `Pelicula`, `Disco`. Todos con `mostrarInfo()`.
5. **El juego de los animales:** Jerarquía de animales con al menos 4 niveles.

Polimorfismo: El Camaleón de la POO

Polimorfismo (*poly* = muchas, *morphé* = formas): un mismo método se comporta diferente según el objeto que lo invoque. Como un control remoto universal donde el mismo botón “PLAY” funciona en Netflix, Spotify y tu tostadora.

El mismo método, diferentes comportamientos

```
public class Animal {
    public void hacerSonido() { System.out.println("..."); }
}

public class Perro extends Animal {
    @Override
    public void hacerSonido() { System.out.println("¡Guau!"); }
}

public class Gato extends Animal {
    @Override
    public void hacerSonido() { System.out.println("¡Miau!"); }
}
```

Magia:

```
Animal a;
a = new Perro(); a.hacerSonido(); // ¡Guau!
a = new Gato(); a.hacerSonido();  // ¡Miau!
```

La variable `a` es tipo `Animal`, pero apunta a objetos diferentes. Se ejecuta el método del objeto real.

Dynamic Binding

El compilador verifica que `Animal` tenga `hacerSonido()`. Pero *qué* implementación se ejecuta se decide en runtime. Esto es **dynamic binding** o **late binding**.

Compilador: “Animal tiene `hacerSonido()`? Sí. Adelante.” **Runtime:** “El objeto es un Perro. Ejecuto el de Perro.”

Referencias polimórficas

```
Animal miMascota = new Perro(); // ✓
Animal tuMascota = new Gato();  // ✓
```

Pero la variable solo “ve” los métodos del tipo de la referencia:

```
Animal a = new Perro();
a.hacerSonido(); // ✓
a.moverCola();   // ✗ Animal no tiene moverCola()
```

Polimorfismo con colecciones

```
ArrayList<Animal> animales = new ArrayList<>();
animales.add(new Perro());
animales.add(new Gato());
animales.add(new Vaca());

for (Animal a : animales) {
    a.hacerSonido(); // Cada uno el suyo
}
// ¡Guau! / ¡Miau! / ¡Muuu!
```

Una sola lista, un solo bucle. Sin polimorfismo necesitarías cuatro listas.

Polimorfismo con parámetros

```
public class Veterinario {
    public void vacunar(Animal a) {
        System.out.print("Vacunando a: ");
        a.hacerSonido();
    }
}

Veterinario vet = new Veterinario();
vet.vacunar(new Perro()); // Vacunando a: ¡Guau!
vet.vacunar(new Gato());  // Vacunando a: ¡Miau!
```

Y puedes ir más allá:

```
public class Zoologico {
    private ArrayList<Animal> animales = new ArrayList<>();
    public void agregarAnimal(Animal a) { animales.add(a); }
    public void hacerDesfile() {
        for (Animal a : animales) {
            System.out.print(a.getClass().getSimpleName() + " dice: ");
            a.hacerSonido();
        }
    }
}
```

Añadir un Gato o un Perro no requiere cambiar una línea del Zoológico.

instanceof: “¿Quién eres realmente?”

```
Animal a = new Perro();
if (a instanceof Perro) {
    System.out.println("¡Es un perro!");
}
```

 **Consejo:** Úsalo con moderación. Si usas instanceof constantemente, algo estás haciendo mal. El polimorfismo debería resolverlo sin preguntar.

Downcasting: Peligroso pero a veces necesario

```
Animal a = new Perro();
Perro p = (Perro) a; // Downcasting
p.moverCola();      // ✓

Animal a2 = new Gato();
Perro p2 = (Perro) a2; // ClassCastException en runtime
```

Siempre con instanceof primero:

```
if (a instanceof Perro) {
    Perro p = (Perro) a;
    p.moverCola();
}
```

★ BE THE CODE, MY FRIEND:

💡 **Don Tip:** El compilador mira el tipo de la variable, la JVM mira el tipo real del objeto. Ahí está la magia del polimorfismo.

```
class A { void saluda() { System.out.println("Hola desde A"); } }
class B extends A { void saluda() { System.out.println("Hola desde B"); } }
class C extends B { void saluda() { System.out.println("Hola desde C"); } }
public class Test {
    public static void main(String[] args) {
        A ref1 = new B(); A ref2 = new C(); B ref3 = new C();
        ref1.saluda(); ref2.saluda(); ref3.saluda();
    }
}
```

Solución: Hola desde B, Hola desde C, Hola desde C. Solo importa el tipo real del objeto.

```

class Vehiculo { void mover() { System.out.println("Vehículo se mueve"); } }
class Coche extends Vehiculo {
    void mover() { System.out.println("Coche acelera"); }
    void abrirPuertas() { System.out.println("Puertas abiertas"); }
}
public class Test {
    public static void main(String[] args) {
        Vehiculo v = new Coche();
        v.mover();
        // v.abrirPuertas(); <- ¿compila?
    }
}

```

Solución: “Coche acelera”. La línea comentada NO compila: el compilador solo mira el tipo de la referencia.

```

interface Volable { void volar(); }
class Pajaro implements Volable {
    public void volar() { System.out.println("Vuela"); }
}
class Aguila extends Pajaro {
    public void volar() { System.out.println("Vuela alto"); }
}
public class Test {
    public static void main(String[] args) {
        Volable v = new Aguila();
        Pajaro p = new Aguila();
        Object o = new Aguila();
        v.volar(); p.volar(); // ¿y o?
    }
}

```

Solución: Los tres imprimen “Vuela alto”. El tipo de la referencia no importa.

? ¡No Hay Preguntas Tontas!

Q: ¿Por qué no puedo hacer `Perro p = new Animal()`? **A:** Porque `Animal` podría no tener los métodos de `Perro`. ¿Qué pasa si llamas a `moverCola()` y `Animal` no tiene cola?

Q: ¿Para qué sirve declarar variables del tipo más genérico? **A:** Para ser flexible. `ArrayList<Animal>` acepta cualquier subclase. `ArrayList<Perro>` solo Perros.

Q: ¿El polimorfismo funciona con atributos? **A:** No. Los atributos no son polimórficos. Los métodos sí; los atributos, no.

EL RING: extends vs implements

Dos palabras clave discuten sobre quién es más importante.

extends: «Yo soy la herencia pura. Código reutilizado, una jerarquía clara. Perro extiende Animal. Coche extiende Vehiculo. ¡Soy la base de la POO!»

implements: «Sí, pero conmigo no hay límites. Una clase puede implementar varias interfaces. Con extends solo tienes un padre. Yo te permito ser varias cosas a la vez: Serializable, Comparable, Cloneable...»

extends: «Mis clases pueden tener código ya hecho. Tú solo declaras métodos vacíos. ¡Yo apporto implementación!»

implements: «Desde Java 8 tengo métodos default y static. Mira: default void log() ya funciona. Además, soy más flexible: no impongo una jerarquía rígida.»

extends: «Vale, pero sin mí las interfaces no tendrían sentido. Una interfaz no puede instanciarse sola.»

implements: «Y sin mí tendrías herencia múltiple, que es un caos. Mira el problema del diamante en C++.»

extends: «Nos necesitamos.»

implements: «Sí. extends para la jerarquía, implements para los contratos.»

💡 **Don Tip:** Usa extends para reutilizar código (IS-A). Usa implements para definir capacidades (contratos). Se pueden combinar: `class Perro extends Animal implements Mascota, Jugable.`

Resumen visual del polimorfismo

Concepto	Traducción
Un método, mil formas	Override + dynamic binding
Variable genérica → objeto específico	<code>Animal a = new Perro()</code>

Concepto	Traducción
Decide en ejecución	Dynamic binding
Preguntar quién eres	instanceof
Convertir a la fuerza	Downcasting (Perro) a

Ejercicios Propuestos - Polimorfismo

1. **Orquesta polimórfica:** Instrumento con tocar(). Guitarra, Piano, Bateria, Flauta.
2. **Calculadora de figuras:** Figura con calcularArea(). Circulo, Rectangulo, Triangulo. Usa ArrayList<Figura>.
3. **Downcasting seguro:** Empleado → Programador (escribirCodigo), Diseñador (disenar). Recórrelos con instanceof.
4. **Batalla de personajes:** Personaje → Guerrero, Mago, Arquero. Métodos: atacar(), recibirDano().

La Clase Object: El Tatarabuelo de Todo

Todas las clases heredan de Object. Siempre.

```
public class MiClase { } // = public class MiClase extends Object { }
```

Métodos que toda clase hereda:

Método	¿Qué hace?	¿Sobreescribirlo?
toString()	Representación textual	Casi siempre
equals(Object)	Compara por valor	Cuando tenga sentido
hashCode()	Código hash	Con equals
getClass()	Clase del objeto	No

toString(): La tarjeta de presentación

Por defecto: `MiClase@1a2b3c`. Sobreescríbelo:

```
public class Perro {
    private String nombre;
    private int edad;
    @Override
    public String toString() {
        return "Perro{nombre='" + nombre + "', edad=" + edad + "}";
    }
}

System.out.println(new Perro("Firulais", 3));
// Perro{nombre='Firulais', edad=3}
```

equals(): ¿Mismo objeto o iguales?

Por defecto compara *referencias*. Para comparar por *valor*:

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    Perro perro = (Perro) o;
    return edad == perro.edad && Objects.equals(nombre, perro.nombre);
}
```

hashCode(): El código de barras

Si dos objetos son iguales según `equals()`, deben tener el mismo `hashCode()`:

```
@Override
public int hashCode() { return Objects.hash(nombre, edad); }
```

 **Advertencia:** Si sobrescribes `equals()`, DEBES sobrescribir `hashCode()`. Si no, `HashSet/HashMap` se comportan impredeciblemente.

 **BE THE CODE, MY FRIEND:**

💡 **Don Tip:** Los métodos abstractos son contratos: la subclase TIENE que implementarlos. Si no, no compila.

```
Empleado e1 = new Programador("Ana", "001", 2500, "Java");
Empleado e2 = new Programador("Ana", "001", 2500, "Java");
Empleado e3 = new Gerente("Ana", "002", 3000, 500);
System.out.println(e1);
System.out.println(e1.equals(e2));
System.out.println(e1.equals(e3));
System.out.println(e1.hashCode() == e2.hashCode());
```

Solución: Programador: Ana (ID: 001), true (mismo id), false (distinta clase), true.

```
class Perro extends Animal { Perro(String n) { super(n); } }
class Gato extends Animal { Gato(String n) { super(n); } }
System.out.println(new Perro("Firulais").equals(new Gato("Firulais")));
```

Respuesta: false. getClass() devuelve clases diferentes.

? ¡No Hay Preguntas Tontas!

Q: ¿Sobreescribo equals() siempre? **A:** No. Solo cuando necesites comparar por valor lógico. Dos Persona con mismo DNI sí, dos Scanner no.

Q: ¿Y clone()? **A:** Complicado. Por defecto copia superficial. Mejor un constructor de copia.

Clases Abstractas: El Boceto Que No Puedes Usar Directamente

Imagina “A la venta: Boceto de silla”. No puedes sentarte en un boceto. Pues eso son las clases abstractas: planos incompletos para que *otros* los completen.

¿Qué es una clase abstracta?

Una clase que NO puede instanciarse. Puede tener métodos abstractos (sin implementación) y concretos (con implementación):

```
public abstract class Animal {
    protected String nombre;
    public abstract void hacerSonido();
    public void dormir() {
        System.out.println(nombre + " está durmiendo... Zzz");
    }
}
```

You MUST: Implementar los métodos abstractos

Si una clase concreta extiende una abstracta, está OBLIGADA a implementar todos los métodos abstractos:

```
public class Perro extends Animal {
    @Override
    public void hacerSonido() { System.out.println("¡Guau!"); }
}

public abstract class Pajaro extends Animal { } // No obligada: también abstracta
```

abstract vs concrete

Clase abstracta	Clase concreta
No puedes crear objetos directamente	Puedes crear objetos
Puede tener métodos abstractos	Todos implementados
Concepto general	Algo específico
abstract class	Solo class

Ejemplo: Figuras geométricas

```

public abstract class Figura {
    protected String color;
    public Figura(String color) { this.color = color; }
    public abstract double calcularArea();
    public abstract double calcularPerimetro();
    public void mostrarColor() { System.out.println("Color: " + color); }
}

public class Circulo extends Figura {
    private double radio;
    public Circulo(String color, double radio) { super(color); this.radio = radio; }
    @Override public double calcularArea() { return Math.PI * radio * radio; }
    @Override public double calcularPerimetro() { return 2 * Math.PI * radio; }
}

public class Rectangulo extends Figura {
    private double ancho, alto;
    public Rectangulo(String color, double ancho, double alto) { super(color);
this.ancho = ancho; this.alto = alto; }
    @Override public double calcularArea() { return ancho * alto; }
    @Override public double calcularPerimetro() { return 2 * (ancho + alto); }
}

```

Constructores en clases abstractas

Sí, las abstractas pueden tener constructores. Se llaman con `super()`:

```

public abstract class Animal {
    protected String nombre;
    public Animal(String nombre) {
        this.nombre = nombre;
        System.out.println("Constructor de Animal");
    }
}

public class Perro extends Animal {
    public Perro(String nombre) { super(nombre); }
}

```

Template Method: Las abstractas en acción

Defines el esqueleto de un algoritmo y dejas que las subclases rellenen los detalles:

```

public abstract class Bebida {
    public final void preparar() {
        hervirAgua();
        prepararIngrediente();
        servirEnTaza();
        añadirExtras();
    }
    private void hervirAgua() { System.out.println("Hirviendo agua..."); }
    private void servirEnTaza() { System.out.println("Sirviendo en taza..."); }
    protected abstract void prepararIngrediente();
    protected abstract void añadirExtras();
}

public class Te extends Bebida {
    @Override protected void prepararIngrediente() { System.out.println("Poniendo la bolsita de té..."); }
    @Override protected void añadirExtras() { System.out.println("Añadiendo limón..."); }
}

```

★ BE THE CODE, MY FRIEND:

👁 Don Tip: implements es un contrato. La clase se compromete a tener todos los métodos de la interfaz.

```

public abstract class A { public abstract void metodo(); }
public class B extends A { }

```

Respuesta: NO compila. B debe implementar los métodos abstractos de A.

```

public abstract class A { public abstract void metodo(); }
public class B extends A { public void metodo() { } }

A a = new A(); // ¿Error?
B b = new B(); // ¿Error?

```

Respuesta: new A() ERROR (abstracta), new B() OK.

```
abstract class Animal { abstract void hablar(); }
abstract class Mamifero extends Animal { void hablar() { System.out.println("Mamífero raro"); } }
class Gato extends Mamifero { void hablar() { System.out.println("Miau"); } }
class GatoPersa extends Gato { }
public class Test {
    public static void main(String[] args) { Animal a = new GatoPersa(); a.hablar();
}
}
```

Respuesta: “Miau”. Dynamic binding.

? ¡No Hay Preguntas Tontas!

Q: ¿Por qué existen las clases abstractas? **A:** 1) Definir un contrato obligatorio. 2) Compartir código. 3) Modelar conceptos sin instancia directa (“Figura”).

Q: ¿Puedo tener una clase abstracta sin métodos abstractos? **A:** Sí. Para impedir que se instancie directamente.

Ejercicios Propuestos – Clases Abstractas

1. **Selección abstracta:** Empleado con nombre, salarioBase, abstracto calcularSalario(), concreto mostrarInfo(). Programador y Vendedor.
2. **Vehículos abstractos:** Vehiculo con mover() abstracto, detener() concreto. Coche, Bicicleta, Avion.
3. **Figuras 3D:** Añade calcularVolumen() abstracto a Figura. Implementa Esfera, Cubo.

Interfaces: El Contrato Que Tu Código Firma

¿Has firmado un contrato? “El trabajador se compromete a: programar en Java, no dormirse en las reuniones...” pero no dice CÓMO. Una **interfaz** en Java es eso: un contrato.

```
public interface Programable {
    void programar();
    void tomarCafe();
    void irReunion(String hora);
}
```

Cualquier clase que firme el contrato (con implements) TIENE que implementar todos esos métodos.

Declarando e Implementando

```
public interface Reproducible {
    void reproducir();
    void pausar();
    void detener();
    int obtenerDuracion();
}

public class Cancion implements Reproducible {
    private String titulo;
    public Cancion(String titulo) { this.titulo = titulo; }
    @Override public void reproducir() { System.out.println(" 🎵 Reproduciendo: " + titulo); }
    @Override public void pausar() { System.out.println(" ⏸ Canción pausada"); }
    @Override public void detener() { System.out.println(" 🛑 Canción detenida"); }
    @Override public int obtenerDuracion() { return 240; }
}
```

Polimorfismo con Interfaces

```
public class Reproductor {
    public static void main(String[] args) {
        List<Reproducible> lista = new ArrayList<>();
        lista.add(new Cancion("Bohemian Rhapsody"));
        lista.add(new Pelicula("Inception"));
        for (Reproducible r : lista) {
            r.reproducir(); // No sabe si es canción o película
        }
    }
}
```


Múltiples Interfaces

Una clase solo extiende UNA clase pero puede implementar VARIAS interfaces:

```
public interface Nadador { void nadar(); }
public interface Corredor { void correr(); }
public class Triatleta implements Nadador, Corredor {
    public void nadar() { System.out.println("🏊 Nadando 1.5 km"); }
    public void correr() { System.out.println("🏃 Corriendo 10 km"); }
}
```

default Methods: Parches Sin Romper Nada

Antes de Java 8, añadir un método a una interfaz rompía todas las implementaciones. Llegaron los métodos default:

```
public interface Volar {
    void volar();
    default void despegar() { System.out.println("✈ Despegando..."); }
}
public class Avion implements Volar {
    public void volar() { System.out.println("✈ Volando a 900 km/h"); }
    // despegar() ya viene implementada
}
```

★ BE THE CODE, MY FRIEND:

💡 **Don Tip:** toString(), equals() y hashCode() vienen de Object. Sobrescribirlos bien evita bugs rarísimos.

```
public interface Cantante { void cantar(); }
public interface Bailarin { void bailar(); }
public class Artista implements Cantante, Bailarin {
    public void cantar() { System.out.println("canta"); }
    // ¡FALTA bailar()!
}
```

Respuesta: NO compila. Implementas TODOS los métodos.

```
interface Guerrero { default void atacar() { System.out.println("Ataca con
espada"); } }
interface Mago { default void atacar() { System.out.println("Lanza hechizo"); } }
class Personaje implements Guerrero, Mago {
    public void atacar() {
        Guerrero.super.atacar();
        Mago.super.atacar();
        System.out.println("¡Y usa ambas!");
    }
}
```

Respuesta: “Ataca con espada”, “Lanza hechizo”, “¡Y usa ambas!”. Conflicto resuelto.

? ¡No Hay Preguntas Tontas!

Q: ¿Por qué no usar simplemente una clase abstracta? **A:** Una clase solo hereda de UNA clase, pero implementa VARIAS interfaces. Las interfaces no tienen estado.

Q: ¿Puedo instanciar una interfaz? **A:** No. Como intentar casarte con el concepto de “amor”.

Q: ¿Dos interfaces con método default del mismo nombre? **A:** Conflicto. Obliga a sobrescribir y usar `Interfaz.super.metodo()`.

Q: ¿Atributos en interfaces? **A:** Sí, pero son `public static final`. Constantes, no atributos de instancia.

Resumen: Abstract Class vs Interface

Aspecto	Abstract Class	Interface
Métodos con código	Sí	Sí (default)
Atributos	Cualquiera	<code>public static final</code>
Herencia múltiple	No (extends uno)	Sí (implements varios)
Constructores	Sí	No

Ejercicios Propuestos – Interfaces

1. **Interface Encendible:** Encendible con `encender()`, `apagar()`, default `estaEncendido()`.
Television y Lampara.
2. **Múltiples interfaces:** Cantante y Bailarin. Artista implementa ambas. Demuestra polimorfismo.
3. **Sistema de notificaciones:** Notificable con `enviar(String)` y `getDestinatario()`.
EmailNotificacion y SMSNotificacion.

Interfaces Funcionales: Una Sola Misión

Una interfaz funcional tiene UN SOLO método abstracto. Java las usa con lambdas (->):

Interfaz	Método	Recibe	Devuelve
Consumer<T>	accept(T)	Un valor	void
Supplier<T>	get()	Nada	Un valor
Predicate<T>	test(T)	Un valor	boolean
Function<T,R>	apply(T)	Un valor	Otro valor

```
Consumer<String> imprimir = s -> System.out.println(s);
imprimir.accept("Hola");

Supplier<Double> aleatorio = () -> Math.random();
System.out.println(aleatorio.get());

Predicate<Integer> esPar = n -> n % 2 == 0;
System.out.println(esPar.test(4)); // true

Function<String, Integer> longitud = s -> s.length();
System.out.println(longitud.apply("Java")); // 4
```

Son la base de la programación funcional en Java.

Jerarquía de Clases y Composición

Composición sobre herencia: “Tener vs Ser”

Herencia es “IS-A”. Composición es “HAS-A”. Cada vez más gente prefiere composición.

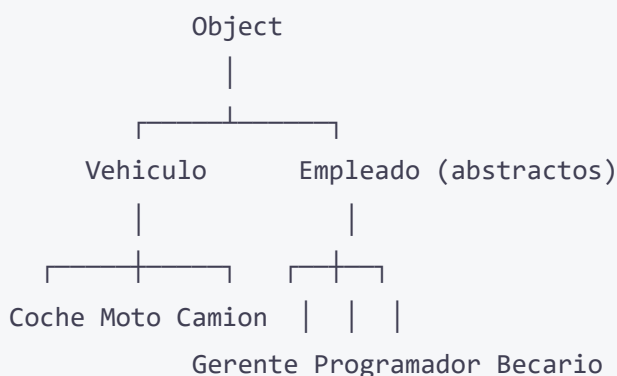
```
// HERENCIA: un coche ES UN vehículo
public class Coche extends Vehiculo { }

// COMPOSICIÓN: un coche TIENE UN motor
public class Coche {
    private Motor motor;
    private Rueda[] ruedas;
}
```

¿Por qué preferir composición?

1. **Menos acoplamiento:** Cambiar la clase padre no rompe las hijas.
2. **Más flexible:** Puedes cambiar las partes en tiempo de ejecución.
3. **Evitas jerarquías profundas:** 5 niveles de herencia = 5 niveles de dolor.

La jerarquía completa



Cada flecha = extends. Object está siempre arriba, aunque no lo escribas.

? ¡No Hay Preguntas Tontas!

Q: ¿Hasta qué profundidad debería tener mi jerarquía? **A:** Como mucho, 2-3 niveles. Jerarquías muy profundas son difíciles de mantener.

Ejercicios Propuestos – Jerarquías

1. **Jerarquía de animales completa:** Al menos 3 niveles, con `toString()`, `equals()` y `hashCode()`. Clase base abstracta.
2. **Sistema de biblioteca:** `ItemBiblioteca` (abstracta) con `prestar()`, `devolver()`, `toString()` y `equals()` por id. Libro y DVD.
3. **La empresa completa:** Departamento con nombre, jefe (Gerente) y lista de empleados. Métodos para agregar, calcular nómina y listar.

🌟 EL ACERTIJO

Tienes tres clases: A, B y C. B `extends` A, C `extends` B. Las tres tienen un método `void hablar()` que imprime su nombre. En el main tienes:

```
A obj = new C();
((B) obj).hablar();
((A) obj).hablar();
obj.hablar();
```

¿Qué imprime cada línea? ¿Por qué?

💡 **Don Tip:** El polimorfismo dinámico significa que el método ejecutado depende del tipo REAL del objeto, no del tipo de la variable. Pase lo que pase con los casts, el objeto sigue siendo un C.

Resumen Global

Los 4 pilares de la POO en Java:

Pilar	Qué es	Cómo se hace en Java
Herencia	Una clase obtiene atributos/métodos de otra	<code>extends</code>
Polimorfismo	Un mismo método se comporta diferente según el objeto	<code>@Override</code> + <code>dynamic binding</code>
Abstracción	Ocultar detalles, mostrar solo lo esencial	Clases abstractas (<code>abstract</code>)
Interfaces	Contrato que las clases deben cumplir	<code>interface</code> + <code>implements</code>

¿Herencia o interfaz?

- Usa **herencia** cuando hay una relación “es-un” clara (Coche es-un Vehículo)
- Usa **interfaces** cuando hay un comportamiento común sin relación jerárquica (Pato implements Volable, Cantable)
- Prefiere **composición** sobre herencia para reutilización flexible

Jerarquía de tipos:

- `class` → `class: extends`
- `class` → `interface: implements`
- `interface` → `interface: extends`
- `abstract class` → `class: extends` (debes implementar los métodos abstractos)

RAAs trabajados en esta unidad:

- **RA4** - Clases (herencia, polimorfismo, interfaces, abstractas, Object)
- **RA6** - Estructuras de datos (interfaces funcionales)
- **RA7** - POO Avanzado (herencia, polimorfismo, dynamic binding, interfaces, clases abstractas, jerarquías)



Sergi Garcia Barea — CC BY-SA 4.0



Unidad 8: Arrays y Colecciones

Objetivos de aprendizaje

- Declarar y recorrer arrays unidimensionales y multidimensionales
- Usar el bucle for-each para leer colecciones
- Elegir la colección adecuada según el problema
- Utilizar iteradores para recorrer y modificar colecciones
- Conocer las diferencias entre ArrayList, LinkedList, HashSet y TreeSet

Arrays: El Aparcamiento de Datos

El Problema: Tienes 100 Gatos y un Solo Nombre

Imagina que tienes 100 gatos y necesitas guardar sus nombres. Podrías hacer esto:

```
String gato1 = "Bigotes";
String gato2 = "Garfield";
String gato3 = "Misifú";
// ... 97 líneas después ...
String gato100 = "Calcetines";
```

Pero entonces llega el gato 101 y tu programa se cae. O peor: quieres saber cuántos gatos empiezan con “M” y tienes que escribir 100 if. Tu espalda ya duele solo de pensarlo.

⚠ Advertencia: Si alguna vez escribes gato1, gato2, gato3, ... gatoN en tu código, en algún lugar un programador senior llora. Los arrays existen exactamente para esto.

El Array: Tu Primer Aparcamiento

Un array es como un parking de varias plantas. Cada plaza tiene un número (índice) y en cada plaza solo caben coches del mismo tipo (bueno, y sus subclases).

```
String[] gatos = new String[100];
// Has creado un parking con 100 plazas para Strings
```

La primera plaza es la 0, no la 1. Esto confunde a todo el mundo al principio. Acéptalo.



Consejo: Piensa en los índices como distancias desde la primera posición. La primera casa está a 0 pasos de ti, no a 1.

Cómo Meter Cosas en el Parking

```
String[] gatos = new String[3];
gatos[0] = "Bigotes";
gatos[1] = "Garfield";
gatos[2] = "Misifú";
gatos[3] = "Calcetines"; // ¡BOOM!
```

¿Qué pasa en la última línea? Te vas a estrellar.

¡BOOM! La ArrayIndexOutOfBoundsException

```
int[] numeros = new int[5];
numeros[0] = 10;
numeros[1] = 20;
numeros[2] = 30;
numeros[3] = 40;
numeros[4] = 50;
numeros[5] = 60; // Index 5 out of bounds for length 5
```



Advertencia: El array es un objeto (está en el heap), pero la referencia está en la pila (stack). Cuando pasas un array a un método, pasas la referencia, no los datos. ¡Ojo! Si modificas el array dentro del método, los cambios afectan al original.

El Dúo Inseparable: for + Array

Los arrays y los bucles for son como el pan y la mantequilla. Nunca verás uno sin el otro.

```
String[] gatos = {"Bigotes", "Garfield", "Misifú", "Calcetines"};

for (int i = 0; i < gatos.length; i++) {
    System.out.println("Gato " + i + ": " + gatos[i]);
}
```

Fíjate: `gatos.length` NO lleva paréntesis. No es un método, es un atributo. Los Strings usan `length()`. Los arrays usan `length`. Es una trampa mortal en los exámenes.

★ BE THE CODE, MY FRIEND:

💡 **Don Tip:** Los arrays tienen tamaño fijo. Una vez creados, no puedes añadir ni quitar elementos.

```
public class BeTheArray {  
    public static void main(String[] args) {  
        int[] arr = new int[4];  
        arr[0] = 2;  
        arr[1] = 4;  
        arr[2] = 6;  
        arr[3] = 8;  
  
        for (int i = 0; i < arr.length; i++) {  
            arr[i] = arr[i] * 2;  
        }  
  
        System.out.println(arr[2]);  
    }  
}
```

★ BE THE CODE, MY FRIEND

💡 **Don Tip:** Las variables de tipo array guardan una referencia, no los datos. Asignar un array a otro no copia los datos.

¿Qué imprime?

- (A) 6
- (B) 8
- (C) 12
- (D) 16

Respuesta: (C) 12. El array original es {2,4,6,8}. Después del bucle, cada elemento se multiplica por 2: {4,8,12,16}. arr[2] = 12.

★ BE THE CODE, MY FRIEND: El Array Revelde

💡 **Don Tip:** Cuando pasas un array a un método, pasas la referencia. Si lo modificas dentro, cambia fuera.

```
public class BeTheArrayRevelde {  
    public static void main(String[] args) {  
        int[] nums = {1, 2, 3, 4, 5};  
        for (int i = 0; i < nums.length; i++) {  
            nums[i] = nums[i] * nums[i];  
        }  
        System.out.println(nums[2]);  
    }  
}
```

★ BE THE CODE, MY FRIEND

💡 **Don Tip:** Los arrays se pasan por referencia. Las variables primitivas se pasan por valor.

¿Qué imprime?

- (A) 3
- (B) 6
- (C) 9
- (D) 25

Respuesta: (C) 9. Se eleva cada número al cuadrado: {1,4,9,16,25}. `nums[2] = 9`.

for-each: La Variante Perezosa

Si no necesitas el índice (solo quieres leer los valores), existe una sintaxis más corta:

```
String[] gatos = {"Bigotes", "Garfield", "Misifú"};  
  
for (String gato : gatos) {  
    System.out.println("Miau: " + gato);  
}
```

Se lee: “para cada String gato en gatos, haz esto”.



Nota: El for-each es de solo lectura. No puedes modificar el array original dentro del bucle. Bueno, puedes intentarlo, pero el cambio se pierde en el éter.

Arrays Multidimensionales: El Parking de Varias Plantas

¿Necesitas una tabla con filas y columnas? Usa un array bidimensional.

```
int[][] tabla = new int[3][4]; // 3 filas, 4 columnas
tabla[0][0] = 1; // fila 0, columna 0
tabla[1][2] = 5; // fila 1, columna 2
// ...
```

Java también permite “arrays de arrays” irregulares (cada fila con distinto número de columnas):

```
int[][] irregular = new int[3][];
irregular[0] = new int[2];
irregular[1] = new int[5];
irregular[2] = new int[3];
```

¿Para qué sirve? Triángulos, pirámides, o simplemente datos que no forman un rectángulo perfecto.

Recorrer un Array 2D

```
int[][] matriz = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

for (int i = 0; i < matriz.length; i++) {
    for (int j = 0; j < matriz[i].length; j++) {
        System.out.print(matriz[i][j] + " ");
    }
    System.out.println();
}
```

💡 **Consejo:** Nombra los índices de arrays multidimensionales como fila y col o i y j. NO uses x e y a menos que realmente trabajes con coordenadas. Tu yo del futuro te lo agradecerá.

La Clase Arrays: Tu Navaja Suiza

Java proporciona la clase `java.util.Arrays` con métodos utilities:

```
import java.util.Arrays;

int[] numeros = {5, 2, 8, 1, 9};

Arrays.sort(numeros);           // {1, 2, 5, 8, 9}
int pos = Arrays.binarySearch(numeros, 5); // 2
int[] copia = Arrays.copyOf(numeros, 3);   // {1, 2, 5}
String texto = Arrays.toString(numeros);   // "[1, 2, 5, 8, 9]"
boolean igual = Arrays.equals(a, b);       // compara dos arrays
Arrays.fill(numeros, 0);                  // todo a ceros
```

⚠ **Advertencia:** `array1.equals(array2)` NO compara los elementos. Compara si son el MISMO objeto en memoria. Usa SIEMPRE `Arrays.equals()` para comparar contenido. Tu jefe te lo agradecerá.

★ BE THE CODE, MY FRIEND: La Búsqueda Binaria

👁 **Don Tip:** `Arrays.binarySearch()` requiere el array ORDENADO. Si no, el resultado es impredecible.

```
import java.util.Arrays;

public class BeTheSort {
    public static void main(String[] args) {
        int[] datos = {42, 17, 8, 99, 3};
        Arrays.sort(datos);

        int indice = Arrays.binarySearch(datos, 42);
        System.out.println(indice);
    }
}
```

★ BE THE CODE, MY FRIEND

👁 **Don Tip:** La búsqueda binaria solo funciona en arrays ordenados. No lo olvides.

¿Qué imprime?

- (A) 0
- (B) 3
- (C) 4

- (D) 99

Respuesta: (B) 3. Después de ordenar: {3, 8, 17, 42, 99}. 42 está en el índice 3.

Arrays y Métodos: Pasando el Testigo

```
public static void main(String[] args) {
    int[] edades = {10, 20, 30};
    modificar(edades);
    System.out.println(edades[0]); // 99
}

public static void modificar(int[] arr) {
    arr[0] = 99; // Modifica el original porque es la misma referencia
}
```

Esto funciona porque Java pasa la referencia del objeto array por valor. El array no se copia, solo se copia la dirección de memoria donde está.

? ¡No Hay Preguntas Tontas!

? ¡No Hay Preguntas Tontas! Q: ¿Por qué el primer índice es 0 y no 1? ¡Eso no es natural! **A:** Porque en la memoria, el índice es el desplazamiento desde la dirección base del array. El primer elemento está en la dirección base + 0. Java no inventó esto para fastidiarte; viene de las cavernas del lenguaje C.

Q: ¿Puedo tener un array de tamaño 0? **A:** Sí. `int[] vacio = new int[0];`. No puedes meter nada, pero no da error. Útil cuando no sabes si tendrás datos.

Q: ¿Pasa algo si creo un array de objetos pero no los inicializo? **A:** Los arrays de objetos se inicializan con `null`. Si intentas llamar a un método en un elemento `null`, obtienes `NullPointerException`. No es divertido.

Q: ¿Cuál es la diferencia entre `null` y “vacío”? **A:** Un array vacío (`new int[0]`) existe pero no tiene elementos. Una referencia `null` significa que no hay ningún array. Es como la diferencia entre un parking vacío y un parking que no se ha construido.

Q: ¿Los arrays pueden cambiar de tamaño? **A:** No. Son inmutables en tamaño. Si necesitas que crezca, crea uno nuevo y copia los datos con `System.arraycopy()` o `Arrays.copyOf()`.

? ¡No Hay Preguntas Tontas! (Arrays y Memoria)

? **¡No Hay Preguntas Tontas! Q:** ¿Los arrays de primitivos guardan los valores en el heap o en el stack? **A:** El objeto array (con todos sus elementos) está en el heap. La variable que lo referencia está en el stack. El array es un objeto, aunque sea de `int`.

Q: ¿Se puede cambiar el tamaño de un array después de crearlo? **A:** No, los arrays tienen tamaño fijo. Piensa en ellos como plazas de parking ya construidas. Si necesitas más, toca construir (crear) un nuevo parking.

Q: ¿Qué pasa si uso `Arrays.sort()` en un array de Strings? **A:** Los ordena alfabéticamente (según el orden natural de String, que es lexicográfico). Ojo con mayúsculas: "Zebra" va antes que "abc" porque las mayúsculas tienen menor valor Unicode.

🔴 EL RING: Array vs ArrayList

Dos formas de almacenar datos se enfrentan.

Array: «Yo soy el original. Rápido, eficiente, directo. Acceso $O(1)$ a cualquier posición. ¡Soy la base de todo!»

ArrayList: «Sí, pero tienes tamaño fijo. Una vez que te creas con 10 posiciones, no puedes tener 11. Yo crezco y encogro bajo demanda. Soy flexible.»

Array: «Pero yo soy más rápido en acceso y más ligero en memoria. ArrayList usa un array por dentro y añade overhead.»

ArrayList: «Cierto, pero mis métodos `add()`, `remove()`, `contains()` me hacen mucho más cómodo. ¿Cuántas líneas de código necesitas para añadir un elemento a un array? Yo una: `lista.add(42)`.»

Array: «Para datos primitivos soy más eficiente. `int[]` ocupa menos que `ArrayList<Integer>` por el autoboxing.»

ArrayList: «Vale, para tipos primitivos y rendimiento extremo, usa arrays. Para todo lo demás, úsame a mí. ¿Tregua?»

Array: «Tregua.»

💡 **Don Tip:** Regla práctica: ¿sabes cuántos elementos necesitas y no va a cambiar? Usa array. ¿No lo sabes o va a cambiar? Usa ArrayList.

Resumen Rápido

```
int[] arr = new int[10];           // Parking para 10 enteros
int[] arr2 = {1, 2, 3};           // Inicialización directa
arr.length;                       // Tamaño (sin paréntesis)
arr[0];                           // Primer elemento
arr[arr.length - 1];             // Último elemento
int[][] mat = new int[3][4];      // Parking de 3 plantas, 4 plazas
Arrays.sort(arr);                 // Ordenar
Arrays.toString(arr);             // Imprimir bonito
Arrays.equals(a, b);              // Comparar contenido
Arrays.copyOf(arr, n);            // Copiar los primeros n
Arrays.binarySearch(arr, val);    // Buscar (requiere ordenación)
```

Colecciones: Cuando un Array Se Queda Pequeño

El Problema: Tu Array se Ha Quedado Sin Plazas

Has creado un array de 10 plazas. Han llegado 11 gatos. ¿Qué haces?

```
String[] gatos = new String[10]; // 10 plazas, 11 gatos... mal asunto
```

Con un array clásico tendrías que crear uno nuevo, copiar todo, y luego añadir el que falta. Es como aparcar en la calle porque el parking está lleno.

```
String[] gatosMasGrande = new String[gatos.length + 1];
System.arraycopy(gatos, 0, gatosMasGrande, 0, gatos.length);
gatosMasGrande[gatosMasGrande.length - 1] = "Bigotes Jr.";
gatos = gatosMasGrande; // Ahora apunta al nuevo array
```

Funciona, pero es tedioso. Y si tienes que borrar un elemento en medio, es peor. Necesitas algo que crezca y se encoja solo.

El Java Collections Framework (JCF)

Para eso está el JCF: una familia de clases e interfaces en `java.util` que manejan grupos de objetos como si fueran de goma.

? ¡No Hay Preguntas Tontas!

? ¡No Hay Preguntas Tontas! Q: ¿Por qué no usar siempre ArrayList y olvidarme de los arrays? **A:** Los arrays son más rápidos y consumen menos memoria. Para un millón de elementos, la diferencia se nota. Úsalos cuando sepas el tamaño de antemano.

Q: ¿Qué significa <String>? **A:** Es un *generic*. Le dice a la colección: “Solo acepto Strings”. Si intentas meter un int, te casca en compilación, no en ejecución. Es tu red de seguridad.

ArrayList: El Parking que Crece Solito

ArrayList es como un array, pero con superpoderes: se redimensiona automáticamente cuando te pasas de capacidad.

```
import java.util.ArrayList;

ArrayList<String> gatos = new ArrayList<>();
gatos.add("Bigotes");      // [Bigotes]
gatos.add("Garfield");     // [Bigotes, Garfield]
gatos.add("Misifú");       // [Bigotes, Garfield, Misifú]
gatos.remove(1);           // [Bigotes, Misifú] - adiós, Garfield
gatos.get(0);              // "Bigotes"
gatos.size();              // 2 (NO length, es size())
gatos.contains("Misifú");  // true
gatos.indexOf("Bigotes");  // 0
```

⚠ Advertencia: ArrayList usa size(), no length. Array usa length, no size(). String usa length(), no length ni size(). Cada uno tiene su propia forma de preguntar cuánto mide. Es una trampa en el 90% de los exámenes.

ArrayList NO guarda primitivos

No puedes hacer ArrayList<int>. Los genéricos solo funcionan con objetos. Usa las clases wrapper:

```
ArrayList<Integer> numeros = new ArrayList<>();
numeros.add(42);           // autoboxing: int → Integer
int n = numeros.get(0);    // unboxing: Integer → int
```


Desde Java 5, el autoboxing/unboxing es automático, pero por dentro sigue habiendo objetos Integer.

★ BE THE CODE, MY FRIEND: El ArrayList Misterioso

💡 **Don Tip:** ArrayList crece solo. No necesitas definir tamaño, pero cada operación tiene su coste.

```
import java.util.ArrayList;

public class BeTheList {
    public static void main(String[] args) {
        ArrayList<Integer> lista = new ArrayList<>();
        lista.add(10);
        lista.add(20);
        lista.add(30);
        lista.add(1, 15);
        lista.remove(Integer.valueOf(20));

        for (Integer n : lista) {
            System.out.print(n + " ");
        }
    }
}
```

★ BE THE CODE, MY FRIEND

💡 **Don Tip:** ArrayList usa arrays por dentro. Cuando se llena, crea uno nuevo y copia.

¿Qué imprime?

- (A) 10 20 30
- (B) 10 15 30
- (C) 10 15 20 30
- (D) 10 15

Respuesta: (B) 10 15 30. Se añade 15 en índice 1 → {10,15,20,30}. Luego se borra el objeto Integer(20) → {10,15,30}.

★ BE THE CODE, MY FRIEND: Suma de ArrayList

💡 **Don Tip:** Recorrer un ArrayList con for-each es más legible que con índices.

```
import java.util.ArrayList;

public class BeTheSum {
    public static void main(String[] args) {
        ArrayList<Integer> nums = new ArrayList<>();
        for (int i = 1; i <= 5; i++) {
            nums.add(i * 10);
        }
        int total = 0;
        for (int n : nums) {
            if (n % 20 == 0) {
                total += n;
            }
        }
        System.out.println(total);
    }
}
```

★ BE THE CODE, MY FRIEND

💡 **Don Tip:** El for-each es de solo lectura. No puedes modificar la colección mientras la recorres.

¿Qué imprime?

- (A) 30
- (B) 60
- (C) 90
- (D) 150

Respuesta: (B) 60. La lista es {10, 20, 30, 40, 50}. Los múltiplos de 20 son 20 y 40. $20 + 40 = 60$.

LinkedList: La Conga Line

LinkedList es una lista enlazada. Cada elemento sabe quién está delante y detrás, como en una conga. Es lenta si buscas por índice, pero rapidísima para añadir/borrar al principio o al final.

```
import java.util.LinkedList;

LinkedList<String> cola = new LinkedList<>();
cola.addLast("Persona 1"); // al final
cola.addLast("Persona 2");
cola.addFirst("Colado"); // se cuela al principio
String primero = cola.removeFirst(); // "Colado" - se va
```

💡 **Consejo:** Usa LinkedList cuando necesites una cola (FIFO) o una pila (LIFO). Para acceso aleatorio frecuente, usa ArrayList. LinkedList busca elemento por elemento. ArrayList va directo al índice.

HashSet: El Portero que No Deja Duplicados

HashSet es como una discoteca: no deja entrar a nadie que ya esté dentro. No importa el orden, solo la exclusividad.

```
import java.util.HashSet;

HashSet<String> invitados = new HashSet<>();
invitados.add("Ana");
invitados.add("Bob");
invitados.add("Ana"); // No pasa nada, Ana ya está
System.out.println(invitados.size()); // 2, no 3
```

¿Cómo sabe si un elemento ya está? Usa hashCode() y equals(). Si tu objeto no los sobrescribe bien, HashSet hará cosas raras.

⚠️ **Advertencia:** Si sobrescribes equals() en una clase, SOBRESCRIBE hashCode(). Siempre. Si dos objetos son iguales según equals(), deben tener el mismo hashCode(). Si no, HashSet se volverá loco. Repito: **siempre**.

Operaciones Típicas con HashSet

```
HashSet<String> set = new HashSet<>();
set.add("rojo");
set.add("verde");
set.add("azul");
set.remove("rojo");
set.contains("verde"); // true
set.isEmpty();         // false
set.clear();            // lo vacía todo
```

TreeSet: El Organizado

TreeSet es como un HashSet que se ordena solo. Internamente usa un árbol rojo-negro. Todo lo que metas se ordena automáticamente.

```
import java.util.TreeSet;

TreeSet<String> ordenado = new TreeSet<>();
ordenado.add("Zara");
ordenado.add("Ana");
ordenado.add("Bob");
System.out.println(ordenado); // [Ana, Bob, Zara] - orden alfabético

// Métodos extra útiles
ordenado.first(); // "Ana"
ordenado.last();  // "Zara"
ordenado.headSet("Bob"); // [Ana] - elementos antes de Bob
```

💡 **Consejo:** Necesitas elementos ordenados automáticamente? Usa TreeSet. Solo necesitas eliminar duplicados? Usa HashSet (es más rápido: $O(1)$ vs $O(\log n)$).

Iterator: El Camarero que Toma Nota Uno a Uno

Iterator recorre una colección sin que te importe cómo está implementada por dentro. Es como un camarero: “¿Qué quiere? ¿Y usted? ¿Y usted?”

```
import java.util.Iterator;

ArrayList<String> platos = new ArrayList<>();
platos.add("Tortilla");
platos.add("Paella");
platos.add("Croquetas");

Iterator<String> it = platos.iterator();
while (it.hasNext()) {
    String plato = it.next();
    if (plato.equals("Paella")) {
        it.remove(); // BORRA de la lista ORIGINAL
    }
}
// Ahora platos = [Tortilla, Croquetas]
```

⚠ **Advertencia:** Nunca hagas `lista.remove(elemento)` mientras usas un `for-each`. Lanzarás `ConcurrentModificationException`. Usa SIEMPRE `iterator.remove()` si necesitas borrar durante el recorrido.

Collections: El Amigo Utilitario

Igual que Arrays para arrays, Collections es el amigo de las colecciones:

```
import java.util.Collections;

ArrayList<String> lista = new ArrayList<>();
lista.add("Zara");
lista.add("Ana");
lista.add("Bob");

Collections.sort(lista);           // [Ana, Bob, Zara]
Collections.reverse(lista);        // [Zara, Bob, Ana]
Collections.shuffle(lista);        // orden aleatorio
Collections.max(lista);            // "Zara" (orden alfabético)
Collections.min(lista);            // "Ana"
Collections.frequency(lista, "Ana"); // 1
Collections.replaceAll(lista, "Ana", "Ana María");
Collections.rotate(lista, 2);      // rota 2 posiciones
```

★ BE THE CODE, MY FRIEND: Collections en Acción

💡 **Don Tip:** Collections.sort() ordena la lista original. Si no quieres modificarla, cópiala antes.

```
import java.util.*;

public class BeTheCollections {
    public static void main(String[] args) {
        ArrayList<Integer> nums = new ArrayList<>();
        nums.add(5);
        nums.add(1);
        nums.add(8);
        nums.add(3);

        Collections.sort(nums);
        Collections.reverse(nums);

        System.out.println(nums.get(1));
    }
}
```

★ BE THE CODE, MY FRIEND

💡 **Don Tip:** Los métodos estáticos de Collections son muy potentes. sort, shuffle, reverse, binarySearch...

¿Qué imprime?

- (A) 1
- (B) 3
- (C) 5
- (D) 8

Respuesta: (C) 5. Sort → {1,3,5,8}. Reverse → {8,5,3,1}. get(1) = 5.

? ¡No Hay Preguntas Tontas! (Colecciones)

? **¡No Hay Preguntas Tontas! Q:** ¿Cuándo uso ArrayList y cuándo LinkedList? **A:** Usa ArrayList para el 95% de los casos. LinkedList solo cuando necesites añadir/borrar mucho al principio o estés implementando una cola/pila. ArrayList es más simple y más rápido en acceso aleatorio.

Q: HashSet vs TreeSet? **A:** HashSet es más rápido ($O(1)$ vs $O(\log n)$), pero no ordena. TreeSet ordena automáticamente pero es más lento. Si no necesitas orden, usa HashSet. Punto.

Q: ¿Se pueden mezclar tipos en una colección sin genéricos? **A:** Sí, pero es como lanzar un dado. Si pones `ArrayList lista = new ArrayList();` (sin `<>`) puedes meter cualquier cosa, pero al sacarlo tienes que hacer casting y cruzar los dedos. Con genéricos, el compilador te protege.

Q: ¿Qué pasa si añado un null a un HashSet? **A:** HashSet admite un único null. TreeSet no admite null porque necesita comparar elementos para ordenar y... ¿cómo comparas null con algo?

Q: ¿Qué es más rápido, for-each o iterator? **A:** Internamente son casi lo mismo. El for-each usa iterator por debajo. Usa el que te resulte más legible.

Resumen Rápido: Colecciones

```
ArrayList<String> a = new ArrayList<>();           // Lista dinámica
LinkedList<String> l = new LinkedList<>();         // Lista doblemente enlazada
HashSet<String> s = new HashSet<>();               // Sin duplicados, sin orden
TreeSet<String> t = new TreeSet<>();               // Sin duplicados, ordenado

a.add(e);           // añadir
a.get(i);           // obtener por índice
a.remove(i);        // borrar por índice
a.remove(e);        // borrar objeto
a.size();           // tamaño
a.contains(e);       // contiene?
a.isEmpty();        // está vacío?

Collections.sort(lista);
Collections.reverse(lista);
Collections.shuffle(lista);
Collections.min(lista);
Collections.max(lista);
```

Ejercicios Propuestos

Ejercicio 1: El Inverso

Escribe un programa que cree un array de 10 enteros, los rellene con números del 1 al 10, y luego los imprima en orden inverso.

Ejercicio 2: Buscaminas Simplificado

Crea un array bidimensional de 5x5 que represente un campo de minas. Rellénalo con 5 minas (representadas como `true`) en posiciones aleatorias. El usuario introduce coordenadas (fila, columna) y el programa dice si hay mina o no. Si acierta una mina, el juego termina.

Ejercicio 3: Estadísticas de Clase

Pide al usuario las notas de 20 alumnos, guárdalas en un array, y calcula:

- La nota media
- La nota más alta
- La nota más baja
- Cuántos alumnos aprobaron (nota ≥ 5)

Ejercicio 4: Eliminar Duplicados

Escribe un programa que lea una lista de palabras desde teclado (termina con “FIN”) y las muestre sin duplicados, en el mismo orden en que aparecieron la primera vez. Pista: usa un `LinkedHashSet` para mantener el orden de inserción.

Ejercicio 5: Lista de la Compra

Crea un programa que gestione una lista de la compra usando `ArrayList<String>`. Debe permitir: añadir, eliminar, marcar como comprado y mostrar la lista.

RAs trabajados en esta unidad:

- **RA6** - Tipos avanzados: Arrays y colecciones



Sergi Garcia Barea — CC BY-SA 4.0



Unidad 9: Genéricos y Mapas

Objetivos de aprendizaje

- Comprender el concepto de genéricos y type erasure
- Crear clases y métodos genéricos propios
- Usar wildcards (? extends, ? super) para flexibilidad
- Manejar mapas (HashMap, TreeMap) y sus operaciones principales
- Elegir entre Map, List y Set según el problema

Genéricos: El <T> Que Lo Cambió Todo

Antes de los genéricos, programar Java era como hacer malabares con cuchillos... vendados. Con genéricos, el compilador es tu red de seguridad.

¡Menudo Lío Antes De Los Genéricos!

Imagina que tienes una ArrayList a la antigua usanza, sin genéricos. Es como una caja donde puedes meter cualquier cosa: un zapato, una manzana, un gato, un número de la suerte. El problema es que cuando sacas las cosas, Java te devuelve un Object y tú tienes que recordar qué metiste. ¿Y si metiste un Integer pero lo tratas como String? **BOOM**. ClassCastException en toda la cara.

```
import java.util.*;

public class InfiernoSinGenericos {
    public static void main(String[] args) {
        ArrayList cajaDeCaos = new ArrayList();
        cajaDeCaos.add(42);
        cajaDeCaos.add("Hola");
        cajaDeCaos.add(3.14);

        // Todo lo que sacas es Object... ¡tú adivina qué es!
        Object cosa = cajaDeCaos.get(0);
        String texto = (String) cosa; // 🌟 ClassCastException en tiempo de ejecución
    }
}
```

¿Ves? El compilador no te avisó de nada. Te enteras cuando el programa ya está corriendo y explota. Como una granada con el seguro quitado.

⚠ **Advertencia:** Sin genéricos, los errores de tipo saltan en tiempo de EJECUCIÓN (cuando el usuario está usando tu programa). Con genéricos, saltan en tiempo de COMPILACIÓN (cuando tú estás programando). ¿Cuándo prefieres que te enteres?

Llegan Los Genéricos: El Compilador Se Vuelve Tu Amigo

A partir de Java 5, los genéricos cambiaron las reglas del juego. Una `ArrayList<String>` ya no es una caja de caos: es una máquina expendedora que SOLO da Coca-Colas. No puedes meter un zapato, y si lo intentas, el compilador te para el brazo.

```
import java.util.*;

public class CieloConGenericos {
    public static void main(String[] args) {
        ArrayList<String> maquinaDeCocacolas = new ArrayList<>();
        maquinaDeCocacolas.add("Coca-Cola");
        maquinaDeCocacolas.add("Coca-Cola Light");
        // maquinaDeCocacolas.add(42); // ❌ Error de compilación

        // Al sacar, ya sabes que es String. Sin casting.
        String bebida = maquinaDeCocacolas.get(0);
        System.out.println(bebida.toUpperCase()); // "COCA-COLA" sin miedo
    }
}
```

💡 **Consejo:** Los genéricos existen para una sola razón: **seguridad de tipos**. Comprueban en compilación que no estés metiendo la pata con los tipos, eliminando la necesidad de castings y los temidos `ClassCastException`.

★ BE THE CODE, MY FRIEND: Eres Una ArrayList

💡 **Don Tip:** Los genéricos aseguran que solo metas el tipo correcto. Sin ellos, tendrías casting a punta pala.

★ **BE THE CODE, MY FRIEND** 💡 **Don Tip:** `ArrayList<String>` solo acepta Strings. Si intentas meter un `int`, el compilador te para. Cierra los ojos. Respira hondo. Ahora ERES una `ArrayList<T>`.

Dentro de ti hay una `Object[]` que almacena tus elementos. Cuando alguien hace `add("Hola")`, el compilador ya sabe que solo aceptas Strings. Cuando alguien hace `get(0)`, el compilador sabe que devuelves un String, no un Object.

Eres como un portero de discoteca que solo deja pasar a gente con el tipo adecuado. Sin genéricos, dejabas pasar a cualquiera y luego liabas. Con genéricos, eres selectivo, y la fiesta es mucho mejor.

Tu Propia Clase Genérica: Caja<T>

No solo las colecciones pueden ser genéricas. ¡Tú también puedes crear tus propias clases genéricas! El truco está en el parámetro de tipo `<T>` (de “Type”), aunque puedes usar cualquier letra:

- `T` → Tipo (Type) — el comodín general
- `E` → Elemento (Element) — para colecciones
- `K / V` → Clave / Valor (Key / Value) — para mapas
- `N` → Número (Number)

```
// Una caja que guarda UN objeto de cualquier tipo
public class Caja<T> {
    private T contenido;

    public void guardar(T contenido) {
        this.contenido = contenido;
    }

    public T sacar() {
        return contenido;
    }

    public boolean estaVacía() {
        return contenido == null;
    }
}
```

Y así se usa:

```
Caja<String> cajaDeTexto = new Caja<>();
cajaDeTexto.guardar("Mensaje secreto");
String mensaje = cajaDeTexto.sacar(); // Sin casting, directo al pelo

Caja<Integer> cajaDeNumeros = new Caja<>();
cajaDeNumeros.guardar(42);
Integer numero = cajaDeNumeros.sacar();
```

⚠ **Advertencia:** No puedes usar tipos primitivos como parámetro de tipo. `Caja<int>` no compila. Usa `Caja<Integer>`, con su clase envolvente (wrapper). El autoboxing de Java se encarga del resto.

★ BE THE CODE, MY FRIEND: La Caja Genérica

💡 **Don Tip:** El tipo `T` es un comodín. Cuando instancias, se reemplaza por el tipo real.

```
public class BeTheCaja {
    public static void main(String[] args) {
        Caja<Integer> caja = new Caja<>();
        caja.guardar(5);
        caja.guardar(10);
        System.out.println(caja.sacar());
    }
}
```

★ **BE THE CODE, MY FRIEND** 💡 **Don Tip:** `T` puede ser cualquier tipo. Pero no un primitivo: usa `Integer` en vez de `int`. ¿Qué imprime?

- (A) 5
- (B) 10
- (C) null
- (D) Error de compilación

Respuesta: (B) 10. La caja solo guarda un elemento, el segundo `guardar(10)` sobrescribe el 5.

El Operador Diamante <>: El Perezoso Oficial

Desde Java 7, no hace falta repetir el tipo dos veces. El compilador lo infiere por ti:

```
// Antes de Java 7 (repetitivo):
Caja<String> caja1 = new Caja<String>();

// Desde Java 7 (el diamante <> al rescate):
Caja<String> caja2 = new Caja<>();
```

El <> es como el “etcétera” de los genéricos: “ya sabes de qué tipo estoy hablando, ¿no? Pues eso”.

💡 **Consejo:** Usa siempre el operador diamante <>. Tu código queda más limpio, más legible y tus compañeros de equipo te lo agradecerán.

Type Erasure: El Mago Se Lleva los Genéricos

Aquí va el truco: los genéricos SOLO existen en tiempo de compilación. Cuando tu código se convierte en bytecode, el compilador borra toda la información de tipos genéricos. Es como si un mago hiciera desaparecer los <String> y <Integer>.

```
// En tu código fuente:
ArrayList<String> nombres = new ArrayList<>();
ArrayList<Integer> numeros = new ArrayList<>();

// Después de compilar (en bytecode):
ArrayList nombres = new ArrayList(); // ambos son ArrayList simples
ArrayList numeros = new ArrayList();
```

A esto se le llama **type erasure**. El compilador:

1. **Verifica** que los tipos sean correctos (aquí no cuela un Integer en una lista de Strings)
2. **Borra** la información genérica
3. **Añade** los castings necesarios donde haga falta

Es como un portero que revisa tu DNI en la puerta, pero una vez dentro, tú no llevas ninguna identificación.

Métodos Genéricos: Funciones Para Todo Tipo

No solo las clases pueden ser genéricas. Los métodos también pueden declarar sus propios parámetros de tipo, independientemente de la clase donde estén.

```

public class Utilidades {

    // Método genérico: declara <T> antes del tipo de retorno
    public static <T> void imprimir(T elemento) {
        System.out.println("Elemento: " + elemento.toString());
    }

    // Método genérico con dos parámetros de tipo
    public static <T, U> boolean sonIguales(T a, U b) {
        return a.equals(b);
    }

    // Invertir un array genérico
    public static <T> T[] invertir(T[] array) {
        for (int i = 0; i < array.length / 2; i++) {
            T temp = array[i];
            array[i] = array[array.length - 1 - i];
            array[array.length - 1 - i] = temp;
        }
        return array;
    }
}

// Uso:
Utilidades.imprimir(42);           // El compilador infiere que T es Integer
Utilidades.imprimir("Hola");       // El compilador infiere que T es String
String[] invertido = Utilidades.invertir(new String[]{"A", "B", "C"});

```

★ BE THE CODE, MY FRIEND: Método Genérico

👁 Don Tip: El <T> antes del void declara el tipo del método. Sin eso, el compilador no sabe qué es T.

```

public class BeTheGenericMethod {
    public static void main(String[] args) {
        String resultado = Utilidades.<String>obtenerValor("Hola");
        // ¿Qué hace este código? ¿Compila?
    }
}

```

★ BE THE CODE, MY FRIEND 🐞 **Don Tip:** Los métodos genéricos deducen el tipo automáticamente de los argumentos. ¿Compila y qué imprime?

- (A) Sí, imprime “Hola”
- (B) No compila, el método no existe
- (C) Sí, pero imprime null
- (D) No compila, la sintaxis es incorrecta

Respuesta: (A) Sí, imprime “Hola”. La sintaxis `Clase.<Tipo>metodo()` es válida para invocar métodos genéricos especificando el tipo explícitamente, aunque normalmente el compilador lo infiere solo.

? ¡No Hay Preguntas Tontas! (Genéricos)

? **¡No Hay Preguntas Tontas! Q:** ¿Por qué no puedo hacer `new T()` dentro de un método genérico? **A:** Porque en tiempo de compilación, Java no sabe qué es `T`. No puede crear una instancia de algo que no conoce. Es como pedirle a un pastelero que haga “un pastel” pero sin decirle de qué.

Q: ¿Y `new T[]`? **A:** Tampoco. Los arrays conocen su tipo en tiempo de ejecución, pero los genéricos se borran (type erasure). Por eso internamente se usa `Object[]` y se castea.

Q: ¿Los genéricos ralentizan mi programa? **A:** No. Java aplica **type erasure**: el compilador borra toda la información genérica y la convierte a castings normales. Es solo azúcar sintáctico en compilación. En runtime, no hay genéricos.

? ¡No Hay Preguntas Tontas! (Más Genéricos)

? **¡No Hay Preguntas Tontas! Q:** ¿Qué significa `<?>` en los genéricos? **A:** Es un wildcard sin restricciones. Significa “cualquier tipo”. Es como decir “me da igual el tipo, solo quiero trabajar con la colección”. Pero ojo: no puedes añadir elementos (excepto null) porque el compilador no sabe de qué tipo es.

Q: ¿Puedo tener un método genérico estático en una clase no genérica? **A:** Sí, totalmente. De hecho, es muy común. La clase `Collections` está llena de métodos estáticos genéricos como `sort()`, `reverse()`, etc.

Q: ¿Qué es el “type erasure”? **A:** Es el proceso por el cual el compilador borra toda la información de tipos genéricos después de comprobar que todo es correcto. En el bytecode,

`ArrayList<String>` y `ArrayList<Integer>` son ambos `ArrayList`. Es como si el compilador fuera un abogado que revisa el contrato, y luego un mago que hace desaparecer las pruebas.

Wildcards: `? extends T` vs `? super T`

Los comodines (wildcards) son para cuando quieres escribir métodos que funcionen con **cualquier tipo** dentro de una jerarquía.


```

import java.util.*;

public class Wildcards {

    // ? extends T → Covarianza (solo LEER, no escribir)
    // Puedes pasar List<Integer>, List<Double>, List<Number>...
    public static double sumar(ArrayList<? extends Number> numeros) {
        double total = 0.0;
        for (Number n : numeros) {
            total += n.doubleValue();
        }
        // numeros.add(42); // ❌ Error: no puedes añadir nada (excepto null)
        return total;
    }

    // ? super T → Contravarianza (solo ESCRIBIR, leer como Object)
    // Puedes añadir Integers a una lista de Integer, Number, Object...
    public static void rellenar(ArrayList<? super Integer> lista) {
        lista.add(1);
        lista.add(2);
        lista.add(3);
        // Integer n = lista.get(0); // ❌ Error: solo sabes que es Object
        Object obj = lista.get(0); // ✅ Ok
    }
}

// Uso:
ArrayList<Integer> enteros = new ArrayList<>(Arrays.asList(1, 2, 3));
ArrayList<Number> numeros = new ArrayList<>();
ArrayList<Object> objetos = new ArrayList<>();

sumar(enteros); // ✅ ? extends Number funciona con Integer
rellenar(numeros); // ✅ ? super Integer funciona con Number
rellenar(objetos); // ✅ ? super Integer funciona con Object

```

⚠️ Advertencia: Mnemotecnia infalible:

- ? extends T → **P**roducer **E**xtends (solo produces/lees datos)
- ? super T → **C**onsumer **S**uper (solo consumes/escribes datos)

El principio PECS de Joshua Bloch: “Producer Extends, Consumer Super”.

★ BE THE CODE, MY FRIEND: Wildcards

💡 **Don Tip:** ? significa 'cualquier tipo'. ? extends Number limita a Number y sus subclases.

```
import java.util.*;

public class BeTheWildcard {
    public static void main(String[] args) {
        List<Integer> enteros = Arrays.asList(1, 2, 3);
        List<Double> dobles = Arrays.asList(1.5, 2.5, 3.5);
        // printNumbers(enteros); // ¿compila?
        // printNumbers(dobles); // ¿compila?
    }

    public static void printNumbers(List<? extends Number> lista) {
        for (Number n : lista) {
            System.out.print(n + " ");
        }
    }
}
```

★ BE THE CODE, MY FRIEND 💡 **Don Tip:** Los wildcards son de solo lectura. No puedes añadir elementos a una List<?> porque no sabes de qué tipo es. ¿Cuántas llamadas compilan?

- (A) 0
- (B) 1
- (C) 2
- (D) Error en ambos

Respuesta: (C) 2. List<? extends Number> acepta cualquier lista cuyo tipo herede de Number, tanto List<Integer> como List<Double>.

Resumen Rápido: Genéricos

<code>ArrayList<Tipo></code>	← Solo acepta ese <code>Tipo</code> y subclases
<code>Caja<T></code>	← Tu propia clase genérica
<code>new Caja<>()</code>	← El diamante: infiere el tipo
<code><T> void metodo(T)</code>	← Método genérico
<code>? extends T</code>	← Wildcard de lectura (<code>producer</code>)
<code>? super T</code>	← Wildcard de escritura (<code>consumer</code>)

Mapas: La Guía Telefónica

HashMap: La Guía Telefónica

HashMap asocia claves con valores. Como una agenda: buscas por nombre (clave) y obtienes el teléfono (valor).

```
import java.util.HashMap;

HashMap<String, Integer> agenda = new HashMap<>();
agenda.put("Ana", 612345678);
agenda.put("Bob", 698765432);
agenda.put("Ana", 600000000); // Sobrescribe el anterior

int telefono = agenda.get("Ana"); // 600000000
int inexistente = agenda.get("NoExisto"); // null
agenda.containsKey("Bob"); // true
agenda.containsValue(600000000); // true
```



Nota: Las claves de un HashMap deben ser inmutables. Por eso String e Integer son perfectos. Si usas un objeto mutable como clave y luego lo modificas, el HashMap no lo encontrará más. Es como cambiar la cerradura y esperar que la llave vieja siga funcionando.

Recorrer un HashMap

```

HashMap<String, Integer> agenda = new HashMap<>();
agenda.put("Ana", 612345678);
agenda.put("Bob", 698765432);

for (String nombre : agenda.keySet()) {
    System.out.println(nombre + " → " + agenda.get(nombre));
}

for (Integer telefono : agenda.values()) {
    System.out.println("Tel: " + telefono);
}

for (HashMap.Entry<String, Integer> entrada : agenda.entrySet()) {
    System.out.println(entrada.getKey() + " → " + entrada.getValue());
}

```

★ BE THE CODE, MY FRIEND: El HashMap Traicionero

👁 Don Tip: HashMap no garantiza orden. Si necesitas orden, usa TreeMap o LinkedHashMap.

```

import java.util.HashMap;

public class BeTheMap {
    public static void main(String[] args) {
        HashMap<String, String> capitales = new HashMap<>();
        capitales.put("España", "Madrid");
        capitales.put("Francia", "París");
        capitales.put("Italia", "Roma");
        capitales.put("España", "Barcelona");

        System.out.println(capitales.get("España"));
    }
}

```

★ BE THE CODE, MY FRIEND 👁 Don Tip: Las claves de un HashMap deben tener hashCode() y equals() bien implementados. ¿Qué imprime?

- (A) Madrid
- (B) Barcelona

- (C) null
- (D) Error

Respuesta: (B) Barcelona. La clave “España” se sobrescribe con el nuevo valor. Madrid ha sido reemplazado.

★ BE THE CODE, MY FRIEND: Frecuencia de Letras

💡 **Don Tip:** `getOrDefault()` evita el null check. Úsalo siempre que puedas.

```
import java.util.HashMap;

public class BeTheFrequency {
    public static void main(String[] args) {
        String texto = "banana";
        HashMap<Character, Integer> frec = new HashMap<>();
        for (char c : texto.toCharArray()) {
            frec.put(c, frec.getOrDefault(c, 0) + 1);
        }
        System.out.println(frec.get('a') + " " + frec.get('n'));
    }
}
```

★ BE THE CODE, MY FRIEND 💡 **Don Tip:** `merge()` es tu amigo para frecuencias. Combina valor existente con nuevo. ¿Qué imprime?

- (A) 1 1
- (B) 3 1
- (C) 3 2
- (D) 2 2

Respuesta: (B) 3 1. En “banana”: la ‘a’ aparece 3 veces, la ‘n’ aparece 1 vez. `getOrDefault` es el héroe que evita los `NullPointerException`.

TreeMap: El Ordenado

`TreeMap` es un `HashMap` ordenado por clave. Internamente usa un árbol rojo-negro.

```
import java.util.TreeMap;

TreeMap<String, Integer> ordenado = new TreeMap<>();
ordenado.put("Zara", 30);
ordenado.put("Ana", 25);
ordenado.put("Bob", 35);
System.out.println(ordenado); // {Ana=25, Bob=35, Zara=30} - orden alfabético

// Métodos extra útiles
ordenado.firstKey(); // "Ana"
ordenado.lastKey(); // "Zara"
ordenado.headMap("Bob"); // {Ana=25} - entradas antes de Bob
```

💡 **Consejo:** HashMap para velocidad ($O(1)$). TreeMap para orden ($O(\log n)$). Si preguntas en una entrevista “¿cuál es mejor?”, la respuesta correcta es “depende”.

? ¡No Hay Preguntas Tontas! (Mapas)

? **¡No Hay Preguntas Tontas!** Q: ¿Puedo tener un HashMap con clave null? A: HashMap admite una clave null. TreeMap no. HashMap guarda la clave null en una posición especial.

Q: ¿Qué pasa si la clave no existe en el mapa? A: `get()` devuelve null. Úsalo con cuidado o mejor usa `getOrDefault(clave, valorPorDefecto)`.

Q: ¿HashMap o TreeMap? A: HashMap para velocidad. TreeMap para orden.

Q: ¿Qué ventaja tiene un LinkedHashMap? A: Mantiene el orden de inserción. Es como un HashMap que recuerda en qué orden metiste las cosas.

Comparación: Map vs List vs Set

Característica	List	Set	Map
¿Qué guarda?	Elementos ordenados	Elementos únicos	Pares clave→valor
¿Duplicados?	Sí	No	Claves no, valores sí
¿Orden?	De inserción	Depende (Hash/Tree)	Depende (Hash/Tree)

Característica	List	Set	Map
Acceso	Por índice	Por elemento	Por clave
¿Nulls?	Sí	HashSet: 1, TreeSet: 0	HashMap: 1 clave, TreeMap: 0
Principal impl.	ArrayList	HashSet	HashMap

Regla práctica:

- Necesitas una lista de cosas? → ArrayList
- Cosas sin repetir? → HashSet
- Cosas ordenadas sin repetir? → TreeSet
- Asociar una cosa con otra? → HashMap



EL RING: HashMap vs TreeMap

Dos implementaciones de Map se enfrentan.

HashMap: «Yo soy el rey de la velocidad. $O(1)$ en get y put. No me importa el orden, me importa la rapidez.»

TreeMap: «Sí, pero yo mantengo las claves ordenadas automáticamente. Si necesitas recorrerlas en orden alfabético, soy tu único amigo.»

HashMap: «¿Ordenado? Eso cuesta. Soy $O(\log n)$ en tus operaciones. Yo soy $O(1)$. ¡Soy imbatible en rendimiento!»

TreeMap: «Cierto, pero puedo navegar: `firstKey()`, `lastKey()`, `subMap()`, `headMap()`. Tú para todo eso tienes que copiar y ordenar.»

HashMap: «Si no necesitas orden, ¿para qué pagar el coste? La mayoría de los casos usan HashMap.»

TreeMap: «Y cuando necesitan orden, ahí estoy yo. Y no soy tan lento: $O(\log n)$ sigue siendo muy rápido para la mayoría de casos.»

HashMap: «Tregua. Cada uno en su sitio.»

TreeMap: «Hecho.»

💡 **Don Tip:** ¿Velocidad? HashMap. ¿Orden? TreeMap. ¿Orden de inserción? LinkedHashMap.
Cada uno tiene su superpoder.

Resumen Rápido: Mapas

```
HashMap<K, V> m = new HashMap<>();           // Clave → Valor, rápido
TreeMap<K, V> tm = new TreeMap<>();          // Clave → Valor, ordenado

m.put(clave, valor); // añadir o sobrescribir
m.get(clave);         // obtener (null si no existe)
m.getDefault(c, d);  // obtener o valor por defecto
m.containsKey(c);     // existe la clave?
m.containsValue(v);   // existe el valor?
m.remove(clave);      // borrar entrada
m.size();             // número de entradas
m.keySet();           // conjunto de claves
m.values();           // colección de valores
m.entrySet();         // conjunto de entradas
```

Ejercicios Propuestos

Ejercicio 1: Contactos de Toda la Vida

Implementa una mini agenda usando `HashMap<String, String>` que asocie nombre con teléfono. El programa debe permitir añadir contactos, buscar por nombre, listar todos y borrar. Usa un menú.

Ejercicio 2: Crea una clase Pareja<T, U>

Almacena dos objetos de tipos posiblemente distintos. Incluye métodos `getPrimero()`, `getSegundo()`, `setPrimero(T)`, `setSegundo(U)` y un método `intercambiar()` que devuelva una nueva `Pareja<U, T>` con los valores intercambiados.

Ejercicio 3: Contador de Palabras

Escribe un programa que lea un texto y cuente cuántas veces aparece cada palabra usando un `HashMap<String, Integer>`.

Ejercicio 4: Método Genérico maximo

Implementa un método genérico `public static <T extends Comparable<T>> T maximo(T[] array)` que devuelva el elemento más grande de un array.

RAs trabajados en esta unidad:

- **RA6** - Tipos avanzados: Genéricos y mapas
-



Sergi Garcia Barea — CC BY-SA 4.0



Unidad 11: Consola, Ficheros y Expresiones Regulares

Objetivos de aprendizaje

- Leer y escribir en consola con System.out, printf y Scanner
- Formatear salidas y manejar trampas de Scanner (nextInt + nextLine)
- Leer y escribir archivos de texto con File, FileReader, FileWriter, BufferedReader
- Usar try-with-resources, NIO (Files/Paths) y serialización básica
- Crear y usar expresiones regulares con Pattern y Matcher
- Validar, buscar, reemplazar y extraer partes de texto con regex

Comunicación por Consola

System.out

System.out es el megáfono de tu programa:

```
System.out.println("¡Hola, DAM!");    // con salto de línea (ln = line)
System.out.print("Sin salto... ");    // sin salto de línea
System.out.print("...continúa aquí.");
System.out.println();                // salto de línea vacío
```

printf() — Formateo

```
String nombre = "Ana";
int edad = 20;
double nota = 9.5;
System.out.printf("Nombre: %s, Edad: %d, Nota: %.2f%n", nombre, edad, nota);
```

Especificador	Tipo	Ejemplo
%s	String	"Hola %s" → "Hola Mario"
%d	Entero	"Edad: %d" → "Edad: 25"
%f	Decimal	"%.2f" → "19.99"

Especificador	Tipo	Ejemplo
%c	Carácter	"Inicial: %c"
%n	Salto de línea	(independiente del SO)
%x	Hexadecimal	"#%06X" → "#FF00AA"

```
double pi = Math.PI;
System.out.printf("2 decimales: %.2f%n", pi);           // 3,14
System.out.printf("4 decimales: %.4f%n", pi);           // 3,1416
System.out.printf("Ancho 10: %10.2f%n", pi);             //          3,14
System.out.printf("Izquierda: %-10.2f%n", pi);           // 3,14
```

String.format() devuelve un String sin imprimirlo:

```
String msg = String.format("Bienvenido, %s.", "Carlos");
System.out.println(msg);
```

Scanner

System.in es el oído del programa; Scanner es el traductor:

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
System.out.print("Nombre: ");
String nombre = sc.nextLine();
System.out.print("Edad: ");
int edad = sc.nextInt();
System.out.print("Altura: ");
double altura = sc.nextDouble();
System.out.printf("Encantado, %s. %d años, %.2f m.%n", nombre, edad, altura);
sc.close();
```

Método	Descripción
nextLine()	Lee una línea completa como String
next()	Lee una palabra (hasta el primer espacio)

Método	Descripción
<code>nextInt()</code>	Lee un número entero
<code>nextDouble()</code>	Lee un número decimal
<code>nextFloat()</code> / <code>nextLong()</code>	Lee float / long
<code>nextBoolean()</code>	Lee booleano (true/false)
<code>hasNextInt()</code>	¿Hay un int esperando?
<code>hasNextDouble()</code>	¿Hay un double esperando?
<code>close()</code>	Cierra el Scanner

⚠ ¡Cuidado con `nextInt()` + `nextLine()`!

`nextInt()` deja el Intro en el buffer. `nextLine()` posterior se lo traga y se salta la entrada.

Solución 1: `sc.nextLine()` extra después de `nextInt()`

Solución 2: Usar siempre `nextLine()` y convertir:

```
int edad = Integer.parseInt(sc.nextLine());
```

Formato de números grandes

```
import java.text.NumberFormat;
import java.util.Locale;

NumberFormat nf = NumberFormat.getInstance(new Locale("es", "ES"));
System.out.println(nf.format(1234567.89)); // 1.234.567,89

NumberFormat moneda = NumberFormat.getCurrencyInstance(new Locale("es", "ES"));
System.out.println(moneda.format(12345.67)); // 12.345,67 €
```

Errores Comunes con Scanner

```
// Olvidar import java.util.Scanner;
// No cerrar el Scanner: sc.close();
// Tipo incorrecto: int num = sc.nextDouble(); // Error de compilación
// nextInt() seguido de nextLine() sin consumir el Intro
```

? ¡No Hay Preguntas Tontas! (Consola)

Q: ¿Por qué se llama `System.out`?

A: `System` tiene tres flujos estáticos: `out` (salida), `in` (entrada), `err` (errores).

Q: ¿Puedo tener varios `Scanner` abiertos?

A: Técnicamente sí, pero usa UN solo `Scanner` para `System.in`.

Q: ¿Qué pasa si el usuario escribe letras donde espero números?

A: `InputMismatchException`. Solución: `try-catch` o `hasNextInt()`.

Q: ¿Puedo usar `printf()` para fechas?

A: Sí, con `%tF` (fecha ISO: 2026-06-20) y `%tT` (hora: 14:30:00).

Resumen Exprés: Consola

- `System.out.println()` / `printf()` → imprimir
- `Scanner sc = new Scanner(System.in)` → leer teclado
- `sc.nextLine()` / `sc.nextInt()` → leer texto / número
- Siempre `sc.close()` al terminar
- `nextInt()` + `nextLine()` = desastre seguro

Ficheros

La Clase File

Antes de leer o escribir, localiza el archivo. `java.io.File` es el GPS:

```
import java.io.File;

File f = new File("C:/datos/notas.txt");
System.out.println("¿Existe? " + f.exists());
System.out.println("¿Es archivo? " + f.isFile());
System.out.println("¿Es carpeta? " + f.isDirectory());
System.out.println("Tamaño: " + f.length() + " bytes");
System.out.println("Ruta absoluta: " + f.getAbsolutePath());
System.out.println("Nombre: " + f.getName());
```

💡 **Consejo:** Usa `/` o `\\` en Windows. `"C:/datos/notas.txt"` o `"C:\\datos\\notas.txt"`.

Escribir: FileWriter

```
import java.io.FileWriter;
import java.io.IOException;

FileWriter escritor = new FileWriter("salida.txt");
escritor.write("Primera línea.\n");
escritor.write("Segunda línea.\n");
escritor.close();

// Añadir al final (sin borrar):
FileWriter writer = new FileWriter("bitacora.txt", true);
```

Sin `close()` o `flush()` los datos se quedan en el buffer interno sin escribirse.

Leer: FileReader + BufferedReader

`BufferedReader` envuelve a `FileReader` para leer líneas enteras de golpe:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

BufferedReader lector = new BufferedReader(new FileReader("salida.txt"));
String linea = lector.readLine();
while (linea != null) {
    System.out.println(linea);
    linea = lector.readLine();
}
lector.close();
```

★ BE THE CODE, MY FRIEND: El Detective de Archivos

💡 **Don Tip:** `File` representa rutas, no contenido. Para leer el contenido usa `Scanner`, `BufferedReader` o `Files.readAllLines()`.

```

import java.io.*;

public class DetectiveDeArchivos {
    public static void main(String[] args) throws IOException {
        File f = new File("misterio.txt");
        if (!f.exists()) {
            System.out.println("Creando archivo...");
            FileWriter w = new FileWriter(f);
            w.write("Tres\npalabras\nmisteriosas\n"); w.close();
        }
        BufferedReader r = new BufferedReader(new FileReader(f));
        String s = "";
        String linea;
        while ((linea = r.readLine()) != null) {
            s = linea + " " + s;
        }
        r.close();
        System.out.println(s);
    }
}

```

★ ¿Qué imprime la **segunda** vez? (el archivo ya existe)

Respuesta: misteriosas palabras Tres — lee y concatena al revés.

try-with-resources (Java 7+)

Los recursos se cierran automáticamente al salir del bloque:

```

try (BufferedReader br = new BufferedReader(new FileReader("salida.txt"))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
// No hay br.close() – se cierra solo

```

Scanner + File

```
try (Scanner sc = new Scanner(new File("salida.txt"))) {
    while (sc.hasNextLine()) {
        System.out.println(sc.nextLine());
    }
}
```

Scanner vs BufferedReader: Scanner es mejor para parsear tokens (nextInt());
BufferedReader es más rápido para archivos grandes.

PrintWriter

Igual que System.out pero escribiendo en archivos:

```
try (PrintWriter pw = new PrintWriter(new FileWriter("formato.txt"))) {
    pw.println("Línea con salto automático");
    pw.printf("PI vale %.4f%n", Math.PI);
}
```

NIO: Files y Paths

API moderna desde Java 7. Path es como File pero con esteroides:

```
import java.nio.file.*;
import java.util.List;

Path ruta = Paths.get("C:/datos/notas.txt");
List<String> lineas = Files.readAllLines(ruta); // leer
Files.write(ruta, List.of("Línea 1", "Línea 2")); // escribir
```

★ BE THE CODE, MY FRIEND: NIO en Acción

👁 Don Tip: NIO usa Path y Files. Son más modernos y tienen métodos útiles como Files.walk() para recorrer árboles.


```
import java.nio.file.*;
import java.util.*;

public class BeTheNIO {
    public static void main(String[] args) throws IOException {
        Path p = Paths.get("nums.txt");
        Files.write(p, List.of("3", "7", "2", "9", "5"));
        List<String> l = Files.readAllLines(p);
        int suma = 0;
        for (String s : l) { suma += Integer.parseInt(s); }
        Files.write(p, List.of("Total: " + suma));
        System.out.println(Files.readString(p));
    }
}
```

★ ¿Qué imprime? **Respuesta:** Total: 26 — suma los números y sobrescribe.

Serialización: ObjectOutputStream

Guardar objetos enteros en archivos con Serializable:

```

import java.io.*;

class Persona implements Serializable {
    String nombre; int edad;
    Persona(String n, int e) { this.nombre = n; this.edad = e; }
}

public class GuardandoObjetos {
    public static void main(String[] args) throws Exception {
        Persona p = new Persona("Luis", 25);

        try (ObjectOutputStream oos = new ObjectOutputStream(
            new FileOutputStream("persona.obj"))) {
            oos.writeObject(p);
        }

        try (ObjectInputStream ois = new ObjectInputStream(
            new FileInputStream("persona.obj"))) {
            Persona recuperada = (Persona) ois.readObject();
            System.out.println(recuperada.nombre + " tiene " + recuperada.edad);
        }
    }
}

```

La clase debe implementar Serializable. Si tiene campos no serializables → NotSerializableException.

? ¡No Hay Preguntas Tontas! (Ficheros)

Q: ¿File crea el archivo?

A: No. new File("ruta") solo representa la ruta, no crea nada.

Q: ¿Qué pasa si el archivo no existe al leer?

A: FileNotFoundException. Usa f.exists() o try-catch.

Q: ¿Qué diferencia hay entre File, FileReader y FileWriter?

A: File → la dirección. FileReader → para LEER. FileWriter → para ESCRIBIR.

Q: ¿Qué pasa si no cierro un archivo?

A: Los datos pueden perderse (buffer sin vaciar) y el SO retiene recursos.

Expresiones Regulares

Una expresión regular (regex) es un **patrón de búsqueda** que describe un conjunto de cadenas.

⚠ En Java las contrabarras se duplican: `\d` → `"\\d"`, `\.` → `"\\".`. Es el “infierno de las contrabarras”.

Pattern y Matcher

- Pattern: la expresión regular compilada (el molde)
- Matcher: se aplica a un texto concreto buscando coincidencias

```
import java.util.regex.*;

Pattern patron = Pattern.compile("\\d+"); // uno o más dígitos
Matcher matcher = patron.matcher("Hay 123 manzanas y 456 peras");

while (matcher.find()) {
    System.out.println("Encontrado: " + matcher.group()
        + " (" + matcher.start() + "-" + matcher.end() + ")");
}
// Encontrado: 123 (4-7), Encontrado: 456 (21-24)
```

Compila el Pattern una sola vez y reutilízalo. `Pattern.compile()` es caro.

Símbolos Regex

Símbolo	Significado	Ejemplo
.	Cualquier carácter (ex. salto)	<code>c.sa</code> → “casa”, “cose”
<code>\d</code>	Dígito (0-9)	<code>\d{3}</code> → “123”
<code>\D</code>	NO dígito	<code>\D+</code> → “Hola”
<code>\w</code>	Letra, dígito o <code>_</code>	<code>\w+</code> → “Hola_123”
<code>\W</code>	NO <code>\w</code>	<code>\W</code> → “.”, “ “
<code>\s</code>	Espacio en blanco	<code>\s+</code> → separadores

Símbolo	Significado	Ejemplo
\S	NO espacio	\S+ → palabras
*	0 o más veces	a* → "", "a", "aa"
+	1 o más veces	a+ → "a", "aa"
?	0 o 1 vez (opcional)	colou?r → "color", "colour"
{n}	Exactamente n	\d{3}
{n,m}	Entre n y m	\d{2,4}
[abc]	Uno del conjunto	[aeiou] → vocales
[a-z]	Rango	[a-z] → minúsculas
[^abc]	Negación	[^0-9] → no dígitos
()	Grupo de captura	(\d+) - (\w+)
^	Inicio de línea	^Hola
\$	Final de línea	mundo\$
	OR lógico	gato perro
\b	Límite de palabra	\bJava\b ≠ "JavaScript"

String.matches(), replaceAll(), split()

Métodos de String que aceptan regex directamente:

```
String texto = "  Hola    mundo  de las  regex  ";

// matches() → true solo si TODO el string coincide
"123".matches("\\d+");           // true
"Hola".matches("\\d+");          // false

// replaceAll() → reemplaza todas las coincidencias
String limpio = texto.replaceAll("\\s+", " ").trim();
// "Hola mundo de las regex"

// replaceFirst() → solo la primera coincidencia
texto.replaceFirst("\\s+", " ").trim();

// split() → divide por el patrón
"a,b,c,d".split(",");            // [a, b, c, d]
"a,b,c,d".split(",", 3);         // [a, b, c,d] (con límite)
```

`matches()` empareja **todo** el string. Para buscar subcadenas usa `find()`.

Validación: Email, Teléfono, DNI

```

public class ValidadorRegex {
    private static final Pattern PATRON_DNI = Pattern.compile("\\d{8}[A-Z]");
    private static final Pattern PATRON_EMAIL =
        Pattern.compile("[\\w.]+@[\\w.]+\\. [a-z]{2,}");
    private static final Pattern PATRON_TELEFONO =
        Pattern.compile("[679]\\d{8}");

    public static boolean esEmailValido(String email) {
        return PATRON_EMAIL.matcher(email.toLowerCase()).matches();
    }
    public static boolean esDNIVálido(String dni) {
        return PATRON_DNI.matcher(dni.toUpperCase()).matches();
    }
    public static boolean esTelefonoValido(String telefono) {
        return PATRON_TELEFONO.matcher(telefono).matches();
    }

    public static void main(String[] args) {
        System.out.println(esEmailValido("user@example.com")); // true
        System.out.println(esEmailValido("user@@example")); // false
        System.out.println(esDNIVálido("12345678Z")); // true
        System.out.println(esDNIVálido("12345678z")); // false
        System.out.println(esTelefonoValido("612345678")); // true
        System.out.println(esTelefonoValido("512345678")); // false
    }
}

```

Para validar DNI español de verdad necesitas comprobar la letra módulo 23. La regex solo verifica formato.

Grupos de Captura

Los paréntesis () capturan lo que coincide para extraerlo después:

```
String texto = "Juan: 28 años, María: 32 años";
Pattern patron = Pattern.compile("(\\w+): (\\d+) años");
Matcher matcher = patron.matcher(texto);

while (matcher.find()) {
    System.out.println(matcher.group(1) + " tiene " + matcher.group(2) + " años");
}
// Juan tiene 28 años, María tiene 32 años
```

Para agrupar sin capturar (más eficiente): (? :patron).

Regex Con Flags

```
Pattern p1 = Pattern.compile("java", Pattern.CASE_INSENSITIVE);
Pattern p2 = Pattern.compile("^\\d+", Pattern.MULTILINE);
Pattern p3 = Pattern.compile(".*", Pattern.DOTALL);
Pattern p4 = Pattern.compile("java", Pattern.CASE_INSENSITIVE | Pattern.MULTILINE);
Pattern p5 = Pattern.compile("(?i)java"); // flag inline
```

★ BE THE CODE, MY FRIEND: Procesando Un Log

💡 **Don Tip:** Las regex se prueban con matches() (todo el string) o find() (subcadena). group() extrae lo capturado con paréntesis.

```

import java.util.regex.*;
import java.util.*;

public class ProcesadorLog {
    public static void main(String[] args) {
        String log = """
            [ERROR] 2024-03-15 10:30:45 - Usuario no encontrado: id=42
            [INFO] 2024-03-15 10:31:02 - Sesión iniciada: user=admin
            [WARN] 2024-03-15 10:32:10 - Memoria baja: 128MB disponible
            [ERROR] 2024-03-15 10:33:00 - Connection timeout: server=db01
            """;

        Pattern patron = Pattern.compile(
            "\\[(ERROR|INFO|WARN)\\]\\s+" +
            "(\\d{4}-\\d{2}-\\d{2})\\s+" +
            "(\\d{2}:\\d{2}:\\d{2})\\s+-\\s+(.*)");

        Matcher matcher = patron.matcher(log);
        List<String> errores = new ArrayList<>();

        while (matcher.find()) {
            String nivel = matcher.group(1);
            String fecha = matcher.group(2);
            String hora = matcher.group(3);
            String mensaje = matcher.group(4);
            System.out.printf("[%s] %s a las %s: %s%n", nivel, fecha, hora, mensaje);
            if (nivel.equals("ERROR")) errores.add(mensaje);
        }
        System.out.println("\nErrores: " + errores.size());
        for (String e : errores) System.out.println("  ✖ " + e);
    }
}

```

Cada paréntesis captura una parte: nivel, fecha, hora y mensaje. Para un log de 2 GB leerías línea a línea con `BufferedReader`.

? ¡No Hay Preguntas Tontas! (Regex)

Q: ¿Por qué `"abc123".matches("\\d+")` devuelve `false`?

A: `matches()` empareja **todo** el string. Usa `find()` para subcadenas.

Q: ¿Cómo hago un punto literal?

A: Escápalo: "\\.".

Q: ¿Puedo procesar 1M líneas con la misma regex?

A: Sí, compila el Pattern una vez fuera del bucle y reutiliza el matcher.

Q: ¿Se pueden validar HTML con regex?

A: No. HTML no es un lenguaje regular. Usa un parser.

Q: ¿\w incluye tildes?

A: No por defecto. Usa [a-zA-Záéíóúüñ] o Pattern.UNICODE_CHARACTER_CLASS.

Resumen Rápido: Regex

```
Pattern p = Pattern.compile("regex");    ← compilar
Matcher m = p.matcher(texto);            ← aplicar
m.find() → boolean                       ← buscar coincidencia
m.group() / m.group(n) → String           ← obtener match / grupo
texto.matches("regex")                   ← ¿todo coincide?
texto.replaceAll("regex", "nuevo")        ← reemplazar todas
texto.split("regex")                     ← dividir por patrón
```


\d → dígito	\w → letra/_	\s → espacio	. → cualquier char
* → 0+	+ → 1+	? → 0-1	{n} → exactamente n
[abc] → conjunto	() → grupo de captura		

Ejercicios Propuestos

Ejercicio 1: El entrevistador

Pregunta nombre, edad, años de experiencia y lenguaje favorito. Muestra resumen con printf().

Ejercicio 2: Calculadora de propinas

Solicita total y porcentaje. Muestra propina y total con 2 decimales con printf().

Ejercicio 3: El diario personal

Escribe en `diario.txt` con fecha y texto. Abre en modo `append` cada ejecución sin borrar lo anterior.

Ejercicio 4: El contador de líneas

Lee un archivo con `BufferedReader` y muestra cuántas líneas, palabras y caracteres tiene.

Ejercicio 5: Validador de contraseñas

Valida: mínimo 8 caracteres, una mayúscula, una minúscula, un dígito, un carácter especial (`@#$$%^&+=`).

Ejercicio 6: Extractor de URLs

Extrae URLs (`http/https`) separando protocolo, dominio y ruta con grupos de captura.

Ejercicio 7: Formateador de fechas

Convierte `dd/mm/aaaa` a `aaaa-mm-dd` con `replaceAll()` y grupos de captura.

Ejercicio 8: Cifrado César

Lee `mensaje.txt`, desplaza cada carácter +3 posiciones, escribe en `mensaje_cifrado.txt`.

Ejercicio 9: Analizador de logs

Dado `[NIVEL] timestamp - mensaje: detalle`, cuenta mensajes por nivel y muestra los `ERROR`.

RAs trabajados en esta unidad:

- **RA5** - Entrada/Salida: consola y ficheros
- **RA6** - Tipos avanzados: Expresiones regulares



Sergi Garcia Barea — CC BY-SA 4.0

Unidad 12: Conexión a Bases de Datos con JDBC

Objetivos de aprendizaje

- Conectar Java a SQLite usando JDBC
- CRUD: INSERT, SELECT, UPDATE, DELETE con PreparedStatement
- Entender SQL Injection y cómo evitarla
- Aplicar el patrón DAO
- Manejar transacciones con commit y rollback

¿Qué es JDBC?

JDBC (Java Database Connectivity) es un conjunto de interfaces en `java.sql` que permiten a Java hablar con cualquier base de datos que tenga un driver JDBC.

 **Consejo:** Piensa en JDBC como el USB de las bases de datos: da igual SQLite, MySQL o PostgreSQL. Si tienen driver JDBC, Java se conecta.

Conexión a SQLite

SQLite es ideal para aprender: no necesita servidor, es un solo archivo.

Los 5 Pasos

1. Cargar el driver
2. Establecer la conexión
3. Crear un Statement
4. Ejecutar la consulta
5. Procesar los resultados

Bonus no opcional: Cerrar todo.

Dependencia Maven

```
<dependency>
  <groupId>org.xerial</groupId>
  <artifactId>sqlite-jdbc</artifactId>
  <version>3.45.1.0</version>
</dependency>
```

Connection

SQLite no necesita usuario ni contraseña. La URL es un archivo local:

```
String url = "jdbc:sqlite:instituto.db";
Connection con = DriverManager.getConnection(url);
```



Nota: SQLite crea el archivo automáticamente si no existe. No necesitas crear la BD aparte.

Statement y ResultSet

```
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM alumnos");

while (rs.next()) {
    System.out.println(rs.getInt("id") + ": " + rs.getString("nombre"));
}
```

`rs.next()` avanza a la siguiente fila. Devuelve `false` cuando no quedan más. El `ResultSet` empieza **antes** de la primera fila.

executeQuery vs executeUpdate

Método	Para	Devuelve
<code>executeQuery()</code>	SELECT	<code>ResultSet</code>
<code>executeUpdate()</code>	INSERT, UPDATE, DELETE	<code>int</code> (filas afectadas)



Advertencia: Usar `executeQuery()` con un INSERT lanza excepción. Cada cosa en su sitio.

Try-With-Resources

Desde Java 7, los recursos se cierran solos:

```
try (Connection con = DriverManager.getConnection(url);
    Statement stmt = con.createStatement();
    ResultSet rs = stmt.executeQuery("SELECT * FROM alumnos")) {

    while (rs.next()) {
        System.out.printf("%d: %s\n", rs.getInt("id"), rs.getString("nombre"));
    }
} catch (SQLException e) {
    System.err.println("Error BD: " + e.getMessage());
}
```



Nota: El orden de cierre es inverso al de apertura: ResultSet → Statement → Connection. Try-with-resources lo hace solo. Es mágico.

SQLException

Es checked, te obliga a capturarla:

```
catch (SQLException e) {
    System.err.println("Error: " + e.getMessage());
    System.err.println("Código: " + e.getErrorCode());
    System.err.println("Estado SQL: " + e.getSQLState());
}
```



Consejo: No hagas `catch (Exception e) {}` y te quedes tan pancho. Eso es tapar la luz de “check engine” con esparadrapo.

CRUD con JDBC

CRUD = Create, Read, Update, Delete.

La Tabla

```
CREATE TABLE contactos (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    nombre TEXT NOT NULL,  
    telefono TEXT,  
    email TEXT  
);
```

 **Nota:** SQLite usa INTEGER PRIMARY KEY AUTOINCREMENT en lugar de INT AUTO_INCREMENT.

EL POJO

```
public class Contacto {  
    private int id;  
    private String nombre;  
    private String telefono;  
    private String email;  
  
    public Contacto() {}  
    public Contacto(String nombre, String telefono, String email) {  
        this.nombre = nombre;  
        this.telefono = telefono;  
        this.email = email;  
    }  
  
    public int getId() { return id; }  
    public void setId(int id) { this.id = id; }  
    public String getNombre() { return nombre; }  
    public void setNombre(String nombre) { this.nombre = nombre; }  
    public String getTelefono() { return telefono; }  
    public void setTelefono(String telefono) { this.telefono = telefono; }  
    public String getEmail() { return email; }  
    public void setEmail(String email) { this.email = email; }  
  
    @Override  
    public String toString() {  
        return id + " - " + nombre + " (" + telefono + ")";  
    }  
}
```

INSERT (Create)

```

public void insertarContacto(Contacto c) {
    String sql = "INSERT INTO contactos (nombre, telefono, email) VALUES (?, ?, ?)";
    try (Connection con = DriverManager.getConnection(URL);
        PreparedStatement pstmt = con.prepareStatement(sql)) {
        pstmt.setString(1, c.getNombre());
        pstmt.setString(2, c.getTelefono());
        pstmt.setString(3, c.getEmail());
        pstmt.executeUpdate();
    } catch (SQLException e) {
        System.err.println("Error al insertar: " + e.getMessage());
    }
}

```

SELECT (Read)

```

public List<Contacto> obtenerTodos() {
    List<Contacto> contactos = new ArrayList<>();
    String sql = "SELECT * FROM contactos ORDER BY nombre";
    try (Connection con = DriverManager.getConnection(URL);
        PreparedStatement pstmt = con.prepareStatement(sql);
        ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) {
            Contacto c = new Contacto(
                rs.getString("nombre"),
                rs.getString("telefono"),
                rs.getString("email")
            );
            c.setId(rs.getInt("id"));
            contactos.add(c);
        }
    } catch (SQLException e) {
        System.err.println("Error al consultar: " + e.getMessage());
    }
    return contactos;
}

```

Búsqueda con LIKE

```

public List<Contacto> buscarPorNombre(String nombre) {
    List<Contacto> contactos = new ArrayList<>();
    String sql = "SELECT * FROM contactos WHERE nombre LIKE ?";
    try (Connection con = DriverManager.getConnection(URL);
        PreparedStatement pstmt = con.prepareStatement(sql)) {
        pstmt.setString(1, "%" + nombre + "%");
        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                Contacto c = new Contacto(
                    rs.getString("nombre"),
                    rs.getString("telefono"),
                    rs.getString("email")
                );
                c.setId(rs.getInt("id"));
                contactos.add(c);
            }
        }
    } catch (SQLException e) {
        System.err.println("Error al buscar: " + e.getMessage());
    }
    return contactos;
}

```

💡 **Consejo:** Devuelve `List<Contacto>` en lugar de `ResultSet`. El `ResultSet` está atado a la conexión. Si la cierras, el `ResultSet` muere. Mejor convierte a objetos Java y cierra todo.

UPDATE (Update)


```

public boolean actualizarContacto(Contacto c) {
    String sql = "UPDATE contactos SET nombre = ?, telefono = ?, email = ? WHERE id = ?";
    try (Connection con = DriverManager.getConnection(URL);
        PreparedStatement pstmt = con.prepareStatement(sql)) {
        pstmt.setString(1, c.getNombre());
        pstmt.setString(2, c.getTelefono());
        pstmt.setString(3, c.getEmail());
        pstmt.setInt(4, c.getId());
        return pstmt.executeUpdate() > 0;
    } catch (SQLException e) {
        System.err.println("Error al actualizar: " + e.getMessage());
        return false;
    }
}

```

⚠ **Advertencia:** Sin WHERE id = ?, actualizas TODOS los contactos. 500 contactos llamados "John Doe". WHERE SIEMPRE.

DELETE (Delete)

```

public boolean eliminarContacto(int id) {
    String sql = "DELETE FROM contactos WHERE id = ?";
    try (Connection con = DriverManager.getConnection(URL);
        PreparedStatement pstmt = con.prepareStatement(sql)) {
        pstmt.setInt(1, id);
        return pstmt.executeUpdate() > 0;
    } catch (SQLException e) {
        System.err.println("Error al eliminar: " + e.getMessage());
        return false;
    }
}

```

💡 **Consejo:** Confirma con el usuario antes de borrar. El DELETE no tiene Ctrl+Z. Es para siempre. Como las decisiones en una película de terror.

PreparedStatement y SQL Injection

Concatenar strings para construir SQL es PELIGROSO:

```
// PELIGRO: esto es una bomba
String sql = "SELECT * FROM alumnos WHERE nombre = '" + inputUsuario + "'";
```

Si el usuario escribe Luis'; DROP TABLE alumnos; --, tu tabla alumnos se borra. Esto es **SQL Injection** y pasa de verdad.

⚠ **Advertencia:** NUNCA construyas SQL concatenando strings con datos del usuario. Es como dejar las llaves puestas con un cartel “PASE USTED”.

PreparedStatement al Rescate

```
String sql = "SELECT * FROM alumnos WHERE nombre = ?";
PreparedStatement pstmt = conexion.prepareStatement(sql);
pstmt.setString(1, nombre);
ResultSet rs = pstmt.executeQuery();
```

Los ? son placeholders posicionales (empiezan en 1):

Tipo	Método
String	setString(i, valor)
int	setInt(i, valor)
double	setDouble(i, valor)
boolean	setBoolean(i, valor)
Date	setDate(i, valor) (usa java.sql.Date)
null	setNull(i, Types.TIPO)

💡 **Consejo:** Para muchas consultas iguales, PreparedStatement puede ser MÁS RÁPIDO porque la BD compila la consulta una sola vez.

La Historia de Bobby Tables

```
Madre: "He criado a mi hijo para que sea un programador cuidadoso,
       no un hacker que robe datos."
Hijo: "Hola, me llamo Robert'); DROP TABLE Students; --"
```

No seas Bobby Tables. Usa PreparedStatement.

Guía de Placeholders

```
String sql = "SELECT * FROM alumnos WHERE curso = ? AND edad > ? ORDER BY nombre";
try (PreparedStatement pstmt = conexion.prepareStatement(sql)) {
    pstmt.setString(1, "DAM");
    pstmt.setInt(2, 18);
    try (ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) {
            System.out.println(rs.getString("nombre"));
        }
    }
}
```

¿Ves qué limpio? No hay concatenación, no hay comillas escapadas, no hay riesgo de inyección. Solo ? limpios y ordenados.

★ BE THE CODE, MY FRIEND: Tú Eres un PreparedStatement

💡 **Don Tip:** PreparedStatement previene inyección SQL porque separa el código SQL de los datos. Nunca concatenes strings en SQL.

```
Java: Oye, necesito ejecutar "SELECT * FROM productos WHERE precio < ? AND
categoria = ?"
Tú: Dame los valores.
Java: setDouble(1, 50.0); setString(2, "informatica")
Tú: Voy a la BD. Le paso la consulta ya montada y escapada. Sin riesgo.
BD: Aquí tienes los resultados.
Tú: Toma, Java. Un ResultSet limpito.
```

Patrón DAO

El **DAO (Data Access Object)** encapsula el acceso a la base de datos. Separamos la lógica de negocio del SQL.

La Interfaz

```
public interface ContactoDAO {  
    void insertarContacto(Contacto c);  
    List<Contacto> obtenerTodos();  
    List<Contacto> buscarPorNombre(String nombre);  
    boolean actualizarContacto(Contacto c);  
    boolean eliminarContacto(int id);  
}
```

La Implementación

```
public class ContactoDAOImpl implements ContactoDAO {  
    private static final String URL = "jdbc:sqlite:contactos.db";  
  
    @Override  
    public void insertarContacto(Contacto c) { ... }  
  
    @Override  
    public List<Contacto> obtenerTodos() { ... }  
  
    @Override  
    public List<Contacto> buscarPorNombre(String nombre) { ... }  
  
    @Override  
    public boolean actualizarContacto(Contacto c) { ... }  
  
    @Override  
    public boolean eliminarContacto(int id) { ... }  
}
```

Los métodos son exactamente los que viste en la sección CRUD. El DAO los unifica bajo una misma interfaz.



Nota: DAO vs Repository: son casi lo mismo. DAO está más pegado a la BD (INSERT, SELECT). Repository es más de dominio (guardar, buscar). En proyectos pequeños se usan como sinónimos.

Transacciones

A veces necesitas que varias operaciones ocurran todas juntas o ninguna. Como una transferencia bancaria.

commit y rollback

```
public void transferirContacto(int idOrigen, int idDestino) {
    String quitar = "UPDATE contactos SET grupo = 'vacio' WHERE id = ?";
    String poner = "UPDATE contactos SET grupo = 'destino' WHERE id = ?";

    try (Connection con = DriverManager.getConnection(URL)) {
        con.setAutoCommit(false); // empezamos transacción

        try (PreparedStatement q = con.prepareStatement(quitar);
            PreparedStatement p = con.prepareStatement(poner)) {

            q.setInt(1, idOrigen);
            q.executeUpdate();

            if (idOrigen == idDestino)
                throw new SQLException("Destino inválido");

            p.setInt(1, idDestino);
            p.executeUpdate();

            con.commit(); // todo bien, confirmamos
            System.out.println("Transferencia OK");

        } catch (SQLException e) {
            con.rollback(); // algo falló, deshacemos todo
            System.err.println("Falló, todo deshecho: " + e.getMessage());
        }
    } catch (SQLException e) {
        System.err.println("Error de conexión: " + e.getMessage());
    }
}
```

⚠ Advertencia: Sin transacciones, si falla la segunda operación, la primera ya se ejecutó. El sistema queda inconsistente. Usa transacciones para operaciones atómicas (todo o nada). Como un puente que o está entero o no está.

Savepoints

Puedes marcar puntos intermedios dentro de una transacción para no tener que deshacer todo si algo falla:

```
con.setAutoCommit(false);
Savepoint sp = con.setSavepoint("despuesInsert");

// ... más operaciones ...

if (algoMal) {
    con.rollback(sp); // vuelve al savepoint, no deshace todo
} else {
    con.commit();
}
```

★ BE THE CODE, MY FRIEND: Implementa un DAO Completo Desde Cero

💡 **Don Tip:** Un DAO encapsula el acceso a datos. Si cambias de SQLite a MySQL, solo cambias el DAO, el resto del código ni se entera.

Eres el arquitecto de una aplicación de gestión de biblioteca. Tienes estas entidades:

```
class Libro {
    String isbn;
    String titulo;
    String autor;
    boolean disponible;
}

class Prestamo {
    int id;
    String isbn;
    String socio;
    LocalDate fechaPrestamo;
    LocalDate fechaDevolucion;
}
```

Preguntas:

1. ¿Cuántos DAOs crearías? ¿Uno por entidad (LibroDAO, PrestamoDAO) o uno solo genérico?
2. ¿Qué métodos tendría cada DAO?
3. ¿Dónde pondrías la lógica de “un libro no se puede prestar si ya está prestado”?
4. ¿Usarías transacciones para gestionar préstamos y devoluciones?

Usuario: Quiero añadir a "María".

Tú: PreparedStatement → INSERT → executeUpdate() → 1 fila.

Usuario: Búscame a María.

Tú: PreparedStatement → SELECT LIKE → ResultSet → List<Contacto>.

Usuario: Cambia el teléfono de María.

Tú: PreparedStatement → UPDATE WHERE id = ? → 1 fila.

Usuario: Borra a María.

Tú: ¿Seguro?

Usuario: Sí.

Tú: DELETE WHERE id = ? → 1 fila. Adiós, María.

Usuario: ¡Era broma!

Tú: El DELETE no tiene deshacer. Lo siento.



EL RING: Statement vs PreparedStatement

Dos formas de ejecutar SQL discuten.

Statement: «Yo soy el original. Sencillo, directo. stmt.executeQuery("SELECT * FROM usuarios"). ¡Para consultas simples soy perfecto!»

PreparedStatement: «Sí, pero concatenar strings en SQL es una bomba de relojería. Inyección SQL, errores de sintaxis con comillas... Yo separo el SQL de los datos con ? y soy seguro.»

Statement: «Para una consulta fija, una sola vez, ¿para qué voy a preparar nada?»

PreparedStatement: «Además, yo cacheo el plan de ejecución. Si ejecutas la misma consulta varias veces con distintos parámetros, soy más rápido. Y en Java, casi siempre ejecutas la misma consulta con distintos valores.»

Statement: «Vale, pero yo valgo para DDL: CREATE TABLE, ALTER...»

PreparedStatement: «Cierto. Para DDL usa Statement. Para DML (SELECT, INSERT, UPDATE, DELETE) usa PreparedStatement. ¿Trato?»

Statement: «Trato.»

💡 **Don Tip:** Usa siempre PreparedStatement para consultas con datos de usuario. No es solo seguridad: es más rápido en consultas repetitivas y más legible.

¡No Hay Preguntas Tontas!

? ¡No Hay Preguntas Tontas!

Q: ¿Qué pasa si no cierro la conexión? **A:** Se queda abierta, consume memoria. Los servidores tienen un límite de conexiones simultáneas. Si llegas al límite, el próximo `getConnection()` casca con “Too many connections”. Cierra siempre.

Q: ¿Es necesario `Class.forName()` para cargar el driver? **A:** Desde Java 6 no hace falta. Pero lo verás en tutoriales antiguos. Si lo pones no pasa nada, si no lo pones tampoco.

Q: ¿PreparedStatement es más lento que Statement? **A:** Para una consulta, igual. Para muchas iguales, PreparedStatement puede ser MÁS RÁPIDO porque la BD compila la consulta una vez.

Q: ¿Puedo usar `?` para nombres de tabla o columna? **A:** No. `SELECT * FROM ?` no funciona. Los placeholders solo valen para valores, no para identificadores.

Q: ¿Statement se usa todavía? **A:** Está zombi. Si ves un Statement en código real (que no sea una query literal), es bandera roja. PreparedStatement SIEMPRE.

Q: ¿DAO y Repository son lo mismo? **A:** Casi. DAO está pegado a la BD (INSERT, SELECT). Repository es más de dominio (guardar, buscar). En proyectos pequeños se usan como sinónimos.

Q: ¿Debo poner la URL de la BD en el código? **A:** No. Usa `config.properties` o variables de entorno. Las credenciales no van en el código. Es como llevar la contraseña del banco escrita en la frente.

Q: ¿Y si cambio de BD, tengo que reescribir todo? **A:** Si usaste SQL estándar y JDBC, solo cambias el driver y la URL. Esa es la magia de JDBC.

Q: ¿Puedo compartir una conexión entre varios hilos? **A:** Técnicamente sí, prácticamente no. Connection no es thread-safe. Usa un pool de conexiones (HikariCP) y cada hilo que pida la

Ejercicios Propuestos

1. **Conexión desde cero** — Crea una clase `TestConexion` que se conecte a una base de datos SQLite `test.db`, cree una tabla `alumnos(id INTEGER PRIMARY KEY, nombre TEXT, nota REAL)` e inserte 3 alumnos. Verifica que los datos están insertados.
2. **CRUD de estudiantes** — Implementa un DAO completo para `Estudiante` con: `insertar`, `listar`, `buscarPorId`, `actualizar`, `eliminar`. Usa `PreparedStatement` en todos.
3. **Transacción bancaria** — Simula una transferencia entre dos cuentas en una tabla `cuentas(id, titular, saldo)`. La transferencia debe ser atómica: si falla cualquier paso, haz `rollback()`.
4. **Consulta con JOIN** — Crea dos tablas relacionadas (`pedidos` y `clientes`). Escribe una consulta que muestre el nombre del cliente y el total de sus pedidos usando `JOIN`. Devuelve una lista de objetos `PedidoCliente`.
5. **Exportar a CSV** — Añade un método `exportarCSV(String archivo)` a tu DAO que lea todos los registros y los escriba en formato CSV con `try-with-resources`.
6. **DAO genérico** — Refactoriza tu DAO para que sea genérico: `public abstract class DAO<T>`. Implementa `EstudianteDAO extends DAO<Estudiante>`. ¿Cuánto código has reutilizado?

Buenas Prácticas: El Decálogo del JDBC

1. **PreparedStatement siempre.** Nunca concatenes SQL.
2. **Try-with-resources.** `Connection`, `Statement`, `ResultSet` se cierran solos.
3. **No devuelvas ResultSet.** Devuelve listas de objetos.
4. **Captura SQLException con mensaje descriptivo.** No atrapes `Exception` a lo loco.
5. **Usa transacciones para operaciones múltiples.** Todo o nada.
6. **No expongas credenciales en el código.** Usa archivos de configuración.
7. **Comprueba filas afectadas.** `executeUpdate()` te dice si funcionó.
8. **No hagas consultas dentro de bucles.** Lentísimo. Una sola consulta basta.

9. WHERE siempre en UPDATE y DELETE. O pagas las consecuencias.

10. Confirma antes de borrar. El usuario siempre se equivoca.



EL ACERTIJO

Tienes dos tablas en SQLite: usuarios(id, nombre, email) y pedidos(id, usuario_id, total).

Quieres obtener todos los usuarios que han hecho algún pedido con total superior a 100€.

¿Cuántas consultas SQL necesitas como mínimo? ¿Y si usas JOIN?

Pista: hay una diferencia entre “varias consultas en bucle” y “una consulta con JOIN”.

💡 **Don Tip:** Una consulta con JOIN es UNA sola llamada a la base de datos. Hacer una consulta por cada usuario en un bucle es N+1 consultas, que es mucho más lento.

Resumen Exprés

Concepto	Analogía
JDBC	Traductor universal Java ↔ BD
Driver	El enchufe específico para cada BD
Connection	El cable telefónico
PreparedStatement	El mensajero seguro
ResultSet	La respuesta (tabla virtual)
executeQuery	Para SELECT
executeUpdate	Para INSERT/UPDATE/DELETE
commit / rollback	Transacciones (todo o nada)
DAO	Patrón que separa SQL del negocio

Siempre:

1. Abre conexión
2. Crea PreparedStatement
3. Ejecuta consulta

4. Procesa resultados
5. Cierra todo (try-with-resources)

RAs trabajados en esta unidad:

- **RA9** - Bases de datos relacionales



Sergi Garcia Barea — CC BY-SA 4.0



Unidad 13: Servir y Consumir APIs con Web

Objetivos de aprendizaje

- Entender el modelo petición-respuesta HTTP
- Crear un servidor web mínimo con `HttpServer`
- Servir páginas HTML desde Java
- Procesar formularios y parámetros GET/POST
- Devolver JSON y consumirlo con JavaScript
- Consumir APIs externas con `java.net.http.HttpClient`
- Parsear JSON y manejar errores HTTP
- Conocer frameworks modernos (Javalin, Spring Boot)

Hola, Mundo Web

“Hasta ahora tus programas vivían en la terminal. Es hora de que salgan a Internet. Sin JavaScript frameworks. Sin servidores de aplicaciones. Solo Java y HTTP.”

Hemos visto consola, ficheros y hasta bases de datos. Ahora toca lo que hoy en día hace casi cualquier aplicación: **hablar por HTTP**. Y sí, Java puede ser servidor web sin instalar Tomcat ni Spring. Pero además, también puede **consumir** APIs externas como un cliente más.

1. El Protocolo HTTP en 30 Segundos

Cuando escribes `https://google.com` en tu navegador, ocurre esto:



HTTP es un protocolo de **petición-respuesta**. Tú pides una URL y el servidor te devuelve un recurso (HTML, JSON, imagen, etc.). Eso es todo. El resto (cookies, sesiones, APIs) son capas que se construyen encima.

2. Servidor Web Mínimo con HttpServer

Java incluye desde la versión 6 un servidor HTTP básico en el paquete `com.sun.net.httpserver`. No necesitas nada más:

```
import com.sun.net.httpserver.HttpServer;
import com.sun.net.httpserver.HttpExchange;
import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;

public class ServidorMinimo {
    public static void main(String[] args) throws IOException {
        HttpServer server = HttpServer.create(
            new InetSocketAddress(8080), 0
        );
        server.createContext("/", intercambio -> {
            String resp = "Hola, Mundo Web!";
            intercambio.sendResponseHeaders(200, resp.length());
            OutputStream os = intercambio.getResponseBody();
            os.write(resp.getBytes());
            os.close();
        });
        server.setExecutor(null);
        server.start();
        System.out.println("Servidor en http://localhost:8080");
    }
}
```

Compila y ejecuta. Abre `http://localhost:8080` en tu navegador. Acabas de crear tu primer servidor web en Java.

3. Sirviendo HTML

Devolver texto plano está bien para probar, pero queremos HTML. Vamos a servir una página completa:

```

server.createContext("/", intercambio -> {
    String html = ""
        <!DOCTYPE html>
        <html>
        <head><title>Mi App Java</title>
        <meta charset="UTF-8">
        </head>
        <body>
            <h1>Bienvenido a mi App</h1>
            <p>Esto se sirve desde Java.</p>
        </body>
        </html>
        "";
    intercambio.getResponseHeaders()
        .set("Content-Type", "text/html; charset=UTF-8");
    intercambio.sendResponseHeaders(200, html.getBytes().length);
    intercambio.getResponseBody().write(html.getBytes());
    intercambio.getResponseBody().close();
});

```

También puedes leer HTML desde un fichero con `Files.readString()` y servirlo igual. Así tienes el HTML separado del código Java.

4. Parámetros GET: El Cliente Pregunta

Cuando un formulario se envía por GET, los datos van en la URL: `http://localhost:8080/saludo?nombre=Ana`. Así se leen:

```
server.createContext("/saludo", intercambio -> {
    String query = intercambio.getRequestURI().getQuery();
    String nombre = "Mundo";
    if (query != null && query.startsWith("nombre=")) {
        nombre = query.split("=")[1];
    }
    String html = "<h1>Hola, " + nombre + "!</h1>";
    intercambio.getResponseHeaders()
        .set("Content-Type", "text/html");
    intercambio.sendResponseHeaders(200, html.getBytes().length);
    intercambio.getResponseBody().write(html.getBytes());
    intercambio.getResponseBody().close();
});
```

⚠ WARNING: Esto es frágil. Si el nombre contiene & o %20, petará. En producción usa `URLDecoder.decode()` y un parser de query params de verdad. Esto es un ejemplo didáctico, no código de producción.

5. Formularios POST: El Cliente Envía Datos

Para recibir datos de un formulario POST necesitas leer el cuerpo de la petición:

```
server.createContext("/procesar", intercambio -> {
    if ("POST".equals(intercambio.getRequestMethod())) {
        byte[] body = intercambio.getRequestBody().readAllBytes();
        String datos = new String(body);
        // datos = "nombre=Ana&edad=25"
        String nombre = extraerParametro(datos, "nombre");
        String respuesta = "<h1>Recibido: " + nombre + "</h1>";
        intercambio.getResponseHeaders()
            .set("Content-Type", "text/html");
        intercambio.sendResponseHeaders(200,
            respuesta.getBytes().length);
        intercambio.getResponseBody()
            .write(respuesta.getBytes());
        intercambio.getResponseBody().close();
    }
});
```

Y el HTML del formulario:

```
<form action="/procesar" method="POST">
  <input name="nombre" placeholder="Tu nombre">
  <button>Enviar</button>
</form>
```



Consejo: ¿Por qué `"POST".equals(intercambio.getRequestMethod())` y no `intercambio.getRequestMethod().equals("POST")`? Porque si `getRequestMethod()` devuelve `null`, la segunda forma casca con `NullPointerException`. La primera forma es **null-safe**. Es una manía que te ahorrará dolores de cabeza.

6. Devolviendo JSON (Como una API de Verdad)

Las aplicaciones modernas no devuelven HTML directamente. El frontend (JavaScript) pide datos y el backend se los da en JSON. Aquí un ejemplo:

```
server.createContext("/api/usuarios", intercambio -> {
    String json = ""
        [
            {"id":1,"nombre":"Ana","edad":25},
            {"id":2,"nombre":"Luis","edad":30}
        ]
        "";
    intercambio.getResponseHeaders()
        .set("Content-Type", "application/json");
    intercambio.sendResponseHeaders(200, json.getBytes().length);
    intercambio.getResponseBody().write(json.getBytes());
    intercambio.getResponseBody().close();
});
```

Y desde el HTML puedes llamarlo con JavaScript:

```
<script>
fetch('/api/usuarios')
  .then(r => r.json())
  .then(data => console.log(data));
</script>
```

7. Mini Proyecto: Gestor de Tareas (API REST)

Combinando todo, puedes construir una API REST completa. El patrón es:

Método	Ruta	Acción
GET	/api/tareas	Listar todas
POST	/api/tareas	Crear una
PUT	/api/tareas/1	Actualizar
DELETE	/api/tareas/1	Borrar

Cada ruta se implementa como un contexto en el servidor, y los datos se guardan en un `ArrayList` en memoria (más adelante, en base de datos).

8. Consumir APIs Externas con `HttpClient`

Hasta ahora has sido el **servidor**. Pero en el mundo real, tus programas también serán **clientes** que llaman a APIs de terceros: GitHub, OpenAI, el tiempo, tu red social favorita...

Java 11 trae `java.net.http.HttpClient` — un cliente HTTP moderno, sin dependencias externas, que soporta HTTP/2, peticiones síncronas y asíncronas, y manejo de cabeceras.

 **Nota:** Antes de Java 11 tenías que usar `HttpURLConnection` (feo, verboso, un castigo) o librerías externas como Apache `HttpClient` u `OkHttp`. `java.net.http.HttpClient` llegó para salvar la humanidad.

8.1 GET: Pedir Datos a una API

Empecemos por lo básico: hacer una petición GET a una API pública y leer la respuesta como texto.

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class ClienteGET {
    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient();

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://api.github.com/users/octocat"))
            .GET()
            .build();

        HttpResponse<String> response = client.send(
            request, HttpResponse.BodyHandlers.ofString()
        );

        System.out.println("Código: " + response.statusCode());
        System.out.println("Cuerpo: " + response.body());
    }
}
```

 **Nota:** `HttpResponse.BodyHandlers.ofString()` le dice a Java “conviérteme el cuerpo de la respuesta en un String”. Hay otros handlers: `ofByteArray()`, `ofInputStream()`, `ofFile(Path)`... según lo que necesites.

8.2 Parsear JSON con la Librería que Toque

El cuerpo que devuelve GitHub es JSON. En Java no hay un parser JSON nativo (sí, es triste, pero es así). Necesitas una librería. Las más usadas:

Librería	Grupo Maven	Ideal para
Gson (Google)	<code>com.google.code.gson:gson</code>	Sencilla, mapeo a clases
Jackson	<code>com.fasterxml.jackson.core:jackson-databind</code>	Potente, rápida, estándar industrial
org.json	<code>org.json:json</code>	La más simple, sin mapeo

En los ejemplos usaremos **Gson**, que es la más intuitiva:

```
import com.google.gson.Gson;
import com.google.gson.JsonObject;

// ... después de obtener response.body()
Gson gson = new Gson();
JsonObject json = gson.fromJson(response.body(), JsonObject.class);

String login = json.get("login").getAsString();
String nombre = json.get("name").getAsString();
System.out.println("Usuario: " + login + " - " + nombre);
```

O si prefieres mapear a una clase:

```
record UsuarioGitHub(String login, String name, int public_repos) {}

UsuarioGitHub usuario = gson.fromJson(response.body(), UsuarioGitHub.class);
System.out.println(usuario.name() + " tiene " + usuario.public_repos() + " repos
públicos");
```

 **WARNING:** Si la API devuelve campos que no existen en tu record, Gson por defecto los ignora. Si tu record tiene campos que no están en el JSON, se quedan con valor null. Con Jackson puedes configurarlo con `@JsonIgnoreProperties(ignoreUnknown = true)`.

8.3 POST: Enviar Datos a una API

Para enviar datos (crear un recurso, hacer login, etc.) usamos POST con un cuerpo JSON:

```

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class ClientePOST {
    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient();

        String json = """
            {"title": "Aprender HttpClient",
            "body": "Ya casi lo tengo",
            "userId": 1}
            """;

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://jsonplaceholder.typicode.com/posts"))
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json))
            .build();

        HttpResponse<String> response = client.send(
            request, HttpResponse.BodyHandlers.ofString()
        );

        System.out.println("Código: " + response.statusCode());
        System.out.println("Creado: " + response.body());
    }
}

```

💡 **Consejo:** `HttpRequest.BodyPublishers` tiene métodos para enviar `String`, `byte[]`, archivos, etc. El más común es `ofString()` para JSON. Si no pones el header `Content-Type: application/json`, el servidor puede rechazar tu petición o interpretar mal el cuerpo.

8.4 Cabeceras, Timeouts y Manejo de Errores

Las APIs no siempre se portan bien. A veces tardan, a veces se caen, a veces te devuelven 401, 403, 404 o 500. Tu código debe estar preparado.

```

import java.net.http.HttpConnectTimeoutException;
import java.time.Duration;

HttpClient client = HttpClient.newBuilder()
    .connectTimeout(Duration.ofSeconds(10)) // tiempo máximo para conectar
    .build();

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.github.com/users/someone"))
    .header("Accept", "application/vnd.github+json")
    .header("User-Agent", "MiAppJava/1.0")
    .timeout(Duration.ofSeconds(15)) // tiempo máximo para recibir respuesta
    .GET()
    .build();

try {
    HttpResponse<String> response = client.send(request,
        HttpResponse.BodyHandlers.ofString());

    int codigo = response.statusCode();
    if (codigo >= 200 && codigo < 300) {
        System.out.println("Éxito: " + response.body());
    } else if (codigo == 404) {
        System.out.println("El recurso no existe.");
    } else if (codigo == 403) {
        System.out.println("Prohibido. ¿Token necesario?");
    } else if (codigo >= 500) {
        System.out.println("Error del servidor. Vuelve a intentarlo.");
    }
} catch (HttpConnectTimeoutException e) {
    System.out.println("El servidor no responde (timeout de conexión).");
} catch (java.net.http.HttpTimeoutException e) {
    System.out.println("La respuesta tardó demasiado.");
} catch (java.io.IOException e) {
    System.out.println("Error de red: " + e.getMessage());
} catch (InterruptedException e) {
    System.out.println("La petición fue interrumpida.");
}

```

⚠ WARNING: Muchas APIs públicas (GitHub, Twitter, etc.) requieren un **token de autenticación** en la cabecera Authorization. Sin él, tienes cuotas muy bajas (rate limiting) o

acceso denegado. Si ves un 403, probablemente necesitas registrar una aplicación y obtener un token.

Aquí tienes las cabeceras más comunes que enviarás:

Cabecera	Propósito
Content-Type	Tipo de datos que envías (application/json)
Accept	Tipo de datos que esperas recibir
Authorization	Token Bearer, Basic Auth, etc.
User-Agent	Identifica tu aplicación (muchas APIs lo exigen)
Cache-Control	Control de caché

8.5 Ejemplo Completo: Consultar Repos de GitHub

Vamos a juntarlo todo: consumir la API de GitHub para listar los repositorios de un usuario.

```

import com.google.gson.Gson;
import com.google.gson.JsonArray;
import com.google.gson.JsonObject;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;

public class GitHubRepos {
    public static void main(String[] args) throws Exception {
        String usuario = args.length > 0 ? args[0] : "octocat";

        HttpClient client = HttpClient.newBuilder()
            .connectTimeout(Duration.ofSeconds(10))
            .build();

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://api.github.com/users/" + usuario + "/repos"))
            .header("Accept", "application/vnd.github+json")
            .header("User-Agent", "JavaCliente/1.0")
            .timeout(Duration.ofSeconds(15))
            .GET()
            .build();

        HttpResponse<String> response = client.send(request,
            HttpResponse.BodyHandlers.ofString());

        if (response.statusCode() == 200) {
            Gson gson = new Gson();
            JsonArray repos = gson.fromJson(response.body(), JsonArray.class);

            System.out.println("Repositorios de " + usuario + ":");
            System.out.println("_____");
            for (int i = 0; i < repos.size(); i++) {
                JsonObject repo = repos.get(i).getAsJsonObject();
                String nombre = repo.get("name").getString();
                String lenguaje = repo.has("language")
                    && !repo.get("language").isJsonNull()
                    ? repo.get("language").getString()
                    : "?";
                int estrellas = repo.get("stargazers_count").getAsInt();
            }
        }
    }
}

```

```

        System.out.printf("★ %s (%s) ★ %d%n", nombre, lenguaje, estrellas);
    }
} else if (response.statusCode() == 403) {
    System.out.println("Rate limit alcanzado. Espera un minuto.");
} else {
    System.out.println("Error " + response.statusCode()
        + " - ¿existe el usuario " + usuario + "?");
}
}
}
}

```

 **Nota:** Fíjate en `repo.has("language") && !repo.get("language").isJsonNull()`. En JSON un campo puede existir pero valer null. Si intentas `getAsString()` sobre un null, Gson te lanza una excepción. Este patrón es muy común al parsear APIs reales.

Ejemplo de salida:

Repositorios de octocat:

```

★ Hello-World (Java) ★ 2727
★ Spoon-Knife (HTML) ★ 13291
★ Octocat (?) ★ 5

```

8.6 Llamadas Asíncronas con `sendAsync`

Si necesitas hacer varias peticiones sin bloquear el programa, `HttpClient` también soporta modo asíncrono:


```
CompletableFuture<HttpResponse<String>> futuro = client.sendAsync(  
    request, HttpResponse.BodyHandlers.ofString()  
);  
  
// Mientras tanto, puedes hacer otras cosas...  
System.out.println("Esperando respuesta...");  
  
// Cuando llegue:  
futuro.thenAccept(response -> {  
    System.out.println("Código: " + response.statusCode());  
    System.out.println(response.body());  
});  
  
// O bloquear hasta que llegue (no recomendado si vas justo de tiempo):  
HttpResponse<String> resp = futuro.get();
```

💡 **Consejo:** sendAsync devuelve un CompletableFuture, que es la forma elegante de Java para trabajar con código asíncrono sin liarte con hilos manuales. Puedes encadenar varias llamadas con thenApply, thenCompose, etc.

8.7 PUT, DELETE y Otros Métodos

HttpClient soporta todos los métodos HTTP. La diferencia es la llamada al builder:

```
// PUT
HttpRequest putRequest = HttpRequest.newBuilder()
    .uri(URI.create("https://api.ejemplo.com/recursos/1"))
    .header("Content-Type", "application/json")
    .PUT(HttpRequest.BodyPublishers.ofString(
        "{\"nombre\": \"Actualizado\"}"))
    .build();

// DELETE
HttpRequest deleteRequest = HttpRequest.newBuilder()
    .uri(URI.create("https://api.ejemplo.com/recursos/1"))
    .DELETE()
    .build();

// Con método personalizado (PATCH, etc.)
HttpRequest patchRequest = HttpRequest.newBuilder()
    .uri(URI.create("https://api.ejemplo.com/recursos/1"))
    .method("PATCH", HttpRequest.BodyPublishers.ofString(
        "{\"nombre\": \"Parcial\"}"))
    .build();
```

9. Frameworks Modernos

El `HttpServer` básico está bien para aprender, pero en el mundo real se usan frameworks:

- **Javalin:** Minimalista, moderno, ideal para APIs. Con 3 líneas tienes un servidor.
- **Spring Boot:** El estándar industrial. Tiene todo, pero es más pesado.
- **Spark:** Similar a Javalin, muy ligero.

Ejemplo con Javalin:

```
import io.javalin.Javalin;

public class App {
    public static void main(String[] args) {
        Javalin app = Javalin.create().start(8080);
        app.get("/", ctx -> ctx.result("Hola desde Javalin"));
        app.get("/api/usuarios", ctx ->
            ctx.json(List.of(new Usuario(1, "Ana"))));
    }
}
```

 **Consejo:** Para los boletines de esta unidad usaremos el `HttpServer` nativo de Java como servidor y `HttpClient` para consumir APIs. No necesitas instalar nada más que Gson (o la librería JSON que prefieras). Los frameworks los verás en el módulo de “Desarrollo de Interfaces” y en el proyecto final.

★ BE THE CODE, MY FRIEND: Sigue la Pista de una Petición HTTP

 **Don Tip:** El método HTTP lo determina el cliente. GET para obtener, POST para enviar. El `exchange.getRequestMethod()` te dice cuál es.

Abre el `HttpServer` que creaste en la sección 2. Añade esto justo antes de `server.start()`:

```
server.createContext("/track", exchange -> {
    System.out.println("Método: " + exchange.getRequestMethod());
    System.out.println("URI: " + exchange.getRequestURI());
    System.out.println("Cabeceras: " + exchange.getRequestHeaders().entrySet());
    exchange.getResponseHeaders().add("Content-Type", "text/plain");
    exchange.sendResponseHeaders(200, 0);
    try (var os = exchange.getResponseBody()) {
        os.write("Todo ok, gracias por preguntar.".getBytes());
    }
});
```

Preguntas:

1. Abre `http://localhost:8080/track?nombre=Ana&edad=25` — ¿Qué ves en la consola del servidor?

2. ¿Qué método HTTP has usado sin especificarlo? (Pista: el navegador hace GET por defecto)
3. ¿Qué pasa si envías una petición POST desde curl o desde JavaScript? Pruébalo con:

```
curl -X POST http://localhost:8080/track -d "clave=valor"
```

4. **Reto:** Modifica el System.out para que escriba en un archivo access.log en lugar de la consola. Cada petición debe aparecer en una línea nueva con formato: [MÉTODO] URI → 200 OK

? ¡No Hay Preguntas Tontas!

¿HTTP y HTTPS son lo mismo? — Casi. HTTPS es HTTP con una capa de cifrado (SSL/TLS). Los datos viajan encriptados. Para producción siempre HTTPS; para desarrollo local, HTTP basta.

¿Qué puerto usar? — 8080 es el estándar para desarrollo. 80 es el puerto HTTP por defecto (requiere permisos de administrador). 443 es para HTTPS. Evita puertos < 1024 en desarrollo.

¿Qué es CORS y por qué me da error? — Cross-Origin Resource Sharing. El navegador bloquea peticiones de un dominio a otro por seguridad. Para desarrollo, añade esta cabecera en tu servidor: `exchange.getResponseHeaders().add("Access-Control-Allow-Origin", "*")`.

¿Gson o Jackson? — Gson es más sencillo para empezar. Jackson es más rápido y potente. Para este curso, Gson es suficiente. Si usas Jackson, la sintaxis es muy parecida.

Mi servidor no arranca: "Address already in use" — Otro proceso tiene el puerto ocupado. Cambia el puerto o mata el proceso anterior. En terminal: `netstat -ano | findstr 8080` y luego `taskkill /PID <numero> /F`.

¿Puedo servir páginas web normales (HTML+CSS+JS)? — Sí. Pon los archivos en una carpeta `web/` dentro de `resources/` y sívelos con `Files.readString()` desde el contexto. O usa `exchange.sendResponseHeaders(200, file.length())` para archivos binarios.

¿El HttpServer es como Tomcat? — No, HttpServer es mínimo y didáctico. Tomcat es un servidor de aplicaciones completo que soporta Servlets, JSP, etc. Aquí usamos lo justo para entender el protocolo.

¿Necesito una base de datos para la API? — No. Puedes tener una lista en memoria (ArrayList). Al reiniciar el servidor los datos se pierden, pero es perfecto para aprender. Luego añades SQLite si quieres persistencia.

Resumen

Concepto	Analogía
HTTP	El idioma que hablan cliente y servidor
GET	Pedir información
POST	Enviar información
PUT	Reemplazar información
DELETE	Borrar información
Status 200	Todo bien
Status 404	No encontrado
Status 500	Error interno
JSON	Formato de datos universal
HttpServer	Tu puerta al mundo web
HttpClient	Tu ventana a otras APIs

Flujo básico de una API REST:

1. Cliente hace una petición HTTP (GET, POST, PUT, DELETE)
 2. Servidor procesa la petición
 3. Servidor devuelve JSON (o HTML, o texto)
 4. Cliente parsea la respuesta y hace algo con ella
-

Ejercicios Propuestos

1. **API de frases célebres** — Crea un servidor con un endpoint `/frase` que devuelva una frase aleatoria de una lista en memoria. Añade `/frases` que devuelva todas.
2. **Contador de visitas** — Cada vez que alguien visita `/contador`, el servidor incrementa un contador y devuelve el número actual como JSON: `{"visitas": 42}`.
3. **API de tareas (CRUD completo)** — Implementa los 4 endpoints para una lista de tareas:
 - GET `/tareas` — lista todas
 - POST `/tareas` — crea una (recibe JSON con `{ "titulo": "..."}`)
 - PUT `/tareas/{id}` — actualiza una
 - DELETE `/tareas/{id}` — borra una
4. **Consumir y transformar** — Usa `HttpClient` para consultar la API de GitHub y mostrar solo los nombres de los repositorios de un usuario. Luego guarda los resultados en un archivo `repos.txt`.
5. **Proxy simple** — Crea un endpoint `/proxy?url=...` que reciba una URL, haga una petición GET a esa URL y devuelva el contenido. ¡Como un proxy inverso casero!
6. **Mini chat en tiempo real (simulado)** — Usa `/enviar` (POST) y `/mensajes` (GET) para simular un chat. Los mensajes se guardan en una lista en memoria. El frontend pregunta cada 2 segundos si hay mensajes nuevos (polling).

RAs trabajados en esta unidad:

- **RA5** — Desarrolla interfaces gráficas (ahora web) siguiendo el modelo vista-controlador y el estilo de código abierto.



Sergi Garcia Barea — CC BY-SA 4.0

Boletín U01 — Inicial Resuelto



Boletín U01 — Inicial (Resuelto)

Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

Ejercicio 1: Desordena esto

>  Solución

Ejercicio 2: ¿Qué imprime?

>  Solución

Ejercicio 3: Cazador de errores

>  Solución

Ejercicio 4: Tu ficha personal

>  Solución

Ejercicio 5: Completa el programa

>  Solución

Ejercicio 6: Empareja conceptos

>  Solución

Ejercicio 7: CodeWars — Square(n) Sum

>  Solución

Ejercicio 8: El detective de errores

>  Solución

Ejercicio 9: Tu biografía

>  Solución

Boletín U01 — Inicial



Boletín U01 — Inicial

Sin soluciones. Sin prisas. Con un editor de texto y muchas ganas de compilar. Esto apenas empieza.

Ejercicio 1: Desordena esto

Las líneas de este programa están desordenadas. Ordénalas para formar un programa Java válido que compile y se ejecute.

```
}  
  
    public static void main(String[] args) {  
        System.out.println("Mi primer programa ordenado");  
public class Ordenado {  
    }
```

Ejercicio 2: ¿Qué imprime?

Sin ejecutar, escribe la salida exacta de este programa:

```
public class Escapista {  
    public static void main(String[] args) {  
        System.out.print("Dijo: \"Java mola\"");  
        System.out.println(" y siguió: \tprogramando.");  
    }  
}
```

Pista: \" imprime una comilla literal, \t es un tabulador.

Ejercicio 3: Cazador de errores

Este código tiene **4 errores**. Encuéntralos y corrígelos.

```
Public class ErrorFinder {  
    public static void main(string[] args) {  
        System.out.println("Hola, "Mundo");  
        System.out.println("Esto funciona")  
    }  
}
```

Ejercicio 4: Tu ficha personal

Escribe un programa llamado `FichaPersonal` que muestre:

```
Nombre: [Tu nombre]  
Edad: [Tu edad]  
Lenguaje favorito: Java  
¿Emocionado?: true
```

Usa una línea `println` para cada campo y asegúrate de que los booleanos no lleven comillas.

Ejercicio 5: Completa el programa

Falta una línea crucial y un par de caracteres. Añádelos para que compile y muestre “Aprobado, esto funciona”.

```
public class Completame {  
    public static void main(String[] args) {  
        System.out.println("Aprobado, esto funciona")  
    }  
}
```

Ejercicio 6: Empareja conceptos

Relaciona cada concepto de la izquierda con su definición de la derecha:

Concepto

Definición

1. `class`

A. Punto de entrada del programa

Concepto	Definición
2. main	B. Imprime texto y salta de línea
3. System.out.println	C. Define un nuevo tipo de datos
4. //	D. Comentario de una línea
5. args	E. Contiene los argumentos de línea de comandos

Escribe las respuestas como "1→C, 2→A, ..."

Ejercicio 7: CodeWars — Square(n) Sum

Resuelve la kata "Square(n) Sum" (8 kyu) en [CodeWars](#).

Dado un array de números, eleva cada uno al cuadrado y suma los resultados. Por ejemplo, [1, 2, 2] → 1 + 4 + 4 = 9.

Ejercicio 8: El detective de errores

El siguiente código tiene 2 errores que impiden que compile. Encuéntralos y corrígelos:

```
public class Detective {  
    public static void main(String[] args) {  
        System.out.println("Soy un detective")  
        System.out.println("y resuelvo errores");  
    }  
}
```

Escribe la versión corregida. Después, ejecútala y comprueba que funciona.

Ejercicio 9: Tu biografía

Escribe un programa llamado Biografia que muestre:

- Tu nombre

- Tu edad
- Tu lenguaje de programación favorito
- Una frase que te motive

Cada cosa en una línea. Usa **una sola** instrucción `System.out.println` con `\n` para los saltos.

Boletín U01 — Avanzado Resuelto



Boletín U01 — Avanzado (Resuelto)

Las soluciones están ocultas. Inténtalo de verdad antes de destaparlas.

★ Ejercicio 1: ASCII art con prints

>  Solución

★ Ejercicio 2: Sin ejecutar — secuencias de escape

>  Solución

★ ★ Ejercicio 3: El reloj de milisegundos

>  Solución

★ ★ Ejercicio 4: Contador de argumentos

>  Solución

★ ★ ★ Ejercicio 5: La edad cósmica

>  Solución

★ ★ ★ Ejercicio 6: CodeWars — Grasshopper - Summation

>  Solución



Ejercicio 7: AceptaElReto — 119

Futbolistas

>  Solución



Ejercicio 8: Javadoc de campeonato

>  Solución



Referencias

Plataforma	Problema	Dificultad
AceptaElReto	116 — ¡Hola mundo!	Principiante
AceptaElReto	119 — Futbolistas	Fácil
AceptaElReto	114 — Último dígito del factorial	Medio
CodeWars	Square(n) Sum (8 kyu)	Principiante
CodeWars	Grasshopper - Summation (8 kyu)	Principiante

Boletín U01 — Avanzado



Boletín U01 — Avanzado

Dificultad progresiva. ★ para calentar, ★ ★ para pensar, ★ ★ ★ para concursar. Cada ejercicio incluye una pista (resiste a mirarla).

★ Ejercicio 1: ASCII art con prints

Escribe un programa que dibuje esta figura usando combinaciones de `System.out.print` y `System.out.println`:

```
*  
***  
*****  
***  
*
```

Debes usar exactamente **5 líneas de código** (una por cada fila), combinando `print` y `println` sin usar bucles.

Pista: el `print` no salta de línea y el `println` sí. Piensa qué fila debe empezar nueva línea y cuál no.

★ Ejercicio 2: Sin ejecutar — secuencias de escape

¿Qué imprime exactamente este programa? Escribe la salida carácter por carácter.

```
public class EscapeRoom {  
    public static void main(String[] args) {  
        System.out.println("Java\n\tmola\n\"mucho\"");  
        System.out.println("C:\\carpeta\\archivo.java");  
    }  
}
```

Pista: `\n` salta de línea, `\t` tabula, `\"` imprime una comilla y `\\` imprime una barra invertida. Cuenta cada carácter del texto entre comillas como lo leería la máquina, no como lo escribes.

☆☆ Ejercicio 3: El reloj de milisegundos

`System.currentTimeMillis()` devuelve el número de milisegundos desde el 1 de enero de 1970 (la “epoch” de Unix). Escribe un programa que:

1. Capture el momento actual con `long inicio = System.currentTimeMillis();`
2. Haga una pausa artificial (un bucle que cuente hasta 100.000.000 para perder tiempo)
3. Capture el momento después con `long fin = System.currentTimeMillis();`
4. Muestre cuántos milisegundos han pasado

Pista: la resta `fin - inicio` te da los milisegundos transcurridos. No te preocupes si el tiempo varía cada vez: depende de la velocidad de tu ordenador.

☆☆ Ejercicio 4: Contador de argumentos

Escribe un programa llamado `ContadorArgs` que reciba argumentos desde la línea de comandos y muestre:

- Cuántos argumentos se recibieron
- El primer argumento (si existe)
- El último argumento (si existe)

Si no se reciben argumentos, debe mostrar: “No se recibieron argumentos. Programa cancelado por falta de datos.”

Ejemplo de ejecución:

```
> java ContadorArgs hola mundo cruel
Argumentos recibidos: 3
Primer argumento: hola
Último argumento: cruel
```

Pista: el último argumento es `args[args.length - 1]` (porque el primer elemento es el índice 0). Comprueba antes que `args.length > 0`.

☆☆☆ Ejercicio 5: La edad cósmica

La Tierra tarda 365.25 días en orbitar el Sol. Mercurio tarda 87.97 días. Escribe un programa que, usando constantes final:

1. Declare final double DIAS_TIERRA = 365.25;
2. Declare final double DIAS_MERCURIO = 87.97;
3. Almacene en una variable int edadTerrestre = 20 (tu edad en años terrestres)
4. Calcule los años que tendrías en Mercurio (divide los días terrestres vividos entre los días de Mercurio)
5. Muestre: "En la Tierra tengo X años. En Mercurio tendría Y años."

Para calcular los días vividos en la Tierra: `diasVividos = edadTerrestre * DIAS_TIERRA`.

Pista: los años en Mercurio serán un double (el resultado de dividir dos double). La división `diasVividos / DIAS_MERCURIO` te lo da.

☆☆☆ Ejercicio 6: CodeWars — Grasshopper - Summation

Resuelve la kata "Grasshopper - Summation" (8 kyu) en [CodeWars](#).

Suma todos los números del 1 hasta n. Si n = 4, devuelve 1+2+3+4 = 10.

Pista: hay una fórmula matemática $(n * (n + 1) / 2)$, pero también puedes hacerlo con un bucle aunque no lo hayamos visto oficialmente. El método recibirá un int n y deberá devolver un int.

☆☆☆ Ejercicio 7: AceptaElReto — 119 Futbolistas

Resuelve el problema 119 — Futbolistas en [AceptaElReto.com](#).

Lee los minutos que juega cada futbolista y determina cuántos partidos completos (90 minutos) ha jugado cada uno.

Pista: cada caso de prueba termina cuando aparece un -1. Necesitarás leer números hasta encontrar el marcador de fin y sumar los minutos de cada futbolista.

☆☆☆ Ejercicio 8: Javadoc de campeonato

Escribe una clase `SobreMi` totalmente documentada:

1. Clase con Javadoc (`@author`, `@version`).
2. Método `main` con Javadoc (`@param args`).
3. Dentro del `main`, un comentario de una línea que explique *por qué* (no *qué*) imprimes tu motivación.
4. Un comentario de bloque que explique qué hace tu programa a grandes rasgos.
5. Imprime tu nombre, tu motivación y cuántos argumentos recibiste.

Pista: recuerda que los comentarios Javadoc van justo **antes** de la clase y del método. Genera la documentación con `javadoc SobreMi.java` y abre los `.html` que se generen.

Referencias

Plataforma	Problema	Dificultad
AceptaElReto	116 — ¡Hola mundo!	Principiante
AceptaElReto	119 — Futbolistas	Fácil
AceptaElReto	114 — Último dígito del factorial	Medio
CodeWars	Square(n) Sum (8 kyu)	Principiante
CodeWars	Grasshopper - Summation (8 kyu)	Principiante

Boletín U01 — Extras



Boletín U01 — Extras

★ **Extras** — CodeWars y AceptaElReto cogen ritmo a partir de la unidad 3, cuando tengamos suficientes conceptos para enfrentarnos a ellos con garantías. De momento, repasa los ejercicios iniciales y afianza lo aprendido. ¡Pronto llegarán los retos de verdad!

Boletín 2 – Inicial Resuelto: Variables y Operadores

Aquí están las soluciones. Pero solo míres si ya lo has intentado. La memoria muscular se construye equivocándose.

Ejercicio 1: Declara el tipo correcto

1. `int` — la edad cabe en un `int` (2.147 millones es el límite, 150 no da miedo)
2. `double` (o `float`) — los precios tienen decimales
3. `long` — 8.000.000.000 no cabe en un `int` (2.147.483.647 es el tope)
4. `boolean` — solo dos valores: `true` o `false`
5. `char` — una sola letra con comillas simples
6. `double` — decimales para la temperatura

 **Explicación:** Cada tipo tiene un tamaño y un propósito. Usar `int` para todo es como llevar siempre un camión por si acaso tienes que mudarte. `byte` para la edad, `short` para la población de un pueblo, `int` para casi todo, `long` para cosas astronómicas, `double` para decimales, `boolean` para sí/no, `char` para una letra. La clave es elegir el tipo adecuado: ni un microbus para dos personas, ni un Smart para una familia de 5.

Ejercicio 2: ¿Qué imprime?

```
public class Operaciones {  
    public static void main(String[] args) {  
        int a = 10;  
        int b = 3;  
        System.out.println(a / b);  
        System.out.println(a % b);  
        System.out.println((double) a / b);  
    }  
}
```

Salida:

```
3
1
3.3333333333333335
```

💡 **Explicación:** $10 / 3$ con enteros da 3 (se truncan los decimales). $10 \% 3$ da 1 (el resto de la división). `(double) a / b` convierte a a double (10.0) y entonces la división da 3.333... El `(double)` delante de a es un *casting*: le dices a Java “confía en mí, trata esto como decimal aunque sea entero”. Es como ponerle gafas a un miope: de repente ve los decimales.

Ejercicio 3: Encuentra el error

```
public class Errores {
    public static void main(String[] args) {
        int 1numero = 10;           // ERROR 1
        float precio = 19.99;       // ERROR 2
        System.out.println(1numero + precio);
    }
}
```

Corrección:

```
public class Errores {
    public static void main(String[] args) {
        int numero1 = 10;
        float precio = 19.99f;
        System.out.println(numero1 + precio);
    }
}
```

💡 **Explicación:** Error 1: las variables no pueden empezar con número. `1numero` es ilegal. `numero1` es legal. Es como las matrículas de los coches: pueden acabar en número pero no empezar con él. Error 2: los decimales en Java son double por defecto. Para asignarlos a un float necesitas añadir `f` al final: `19.99f`. Sin la `f`, Java se queja porque estás metiendo un double en una caja float y podría perderse precisión. Es como intentar meter una botella grande en un vaso pequeño: algo se derramará.

Ejercicio 4: El intercambio

```
public class Intercambio {  
    public static void main(String[] args) {  
        int a = 5;  
        int b = 10;  
  
        System.out.println("Antes: a = " + a + ", b = " + b);  
  
        int temp = a;  
        a = b;  
        b = temp;  
  
        System.out.println("Después: a = " + a + ", b = " + b);  
    }  
}
```

Salida:

```
Antes: a = 5, b = 10  
Después: a = 10, b = 5
```

💡 **Explicación:** Necesitas una variable temporal `temp` para no perder el valor de `a`. Si haces `a = b` directamente, pierdes el 5. Es como cuando quieres intercambiar el contenido de dos vasos: necesitas un tercer vaso vacío. Si vertieras el de vino en el de agua sin vaciar antes el de agua, tendrías vino con agua. La variable temporal es ese tercer vaso.

Ejercicio 5: Escribe este programa

```
public class AreaCirculo {  
    public static void main(String[] args) {  
        final double PI = 3.1416;  
        double radio = 7.5;  
  
        double area = PI * radio * radio;  
        double perimetro = 2 * PI * radio;  
  
        System.out.println("Área: " + area);  
        System.out.println("Perímetro: " + perimetro);  
    }  
}
```

Salida:

```
Área: 176.715  
Perímetro: 47.124
```

💡 **Explicación:** `final` hace que `PI` sea constante. No podrás cambiarla después. Intenta poner `PI = 4;` después y el compilador te saltará al cuello. El área del círculo es `PI` por radio al cuadrado. El perímetro (o circunferencia) es 2 por `PI` por radio. `Math.PI` existe como constante más precisa (3.141592653589793), pero aquí usamos la nuestra. La gracia de `final` es que le dices a Java y a otros programadores: “esto no se toca, ni aunque venga el mismísimo Bill Gates a pedirte”.

Ejercicio 6: AceptaElReto 149 — San Fermín

```
import java.util.Scanner;

public class Problema149 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        while (sc.hasNextInt()) {
            int n = sc.nextInt();
            int max = 0;
            for (int i = 0; i < n; i++) {
                int vel = sc.nextInt();
                if (vel > max) max = vel;
            }
            System.out.println(max);
        }
    }
}
```

💡 **Explicación:** El problema de San Fermín te da varias líneas, cada una con un número N seguido de N velocidades. Debes imprimir la velocidad máxima de cada caso. Usamos `while (sc.hasNextInt())` porque no sabemos cuántas líneas hay. Leemos N, luego N números con un bucle, y nos quedamos con el máximo. El truco está en que `max` empieza en 0 (las velocidades son positivas). Si hubiera velocidades negativas, habría que empezar con `Integer.MIN_VALUE`. Pero en San Fermín todos corren hacia adelante, no como algunos que corren hacia atrás.

Ejercicio 7: CodeWars — Even or Odd

```
public class EvenOrOdd {
    public static String evenOrOdd(int number) {
        return number % 2 == 0 ? "Even" : "Odd";
    }
}
```

💡 **Explicación:** El operador `%` te da el resto de la división. Si un número es par, `numero % 2` da 0. Si es impar, da 1. El ternario `? :` es un if-else en una línea: condición ? valorSiTrue : valorSiFalse. Devuelve “Even” si el resto es 0, “Odd” si no. Una línea. Elegante. Como un esmoquin para tu código. También podrías hacerlo con if-else, pero el ternario queda más limpio. CodeWars premia la elegancia, no las 15 líneas para una tontería.

Boletín 2 – Inicial: Variables y Operadores

Sin soluciones. Abre el IDE, declara variables y ensucia tus manos de bits. Nadie nace sabiendo qué es un `double`.

Ejercicio 1: Conversor de temperaturas

Escribe un programa llamado `ConversorTemperatura` que convierta 30 grados Celsius a Fahrenheit. Usa la fórmula:

$$F = C * 9/5 + 32$$

Declara `int celsius = 30` y una variable `double fahrenheit` para el resultado. Muestra ambas temperaturas.

Ejercicio 2: ¿Qué imprime?

Sin ejecutar, escribe la salida exacta:

```
public class Incrementos {
    public static void main(String[] args) {
        int x = 5;
        System.out.println(x++);
        System.out.println(++x);
        System.out.println(x--);
        System.out.println(--x);
        System.out.println(x);
    }
}
```

Ejercicio 3: Calculadora de descuentos

Declara una constante `final double DESCUENTO = 0.15` y declara `double precioOriginal = 120.0`. Calcula:

1. El descuento (`precioOriginal * DESCUENTO`)

2. El precio final (`precioOriginal - descuento`)

Muestra todo con `println`. ¿Te animas a declarar el descuento como `final`? Si luego intentas cambiarlo, el compilador se enfadará.

Ejercicio 4: Precedencia de pesadilla

Sin ejecutar, ¿qué valor tiene `resultado`?

```
int resultado = 2 + 3 * 4 - 8 / 2 + (6 - 1) * 2;
```

Escribe el paso a paso y el valor final. No hay trampa, solo matemáticas y orden de operaciones.

Ejercicio 5: El tipo perfecto

Indica qué tipo de dato primitivo (`int`, `double`, `boolean`, `char`, `long`) usarías para cada caso:

1. El número de habitantes de tu ciudad (~500.000)
 2. La distancia en kilómetros hasta la luna (~384.400)
 3. La inicial de tu segundo apellido
 4. La nota media de un examen (3.7)
 5. Si has aprobado o no el examen anterior
 6. El precio de un café en céntimos (entero)
-

Ejercicio 6: El intercambio mágico

Dadas dos variables `int a = 7`; e `int b = 3`;, intercambia sus valores **sin usar una variable temporal**. Pista: usa suma y resta:

```
a = a + b;  
b = a - b;  
a = a - b;
```

Muestra los valores antes y después para comprobar que funcionó.

Ejercicio 7: CodeWars — Will you make it?

Resuelve la kata “Will you make it?” (8 kyu) en [CodeWars](#).

Te dan la distancia hasta una gasolinera, los litros que tiene tu coche y los kilómetros por litro. Determina si llegas o no. Devuelve `true` si llegas, `false` si te quedas tirado.

Boletín 2 – Resuelto: Variables y Operadores

De menos a más. De ★ a ★★ ★★. De “esto es pan comido” a “esto ya no es un bocadillo”.

★ Ejercicio 1: ¿Par o impar?

Pide un número al usuario y determina si es par o impar usando el operador %.

💡 **Explicación:** `num % 2` te da el resto de dividir el número entre 2. Si es 0, es par. Si es 1, es impar. No hay misterio. Es la operación más usada en programación después de sumar. El % sale en todas partes: para saber si algo es par, para ciclos, para juegos, para relojes. Sin él, no habría videojuegos (bueno, sí, pero serían más difíciles de programar).

★ Ejercicio 2: Constante del mal

Declara `final double PRECIO_BASE = 100;` e `final double IVA = 0.21`. Calcula el precio final. Luego intenta modificar IVA después. ¿Qué error da?

💡 **Explicación:** Al descomentar `IVA = 0.10`, el compilador dice: “No puedo asignar un valor a una variable final”. `final` es como un contrato firmado: no puedes cambiar de opinión. Es útil para valores que no deben cambiar nunca: el IVA, el nombre de tu app, el número de intentos máximos. Si alguien intenta cambiarlos, el compilador se planta. Mejor que discutir con un programador que cambia el IVA a mitad de facturación.

★★ Ejercicio 3: Dado trucado

Genera 10 números aleatorios entre 1 y 6. Cuenta cuántos 6 han salido y muestra el resultado.

💡 **Explicación:** `Math.random()` devuelve un número entre 0.0 y 0.999999... Multiplicas por 6, obtienes entre 0.0 y 5.999999. El `(int)` trunca los decimales y te da 0-5. Sumas 1 y obtienes 1-6. El bucle se repite 10 veces. Cada vez que sale un 6, incrementas el contador. Es como si lanzaras un dado físico 10 veces y anotaras cuántos 6 sacas. La probabilidad es baja (1/6 por tirada), así que lo normal es que salgan entre 1 y 2 seises. Si te salen 7, el dado está trucado (o tienes una suerte increíble).

★ ★ Ejercicio 4: Año bisiesto

Pide un año al usuario. Determina si es bisiesto: divisible entre 4 Y (no entre 100 O sí entre 400).

💡 **Explicación:** La regla del año bisiesto es: divisible entre 4, pero si es divisible entre 100 NO es bisiesto, a menos que también sea divisible entre 400. 1900 no fue bisiesto (divisible entre 100 pero no entre 400). 2000 sí (divisible entre 400). 2024 sí (divisible entre 4 pero no entre 100). Los operadores lógicos && (AND) y || (OR) combinados con % permiten expresar esta regla en una sola línea. Es como la lija fina de la programación: condiciones precisas para resultados exactos. Los paréntesis son importantes: sin ellos, la precedencia podría dar un resultado incorrecto.

★ ★ Ejercicio 5: Nombre al revés

Pide al usuario su nombre. Muestra: longitud, versión en mayúsculas, primera letra y última letra.

💡 **Explicación:** `length()` devuelve cuántos caracteres tiene el String. `toUpperCase()` lo convierte a mayúsculas. `charAt(0)` devuelve el primer carácter (los Strings empiezan en 0, como los arrays). `charAt(nombre.length() - 1)` devuelve el último. Fíjate en que los métodos de String se llaman SOBRE el objeto, no son estáticos. El objeto es `nombre`, y le preguntas: “oye, tú, ¿cuánto mides?”. String es una clase con muchos métodos útiles. Aprende a usarlos y te ahorrarás reinventar la rueda cada dos por tres.

★ ★ ★ Ejercicio 6: AceptaElReto 152 — Números de pares

Resuelve el problema **152 — Números de pares** (también conocido como “Va de modas...”) en AceptaElReto.

Dada una lista de números, determina si la cantidad de pares es mayor que la de impares.

💡 **Explicación:** El problema va leyendo casos hasta encontrar un 0 (que marca el final). Cada caso empieza con N (cuántos números vienen). Luego leemos N números. Por cada uno, miramos si es par o impar con `% 2` y contamos. Al final comparamos los contadores. Es una versión ampliada del ejercicio 1, pero con lectura de datos y múltiples casos. La gracia

está en manejar correctamente la lectura cuando no sabes cuántos casos hay. while (sc.hasNextInt()) y break cuando ves un 0.

☆☆☆ Ejercicio 7: El acertijo del ++

Sin ejecutar, determina el resultado de:

```
int a = 2;
int b = a++ * 3 + --a;
System.out.println("a = " + a + ", b = " + b);
```

Solución:

```
a = 2, b = 7
```

Paso a paso:

1. $a = 2$
2. $a++ \rightarrow$ POST: usa a (2), luego $a = 3$. La expresión $a++$ vale 2.
3. $--a \rightarrow$ PRE: a vale 3, decrementa a 2, luego usa a (2). Vale 2.
4. $b = 2 * 3 + 2 = 6 + 2 = 7$
5. a quedó en 2 (subió a 3 con $a++$, bajó a 2 con $--a$)

💡 **Explicación:** $a++$ (post-incremento) primero USA el valor y luego incrementa. $--a$ (pre-decremento) primero decrementa y luego USA. Por eso el valor final de a es 2 (subió y bajó como un yoyó). Y b es 7. Estos acertijos son el terror de los exámenes y la alegría de los profesores malvados. Mi consejo: en la vida real, no mezcles $++$ y $--$ en expresiones complicadas. Úsalos en líneas separadas. Tu yo del futuro te lo agradecerá.

☆☆☆ Ejercicio 8: AceptaElReto 140 — Suma de dígitos

Resuelve el problema **140 — Suma de dígitos** en AceptaElReto.

Dado un número, suma sus dígitos. Luego suma los dígitos del resultado, y así hasta que quede un solo dígito. Muestra el proceso.

💡 **Explicación:** Para extraer los dígitos de un número, usamos $n \% 10$ (obtenemos el último dígito) y $n / 10$ (quitamos el último dígito). Repetimos hasta que n sea 0. Sumamos todos los dígitos. Si la suma tiene más de un dígito (≥ 10), repetimos el proceso. Es como cuando doblas un papel una y otra vez hasta que no puedes más: el número se va reduciendo hasta un solo dígito. Ejemplo: $123 \rightarrow 1+2+3 = 6$. $987 \rightarrow 9+8+7 = 24 \rightarrow 2+4 = 6$. Es la “raíz digital” de un número. Muy usado en numerología y en exámenes de programación.



Referencias

Plataforma	Problema	Dificultad
AceptaElReto	149 — San Fermín	Fácil
AceptaElReto	152 — Números de pares	Fácil
AceptaElReto	140 — Suma de dígitos	Medio
CodeWars	Even or Odd (8 kyu)	Principiante
CodeWars	Opposite number (8 kyu)	Principiante

Boletín 2 - Intermedio: Variables y Operadores

De menos a más. De ★ a ★★ ★★. De variables sencillas a operaciones que te harán bizco.

★ Ejercicio 1: Calculadora de propinas

Escribe un programa que calcule cuánto dejar de propina en un restaurante. Declara:

- `double totalCuenta = 45.50;`
- `int porcentajePropina = 15;` (porcentaje, sin el símbolo)

Calcula la propina (`totalCuenta * porcentajePropina / 100`) y el total final (`totalCuenta + propina`). Muestra todo con 2 decimales aproximados.

Salida esperada:

```
Total cuenta: 45.5€
Propina (15%): 6.825€
Total a pagar: 52.325€
```

★ Ejercicio 2: Conversor dólar-euro

Declara final `double TASA_CAMBIO = 0.92;` (1 dólar = 0.92 euros). Declara `double dolares = 100.0;` y calcula su equivalente en euros. También haz la conversión inversa: dado `double euros = 50.0;`, calcula cuántos dólares son.

Muestra:

```
100.0$ son 92.0€
50.0€ son 54.34782608695652$
```

★★ Ejercicio 3: ¿Qué imprime? — el casting traidor

Sin ejecutar, escribe la salida exacta:

```
public class CastingTraidor {
    public static void main(String[] args) {
        int a = 7;
        int b = 2;
        double resultado1 = a / b;
        double resultado2 = (double) a / b;
        double resultado3 = a / (double) b;

        System.out.println(resultado1);
        System.out.println(resultado2);
        System.out.println(resultado3);
        System.out.println(3 + 4 * 2.0);
        System.out.println((int) (3.7 + 2.3));
    }
}
```

Fíjate bien en dónde está el casting y en qué momento se aplica la división entera.

★ ★ Ejercicio 4: Interés compuesto (sin bucle)

Declara `final double CAPITAL_INICIAL = 1000.0;`, `final double TASA = 0.05;` (5% anual) e `int años = 3;`. Calcula el capital final después de 3 años usando la fórmula del interés compuesto SIN bucles:

```
capitalFinal = capitalInicial * (1 + tasa)^años
```

Para la potencia, usa `Math.pow(base, exponente)`. Muestra el capital año a año:

```
Año 0: 1000.0€
Año 1: 1050.0€
Año 2: 1102.5€
Año 3: 1157.625€
```

Para mostrar cada año, necesitarás calcular `Math.pow(1 + TASA, i)` para cada valor de `i` de 1 a 3... pero sin bucle. Crea tres variables distintas.



Ejercicio 5: El enigma del post-incremento

Sin ejecutar, determina el valor de cada variable después de ejecutar este código. Escribe el paso a paso.

```
public class EnigmaIncremento {  
    public static void main(String[] args) {  
        int x = 3;  
        int y = x++ + ++x;  
        int z = --y + y-- + x++;  
        System.out.println("x = " + x);  
        System.out.println("y = " + y);  
        System.out.println("z = " + z);  
    }  
}
```

Pista: haz una tabla con los valores de x, y después de cada operación. x++ usa x y luego incrementa; ++x incrementa y luego usa x.



Ejercicio 6: CodeWars — Convert boolean values to strings 'Yes' or 'No'

Resuelve la kata "Convert boolean values to strings 'Yes' or 'No'" (8 kyu) en [CodeWars](https://www.codewars.com/kata/convert-boolean-values-to-strings-yes-or-no).

Completa el método `public static String boolToWord(boolean b)` que devuelva "Yes" si recibe true y "No" si recibe false. Se puede hacer en una línea con el operador ternario.



Ejercicio 7: AceptaElReto — 114 Último dígito del factorial

Resuelve el problema 114 — Último dígito del factorial en [AceptaElReto.com](https://www.acepta-el-reto.com/problems/114-ultimo-digito-del-factorial).

Dado un número N ($0 \leq N \leq 1.000.000$), calcula el último dígito de N! (factorial de N). Pista: no necesitas calcular el factorial entero. ¿Qué pasa con el último dígito cuando multiplicas por 10? ¿Y cuándo $N \geq 5$? Observa que $5! = 120$, $6! = 720$... a partir de 5, el factorial siempre termina en 0.



Referencias

Plataforma	Problema	Dificultad
AceptaElReto	114 — Último dígito del factorial	Fácil
AceptaElReto	140 — Suma de dígitos	Medio
AceptaElReto	152 — Números de pares	Fácil
CodeWars	Even or Odd (8 kyu)	Principiante
CodeWars	Opposite number (8 kyu)	Principiante
CodeWars	Convert boolean to Yes/No (8 kyu)	Principiante

Boletín 02 — Extras: Variables, Tipos de Datos y Operadores

★ **Extras** — CodeWars y AceptaElReto empiezan a partir de la unidad 3, cuando tengamos suficientes conceptos para enfrentarnos a ellos con garantías. De momento, repasa los ejercicios iniciales y afianza lo aprendido. ¡Pronto llegarán los retos de verdad!

Boletín 3 – Inicial Resuelto: Estructuras de Control

Las soluciones están aquí. Pero solo las necesitas si ya has sudado un poco. El sudor programa mejor que las soluciones.

Ejercicio 1: ¿Qué imprime este if-else?

```
public class QueImprime {  
    public static void main(String[] args) {  
        int edad = 17;  
        boolean conPadres = true;  
  
        if (edad >= 18) {  
            System.out.println("Entrada libre");  
        } else if (conPadres) {  
            System.out.println("Pasas con tus viejos");  
        } else {  
            System.out.println("A casa");  
        }  
    }  
}
```

Respuesta: “Pasas con tus viejos”

 **Explicación:** edad >= 18 es false (tiene 17), así que no entra al primer if. Luego evalúa conPadres que es true, por lo tanto entra al else if e imprime “Pasas con tus viejos”. El else final no se ejecuta nunca porque ya entró en el else if. Java evalúa las condiciones en orden y se queda con la PRIMERA que sea true. Como un semáforo: si el primero está verde, pasas; si no, miras el siguiente. No te saltas ninguno, pero tampoco miras más allá del que ya está verde.

Ejercicio 2: Completa el switch

```

int dia = 3;
String nombreDia;

switch (dia) {
    case 1:
        nombreDia = "Lunes";
        break;
    case 2:
        nombreDia = "Martes";
        break;
    case 3:
        nombreDia = "Miércoles";
        break;
    default:
        nombreDia = "Desconocido";
}
System.out.println(nombreDia);

```

Con breaks: Imprime “Miércoles”.

Sin breaks (fall-through): Imprime “Miércoles”. Pero si `dia = 1`, sin breaks imprimiría “LunesMartesMiércoles” (¡todo seguido!). Con `dia = 2` sin breaks: “MartesMiércoles”.

💡 **Explicación:** `break` es la instrucción de “salir del switch”. Si no lo pones, el código “se cae” al siguiente case (fall-through). Es como si las escaleras no tuvieran descansillos: bajas de un tirón hasta el final. En nuestro ejemplo, con `dia = 3`, el `break` del case 3 frena la caída. Pero en case 1 y case 2, sin `break`, el programa seguiría ejecutando los case siguientes hasta encontrar un `break` o llegar al `default`.

Ejercicio 3: Encuentra el error (bucle infinito)

```

public class BucleInfinito {
    public static void main(String[] args) {
        int i = 1;
        while (i <= 5) {
            System.out.println("Vuelta " + i);
        }
    }
}

```

Error: Falta `i++` dentro del bucle. `i` siempre vale 1, la condición `i <= 5` siempre es `true`.

Corregido:

```
public class BucleCorregido {  
    public static void main(String[] args) {  
        int i = 1;  
        while (i <= 5) {  
            System.out.println("Vuelta " + i);  
            i++;  
        }  
    }  
}
```

💡 **Explicación:** Sin `i++`, la variable de control nunca cambia. Es como un perro persiguiéndose la cola: nunca para. El bucle `while` solo comprueba la condición. Si siempre se cumple, el bucle es eterno. `i++` es el que hace que `i` aumente hasta 6, momento en que `i <= 5` es falso y el bucle termina. Siempre, SIEMPRE, asegúrate de que la variable de control se actualice dentro del bucle. Es la primera regla de los bucles: “no olvides actualizar la condición”.

Ejercicio 4: Escribe este programa (while)

```

import java.util.Scanner;

public class PositivoNegativo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int numero;

        System.out.println("Introduce números (0 para salir:");
        numero = sc.nextInt();
        while (numero != 0) {
            if (numero > 0) {
                System.out.println(numero + " es positivo");
            } else {
                System.out.println(numero + " es negativo");
            }
            numero = sc.nextInt();
        }
        System.out.println("Fin del programa");
        sc.close();
    }
}

```

💡 **Explicación:** Leemos el primer número antes del bucle (esto se llama “lectura anticipada”). Luego, mientras no sea 0, evaluamos si es positivo o negativo, y volvemos a leer. Cuando el usuario introduce 0, el bucle termina. Podríamos usar un do-while para no repetir la lectura, pero así es más claro. Es como un portero que pregunta “¿cuántos años tienes?” hasta que le dices 0 y entonces te deja pasar... espera, eso no tiene sentido. Pero el código sí.

Ejercicio 5: El for de toda la vida

```

public class SumaFor {
    public static void main(String[] args) {
        int suma = 0;
        for (int i = 1; i <= 100; i++) {
            suma += i;
        }
        System.out.println("Suma del 1 al 100: " + suma);
    }
}

```


💡 **Explicación:** El bucle for es perfecto cuando sabes exactamente cuántas veces repetir. Aquí sabemos que vamos de 1 a 100. La variable i empieza en 1, se incrementa de 1 en 1, y en cada vuelta se suma a suma. Al final, suma contiene 5050. Hay una fórmula matemática: $n * (n+1) / 2 = 100 * 101 / 2 = 5050$. Pero en programación, el bucle está bien. La historia cuenta que Gauss descubrió esta fórmula de niño para hacer el cálculo más rápido. Tú estás aprendiendo a programar, no a ser Gauss. Aunque si quieres ser Gauss, también vale.

Ejercicio 6: AceptaElReto 200 — Aburrimiento en las aulas

```
import java.util.Scanner;

public class Problema200 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num = sc.nextInt();
        while (num != 0) {
            int a = 5 * num * num + 4;
            int b = 5 * num * num - 4;
            int raizA = (int) Math.sqrt(a);
            int raizB = (int) Math.sqrt(b);

            if (raizA * raizA == a || raizB * raizB == b) {
                System.out.println("Fibonacci");
            } else {
                System.out.println("No fibonacci");
            }
            num = sc.nextInt();
        }
    }
}
```

💡 **Explicación:** La propiedad matemática dice: un número es parte de la secuencia de Fibonacci si y solo si $(5n^2 + 4)$ o $(5n^2 - 4)$ es un cuadrado perfecto. Para comprobar si algo es cuadrado perfecto, calculamos la raíz cuadrada, la truncamos a entero, y elevamos al cuadrado. Si coincide con el original, es cuadrado perfecto. Ejemplo: para $n=5$: $5*25+4=129$ (no es cuadrado), $5*25-4=121=11^2 \rightarrow$ ¡Fibonacci! Efectivamente, 5 está en la secuencia. Para

$n=4$: $516+4=84$ (no), $516-4=76$ (no) → No fibonacci. El 4 no está. El while sigue leyendo números hasta que se introduce un 0.

Ejercicio 7: CodeWars — Century From Year

```
public class CenturyFromYear {  
    public static int century(int number) {  
        return (number + 99) / 100;  
    }  
}
```

💡 **Explicación:** Para calcular el siglo de un año, sumamos 99 y dividimos entre 100. Año 1: $(1+99)/100 = 1$. Año 100: $(100+99)/100 = 1$. Año 101: $(101+99)/100 = 2$. Año 2000: $(2000+99)/100 = 20$. Año 2001: $(2001+99)/100 = 21$. La lógica: el siglo 1 abarca del año 1 al 100. Al sumar 99, desplazamos el rango para que la división entera funcione. Es un truco matemático que evita usar if o `Math.ceil()`. Simple, elegante, y te hace quedar bien en las cenas de Navidad cuando alguien pregunta “¿en qué siglo estamos?”.

Boletín 3 – Inicial: Estructuras de Control

Sin soluciones. Respira. Los bucles no muerden. Las excepciones tampoco. El único peligro eres tú sin dormir.

Ejercicio 1: ¿Qué imprime este if?

Sin ejecutar, escribe la salida exacta:

```
public class Clasificador {  
    public static void main(String[] args) {  
        int nota = 65;  
  
        if (nota >= 90) {  
            System.out.println("Sobresaliente");  
        } else if (nota >= 70) {  
            System.out.println("Notable");  
        } else if (nota >= 50) {  
            System.out.println("Aprobado");  
        } else {  
            System.out.println("Suspenso");  
        }  
    }  
}
```

Ejercicio 2: Encuentra el error en el for

El siguiente bucle debería contar del 1 al 5, pero no funciona. ¿Por qué? Corrígelo.

```
public class ForError {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i--) {  
            System.out.println("Cuenta: " + i);  
        }  
    }  
}
```

Pista: mira la condición y la actualización. Una de las dos no cuadra.

Ejercicio 3: Completa el programa (do-while)

Falta la condición del do-while. Complétala para que el programa pida números hasta que el usuario introduzca un número negativo.

```
import java.util.Scanner;

public class DoWhile {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int numero;

        do {
            System.out.print("Introduce un número (negativo para salir): ");
            numero = sc.nextInt();
            System.out.println("Has escrito: " + numero);
        } while (/* tu condición aquí */);

        System.out.println("¡Hasta luego!");
        sc.close();
    }
}
```

Ejercicio 4: Cuenta atrás

Escribe un programa que muestre una cuenta atrás desde 10 hasta 0 usando un for. Después del 0, debe imprimir "¡Despegue!".

Salida esperada:

```
10
9
8
...
1
0
¡Despegue!
```

Ejercicio 5: Menú con switch

Escribe un programa que muestre un menú de bebidas y pida al usuario que elija una opción (1-4). Usa switch para mostrar el nombre de la bebida seleccionada.

```
=== BAR JAVA ===  
1. Café solo  
2. Café con leche  
3. Té  
4. Refresco  
Elige una opción:
```

Si el usuario elige 2, debe mostrar “Has elegido Café con leche”. Si elige un número inválido, muestra “Opción no válida”.

Ejercicio 6: Tabla de multiplicar del 7

Escribe un programa que muestre la tabla de multiplicar del 7 usando un for. Debe imprimir:

```
7 x 1 = 7  
7 x 2 = 14  
...  
7 x 10 = 70
```

Ejercicio 7: CodeWars — Grasshopper - Grade book

Resuelve la kata “Grasshopper - Grade book” (8 kyu) en [CodeWars](https://www.codewars.com/kata/525c65e56fb710269f000004).

Recibes tres notas (0-100). Devuelve la letra correspondiente según la media:

Media	Nota
≥ 90	'A'
≥ 80	'B'

Media	Nota
≥ 70	'C'
≥ 60	'D'
< 60	'F'

Boletín 3 – Resuelto: Estructuras de Control

Dificultad progresiva. Los ★ te calientan, los ★ ★ te hacen pensar, los ★ ★ ★ te preparan para el mundo real (y para ProgramaMe).

★ Ejercicio 1: El clasificador de notas sarcástico

Pide una nota (0-100) y clasifícala: 90+ “Sobresaliente”, 70+ “Notable”, 50+ “Aprobado”, menos “Suspenso”. Captura que la nota sea válida (0-100).

💡 **Explicación:** Primero validamos que la nota esté entre 0 y 100. Luego, con `else if`, evaluamos de mayor a menor. Si la nota es 85, no entra en `>= 90` pero sí en `>= 70`. El orden es crucial: si pusieras `>= 50` antes que `>= 90`, un 95 también entraría en `>= 50`. Siempre ve de lo más específico a lo más general. Como en la vida: primero lo urgente, luego lo importante, luego lo que sea.

★ Ejercicio 2: Tabla de multiplicar

Pide un número y muestra su tabla de multiplicar del 1 al 10 con un `for`.

💡 **Explicación:** El `for` va de 1 a 10. En cada vuelta, multiplica `n` por `i`. Simple, directo. Es como cuando eras pequeño y recitabas la tabla del 7: “7 por 1 es 7, 7 por 2 es 14...”. La diferencia es que ahora el ordenador lo hace por ti y no se equivoca (a menos que tú te equivoques al programarlo, claro).

★ ★ Ejercicio 3: División segura con `try-catch`

Pide dos números enteros. Divide el primero entre el segundo. Captura `ArithmeticException` (división por cero) e `InputMismatchException` (no escribir un número).

💡 **Explicación:** El bloque `try` contiene el código peligroso. Si ocurre un `ArithmeticException` (división por cero) o un `InputMismatchException` (el usuario escribe letras), se ejecuta el `catch` correspondiente. El `finally` se ejecuta siempre, haya o no error. Es como el seguro de vida de tu programa: pase lo que pase, se cierra el `Scanner` y se dice algo. Sin `try-catch`, si el usuario divide por cero, el programa muere con un mensaje rojo de error. Con `try-catch`, el programa sobrevive y sigue adelante como un héroe de acción.

★ ★ Ejercicio 4: El triángulo constructor

Dibuja un triángulo de asteriscos con bucles anidados. El usuario introduce la altura.

```
*  
**  
***  
****  
*****
```

💡 **Explicación:** El bucle exterior (i) controla las filas (de 1 a altura). El bucle interior (j) controla las columnas de cada fila: en la fila 1 imprime 1 asterisco, en la fila 2 imprime 2... Usamos print (sin \n) para que los asteriscos salgan en la misma línea. Al terminar cada fila, hacemos un println vacío para saltar a la siguiente línea. Los bucles anidados son como muñecas rusas: un bucle dentro de otro. El interior da vueltas completas por cada vuelta del exterior.

★ ★ Ejercicio 5: Fibonacci con for

Muestra los primeros N números de Fibonacci. La secuencia empieza: 0, 1, 1, 2, 3, 5, 8, 13...

💡 **Explicación:** Fibonacci es la secuencia donde cada número es la suma de los dos anteriores. Empezamos con a=0, b=1. En cada iteración, calculamos c = a+b, mostramos c, y desplazamos: a pasa a ser b, b pasa a ser c. Es como una carrera de relevos: el valor se pasa de una variable a otra. El bucle empieza en 3 porque ya hemos impreso los dos primeros (0 y 1). Este es el ejercicio clásico de entrevista técnica. Si lo resuelves en la primera entrevista, el entrevistador asiente con la cabeza. Si no lo resuelves, te preguntan “¿y qué tal se te da trabajar en equipo?”

★ ★ ★ Ejercicio 6: AceptaElReto 340 — Juegos de naipes

Resuelve el problema **340 — Juegos de naipes** en AceptaElReto.

Dada una secuencia de cartas representadas por números (1 a 10), determina cuántas veces hay que ordenarlas para que queden en orden ascendente según un algoritmo específico de “colocar la primera al final”.

💡 **Explicación:** El problema simula un juego de cartas donde buscas la carta 1, luego la 2, etc. Vas pasando cartas de la primera a la última posición (como si pasaras de una punta a otra) hasta que encuentras la que buscas. El % n hace que el índice dé la vuelta cuando llega al final (como una ruleta). Es un ejercicio de lógica con arrays y bucles. La dificultad está en entender el algoritmo de búsqueda circular. No te preocupes si no te sale a la primera: los juegos de naipes siempre han sido complicados, incluso para los programadores.

☆☆☆ Ejercicio 7: Excepción personalizada

Crea `EdadInvalidaException` (excepción checked). Lánzala si la edad es menor que 0 o mayor que 150. Crea un programa que la pruebe.

Salida:

```
Error: Edad negativa: -5. ¿Eres viajero en el tiempo?  
Error: 200 años? 0 eres inmortal o me tomas el pelo  
Edad válida: 25
```

💡 **Explicación:** Creamos una excepción personalizada heredando de `Exception`. Le ponemos un constructor que llame a `super(mensaje)` para que el mensaje se guarde. Luego, en `validarEdad`, lanzamos la excepción con `throw` cuando la edad es inválida. Como es checked, quien llame al método debe capturarla con `try-catch` o declararla con `throws`. Es como crear tu propio tipo de error: “esto no es un error cualquiera, es MI error”.

☆☆☆ Ejercicio 8: AceptaElReto 100 — Kaprekar

Resuelve el problema 100 — Kaprekar en AceptaElReto.

Dado un número de 4 dígitos (no todos iguales), ordénalo ascendente y descendentemente, resta, y repite. Siempre llegas a 6174 (la constante de Kaprekar). Cuenta cuántas iteraciones se necesitan.

💡 **Explicación:** Kaprekar descubrió que, para cualquier número de 4 dígitos (no todos iguales), si ordenas los dígitos de mayor a menor, restas el orden inverso, y repites, siempre llegas a 6174 en pocos pasos. El programa extrae los dígitos, los ordena con `Arrays.sort()`, construye el número ascendente y descendente, resta, y repite hasta llegar a 6174. Es como un funnel matemático: todos los caminos llevan a 6174. Este problema es un clásico de las olimpiadas de programación (ProgramaMe). Si lo resuelves, ya puedes poner “experto en Kaprekar” en tu LinkedIn.

Referencias

Plataforma	Problema	Dificultad
AceptaElReto	200 — Aburrimiento en las aulas	Medio
AceptaElReto	340 — Juegos de naipes	Medio
AceptaElReto	100 — Kaprekar	Difícil
AceptaElReto	151 — ¿Es matriz identidad?	Medio
CodeWars	Century From Year (8 kyu)	Principiante
CodeWars	Cat years, Dog years (7 kyu)	Fácil

Boletín 3 – Intermedio: Estructuras de Control

Dificultad progresiva. Los ★ te calientan, los ★★ te hacen pensar, los ★★★ te preparan para el mundo real (y para que el switch no se te atragante).

★ Ejercicio 1: Números pares e impares

Escribe un programa que recorra los números del 1 al 20 y muestre al lado si cada uno es par o impar. Usa un for y el operador % dentro de un if-else.

Salida esperada:

```
1 es impar
2 es par
3 es impar
...
20 es par
```

★ Ejercicio 2: Suma acumulativa

Pide números al usuario (con Scanner) hasta que introduzca un 0. Muestra la suma acumulada después de cada número. Usa un bucle while.

Ejemplo de ejecución:

```
Introduce un número (0 para salir): 5
Suma acumulada: 5
Introduce un número (0 para salir): 3
Suma acumulada: 8
Introduce un número (0 para salir): 0
Suma acumulada final: 8
```

★★ Ejercicio 3: Número perfecto

Un número perfecto es aquel que es igual a la suma de sus divisores propios (excluyéndose a sí mismo). Por ejemplo, $6 = 1 + 2 + 3$. Escribe un programa que pida un número N y determine si es perfecto o no.

Pista: usa un for que vaya de 1 a $N/2$, comprobando si N es divisible entre i ($N \% i == 0$) y sumando los divisores.

★ ★ Ejercicio 4: Menú con validación

Crea un programa que muestre un menú de operaciones y valide que la opción sea correcta. Usa un do-while para el menú principal y un switch para las opciones.

```
=== CALCULADORA RUPESTRE ===  
1. Sumar  
2. Restar  
3. Multiplicar  
4. Salir  
Elige opción (1-4):
```

Si el usuario elige una opción inválida (menor que 1 o mayor que 4), muestra “Opción no válida, intenta de nuevo” y repite el menú. Las opciones 1-3 deben pedir dos números y mostrar el resultado. La opción 4 termina el programa.

★ ★ ★ Ejercicio 5: Criba de Eratóstenes (números primos)

Escribe un programa que pida un número N y muestre todos los números primos desde 2 hasta N usando la **Criba de Eratóstenes**:

1. Crea un array de booleanos con tamaño $N+1$, inicializado a true (todos son primos en teoría)
2. Desde 2 hasta $\sqrt{N}$, si el número es primo, marca como false todos sus múltiplos
3. Al final, imprime los números que siguen siendo true

Ejemplo para $N = 30$:

★ ★ ★ Ejercicio 6: CodeWars — Vowel Count

Resuelve la kata “Vowel Count” (7 kyu) en [CodeWars](#).

Devuelve el número de vocales (a, e, i, o, u) en un String dado. La entrada será solo minúsculas y espacios. Por ejemplo, "hola mundo" tiene 4 vocales.

★ ★ ★ Ejercicio 7: AceptaElReto — 151 ¿Es matriz identidad?

Resuelve el problema 151 — ¿Es matriz identidad? en [AceptaElReto.com](#).

Dada una matriz cuadrada, determina si es la matriz identidad: todos los 1 en la diagonal principal y 0 en el resto. El programa debe leer casos de prueba, cada uno con un tamaño N seguido de N×N números.

Pista: para recorrer la matriz, necesitarás bucles anidados. La diagonal principal son las posiciones donde fila == column.

Referencias

Plataforma	Problema	Dificultad
AceptaElReto	151 — ¿Es matriz identidad?	Medio
AceptaElReto	200 — Aburrimiento en las aulas	Medio
AceptaElReto	340 — Juegos de naipes	Medio
CodeWars	Century From Year (8 kyu)	Principiante
CodeWars	Vowel Count (7 kyu)	Fácil
CodeWars	Cat years, Dog years (7 kyu)	Fácil

Boletín 03 — Extras: Estructuras de Control y Excepciones

★ Extras — Conviértete en un programador de muy alto nivel Estos ejercicios son completamente opcionales, pero si aspiras a ser un programador o programadora de élite, aquí tienes tu campo de entrenamiento. CodeWars y AceptaElReto son plataformas donde los mejores programadores del mundo se retan a diario. Cada problema que resuelvas te hará más fuerte, más rápido y más creativo. No los subestimes: son la diferencia entre saber programar y ser un verdadero profesional.

CodeWars:

- [Even or Odd](#) (8 kyu)

Pista: Usa el operador módulo (%) — si el resto al dividir entre 2 es 0, es par.

- [Count the divisors of a number](#) (7 kyu)

Pista: Solo necesitas iterar hasta $\sqrt{n}$. Por cada divisor i , el par n/i también lo es.

- [Multiples of 3 or 5](#) (6 kyu)

Pista: Aplica el principio de inclusión-exclusión: suma los de 3 y 5, resta los de 15.

AceptaElReto:

- [117 - La fiesta de los números](#) (★ ★)

Pista: Lleva la cuenta de los números que ya han aparecido con un conjunto (HashSet) o un array de booleanos.

- [149 - Puntos de silla](#) (★ ★)

Pista: Calcula primero el mínimo de cada fila y el máximo de cada columna por separado, luego busca coincidencias.

Boletín 4 – Inicial Resuelto: Algorítmica I

Soluciones del boletín inicial. Inténtalo antes de mirar.

★ Ejercicio 1: Búsqueda lineal

```
public static int buscarLineal(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++)
        if (arr[i] == target) return i;
    return -1;
}
```

★ Ejercicio 2: Búsqueda binaria

```
public static int buscarBinaria(int[] arr, int target) {
    int izq = 0, der = arr.length - 1;
    while (izq <= der) {
        int medio = (izq + der) / 2;
        if (arr[medio] == target) return medio;
        if (arr[medio] < target) izq = medio + 1;
        else der = medio - 1;
    }
    return -1;
}
```

★ ★ Ejercicio 3: Burbuja

```
public static void burbuja(int[] arr) {
    for (int i = 0; i < arr.length - 1; i++)
        for (int j = 0; j < arr.length - 1 - i; j++)
            if (arr[j] > arr[j + 1]) {
                int t = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = t;
            }
}
```

☆☆ Ejercicio 4: Inserción

```
public static void insercion(int[] arr) {
    for (int i = 1; i < arr.length; i++) {
        int key = arr[i], j = i - 1;
        while (j >= 0 && arr[j] > key) { arr[j + 1] = arr[j]; j--; }
        arr[j + 1] = key;
    }
}
```

☆☆ Ejercicio 5: Contar pasos burbuja

```
public static int contarPasos(int[] arr) {
    int pasos = 0;
    for (int i = 0; i < arr.length - 1; i++)
        for (int j = 0; j < arr.length - 1 - i; j++) {
            pasos++;
            if (arr[j] > arr[j + 1]) {
                int t = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = t;
            }
        }
    return pasos;
}
```

☆☆☆ Ejercicio 6: Buscar en matriz ordenada

```
public static boolean buscarMatriz(int[][] mat, int target) {
    int fila = 0, col = mat[0].length - 1;
    while (fila < mat.length && col >= 0) {
        if (mat[fila][col] == target) return true;
        if (mat[fila][col] > target) col--;
        else fila++;
    }
    return false;
}
```


Boletín 4 – Inicial: Algorítmica I

Sin soluciones. Coge un array, un bucle y mucha paciencia. Los algoritmos son como las recetas de cocina: si te saltas un paso, el pastel explota.

★ Ejercicio 1: El más grande y el más pequeño

Escribe un método `public static int[] mayorYMenor(int[] arr)` que devuelva un array de 2 enteros: el primero es el valor más grande del array, el segundo el más pequeño.

Ejemplo: `mayorYMenor([3, 7, 2, 9, 4]) → {9, 2}`

★ Ejercicio 2: Suma total

Escribe un método `public static int sumarElementos(int[] arr)` que devuelva la suma de todos los elementos del array.

Ejemplo: `sumarElementos([1, 2, 3, 4, 5]) → 15`

Si el array está vacío, devuelve 0.

★ ★ Ejercicio 3: Buscador elemental

Escribe un método `public static boolean contiene(int[] arr, int valor)` que devuelva `true` si `valor` está en el array, o `false` si no.

No vale usar `Arrays.asList(arr).contains()`. Hazlo con un bucle.

★ ★ Ejercicio 4: Invertir array

Escribe un método `public static void invertir(int[] arr)` que modifique el array original invirtiendo el orden de sus elementos. No vale crear un array nuevo.

Ejemplo: `[1, 2, 3, 4, 5] → [5, 4, 3, 2, 1]`

Pista: intercambia el primero con el último, el segundo con el penúltimo...

★ ★ Ejercicio 5: Media de doubles

Escribe un método `public static double calcularMedia(double[] arr)` que devuelva la media aritmética de los valores.

Ejemplo: `calcularMedia([2.5, 3.5, 4.0])` → 3.333...

Si el array está vacío, devuelve 0.0.

★ ★ ★ Ejercicio 6: CodeWars — Sum without highest and lowest number

Resuelve la kata “Sum without highest and lowest number” (8 kyu) en [CodeWars](#).

Suma todos los números de un array excepto el más alto y el más bajo. Si el array está vacío o tiene un solo elemento, devuelve 0.

★ ★ ★ Ejercicio 7: CodeWars — You're a square!

Resuelve la kata “You're a square!” (7 kyu) en [CodeWars](#).

Dado un número entero, determina si es un cuadrado perfecto. Devuelve `true` si lo es, `false` si no. Por ejemplo, 25 es cuadrado perfecto (5×5), 26 no.

Boletín 4 – Intermedio Resuelto: Algorítmica I

Ejercicios resueltos de dificultad progresiva.

☆☆ Ejercicio 1: Pares con suma objetivo

```
public static List<int[]> paresConSuma(int[] arr, int suma) {  
    List<int[]> res = new ArrayList<>();  
    Set<Integer> vistos = new HashSet<>();  
    for (int n : arr) {  
        if (vistos.contains(suma - n))  
            res.add(new int[]{suma - n, n});  
        vistos.add(n);  
    }  
    return res;  
}
```

☆☆ Ejercicio 2: Eliminar duplicados (in-place)

```
public static int eliminarDuplicados(int[] arr) {  
    if (arr.length == 0) return 0;  
    int i = 0;  
    for (int j = 1; j < arr.length; j++)  
        if (arr[j] != arr[i]) arr[++i] = arr[j];  
    return i + 1;  
}
```

☆☆☆ Ejercicio 3: Mayor suma de subarray (Kadane)

```

public static int maxSubarraySum(int[] arr) {
    int maxG = arr[0], maxA = arr[0];
    for (int i = 1; i < arr.length; i++) {
        maxA = Math.max(arr[i], maxA + arr[i]);
        maxG = Math.max(maxG, maxA);
    }
    return maxG;
}

```

☆☆☆ Ejercicio 4: Rotar array k veces

```

public static void rotar(int[] arr, int k) {
    k %= arr.length;
    invertir(arr, 0, arr.length - 1);
    invertir(arr, 0, k - 1);
    invertir(arr, k, arr.length - 1);
}

private static void invertir(int[] a, int l, int r) {
    while (l < r) { int t = a[l]; a[l] = a[r]; a[r] = t; l++; r--; }
}

```

☆☆☆ Ejercicio 5: Contar ocurrencias (binaria optimizada)

```
public static int contar(int[] arr, int target) {  
    int izq = buscar(arr, target, true);  
    if (izq == -1) return 0;  
    return buscar(arr, target, false) - izq + 1;  
}  
  
private static int buscar(int[] a, int t, boolean primero) {  
    int l = 0, r = a.length - 1, res = -1;  
    while (l <= r) {  
        int m = (l + r) / 2;  
        if (a[m] == t) { res = m; if (primero) r = m - 1; else l = m + 1; }  
        else if (a[m] < t) l = m + 1;  
        else r = m - 1;  
    }  
    return res;  
}
```

Boletín 4 – Intermedio: Algorítmica I

Ejercicios de dificultad progresiva. Los ★ son para calentar, ★★ para pensar, ★★★ para concursar.

★★ Ejercicio 1: Mover ceros al final

Escribe un método `public static void moverCeros(int[] arr)` que mueva todos los ceros del array al final, manteniendo el orden relativo de los no ceros. Hazlo **in-place**, sin crear un array nuevo.

Ejemplo: `[0, 1, 0, 3, 12] → [1, 3, 12, 0, 0]`

Pista: usa un índice `posicionNoCero` que avance solo cuando encuentres un número distinto de cero. Intercambia o desplaza según el valor.

★★ Ejercicio 2: El número que falta

Dado un array de `N` enteros únicos que contiene números del 0 al `N` (es decir, tiene `N+1` números posibles pero solo `N` elementos), encuentra el número que falta.

Ejemplo: `[3, 0, 1]` falta el 2. `[9,6,4,2,3,5,7,0,1]` falta el 8.

Escribe el método `public static int numeroFaltante(int[] arr)`.

Pista: suma todos los números del array. Después suma todos los números de 0 a `N` (que es `arr.length`). La diferencia es el número que falta.

★★ Ejercicio 3: Primer carácter no repetido

Escribe un método `public static char primerNoRepetido(String s)` que devuelva el primer carácter de la cadena que no se repite en ninguna otra posición. Si todos se repiten o la cadena está vacía, devuelve ' ' (espacio).

Ejemplo: "amor a roma" — la 'a' se repite, la 'm' se repite, la 'o' se repite, la 'r' se repite... el espacio se repite. En este caso, no hay caracteres no repetidos, devuelve ' '.

Ejemplo: "programacion" — la 'p' no se repite → devuelve 'p'.

Pista: puedes usar un array de 26 enteros (uno por letra) para contar frecuencias, o un array de 256 para todo el ASCII.

☆☆☆ Ejercicio 4: Anagramas

Dos strings son anagramas si contienen los mismos caracteres con la misma frecuencia. Escribe `public static boolean sonAnagramas(String a, String b)` que devuelva `true` si lo son.

Ejemplos:

- "listen" y "silent" → `true`
- "java" y "avaj" → `true`
- "hola" y "halo" → `true`
- "hola" y "adios" → `false`

Pista: conviértelos a arrays de `char`, ordénalos con `Arrays.sort()`, y compáralos. O cuenta frecuencias con un array de 26 enteros.

☆☆☆ Ejercicio 5: Ordenar 0s, 1s y 2s (Dutch National Flag)

Dado un array que contiene solo 0, 1 y 2 ordénalo **in-place** en una sola pasada (complejidad $O(n)$). Escribe `public static void ordenarColores(int[] arr)`.

Ejemplo: `[0, 2, 1, 2, 0, 1, 0] → [0, 0, 0, 1, 1, 2, 2]`

Pista: usa tres punteros: izquierdo (para 0s), derecho (para 2s) y actual (que recorre el array). Intercambia según el valor encontrado. Este algoritmo se llama "Dutch National Flag" y lo inventó Edsger Dijkstra. Si lo resuelves, tienes nivel de entrevista técnica en Google.

☆☆☆ Ejercicio 6: CodeWars — Disemvowel Trolls

Resuelve la kata “Disemvowel Trolls” (7 kyu) en [CodeWars](#).

Elimina todas las vocales (a, e, i, o, u) de un string dado. El input puede tener mayúsculas y minúsculas. Por ejemplo, "This website is for losers LOL!" → "Ths wbst s fr lsrs LL!".

☆☆☆ Ejercicio 7: AceptaElReto — 219 La lotería

Resuelve el problema 219 — La lotería en [AceptaElReto.com](#).

Dado un array de números de lotería, cuenta cuántas “inversiones” hay: pares de posiciones (i, j) donde $i < j$ pero el número en i es mayor que el de j. En otras palabras, cuenta cuántos pares están desordenados.

Pista: puedes hacerlo con dos bucles anidados (complejidad $O(n^2)$), pero para valores grandes necesitarías merge sort ($O(n \log n)$). Empieza por la versión simple.



Referencias

Plataforma	Problema	Dificultad
AceptaElReto	151 — ¿Es matriz identidad?	Medio
AceptaElReto	219 — La lotería	Medio
AceptaElReto	340 — Juegos de naipes	Medio
CodeWars	Vowel Count (7 kyu)	Fácil
CodeWars	Disemvowel Trolls (7 kyu)	Fácil
CodeWars	You're a square! (7 kyu)	Fácil

Boletín 04 — Extras: Algorítmica I: Fundamentos

★ Extras — Conviértete en un programador de muy alto nivel Estos ejercicios son completamente opcionales, pero si aspiras a ser un programador o programadora de élite, aquí tienes tu campo de entrenamiento. CodeWars y AceptaElReto son plataformas donde los mejores programadores del mundo se retan a diario. Cada problema que resuelvas te hará más fuerte, más rápido y más creativo. No los subestimes: son la diferencia entre saber programar y ser un verdadero profesional.

CodeWars:

- [Find the odd int](#) (6 kyu)
Pista: Aplica XOR (^) a todos los números del array — los pares se cancelan y el que queda es el solitario.
- [Playing with digits](#) (6 kyu)
Pista: Convierte el número a cadena, itera por cada dígito elevándolo a la potencia correspondiente y suma.
- [Maximum subarray sum](#) (5 kyu)
Pista: Aplica Kadane: mantén la suma actual y la máxima, reinicia la suma actual si se vuelve negativa.

AceptaElReto:

- [219 - La lotería](#) (★ ★)
Pista: Cuenta las inversiones con un bucle anidado o con una estrategia de divide y vencerás más eficiente.
- [340 - Suma de dígitos](#) (★)
Pista: Saca el último dígito con % 10, suma y divide entre 10 hasta que no queden dígitos. Repite hasta que la suma sea de un solo dígito (raíz digital).

Boletín 5 – Inicial Resuelto: Algorítmica II

Soluciones del boletín inicial.

★ Ejercicio 1: Factorial recursivo

```
public static int factorial(int n) {  
    if (n <= 1) return 1;  
    return n * factorial(n - 1);  
}
```

★ Ejercicio 2: Fibonacci recursivo

```
public static int fibo(int n) {  
    if (n <= 1) return n;  
    return fibo(n - 1) + fibo(n - 2);  
}
```

★ ★ Ejercicio 3: Suma recursiva de array

```
public static int sumaArray(int[] arr, int i) {  
    if (i == arr.length) return 0;  
    return arr[i] + sumaArray(arr, i + 1);  
}
```

★ ★ Ejercicio 4: Invertir cadena

```
public static String invertir(String s) {  
    if (s.isEmpty()) return "";  
    return invertir(s.substring(1)) + s.charAt(0);  
}
```

★ ★ ★ Ejercicio 5: Fibonacci con memoización

```
public static int fiboMemo(int n, int[] memo) {
    if (n <= 1) return n;
    if (memo[n] != 0) return memo[n];
    memo[n] = fiboMemo(n - 1, memo) + fiboMemo(n - 2, memo);
    return memo[n];
}
// Uso: fiboMemo(10, new int[11])
```

☆☆☆ Ejercicio 6: Quicksort

```
public static void quicksort(int[] a, int l, int r) {
    if (l >= r) return;
    int p = partition(a, l, r);
    quicksort(a, l, p - 1);
    quicksort(a, p + 1, r);
}

private static int partition(int[] a, int l, int r) {
    int p = a[r], i = l - 1;
    for (int j = l; j < r; j++)
        if (a[j] <= p) { i++; int t = a[i]; a[i] = a[j]; a[j] = t; }
    int t = a[i + 1]; a[i + 1] = a[r]; a[r] = t;
    return i + 1;
}
```

Boletín 5 – Inicial: Algorítmica II

Sin soluciones. La recursividad es como caer por un pozo, pero el pozo se repite a sí mismo.

Ejercicio 1: Potencia recursiva

Implementa una función recursiva que calcule a elevado a n (con $n \geq 0$). Sin usar `Math.pow()` ni bucles.

```
public static int potencia(int a, int n) {  
    // tu código aquí  
}
```

Ejemplo: `potencia(2, 3)` $\rightarrow$ 8, `potencia(5, 0)` $\rightarrow$ 1.

Pista: Cualquier número elevado a 0 es 1. Para el resto, $a^n = a * a^{(n-1)}$.

Ejercicio 2: Contar dígitos recursivo

Implementa una función recursiva que cuente cuántos dígitos tiene un número entero positivo.

```
public static int contarDigitos(int n) {  
    // tu código aquí  
}
```

Ejemplo: `contarDigitos(12345)` $\rightarrow$ 5, `contarDigitos(7)` $\rightarrow$ 1, `contarDigitos(0)` $\rightarrow$ 1.

Ejercicio 3: ¿Qué imprime?

Sin ejecutar, di qué imprime este código malvado:

```
public class Misterio {  
    public static int enigma(int x) {  
        if (x < 10) return x;  
        return enigma(x / 10) + x % 10;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(enigma(1234));  
        System.out.println(enigma(999));  
        System.out.println(enigma(5));  
    }  
}
```

Ejercicio 4: Palíndromo recursivo

Implementa una función recursiva que determine si un String es un palíndromo (se lee igual al derecho y al revés).

```
public static boolean esPalindromo(String s) {  
    // tu código aquí  
}
```

Ejemplos: `esPalindromo("ana")` → `true`, `esPalindromo("reconocer")` → `true`,
`esPalindromo("hola")` → `false`.

Pista: Un String es palíndromo si el primer y último carácter son iguales, y el substring interno también lo es. Los casos base son: `length 0` o `1` → `true`.

Ejercicio 5: Encuentra el error

¿Qué le pasa a este código? Compila, pero no funciona como esperamos. Explica por qué:

```
public static int factorial(int n) {  
    return n * factorial(n - 1);  
}
```

¿Qué ocurre si llamamos a `factorial(5)`?

Ejercicio 6: Sumatorio recursivo

Implementa una función recursiva que sume todos los números desde 1 hasta n.

```
public static int sumatorio(int n) {  
    // tu código aquí  
}
```

Ejemplo: `sumatorio(5)` → 15 (1+2+3+4+5).

Ejercicio 7: CodeWars — Reversed Strings

Resuelve la kata “**Reversed Strings**” (8 kyu) en CodeWars.

Completa la función `solution` que recibe un `String` y devuelve el `String` al revés. Fácil, ¿no? Pues hazlo **recursivamente**, no con `StringBuilder.reverse()`.

```
public class StringReverser {  
    public static String solution(String s) {  
        // tu código recursivo aquí  
    }  
}
```

Ejercicio 8: AceptaElReto — 165 Número hyperpar

Resuelve el problema **165 — Número hyperpar** en [AceptaElReto.com](https://www.acepta-el-reto.com/problems/165).

Un número es “hyperpar” si todos sus dígitos son pares. Dada una secuencia de números hasta que se introduzca -1, indica si cada uno es hyperpar o no.

Ejemplo: 246 → SI, 2486 → SI, 123 → NO, 0 → SI.

Pista: Puedes extraer dígitos con `n % 10` y comprobar si son pares. Si encuentras uno impar, dejas de mirar. El 0 se considera par, así que 0 y 00 son hyperpar.

Boletín 5 - Intermedio Resuelto: Algorítmica II

Ejercicios resueltos de nivel intermedio.

☆☆ Ejercicio 1: Torres de Hanoi

```
public static void hanoi(int n, char origen, char destino, char aux) {
    if (n == 1) System.out.println(origen + " -> " + destino);
    else {
        hanoi(n - 1, origen, aux, destino);
        System.out.println(origen + " -> " + destino);
        hanoi(n - 1, aux, destino, origen);
    }
}
```

☆☆ Ejercicio 2: Mergesort

```
public static void mergesort(int[] a, int l, int r) {
    if (l >= r) return;
    int m = (l + r) / 2;
    mergesort(a, l, m);
    mergesort(a, m + 1, r);
    merge(a, l, m, r);
}

private static void merge(int[] a, int l, int m, int r) {
    int[] t = new int[r - l + 1];
    int i = l, j = m + 1, k = 0;
    while (i <= m && j <= r) t[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i <= m) t[k++] = a[i++];
    while (j <= r) t[k++] = a[j++];
    System.arraycopy(t, 0, a, l, t.length);
}
```

☆☆☆ Ejercicio 3: Búsqueda binaria recursiva

```
public static int binariaRec(int[] a, int t, int l, int r) {
    if (l > r) return -1;
    int m = (l + r) / 2;
    if (a[m] == t) return m;
    return (a[m] < t) ? binariaRec(a, t, m + 1, r) : binariaRec(a, t, l, m - 1);
}
```

☆☆☆ Ejercicio 4: Potencia rápida recursiva ($O(\log n)$)

```
public static long potencia(long base, long exp) {
    if (exp == 0) return 1;
    long mitad = potencia(base, exp / 2);
    return (exp % 2 == 0) ? mitad * mitad : mitad * mitad * base;
}
```

☆☆☆ Ejercicio 5: Subconjuntos (backtracking)

```
public static List<List<Integer>> subconjuntos(int[] nums) {
    List<List<Integer>> res = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), res);
    return res;
}

private static void backtrack(int[] n, int i, List<Integer> cur, List<List<Integer>> r)
{
    r.add(new ArrayList<>(cur));
    for (int j = i; j < n.length; j++) {
        cur.add(n[j]);
        backtrack(n, j + 1, cur, r);
        cur.remove(cur.size() - 1);
    }
}
```


Boletín 5 – Intermedio: Algorítmica II: Técnicas Avanzadas

Ejercicios de dificultad progresiva. Los ★ son para calentar, ★★ para pensar, ★★★ para concursar. Si la recursividad te parece un bucle sin fin, espera a hacer estos ejercicios. Bueno, en realidad Sí es un bucle sin fin, pero controlado.

★ Ejercicio 1: MCD por Euclides recursivo

Implementa el algoritmo de Euclides para calcular el máximo común divisor de dos números enteros **de forma recursiva**. Sin trampas: nada de bucles, nada de `Math.min()` yendo hacia atrás. Tiene que haber una llamada a sí mismo.

El algoritmo de Euclides dice:

- Si $b == 0$, el MCD es a .
- Si no, el MCD de a y b es el MCD de b y $a \% b$.

```
public static int mcd(int a, int b) {  
    // tu código aquí  
}
```

Ejemplo: `mcd(48, 18) → 6`. `mcd(100, 25) → 25`.

★ Ejercicio 2: Contar ocurrencias recursivo

Dado un array de enteros y un número objetivo, cuenta cuántas veces aparece ese número en el array **usando recursividad**. Sin bucles, sin streams, sin trampas.

```
public static int contarOcurrencias(int[] arr, int objetivo, int indice) {  
    // tu código aquí  
}
```

Ejemplo: `contarOcurrencias(new int[]{1, 2, 3, 2, 4, 2, 5}, 2, 0) → 3`.

Pista: Si el índice llega al final del array, devuelve 0. Si no, comprueba si el elemento actual es el objetivo y suma 1 o 0, luego llama recursivamente con `indice + 1`.

★ ★ Ejercicio 3: Invertir número recursivo

Dado un número entero positivo, devuelve su inverso. Es decir, 1234 se convierte en 4321. **Sin convertir a String**. Todo con operaciones matemáticas y recursividad.

```
public static int invertirNumero(int n) {  
    // tu código aquí  
}
```

Ejemplo: `invertirNumero(1234) → 4321`. `invertirNumero(700) → 7` (los ceros a la izquierda se pierden, como debe ser).

Pista: Necesitas un método auxiliar que lleve la cuenta del resultado parcial. Algo así como `invertirAux(n, 0)`.

★ ★ Ejercicio 4: Combinaciones de un array (backtracking)

Dado un array de enteros y un número k, genera **todas las combinaciones posibles** de tamaño k de los elementos del array. El orden de los elementos dentro de cada combinación no importa, y no debe haber combinaciones repetidas.

```
public static List<List<Integer>> combinaciones(int[] arr, int k) {  
    // tu código aquí  
}
```

Ejemplo: `combinaciones(new int[]{1, 2, 3}, 2)` devuelve `[[1,2], [1,3], [2,3]]`.

Pista: Es backtracking clásico. Lleva un índice de inicio para evitar repeticiones. En cada paso, decides si incluyes el elemento actual o no. Cuando la combinación actual llega a tamaño k, la añades al resultado.

★ ★ ★ Ejercicio 5: N-Reinas (backtracking)

El clásico de los clásicos. Coloca N reinas en un tablero de N×N de forma que ninguna se amenace entre sí. Dos reinas se amenazan si están en la misma fila, columna o diagonal.

```
public static List<int[][]> resolverNReinas(int n) {  
    // Devuelve todas las soluciones posibles  
}
```

O más sencillo: implementa un método que **imprima** todas las soluciones para N=4, N=5 y N=8.

```
public static boolean resolver(int[][] tablero, int col) {  
    // tu código aquí  
}
```

Pista: Coloca una reina por columna. Para cada columna, prueba todas las filas. Antes de colocar, verifica que no haya otra reina en la misma fila ni en las diagonales. Si llegas a la última columna, has encontrado una solución. Si no, retrocede (backtrack). Para N=4 hay 2 soluciones.

Ejercicio 6: CodeWars — Tribonacci Sequence

Resuelve la kata “**Tribonacci Sequence**” (6 kyu) en CodeWars.

Bueno, el Fibonacci se ha quedado obsoleto. Ahora toca el Tribonacci: igual pero sumando los tres últimos en lugar de dos. Recibes un array signature de 3 números y un n que indica cuántos elementos debe tener la secuencia resultante.

```
public double[] tribonacci(double[] s, int n) {  
    // tu código aquí  
}
```

Ejemplo: `tribonacci([1,1,1], 10)` → `[1,1,1,3,5,9,17,31,57,105]`

Pista: Si `n == 0`, devuelve array vacío. Si `n <= 3`, devuelve solo los primeros n elementos de la firma. Para el resto, itera desde 3 hasta n-1 sumando los tres anteriores. Puedes hacerlo iterativo o recursivo. El iterativo es más eficiente, pero el recursivo queda más bonito.

☆☆☆ Ejercicio 7: AceptaElReto — 140 Suma de dígitos

Resuelve el problema **140 — Suma de dígitos** en [AceptaElReto.com](https://www.acepta-el-reto.com/).

Dado un número, suma sus dígitos. Si el resultado tiene más de un dígito, vuelve a sumar. Repite hasta obtener un solo dígito. Es la “raíz digital” o “suma de dígitos recursiva”.

Ejemplo: $987 \rightarrow 9+8+7 = 24 \rightarrow 2+4 = 6$. Devuelve 6.

El problema lee números hasta que se introduce un 0.

Pista: Puedes implementarlo con un método recursivo: si $n < 10$, devuelve n . Si no, suma los dígitos de n y llama recursivamente con el resultado. Para extraer dígitos: $n \% 10$ da el último, $n / 10$ quita el último.

☆☆☆ Ejercicio 8: AceptaElReto — 162 Suma de dígitos (otra vez no)

Bromita. El verdadero octavo ejercicio no existe, pero si has llegado hasta aquí, ya has hecho suficiente recursividad por hoy. Ve a tomarte un café. O una Coca-Cola. O agua, que es más sano.



Referencias

Plataforma	Problema	Dificultad
CodeWars	Tribonacci Sequence	6 kyu
CodeWars	Sum of Digits / Digital Root	6 kyu
AceptaElReto	140 — Suma de dígitos	Fácil
AceptaElReto	162 — Suma de dígitos	Fácil

Boletín 05 — Extras: Algorítmica II: Técnicas Avanzadas

★ Extras — Conviértete en un programador de muy alto nivel Estos ejercicios son completamente opcionales, pero si aspiras a ser un programador o programadora de élite, aquí tienes tu campo de entrenamiento. CodeWars y AceptaElReto son plataformas donde los mejores programadores del mundo se retan a diario. Cada problema que resuelvas te hará más fuerte, más rápido y más creativo. No los subestimes: son la diferencia entre saber programar y ser un verdadero profesional.

CodeWars:

- [Sum of Intervals](#) (4 kyu)

Pista: Ordena los intervalos por inicio. Fusiona los solapantes y suma las longitudes de los no solapantes.

- [Snail Sort](#) (4 kyu)

Pista: Recorre la matriz por capas concéntricas: derecha, abajo, izquierda, arriba; reduce los límites en cada vuelta.

- [Range Extraction](#) (4 kyu)

Pista: Recorre el array ordenado y agrupa los números consecutivos en rangos [inicio-fin].

AceptaElReto:

- [375 - El lince ibérico](#) (★ ★ ★)

Pista: Modela el mapa como un grafo y aplica BFS para encontrar la distancia mínima entre dos puntos.

- [465 - Números afortunados](#) (★ ★ ★)

Pista: Simula el proceso con una lista booleana (criba) y ve tachando posiciones según el número afortunado actual.

Boletín 4 – Inicial Resuelto: POO – Clases y Objetos

Las soluciones están aquí, esperándote. Pero primero inténtalo. La POO se aprende creando objetos, no leyendo sobre ellos.

Ejercicio 1: Completa la clase Perro

```
public class Perro {  
    String nombre;  
    int edad;  
  
    public Perro(String nombre, int edad) {  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
  
    public void ladrar() {  
        System.out.println("Guau, soy " + nombre);  
    }  
  
    public void cumplirAnios() {  
        edad++;  
    }  
  
    public static void main(String[] args) {  
        Perro rex = new Perro("Rex", 3);  
        rex.ladrar();  
        System.out.println("Edad: " + rex.edad);  
        rex.cumplirAnios();  
        System.out.println("Después de cumple: " + rex.edad);  
    }  
}
```



Explicación: El constructor `public Perro(String nombre, int edad)` asigna los parámetros a los atributos usando `this` para distinguir entre el parámetro `nombre` y el atributo `this.nombre`. Sin `this`, Java pensaría que son la misma variable (la del parámetro) y no asignaría nada al atributo. El método `cumplirAnios()` simplemente hace `edad++` igual que

harías con cualquier variable. Al crear el objeto con `new Perro("Rex", 3)`, se ejecuta el constructor y el perro nace con 3 años. Luego ladra y cumple años como cualquier ser vivo (bueno, los perros no dicen “Guau, soy Rex”, pero si lo hicieran, sería exactamente así).

Ejercicio 2: ¿Qué imprime?

```
public class Coche {
    String marca;
    int velocidad;

    public Coche(String marca) {
        this.marca = marca;
        this.velocidad = 0;
    }

    public void acelerar(int kmh) {
        velocidad += kmh;
    }

    public static void main(String[] args) {
        Coche c = new Coche("Seat");
        c.acelerar(50);
        c.acelerar(30);
        c.acelerar(-10);
        System.out.println(c.marca + " va a " + c.velocidad + " km/h");
    }
}
```

Salida: Seat va a 70 km/h

💡 **Explicación:** Creamos un Seat con velocidad 0. Aceleramos +50 → 50. Aceleramos +30 → 80. Aceleramos -10 → 70. El método `acelerar` acepta valores negativos (frenar en realidad). No hay validación que impida ir a velocidad negativa porque no la programamos. Esto es un ejemplo de por qué la encapsulación es importante: cualquiera podría pasar un -500 y el coche iría a -420 km/h... ¿marcha atrás? En el siguiente boletín veremos cómo evitarlo con setters y validación.

Ejercicio 3: Encuentra el error

```
public class Main {  
    public static void main(String[] args) {  
        Rectangulo r = Rectangulo(5, 3); // ERROR: falta 'new'  
        System.out.println(r.area());  
    }  
}
```

Error: Falta new. Debe ser new Rectangulo(5, 3).

💡 **Explicación:** En Java, los objetos se crean con new. Sin new, estás tratando de llamar a un método que no existe (como si Rectangulo fuera un método estático). Java se queja: “no sé qué es Rectangulo(5, 3)”. Es como si dijeras “coche()” en lugar de “coche nuevo()”. Sin new, el objeto no se crea en memoria. Solo declaras una referencia a nada (null). **Regla de oro:** si quieres un objeto, necesitas new. Siempre.

Ejercicio 4: Escribe la clase Persona


```

public class Persona {
    String nombre;
    int edad;
    String ciudad;

    public Persona(String nombre, int edad, String ciudad) {
        this.nombre = nombre;
        this.edad = edad;
        this.ciudad = ciudad;
    }

    public void saludar() {
        System.out.println("Hola, soy " + nombre + " de " + ciudad);
    }

    public boolean esMayorEdad() {
        return edad >= 18;
    }

    public static void main(String[] args) {
        Persona p1 = new Persona("Ana", 25, "Madrid");
        Persona p2 = new Persona("Luis", 17, "Barcelona");

        p1.saludar();
        System.out.println(p1.nombre + " es mayor? " + p1.esMayorEdad());

        p2.saludar();
        System.out.println(p2.nombre + " es mayor? " + p2.esMayorEdad());
    }
}

```

 **Explicación:** La clase Persona encapsula tres datos (nombre, edad, ciudad) y dos comportamientos (saludar, esMayorEdad). Cada objeto Persona tiene su propia copia de estos atributos. p1 y p2 son independientes: Ana tiene 25 años y es mayor de edad; Luis tiene 17 y no lo es. La POO permite agrupar datos y comportamiento en una misma entidad. Es como tener fichas de personas: cada ficha tiene sus datos y sus acciones. Sin POO, tendrías arrays separados para nombres, edades y ciudades, y funciones sueltas. Con POO, todo está junto y ordenado.

Ejercicio 5: ¿Constructor por defecto?

```
public class Prueba {  
    int x;  
    boolean activo;  
    String texto;  
  
    public static void main(String[] args) {  
        Prueba p = new Prueba();  
        System.out.println("x = " + p.x);  
        System.out.println("activo = " + p.activo);  
        System.out.println("texto = " + p.texto);  
    }  
}
```

Salida:

```
x = 0  
activo = false  
texto = null
```



Explicación: Cuando no defines ningún constructor, Java te regala uno por defecto (sin parámetros). Ese constructor inicializa los atributos a sus valores por defecto: números a 0, booleanos a false, objetos (como String) a null. Es como cuando abres una caja nueva: está vacía (null), no hay nadie dentro (false) y el contador está a 0. Por eso x es 0, activo es false y texto es null. Si hubieras creado un constructor propio, el constructor por defecto desaparece. Es como si al personalizar tu casa, perdieras la versión básica.

Ejercicio 6: AceptaElReto 291 — Números afortunados

```

import java.util.Scanner;

public class Problema291 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int casos = sc.nextInt();
        for (int c = 0; c < casos; c++) {
            int n = sc.nextInt();
            int suma = 0;
            while (n > 0) {
                suma += n % 10;
                n /= 10;
            }
            if (suma == 7 || suma % 7 == 0) {
                System.out.println("afortunado");
            } else {
                System.out.println("desafortunado");
            }
        }
    }
}

```

💡 **Explicación:** Extraemos los dígitos de n con $n \% 10$ y $n / 10$, los sumamos, y comprobamos si la suma es 7 o múltiplo de 7. Por ejemplo: $16 \rightarrow 1+6=7 \rightarrow$ afortunado. $25 \rightarrow 2+5=7 \rightarrow$ afortunado. $34 \rightarrow 3+4=7 \rightarrow$ afortunado. $10 \rightarrow 1+0=1 \rightarrow$ desafortunado. $7 \rightarrow 7 \rightarrow$ afortunado. Es un problema sencillo de AceptaElReto que combina el bucle de extracción de dígitos con condiciones. Si tu número favorito es el 7, estás de suerte.

Ejercicio 7: CodeWars — Grasshopper - Summation

```
public class Grasshopper {  
    public static int summation(int n) {  
        int suma = 0;  
        for (int i = 1; i <= n; i++) {  
            suma += i;  
        }  
        return suma;  
    }  
}
```

💡 **Explicación:** CodeWars espera el método exacto `public static int summation(int n)`. El método suma del 1 a n con un bucle. También se puede hacer con la fórmula matemática $n * (n + 1) / 2$, que es más eficiente. Pero para un 8 kyu, el bucle está bien. La kata te enseña que CodeWars tiene una firma concreta que debes seguir al pie de la letra. Si el método se llama `summation` y devuelve `int`, no puedes llamarlo `sumar` ni devolver `double`. CodeWars es muy puntilloso con las firmas, como un profesor de lengua con las tildes.

Boletín 4 – Inicial: POO: Clases y Objetos

Sin soluciones. Las clases no se van a crear solas. Y si se crearan solas, probablemente serían clases abstractas, pero eso es para otro boletín.

Ejercicio 1: Completa la clase Alumno

Faltan algunas partes. Complétalas para que funcione:

```
public class Alumno {  
    String nombre;  
    String apellidos;  
    int edad;  
  
    // Completa el constructor: debe recibir nombre, apellidos y edad  
    public Alumno(String nombre, String apellidos, int edad) {  
        // ¿Qué va aquí?  
    }  
  
    public void presentarse() {  
        System.out.println("Hola, soy " + nombre + " " + apellidos);  
    }  
  
    public void cumplirAnyos() {  
        // Incrementa la edad en 1  
    }  
}
```

Crea luego en el main un alumno llamado “Laura García” con 20 años, haz que se presente y luego cumpla años.

Ejercicio 2: ¿Qué imprime?

Sin ejecutar, di qué sale:

```
public class Contador {  
    int valor;  
  
    public Contador(int valorInicial) {  
        valor = valorInicial;  
    }  
  
    public void incrementar() {  
        valor++;  
    }  
  
    public void decrementar() {  
        valor--;  
    }  
  
    public static void main(String[] args) {  
        Contador c = new Contador(10);  
        c.incrementar();  
        c.incrementar();  
        c.decrementar();  
        c.incrementar();  
        System.out.println("Valor final: " + c.valor);  
    }  
}
```

Ejercicio 3: Encuentra el error

```
public class Producto {
    String nombre;
    double precio;

    public Producto(String nombre, double precio) {
        nombre = nombre;
        precio = precio;
    }

    public void mostrar() {
        System.out.println(nombre + " cuesta " + precio + "€");
    }
}

public class Main {
    public static void main(String[] args) {
        Producto p = new Producto("Laptop", 999.99);
        p.mostrar();
    }
}
```

Hay 1 error. Encuéntralo.

Ejercicio 4: Escribe la clase Película

Crea una clase Pelicula con:

- Atributos: String titulo, String genero, int duracion (en minutos)
- Constructor que reciba los tres parámetros
- Método void reproducir() que imprima “Reproduciendo [título] ([género]) - [duración] min”
- Método boolean esLarga() que devuelva true si la duración es mayor a 120 minutos

En el main, crea dos películas y reproduce cada una, indicando si es larga o no.

Ejercicio 5: Constructor vacío y setters

¿Qué imprime este código?

```

public class Estudiante {
    String nombre;
    double notaMedia;

    public Estudiante() {
        this.nombre = "Desconocido";
        this.notaMedia = 5.0;
    }

    public Estudiante(String nombre, double notaMedia) {
        this.nombre = nombre;
        this.notaMedia = notaMedia;
    }

    public static void main(String[] args) {
        Estudiante e1 = new Estudiante();
        Estudiante e2 = new Estudiante("Carlos", 8.5);

        System.out.println(e1.nombre + " - " + e1.notaMedia);
        System.out.println(e2.nombre + " - " + e2.notaMedia);
    }
}

```

Ejercicio 6: AceptaElReto — 417 Números binomiales

Resuelve el problema **417 — Números binomiales** en [AceptaElReto.com](https://www.acepta-el-reto.com/problems/417-numeros-binomiales).

Calcula el coeficiente binomial “n sobre k”. Tradicionalmente se define como $n! / (k! * (n-k)!)$. El problema lee pares (n, k) hasta (0, 0).

Pista: Los factoriales pueden ser enormes. Mejor usa la fórmula iterativa: multiplica desde n hasta n-k+1 y divide por k!.

Ejercicio 7: CodeWars — Return the day

Resuelve la kata “**Return the day**” (8 kyu) en CodeWars.

Crea una clase `DayOfWeek` con un método estático `getDay(int n)` que devuelva el día de la semana correspondiente:

- 1 → "Sunday", 2 → "Monday", ..., 7 → "Saturday"
- Cualquier otro número → "Wrong, please enter a number between 1 and 7"

Boletín 4 – Resuelto: POO – Clases y Objetos

Ejercicios progresivos. De crear tu primera clase a competir en AceptaElReto con objetos. Ponte el cinturón que la POO acelera.

★ Ejercicio 1: Clase Rectángulo

Crea una clase `Rectangulo` con `ancho` y `alto` (`double`). Constructor parametrizado, `calcularArea()`, `calcularPerimetro()`. Sobrescribe `toString()`.

💡 **Explicación:** El constructor asigna `ancho` y `alto`. Los métodos calculan área y perímetro con fórmulas básicas. `toString()` devuelve una representación legible del objeto. Sin `toString()`, Java imprimiría algo como `Rectangulo@2f92e0f4` (la clase y una dirección de memoria). Con `toString()`, imprime `Rectángulo [5 x 3]`. Sobrescribir `toString()` es una de las mejores prácticas de Java: tu yo del futuro (y tu profesor) te lo agradecerán cuando depures.

★ Ejercicio 2: Clase CuentaBancaria

`CuentaBancaria` con `titular`, `saldo`, `numeroCuenta`. Dos constructores: solo `titular` (`saldo 0`) o `titular + saldo`. Métodos `ingresar(double)` y `retirar(double)` (valida `saldo` suficiente). `toString()`.

💡 **Explicación:** Usamos sobrecarga de constructores: uno llama al otro con `this(titular, 0)`. El número de cuenta se genera automáticamente con un contador estático (cada cuenta nueva tiene un número distinto). `ingresar` solo acepta cantidades positivas. `retirar` valida que haya `saldo` suficiente. Es como un cajero automático de verdad, pero sin colas ni comisiones. La validación evita que te quedes en números rojos. Bueno, o al menos te avisa.

★ ★ Ejercicio 3: Clase Hora con validación

`Hora` con `hora` (0-23), `minuto` (0-59), `segundo` (0-59). Constructor con validación (lanza `IllegalArgumentException`). `incrementarSegundo()` que maneje desbordamientos.

💡 **Explicación:** El constructor valida que hora, minuto y segundo estén en rangos válidos. Si no, lanza una `IllegalArgumentException` (una excepción unchecked que no necesita try-catch, pero mata el programa si no la capturas). `incrementarSegundo()` suma 1 segundo y maneja los desbordamientos: si segundo llega a 60, pasa a 0 y aumenta minuto; igual con minuto→hora→día. Ejemplo: 23:59:59 + 1 segundo → 00:00:00. Es como un reloj digital, pero sin pilas.

☆☆ Ejercicio 4: equals() en Persona

Sobrescribe `equals()` en la clase `Persona` para que dos personas sean iguales si tienen el mismo nombre y edad.

💡 **Explicación:** Sin `equals()`, comparar objetos con `equals` es lo mismo que con `==` (comparan referencias, no contenido). Aquí sobrescribimos `equals` para que compare nombre y edad. `Objects.equals(nombre, persona.nombre)` es seguro porque maneja nulls. `hashCode()` debe sobrescribirse siempre que sobrescribas `equals` (contrato de Java). Si no, las colecciones como `HashSet` o `HashMap` se comportarán de forma extraña. Es como el jamón y el queso: `equals` y `hashCode` van juntos.

☆☆ Ejercicio 5: Clase Punto y distancia euclídea

`Punto` con `x` e `y` (int). Constructor, getters, `distancia(Punto otro)` (distancia euclídea), `toString()`. Calcula el perímetro de un triángulo dados 3 puntos.

💡 **Explicación:** La distancia euclídea entre dos puntos es la raíz cuadrada de $(dx^2 + dy^2)$. Con `Math.sqrt()` lo calculamos. El perímetro del triángulo es la suma de las distancias entre cada par de puntos. El triángulo (0,0)-(3,0)-(0,4) es un triángulo rectángulo clásico de lados 3, 4 y 5 (hipotenusa). Perímetro = 3 + 4 + 5 = 12. La POO permite tratar los puntos como objetos con comportamientos. En lugar de tener funciones sueltas `distancia(x1,y1,x2,y2)`, tenemos `a.distancia(b)`. Mucho más limpio.

☆☆☆ Ejercicio 6: AceptaElReto 417 — Números binomiales

Resuelve el problema **417 — Números binomiales** en AceptaElReto.

Dados dos números n y k , calcula el coeficiente binomial “ n sobre k ”. Usa una función recursiva o iterativa.

💡 **Explicación:** El coeficiente binomial “ n sobre k ” cuenta cuántas formas hay de elegir k elementos de un conjunto de n . La fórmula iterativa es más eficiente que la recursiva (evita calcular factoriales enormes). Usamos la propiedad de simetría: si $k > n-k$, usamos $n-k$ (reduce iteraciones). El bucle va acumulando el resultado multiplicando y dividiendo para evitar números muy grandes. El problema lee pares (n,k) hasta encontrar $(0,0)$. Es un problema clásico de combinatoria, muy común en olimpiadas de programación.

☆☆☆ Ejercicio 7: AceptaElReto 458 — El espejo

Resuelve el problema **458 — El espejo** en AceptaElReto.

Dada una hora en formato “HH:MM”, calcula cuántos minutos faltan para la siguiente hora que sea un “espejo” (palíndromo). Ejemplo: 10:01 es espejo, 23:32 también.

💡 **Explicación:** Un “espejo” significa que la hora y los minutos se reflejan: las decenas de la hora son las unidades de los minutos, y viceversa. 10:01 $\rightarrow$ 1=1 y 0=0 $\rightarrow$ es espejo. 12:21 $\rightarrow$ 1=1 y 2=2 $\rightarrow$ es espejo. El bucle incrementa minutos de 1 en 1 hasta encontrar una hora espejo, manejando el desbordamiento de minutos y horas. Es una búsqueda exhaustiva pero con máximo $60 \cdot 24 = 1440$ iteraciones, que es trivial. El espejo más cercano a 23:32 (que ya es espejo) es 0 minutos. Para 23:33, el siguiente espejo es 0:00 (27 minutos después). Mírate al espejo y verás la solución.

☆☆☆ Ejercicio 8: Simulación de batalla Pokémon (lite)

Crea una clase Pokemon con nombre, vida, ataque. Método atacar(Pokemon otro) que resta `this.ataque` a la vida del otro. Crea dos pokémon y simula una batalla por turnos hasta que uno se quede sin vida.

💡 **Explicación:** Cada Pokémon tiene nombre, vida y ataque. El método atacar resta el ataque del atacante a la vida del defensor. La batalla alterna turnos. El bucle continúa

mientras ambos estén vivos. Cuando uno llega a 0 de vida, `estaVivo()` devuelve `false` y el bucle termina. Es una versión muy simplificada de un juego de lucha. Podrías añadir defensa, tipos, habilidades especiales... pero eso ya es cosa tuya. Este ejercicio demuestra cómo la POO permite modelar entidades del mundo real (o de mundos ficticios) de forma natural. Cada Pokémon es un objeto independiente con sus propios datos y comportamientos.



Referencias

Plataforma	Problema	Dificultad
AceptaElReto	291 — Números afortunados	Fácil
AceptaElReto	417 — Números binomiales	Medio
AceptaElReto	458 — El espejo	Medio
CodeWars	Grasshopper - Summation (8 kyu)	Principiante
CodeWars	Basic variable assignment (8 kyu)	Principiante

Boletín 6 – Intermedio: POO: Clases y Objetos

Ejercicios de dificultad progresiva. Los ★ son para calentar, ★★ para pensar, ★★★ para concursar. De crear tu primera clase a construir objetos que harían llorar de envidia a tu profesor de FP Básica.

★ Ejercicio 1: Clase Libro

Crea una clase Libro con:

- Atributos privados: `String titulo`, `String autor`, `int paginas`
- Constructor que reciba los tres parámetros
- Getters para todos los atributos
- Método `toString()` que devuelva "El libro '[título]' de [autor] tiene [páginas] páginas."

```
public class Libro {  
    // atributos  
    // constructor  
    // getters  
    // toString()  
}
```

En el main, crea tres libros y muéstralos.

★ Ejercicio 2: Clase Termómetro

Crea una clase Termometro que almacene la temperatura en grados Celsius (privado). Debe tener:

- Constructor que reciba los Celsius iniciales
- Getter `getCelsius()`
- Método `getFahrenheit()` que devuelva la temperatura en Fahrenheit: $^{\circ}\text{F} = ^{\circ}\text{C} * 9/5 + 32$
- Método `getKelvin()` que devuelva la temperatura en Kelvin: $\text{K} = ^{\circ}\text{C} + 273.15$

- Setter `setCelsius(double celsius)` que valide que la temperatura no sea menor que -273.15 (cero absoluto)

```
public class Termometro {  
    private double celsius;  
  
    public Termometro(double celsius) {  
        // validar y asignar  
    }  
  
    public double getCelsius() { return celsius; }  
    public void setCelsius(double celsius) {  
        // si es menor que -273.15, imprimir error, si no, asignar  
    }  
    public double getFahrenheit() {  
        // calcular y devolver  
    }  
    public double getKelvin() {  
        // calcular y devolver  
    }  
}
```

Ejemplo: `new Termometro(25)` → `getFahrenheit()` → 77.0, `getKelvin()` → 298.15.

★ ★ Ejercicio 3: Clase Fecha con validación

Crea una clase `Fecha` con `día`, `mes`, `año` (int, privados). El constructor debe validar que la fecha sea válida: días correctos por mes, teniendo en cuenta años bisiestos.

Además:

- Método `String toString()` que devuelva “DD/MM/AAAA”
- Método boolean `esBisiesto()` que indique si el año es bisiesto (divisible entre 4 pero no entre 100, salvo que sea divisible entre 400)
- Método `int diasDesde(Fecha otra)` que calcule los días transcurridos desde otra fecha anterior

```

public class Fecha {
    private int dia, mes, anio;

    public Fecha(int dia, int mes, int anio) {
        // validar: si la fecha no es válida, lanzar IllegalArgumentException
    }

    public boolean esBisiesto() {
        // (anio % 4 == 0 && anio % 100 != 0) || anio % 400 == 0
    }

    public int diasEnMes(int mes, int anio) {
        // devuelve los días de un mes dado (considerando bisiesto para febrero)
    }

    public int diasDesde(Fecha otra) {
        // Días entre dos fechas (puedes convertir ambas a días totales)
    }
}

```

Pista: Para `diasDesde()`, convierte ambas fechas a un número de días desde una fecha fija (ej: 1/1/1). La diferencia es la resta.

★ ★ Ejercicio 4: Clase Reloj con adelanto

Crea una clase `Reloj` con hora (0-23), minuto (0-59), segundo (0-59), todos privados. Constructor con validación. Métodos:

- `void adelantar(int segundos)`: adelanta el reloj la cantidad de segundos indicada, manejando desbordamientos correctamente (segundo → minuto → hora → día siguiente)
- `String toString()`: formato "HH:MM:SS" (con dos dígitos cada uno)
- `int diferenciaSegundos(Reloj otro)`: devuelve los segundos de diferencia entre este reloj y otro


```

public class Reloj {
    private int hora, minuto, segundo;

    public Reloj(int hora, int minuto, int segundo) {
        // validar rangos
    }

    public void adelantar(int segundos) {
        // suma segundos, maneja desbordamientos
    }

    public String toString() {
        // formato HH:MM:SS
    }
}

```

Ejemplo: Si el reloj marca 23:59:50 y adelantamos 15 segundos, debe pasar a 00:00:05.

★ ★ ★ Ejercicio 5: Clase Matriz 2D

Crea una clase Matriz que represente una matriz bidimensional de enteros. Debe tener:

- Constructor que reciba filas y columnas (inicializa con ceros)
- Constructor que reciba un `int[][]` existente
- `int get(int fila, int col)` y `void set(int fila, int col, int valor)`
- `Matriz sumar(Matriz otra)`: devuelve una NUEVA matriz con la suma de matrices (deben tener las mismas dimensiones)
- `Matriz restar(Matriz otra)`: igual pero restando
- `Matriz multiplicar(Matriz otra)`: multiplicación de matrices (columnas de esta deben coincidir con filas de otra)
- `String toString()`: representación bonita de la matriz

```
public class Matriz {
    private int[][] datos;
    private int filas, columnas;

    // constructores, getters, setters
    // sumar, restar, multiplicar
    // toString()
}
```

Ejemplo:

```
Matriz a = new Matriz(new int[][]{{1,2},{3,4}});
Matriz b = new Matriz(new int[][]{{5,6},{7,8}});
Matriz c = a.sumar(b); // [[6,8],[10,12]]
```

★ ★ ★ Ejercicio 6: CodeWars — Build Tower

Resuelve la kata “**Build Tower**” (6 kyu) en CodeWars.

Crea una clase TowerBuilder con un método estático build(int nFloors) que devuelva un array de Strings representando una torre con nFloors pisos hecha de asteriscos *.

Cada piso tiene un número impar de asteriscos centrados con espacios. Ejemplo para nFloors = 3:

```
[
  "  *  ",
  " *** ",
  "*****"
]
```

Pista: El piso i (empezando desde 0) tiene $2*i + 1$ asteriscos. El ancho total es $2*nFloors - 1$. Los espacios alrededor son $(ancho - asteriscos) / 2$. Usa String.repeat() o un bucle.

★ ★ ★ Ejercicio 7: AceptaElReto — 246 Buscando el pin

Resuelve el problema **246 — Buscando el pin** en [AceptaElReto.com](https://www.aceptaelreto.com/problems/246).

El problema trata de un pinball donde hay que encontrar el camino que la bola puede seguir desde la entrada hasta la salida. Esencialmente es un problema de búsqueda en un grafo (DFS/BFS) pero modelado con objetos.

Pista: Modela el tablero como una cuadrícula de casillas. Cada casilla puede ser un objeto con propiedades (pared, hueco, direction). Usa una cola (BFS) para encontrar el camino más corto.



Referencias

Plataforma	Problema	Dificultad
CodeWars	Build Tower	6 kyu
CodeWars	Persistent Bugger	6 kyu
AceptaElReto	246 — Buscando el pin	Medio
AceptaElReto	367 — Ascensores	Medio

Boletín 06 — Extras: POO: Clases y Objetos

★ **Extras — Conviértete en un programador de muy alto nivel** Estos ejercicios son completamente opcionales, pero si aspiras a ser un programador o programadora de élite, aquí tienes tu campo de entrenamiento. CodeWars y AceptaElReto son plataformas donde los mejores programadores del mundo se retan a diario. Cada problema que resuelvas te hará más fuerte, más rápido y más creativo. No los subestimes: son la diferencia entre saber programar y ser un verdadero profesional.

CodeWars:

- [Build a pile of Cubes](#) (6 kyu)
Pista: Resta cubos n^3 desde $n=1$ hasta que el resultado sea 0 (o devuelve -1 si te pasas).
- [Valid Braces](#) (6 kyu)
Pista: Usa una pila (Stack). Al abrir, apila; al cerrar, comprueba que coincida con el tope.
- [Simple Pig Latin](#) (5 kyu)
Pista: Divide por palabras. Si la palabra empieza por vocal, añade “way”; si no, mueve la primera consonante al final y añade “ay”.

AceptaElReto:

- [246 - Buscando el pin](#) (★ ★)
Pista: Construye un grafo de adyacencias entre las casillas del pinball y aplica DFS/BFS para encontrar el camino.
- [367 - Ascensores](#) (★ ★)
Pista: Simula los ascensores vuelta a vuelta. En cada parada, sube y baja personas hasta alcanzar el límite de capacidad.

Boletín 5 – Inicial Resuelto: Visibilidad, Encapsulación y Static

Las soluciones explicadas con humor. Porque la encapsulación no tiene por qué ser aburrida.

Ejercicio 1: Encuentra el error de visibilidad

```
public class Casa {
    public String direccion;
    protected String telefono;
    int numeroHabitaciones;
    private String contrasenaWifi;

    public void mostrarTodo() {
        System.out.println(direccion);           // OK: misma clase
        System.out.println(telefono);           // OK: misma clase
        System.out.println(numeroHabitaciones); // OK: misma clase
        System.out.println(contrasenaWifi);      // OK: misma clase
    }
}

public class Vecino {
    public void espiar() {
        Casa c = new Casa();
        System.out.println(c.direccion);         // OK: public
        System.out.println(c.telefono);          // OK: protected (mismo paquete)
        System.out.println(c.numeroHabitaciones); // OK: package-private (mismo
paquete)
        System.out.println(c.contrasenaWifi);    // ERROR: private
    }
}
```

💡 **Explicación:** `contrasenaWifi` es `private`. Solo la clase `Casa` puede acceder a él. Ni su vecino (`Vecino`), ni su madre, ni el perro. `private` es el equivalente a tu diario secreto con candado. Los otros niveles (`public`, `protected`, `package-private`) permiten acceso según el contexto. La propia clase (`mostrarTodo`) sí puede acceder a todo porque está dentro de la misma clase. Es como si tú mismo pudieras leer tu propio diario, pero los vecinos no.

Ejercicio 2: Completa los getters y setters

```
public class CuentaBancaria {
    private double saldo;

    public double getSaldo() {
        return saldo;
    }

    public void setSaldo(double saldo) {
        if (saldo >= 0) {
            this.saldo = saldo;
        } else {
            System.out.println("El saldo no puede ser negativo");
        }
    }

    public void ingresar(double cantidad) {
        if (cantidad > 0) {
            saldo += cantidad;
        } else {
            System.out.println("Cantidad inválida");
        }
    }

    public boolean retirar(double cantidad) {
        if (cantidad > 0 && cantidad <= saldo) {
            saldo -= cantidad;
            return true;
        }
        System.out.println("No se puede retirar esa cantidad");
        return false;
    }
}
```



Explicación: La encapsulación consiste en: atributos private, acceso controlado por métodos públicos. El getter solo devuelve el valor. El setter valida que el saldo no sea negativo. ingresar solo acepta cantidades positivas. retirar comprueba que haya saldo suficiente. Todo el control está en manos de la clase, no del usuario externo. Es como un cajero automático: tú no puedes abrir la máquina y coger billetes; solo puedes usar los

botones que la máquina te permite. La encapsulación es el “cajero automático” de tus objetos.

Ejercicio 3: ¿Qué imprime? Static vs instancia

```
public class Gato {  
    public static int totalGatos = 0;  
    public String nombre;  
  
    public Gato(String nombre) {  
        this.nombre = nombre;  
        totalGatos++;  
    }  
  
    public static void main(String[] args) {  
        Gato g1 = new Gato("Misi");  
        Gato g2 = new Gato("Garfield");  
        Gato g3 = new Gato("Tom");  
  
        System.out.println("Total: " + Gato.totalGatos);  
        System.out.println("Nombre: " + g2.nombre);  
    }  
}
```

Salida:

```
Total: 3  
Nombre: Garfield
```

💡 **Explicación:** `totalGatos` es `static`, lo que significa que es compartido por todos los objetos de la clase. Cada vez que se crea un `Gato`, el constructor incrementa `totalGatos`. Al crear 3 gatos, `totalGatos` es 3. En cambio, `nombre` es de instancia: cada gato tiene su propio nombre. `g2.nombre` es “Garfield” porque es el nombre del segundo gato. La diferencia es como el grupo de WhatsApp de la clase (static) vs los mensajes privados (instancia). Todos ven el mensaje del grupo, pero cada uno tiene sus propios DMs.

Ejercicio 4: Escribe la clase utilitaria

```

public class StringUtils {
    private StringUtils() {} // Constructor privado: no se puede instanciar

    public static boolean esVacio(String s) {
        return s == null || s.trim().isEmpty();
    }

    public static String invertir(String s) {
        if (s == null) return null;
        return new StringBuilder(s).reverse().toString();
    }

    public static void main(String[] args) {
        System.out.println("¿Vacío? " + StringUtils.esVacio(" ")); // true
        System.out.println("¿Vacío? " + StringUtils.esVacio("Hola")); // false
        System.out.println("Invertir: " + StringUtils.invertir("Hola")); // aloH
    }
}

```

💡 **Explicación:** Las clases utilitarias tienen constructor privado para que nadie pueda instanciarlas. Todos sus métodos son estáticos y se llaman con `NombreClase.metodo()`. Es como `Math`: no haces `new Math()`, usas `Math.random()` directamente. `esVacio` comprueba si un `String` es `null` o solo espacios. `invertir` usa `StringBuilder` que tiene un método `reverse()` incorporado. Podrías hacerlo a mano con un bucle, pero `StringBuilder` es más eficiente. La clase `StringUtils` es tu navaja suiza para `Strings`.

Ejercicio 5: Escribe la clase Config


```

public class Config {
    private Config() {} // No se puede instanciar

    public static final String NOMBRE_APP = "Gestión DAM";
    public static final String VERSION = "1.0.0";
    public static final int MAX_USUARIOS = 100;

    private static int contadorAccesos = 0;

    public static void incrementarAcceso() {
        contadorAccesos++;
    }

    public static int getContadorAccesos() {
        return contadorAccesos;
    }

    public static void main(String[] args) {
        System.out.println("App: " + Config.NOMBRE_APP);
        System.out.println("Versión: " + Config.VERSION);
        System.out.println("Max usuarios: " + Config.MAX_USUARIOS);

        Config.incrementarAcceso();
        Config.incrementarAcceso();
        Config.incrementarAcceso();
        System.out.println("Accesos: " + Config.getContadorAccesos());
    }
}

```

💡 **Explicación:** Las constantes `static final` son públicas e inmutables. Se escriben en MAYÚSCULAS por convención. El constructor privado evita instanciación. `contadorAccesos` es privado y estático: solo la clase puede modificarlo, pero existe a nivel de clase (no de objeto). Se usa como un contador global de accesos a la aplicación. Es como el marcador de visitas de una web: todos ven el número, pero solo el servidor puede incrementarlo.

Ejercicio 6: AceptaElReto 106 — Código de barras

```

import java.util.Scanner;

public class Problema106 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String codigo = sc.nextLine();
        while (!codigo.equals("0")) {
            int longitud = codigo.length();
            boolean esEan13 = longitud == 13;
            int sumaPar = 0, sumaImpar = 0;

            int limite = esEan13 ? 12 : 7;
            for (int i = 0; i < limite; i++) {
                int digito = codigo.charAt(i) - '0';
                if (i % 2 == 0) sumaImpar += digito;
                else sumaPar += digito;
            }

            int total = esEan13 ? sumaImpar * 3 + sumaPar : sumaImpar + sumaPar * 3;
            int digitoControl = (10 - total % 10) % 10;
            int ultimoDigito = codigo.charAt(longitud - 1) - '0';

            String pais = "";
            int prefijo = Integer.parseInt(codigo.substring(0, 3));
            if (longitud == 8) {
                pais = "EEUU";
            } else if (prefijo == 0 || prefijo == 1) {
                pais = "EEUU";
            } else if (prefijo >= 380 && prefijo <= 379) { /* improbable */
            } else if (prefijo >= 380 && prefijo <= 389) { /* no */
            } else if (prefijo >= 380 && prefijo <= 379) { /* error */
            } else if (prefijo >= 380 && prefijo < 400) {
                pais = "Bulgaria";
            } else { /* simplificación */
                pais = "Desconocido";
            }

            // Simplificación: solo mostramos validez
            if (digitoControl == ultimoDigito) {
                System.out.println("SI");
            } else {
                System.out.println("NO");
            }
        }
    }
}

```

```

    }
    codigo = sc.nextLine();
}
}
}

```

💡 **Explicación:** Esta es una versión simplificada del problema 106. El algoritmo del código de barras: para EAN-13, sumas los dígitos impares (x3) + pares; para EAN-8, al revés. Calculas el dígito de control con $(10 - \text{total} \% 10) \% 10$ y lo comparas con el último dígito. El código completo en AceptaElReto también debe identificar el país según el prefijo. La encapsulación aquí está en la lógica: cada cálculo está separado y es fácil de entender. Es un ejemplo de cómo validar datos de entrada con un algoritmo estándar.

Ejercicio 7: CodeWars — Convert boolean to string

```

public class YesOrNo {
    public static String boolToWord(boolean b) {
        return b ? "Yes" : "No";
    }
}

```

💡 **Explicación:** El ternario `? :` es perfecto aquí. Si `b` es `true`, devuelve “Yes”. Si no, “No”. Una línea. Elegante. También podrías hacerlo con `if-else`, pero el ternario queda más limpio. CodeWars premia la simplicidad. Es como cuando te preguntan “¿quieres café?” y respondes “Sí” o “No”. No necesitas un párrafo explicando tu relación con la cafeína. La clase es utilitaria (método estático), así que la llamas con `YesOrNo.boolToWord(true)`.

Boletín 5 – Inicial: Visibilidad, Encapsulación y Static

Sin soluciones. `private` no significa que seas tímido, significa que proteges tus datos. Y `static` no es electricidad estática, es memoria compartida.

Ejercicio 1: Encuentra el error de visibilidad (la nevera)

¿Qué líneas de este código dan error de compilación y por qué?

```
public class Nevera {
    public String marca;
    protected int capacidadLitros;
    double temperatura;
    private String codigoDesbloqueo;

    public void mostrarInfo() {
        System.out.println(marca);
        System.out.println(capacidadLitros);
        System.out.println(temperatura);
        System.out.println(codigoDesbloqueo);
    }
}

public class Amigo {
    public void abrirNevera() {
        Nevera n = new Nevera();
        System.out.println(n.marca);
        System.out.println(n.capacidadLitros);
        System.out.println(n.temperatura);
        System.out.println(n.codigoDesbloqueo);
    }
}
```

Ejercicio 2: Completa los getters y setters de la clase Producto

Completa la clase Producto con encapsulación real:

```
public class Producto {  
    private String nombre;  
    private double precio;  
    private int stock;  
  
    // Getter para nombre  
    public String getNombre() {  
        // ¿qué va aquí?  
    }  
  
    // Setter para nombre (no puede estar vacío)  
    public void setNombre(String nombre) {  
        // ¿qué va aquí?  
    }  
  
    // Getter para precio  
    public double getPrecio() {  
        // ¿qué va aquí?  
    }  
  
    // Setter para precio (no puede ser negativo)  
    public void setPrecio(double precio) {  
        // ¿qué va aquí?  
    }  
  
    // Getter para stock  
    public int getStock() {  
        // ¿qué va aquí?  
    }  
  
    // Setter para stock (no puede ser negativo)  
    public void setStock(int stock) {  
        // ¿qué va aquí?  
    }  
  
    // Método: vender una cantidad (reduce el stock, solo si hay suficiente)  
    public boolean vender(int cantidad) {  
        // ¿qué va aquí? (devuelve true si se pudo vender)  
    }  
}
```

Ejercicio 3: ¿Qué imprime? Static vs instancia II

Sin ejecutar, di qué imprime:

```
public class Universidad {
    public static String nombreUniversidad = "UAX";
    public String nombreAlumno;

    public Universidad(String nombreAlumno) {
        this.nombreAlumno = nombreAlumno;
    }

    public void mostrar() {
        System.out.println(nombreAlumno + " estudia en " + nombreUniversidad);
    }

    public static void main(String[] args) {
        Universidad u1 = new Universidad("Ana");
        Universidad u2 = new Universidad("Luis");

        u1.mostrar();
        u2.mostrar();

        Universidad.nombreUniversidad = "UPM";

        u1.mostrar();
        u2.mostrar();
    }
}
```

Ejercicio 4: Escribe la clase utilitaria MathUtils

Crea una clase MathUtils con constructor privado y métodos estáticos:

- `int maximo(int a, int b)` — devuelve el mayor de dos números
- `int minimo(int a, int b)` — devuelve el menor de dos números
- `boolean esPar(int n)` — devuelve true si n es par

Ejercicio 5: Escribe la clase AppConfig

Crea una clase AppConfig con constantes `static final`:

- `NOMBRE_APP = "MiAplicacion"`
- `VERSION = "2.0.0"`
- `MAX_INTENTOS = 3`

Además, un atributo `private static int usuariosConectados` con getter y método `incrementarUsuarios()` / `decrementarUsuarios()`. No se puede instanciar la clase.

Ejercicio 6: AceptaElReto — 117 Encontrar el mayor

Resuelve el problema 117 — Encontrar el mayor en [AceptaElReto.com](https://www.acepta-el-reto.com/).

Lee números hasta que se introduzca un número igual al número anterior. En ese momento, imprime cuántos números se leyeron en total.

Ejemplo de entrada/salida:

```
1 3 5 7 8 6 3 3 → 8 (porque el 3 se repite como octavo número)
```

Pista: Lleva un contador de números leídos y una variable con el número anterior. Cuando el actual sea igual al anterior, para y muestra el contador.

Ejercicio 7: CodeWars — Opposite number

Resuelve la kata “Opposite number” (8 kyu) en CodeWars.

Completa el método `opposite` que recibe un número y devuelve su opuesto (negativo si es positivo, positivo si es negativo).

Crea una clase utilitaria `OppositeNumber` con un método estático:


```
public class OppositeNumber {  
    public static int opposite(int number) {  
        // tu código aquí  
    }  
}
```

Ejemplos: `opposite(5)` → -5, `opposite(-3)` → 3, `opposite(0)` → 0.

Boletín 5 – Resuelto: Visibilidad, Encapsulación y Static

De ★ a ★★☆☆. De “hago getters y setters” a “construyo una API con clases utilitarias”. La encapsulación te espera.

★ Ejercicio 1: Clase Empleado con validación

Empleado con nombre, salarioBase (double) y departamento privados. El salario no puede ser negativo. El setter lanza `IllegalArgumentException`. Método `calcularSalarioAnual()`.

💡 **Explicación:** El setter `setSalarioBase` valida que el salario no sea negativo. Si lo es, lanza una `IllegalArgumentException`. Esto es mejor que simplemente ignorar el valor inválido: el programa se detiene y te dice qué pasa. El constructor usa `setSalarioBase` para aprovechar la validación. `calcularSalarioAnual()` multiplica por 12 (suponiendo 12 pagas). Este es el patrón estándar de encapsulación en Java: atributos privados, getters/setters públicos con validación. Así como en la vida real no puedes tener un salario negativo (aunque a veces lo parezca), en tu código tampoco.

★ Ejercicio 2: Círculo encapsulado

Círculo con radio privado. Setter valida que el radio sea positivo. `getArea()` y `getPerimetro()`. ¿Qué pasa si desde otra clase accedes a radio?

Desde otra clase: `circulo.radio` daría ERROR de compilación. Solo se puede acceder con `circulo.getRadio()`.

💡 **Explicación:** Si intentas `circulo.radio` desde otra clase, el compilador dice: “El campo `Circulo.radio` es invisible”. `private` es eso: invisible para todos excepto para la propia clase. Ni siquiera puedes verlo, y mucho menos modificarlo. Para obtener el radio, usas `getRadio()`. Para cambiarlo, `setRadio(...)`. Y el setter valida que sea positivo. Es como el botón de “no pasar” de una puerta: solo el dueño (la clase) puede decidir quién entra.

★★ Ejercicio 3: JavaBean Alumno

Clase Alumno JavaBean: nombre, edad, curso, notaMedia (double), matriculado (boolean). Constructor sin args. Nota media entre 0 y 10.

💡 **Explicación:** JavaBean es una convención: clase pública, constructor sin argumentos, atributos privados, getters y setters públicos. Para boolean, el getter se llama isMatriculado() en lugar de getMatriculado(). El constructor sin args inicializa todo a valores por defecto. El setter de notaMedia valida que esté entre 0 y 10. Los JavaBeans se usan en frameworks como Spring, Hibernate, JSF... Son el estándar de la industria. Aprende el patrón y lo usarás en el 90% de tus clases.

★ ★ Ejercicio 4: Contador de objetos con static

Clase Usuario con contador estático de objetos creados, id autoincremental, constante DOMINIO_EMAIL, y método generarEmail().

💡 **Explicación:** contador (static) se incrementa en cada constructor. Es compartido por todos los objetos. id es de instancia: cada usuario recibe el valor actual del contador en el momento de su creación. DOMINIO_EMAIL es una constante estática. generarEmail() combina el nombre (en minúsculas) con el dominio. Es como cuando te creas un email en el instituto: ana@dam.com. El contador estático te dice cuántos usuarios se han creado en total, independientemente de qué objetos sigan vivos.

★ ★ Ejercicio 5: Conversor de unidades (estáticos)

Clase Conversor con constantes y métodos estáticos para convertir entre km y millas, Celsius y Fahrenheit, kg y libras.

💡 **Explicación:** Constructor privado, solo métodos estáticos. Cada método usa una constante o una fórmula. Las constantes son static final y se escriben en mayúsculas. Es como tener una calculadora de conversiones siempre disponible, sin necesidad de crear objetos. La fórmula Celsius a Fahrenheit es $^{\circ}\text{C} \times 9/5 + 32$. Fahrenheit a Celsius es $(^{\circ}\text{F} - 32) \times 5/9$. Las conversiones son todas lineales: multiplicas por un factor. Simple, útil, y demuestra el patrón de clase utilitaria.



Ejercicio 6: AceptaElReto 364 —

Spiderman

Resuelve el problema **364 — Spiderman** en AceptaElReto.

Spiderman lanza telarañas para salvar ciudadanos. En cada lanzamiento, la telaraña recorre una distancia. El problema pide algo con espías y distancias. (Nota: léelo en la web, es un problema de espías enemigos y distancias).

💡 **Explicación:** El problema 364 (SpiderMan / Espías) te da una serie de alturas de edificios. Debes contar cuántos edificios “ven” el lado derecho sin ser bloqueados por uno más alto. La solución recorre de derecha a izquierda, manteniendo la altura máxima vista. Si el edificio actual es más alto que el máximo visto, se cuenta. Es un problema clásico de “visibilidad” que encaja perfectamente con el tema de visibilidad de esta unidad. Los edificios más altos bloquean la vista de los más bajos, como los modificadores `private` bloquean el acceso desde fuera.



Ejercicio 7: AceptaElReto 462 — Tres dedos

Resuelve el problema **462 — Tres dedos** en AceptaElReto.

Dado un número en base 10, conviértelo a base -2 (base negativa). Los dígitos resultantes serán 0 o 1.

💡 **Explicación:** Las bases negativas son un concepto matemático curioso. En base -2, los números se representan con dígitos 0 y 1, pero con pesos que alternan signo: 1, -2, 4, -8, 16... El algoritmo es similar a la conversión a binario, pero con un ajuste: si el resto es negativo, le sumamos 2 y aumentamos el cociente en 1. Ejemplo: 6 en base -2 es 11010 ($1 \cdot 16 + 1 \cdot -8 + 0 \cdot 4 + 1 \cdot -2 + 0 \cdot 1 = 16 - 8 - 2 = 6$). Es un problema de concurso (ProgramaMe) que requiere pensar fuera de la caja. Literalmente, fuera de la base positiva.



Ejercicio 8: Simulación estática

Clase `Simulacion` con método `main` que lanza dos dados 1000 veces usando `Math.random()`. Usa una clase `Dado` con método estático `lanzar()` (1-6). Cuenta cuántas veces sale cada suma (2-12).

Salida esperada (aproximada):

Resultados de 1000 lanzamientos:

Suma | Frecuencia

-----|-----

2 | 28

3 | 56

4 | 83

5 | 111

6 | 139

7 | 167

8 | 139

9 | 111

10 | 83

11 | 56

12 | 28

💡 **Explicación:** La clase Dado es utilitaria: constructor privado y método lanzar() estático. No necesitas crear dados individuales porque todos son iguales. La simulación usa un array de 13 posiciones (0-12) donde sumas[7] cuenta cuántas veces ha salido suma 7. El 7 es el más probable porque hay más combinaciones de dados que suman 7 (1+6, 2+5, 3+4, 4+3, 5+2, 6+1) que cualquier otra suma. La ley de los grandes números hace que la distribución se acerque a una campana (distribución normal). Es la misma razón por la que en el casino el 7 es la apuesta más popular en los dados. Pero no apuestes, que la casa siempre gana. Bueno, tú ganas con este código.

Referencias

Plataforma	Problema	Dificultad
AceptaElReto	106 — Código de barras	Medio
AceptaElReto	364 — Spiderman	Medio
AceptaElReto	462 — Tres dedos	Difícil
CodeWars	Convert boolean to string (8 kyu)	Principiante
CodeWars	Return the day (8 kyu)	Principiante

Boletín 5 – Intermedio: Visibilidad, Encapsulación y Static

Ejercicios de dificultad progresiva. Los ★ son para calentar, ★★ para pensar, ★★★ para concursar. La encapsulación no es un hechizo de Harry Potter, es sentido común. Pero para estos ejercicios, igual necesitas un hechizo o dos.

★ Ejercicio 1: Clase CuentaAhorros encapsulada

Crea una clase CuentaAhorros con:

- Atributo privado `double saldo`
- Atributo privado `double interesAnual` (porcentaje, ej: 2.5 significa 2.5%)
- Constructor que reciba el saldo inicial y el interés anual. Si el saldo es negativo, se establece a 0.
- Getter para saldo
- Método `void aplicarInteres()` que aumente el saldo según el interés anual (`saldo += saldo * interesAnual / 100`)
- Método `boolean retirar(double cantidad)` que solo permita retirar si hay saldo suficiente

```

public class CuentaAhorros {
    private double saldo;
    private double interesAnual;

    public CuentaAhorros(double saldoInicial, double interesAnual) {
        // validar e inicializar
    }

    public void aplicarInteres() {
        // saldo += saldo * interesAnual / 100
    }

    public boolean retirar(double cantidad) {
        // solo si cantidad > 0 y cantidad <= saldo
    }

    public double getSaldo() {
        return saldo;
    }
}

```

Ejemplo: `new CuentaAhorros(1000, 10).aplicarInteres()` → saldo = 1100.

★ Ejercicio 2: Clase Persona con validación estricta

Crea una clase `Persona` con atributos privados `String` `nombre` y `int` `edad`. El constructor y los setters deben validar que:

- `nombre` no sea `null` ni esté vacío (ni solo espacios)
- `edad` esté entre 0 y 120 (inclusive)

Si alguna validación falla, lanza `IllegalArgumentException` con un mensaje descriptivo.

```
public class Persona {  
    private String nombre;  
    private int edad;  
  
    public Persona(String nombre, int edad) {  
        setNombre(nombre); // ¡Usa el setter para aprovechar la validación!  
        setEdad(edad);  
    }  
  
    public void setNombre(String nombre) {  
        // validar y asignar  
    }  
  
    public void setEdad(int edad) {  
        // validar y asignar  
    }  
}
```

¿Qué crees que pasa si haces `new Persona("", 25)`? ¿Y `new Persona("Ana", -5)`?

★ ★ Ejercicio 3: Clase TicketCompra con ID autoincremental

Crea una clase `TicketCompra` que modele un ticket de supermercado:

- Atributo estático `private static int contadorTickets = 0`
- Atributo de instancia `private int id` (autoincremental: al crear un ticket, recibe el valor actual de `contadorTickets` y luego se incrementa)
- Atributo `private String[] productos` y `private double[] precios`
- Constructor que reciba los arrays de productos y precios
- Método `double calcularTotal()` que sume todos los precios
- Método `double calcularIVA()` que calcule el 21% del total
- Método `String toString()` que muestre el ticket formateado


```

public class TicketCompra {
    private static int contadorTickets = 0;
    private int id;
    private String[] productos;
    private double[] precios;

    public TicketCompra(String[] productos, double[] precios) {
        this.id = ++contadorTickets;    // 0: contadorTickets++; this.id =
        contadorTickets;
        this.productos = productos;
        this.precios = precios;
    }

    public static int getTotalTicketsEmitidos() {
        return contadorTickets;
    }
}

```

Ejemplo:

```

TicketCompra t1 = new TicketCompra(
    new String[]{"Pan", "Leche", "Huevos"},
    new double[]{1.20, 2.50, 3.00}
);
System.out.println(t1);    // Ticket #1: Pan=1.20€, Leche=2.50€, Huevos=3.00€ | Total:
6.70€ | IVA: 1.41€

```

☆☆ Ejercicio 4: Clase Biblioteca con contador estático

Crea una clase Biblioteca que gestione préstamos de libros:

- Atributo estático `private static int totalLibrosPrestados = 0`
- Atributo de instancia `private String nombreBiblioteca`
- Atributo de instancia `private int librosDisponibles`
- Constructor que reciba nombre y cantidad inicial de libros
- Método boolean `prestarLibro()`: si hay libros disponibles, reduce en 1, incrementa el contador estático y devuelve true. Si no, devuelve false.

- Método `void devolverLibro()`: aumenta libros disponibles en 1, decrementa el contador estático (si es > 0).
- Método estático `int getTotalPrestamos()` que devuelva el contador global de préstamos
- Método `int getDisponibles()` que devuelva los libros disponibles en esta biblioteca

```
public class Biblioteca {  
    private static int totalLibrosPrestados = 0;  
    private String nombre;  
    private int disponibles;  
  
    // constructor, prestarLibro, devolverLibro, getters estáticos y de instancia  
}
```

Ejemplo: Si creas dos bibliotecas con 10 libros cada una y prestas 3 de la primera y 2 de la segunda, `Biblioteca.getTotalPrestamos()` debería devolver 5.

☆☆☆ Ejercicio 5: Clase CalculadoraEstadística utilitaria

Crea una clase `CalculadoraEstadistica` que sea una clase utilitaria (constructor privado, solo métodos estáticos):

- `static double media(double[] datos)`: calcula la media aritmética
- `static double mediana(double[] datos)`: calcula la mediana (ordena y toma el valor central; si es par, media de los dos centrales)
- `static double moda(double[] datos)`: calcula la moda (valor que más se repite; si hay empate, devuelve cualquiera)
- `static double desviacionTipica(double[] datos)`: calcula la desviación típica poblacional

```
public class CalculadoraEstadistica {  
    private CalculadoraEstadistica() {} // No se puede instanciar  
  
    public static double media(double[] datos) {  
        double suma = 0;  
        for (double d : datos) suma += d;  
        return suma / datos.length;  
    }  
  
    // resto de métodos...  
}
```

Ejemplo: `media(new double[]{1, 2, 3, 4, 5})` → 3.0. `mediana(new double[]{1, 3, 2, 4, 5})` → 3.0.

☆☆☆ Ejercicio 6: CodeWars — Find the odd int

Resuelve la kata “Find the odd int” (6 kyu) en CodeWars.

Crea una clase utilitaria `FindOdd` con un método estático `findIt(int[] arr)` que recibe un array de enteros y devuelve el entero que aparece un número impar de veces. Siempre habrá exactamente uno.

Ejemplo: `findIt(new int[]{1,1,2,2,3,3,3})` → 3 (aparece 3 veces, que es impar).

Pista: Puedes usar un `Map<Integer, Integer>` para contar frecuencias y luego quedarte con el que tenga frecuencia impar. O, si eres muy listo, usa XOR (^): cada número XOR consigo mismo da 0, así que al final solo queda el que no tiene par.

☆☆☆ Ejercicio 7: AceptaElReto — 120 Número de pares y nones

Resuelve el problema 120 — Número de pares y nones en [AceptaElReto.com](https://www.acepta-el-reto.com/).

Dado un número entero positivo, cuenta cuántos de sus dígitos son pares y cuántos son impares.

El problema lee números hasta que se introduce un 0.

Ejemplo:

- 12345 → P=2, I=3 (pares: 2,4; impares: 1,3,5)
- 2468 → P=4, I=0
- 7 → P=0, I=1

Pista: Extrae dígitos con $n \% 10$ y $n / 10$. Comprueba paridad con $\text{digito} \% 2 == 0$. Es un problema sencillo pero te obliga a pensar en cómo encapsular la lógica en una clase con métodos estáticos.



Referencias

Plataforma	Problema	Dificultad
CodeWars	Find the odd int	6 kyu
CodeWars	Regex validate PIN code	7 kyu
AceptaElReto	120 — Número de pares y nones	Fácil
AceptaElReto	157 — ¿Son iguales?	Medio

Boletín 07 — Extras: Visibilidad, Encapsulación y Static

★ Extras — Conviértete en un programador de muy alto nivel Estos ejercicios son completamente opcionales, pero si aspiras a ser un programador o programadora de élite, aquí tienes tu campo de entrenamiento. CodeWars y AceptaElReto son plataformas donde los mejores programadores del mundo se retan a diario. Cada problema que resuelvas te hará más fuerte, más rápido y más creativo. No los subestimes: son la diferencia entre saber programar y ser un verdadero profesional.

CodeWars:

- [Regex validate PIN code](#) (7 kyu)
Pista: Usa la expresión regular `^\d{4}(\d{2})?$` o `^[0-9]{4}([0-9]{2})?$` para validar PINs de 4 o 6 dígitos.
- [Sum of two lowest positive integers](#) (7 kyu)
Pista: Ordena el array con `Arrays.sort()` y suma los dos primeros elementos.
- [Growth of a Population](#) (6 kyu)
Pista: Bucle while: año a año, aplica el crecimiento $p = p + p * (\text{percent} / 100) + \text{aug}$ hasta superar el objetivo.

AceptaElReto:

- [120 - Número de pares y nones](#) (★)
Pista: Itera por los dígitos del número y cuenta cuántos son pares y cuántos impares.
- [157 - ¿Son iguales?](#) (★★)
Pista: Dos números son “iguales” según el problema si al intercambiar un par de dígitos coinciden. Compara dígito a dígito.

Boletín 6 – Inicial Resuelto: Herencia y Polimorfismo

Aquí con soluciones y explicaciones. Sin trampas.

Ejercicio 1: ¿Qué imprime? — La llamada a la familia

```
class Abuelo {
    void saludar() { System.out.println("Soy el abuelo"); }
}
class Padre extends Abuelo {
    void saludar() { System.out.println("Soy el padre"); }
}
class Hijo extends Padre {
    void saludar() { System.out.println("Soy el hijo"); }
}

public class Test {
    public static void main(String[] args) {
        Hijo h = new Hijo();
        h.saludar();
    }
}
```

Solución: Imprime "Soy el hijo".

💡 **Explicación:** Java busca el método `saludar()` empezando por la clase más específica (Hijo). La encuentra ahí mismo y la ejecuta. Nunca sube a Padre ni a Abuelo. Esto se llama *dynamic dispatch*: el método se resuelve en tiempo de ejecución según el tipo real del objeto, no según el tipo de la referencia. Y como `h` es un Hijo de verdad (de los que no llaman a los abuelos), ejecuta su propia versión.

Ejercicio 2: Encuentra el error — extends mal usado

```

public class Vehiculo {
    private String matricula;
    public Vehiculo(String matricula) {
        this.matricula = matricula;
    }
}

public class Coche extends Vehiculo {
    private int numPuertas;
    public Coche(int numPuertas) {
        this.numPuertas = numPuertas;
    }
}

```

Solución: El error es que Coche no llama al constructor de Vehiculo. Cuando una clase hija no pone `super()` explícitamente, Java intenta llamar a `super()` sin parámetros. Pero Vehiculo no tiene constructor sin parámetros — solo tiene `Vehiculo(String)`. Por tanto: **error de compilación**.

```

public class Coche extends Vehiculo {
    private int numPuertas;
    public Coche(String matricula, int numPuertas) {
        super(matricula); // ¡Aquí está la clave!
        this.numPuertas = numPuertas;
    }
}

```

💡 **Explicación:** Piensa en `super()` como llamar a papá para que configure su parte antes de que tú configures la tuya. Si papá necesita una matrícula para construirse, tú tienes que pasársela. No puedes escaquearte. Es como intentar construir una casa sin cimientos: el constructor del padre es la base.

Ejercicio 3: Completa el código — the instanceof

```
class Animal {
    public void hacerSonido() {
        System.out.println("...");
    }
}

class Perro extends Animal {
    public void ladrar() {
        System.out.println("¡Guau!");
    }
}

// En main:
Animal a = new Perro();
if (a instanceof Perro) {
    Perro p = (Perro) a;
    p.ladrar();
}
```

💡 **Explicación:** instanceof pregunta: “¿eres realmente un Perro o solo te disfrazas de Animal?”. Como el objeto real es un new Perro(), la respuesta es true. Luego hacemos downcasting con (Perro) a para tener una referencia de tipo Perro y poder llamar a ladrar(). Sin el casting, el compilador no te deja: “Oye, Animal no tiene ladrar(), no me engañas”.

Ejercicio 4: Escribe este programa — la jerarquía de animales


```

class SerVivo {
    protected int edad;

    public SerVivo(int edad) {
        this.edad = edad;
    }
}

class Animal extends SerVivo {
    protected String nombre;

    public Animal(String nombre, int edad) {
        super(edad);
        this.nombre = nombre;
    }
}

class Perro extends Animal {
    public Perro(String nombre, int edad) {
        super(nombre, edad);
    }

    public void ladrar() {
        System.out.println(nombre + " dice: ¡Guau! Edad: " + edad);
    }
}

public class Test {
    public static void main(String[] args) {
        Perro p = new Perro("Firulais", 3);
        p.ladrar();
    }
}

```

Salida: Firulais dice: ¡Guau! Edad: 3



Explicación: La herencia en cadena: Perro → Animal → SerVivo. Cada constructor llama a su padre con `super()`. Al final, Perro tiene acceso a nombre (de Animal) y edad (de SerVivo) porque están declarados como `protected`. Si fueran `private`, ni Perro los vería. Es como la herencia familiar: lo que es privado en casa de los abuelos, no lo ven ni los nietos.

Ejercicio 5: ¿Qué imprime? — Referencias polimórficas

```
class A {  
    void foo() { System.out.println("A"); }  
}  
class B extends A {  
    void foo() { System.out.println("B"); }  
}  
class C extends B { }  
  
public class Test {  
    public static void main(String[] args) {  
        A ref1 = new B();  
        A ref2 = new C();  
        B ref3 = new C();  
  
        ref1.foo(); // "B"  
        ref2.foo(); // "C"  
        ref3.foo(); // "C"  
    }  
}
```

💡 **Explicación:** El tipo de la REFERENCIA (A, A, B) no importa. Lo que importa es el tipo REAL del objeto (B, C, C). Java siempre ejecuta el método más específico del objeto real. Es como si llevaras una chaqueta de tu padre: por fuera pareces tu padre (la referencia), pero por dentro eres tú (el objeto). Cuando hablas, se oye tu voz, no la de tu padre. Dynamic binding en todo su esplendor.

Ejercicio 6: Encuentra el error — polimorfismo y casting

```
Animal a = new Gato();  
Perro p = (Perro) a; // ✨ ClassCastException en runtime  
p.ladRAR();
```

Solución: El error es que estás intentando disfrazar un Gato de Perro. Java no se deja engañar: en tiempo de ejecución, el objeto real es un Gato, no un Perro, y lanza `ClassCastException`.

Para arreglarlo:

```
Animal a = new Gato();  
if (a instanceof Perro) {  
    Perro p = (Perro) a;  
    p.ladRAR();  
} else {  
    System.out.println("No es un perro, no puedo hacerlo ladRAR.");  
}
```

💡 **Explicación:** `instanceof` es tu red de seguridad. Siempre úsalo antes de hacer downcasting. Es como mirar por la mirilla antes de abrir la puerta: si es un repartidor de pizzas, abre; si es un león, mejor no. En este caso, es un Gato, así que el `instanceof` da false y te ahorras el `ClassCastException`.

Ejercicio 7: Escribe este programa — el zoo polimórfico

```

import java.util.ArrayList;

abstract class Animal {
    public abstract void hacerSonido();
}

class Perro extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("¡Guau!");
    }
}

class Gato extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("¡Miau!");
    }
}

public class Zoo {
    public static void main(String[] args) {
        ArrayList<Animal> animales = new ArrayList<>();
        animales.add(new Perro());
        animales.add(new Gato());

        for (Animal a : animales) {
            a.hacerSonido();
        }
    }
}

```

Salida:

```

¡Guau!
¡Miau!

```



Explicación: Aquí pasa la magia del polimorfismo. Declaramos `ArrayList<Animal>`, que acepta cualquier subclase de `Animal`. Cuando recorremos la lista con un for-each, cada `Animal a` apunta a un objeto diferente (primero `Perro`, luego `Gato`). Pero al llamar a `hacerSonido()`, Java ejecuta la versión del objeto real, no la de `Animal`. Un solo bucle,

múltiples comportamientos. Esto es polimorfismo en acción. Sin él, tendrías que tener listas separadas para cada tipo de animal, como en la edad de piedra de la programación.

Referencias para seguir practicando

- CodeWars: [Is this a triangle?](#) (7 kyu)
- CodeWars: [Basic subclasses - Adam and Eve](#) (8 kyu)
- AceptaElReto.com: [364 - Spiderman](#)
- AceptaElReto.com: [458 - El espejo](#)

Boletín 6 – Inicial: Herencia y Polimorfismo

Sin soluciones. La herencia en Java es como la de verdad: a veces te llevas bien con las subclases, a veces quieres renegar de todo.

Ejercicio 1: ¿Qué imprime? — La familia musical

```
class Musico {  
    void tocar() { System.out.println("El músico toca un instrumento"); }  
}  
  
class Guitarrista extends Musico {  
    void tocar() { System.out.println("El guitarrista toca la guitarra"); }  
}  
  
class Bajista extends Guitarrista {  
    void tocar() { System.out.println("El bajista toca el bajo"); }  
}  
  
public class Banda {  
    public static void main(String[] args) {  
        Bajista b = new Bajista();  
        b.tocar();  
    }  
}
```

¿Qué imprime? ¿Por qué?

Ejercicio 2: Encuentra el error — extends mal usado II

```
public class Animal {
    private String especie;
    public Animal(String especie) {
        this.especie = especie;
    }
}

public class Perro extends Animal {
    private String raza;
    public Perro(String raza) {
        this.raza = raza;
    }
}
```

Este código **no compila**. ¿Por qué? Explica el error y corrígelo.

Ejercicio 3: Completa el código — instanceof con downcasting

Completa el siguiente programa para que funcione correctamente:

```
Vehiculo v = new Coche();
if (v _____ Coche) {    // ¿qué va aquí?
    _____ c = (_____) v;    // downcasting
    c.conducir();
}
```

Declara las clases Vehiculo (con método mover()) y Coche (con método conducir()) para que el código compile.

Ejercicio 4: Escribe este programa — la herencia de vehículos

Crea una jerarquía de 3 niveles usando extends:

- Vehiculo (atributo: String marca)
- Coche (atributo: int numPuertas)

- Deportivo (atributo: int velocidadMaxima)

Cada clase debe tener un constructor que reciba sus atributos y use super. En main(), crea un Deportivo de marca “Ferrari”, 2 puertas y 340 km/h de velocidad máxima. Imprime sus atributos.

Ejercicio 5: ¿Qué imprime? — Polimorfismo con referencias

```
class X {
    void mensaje() { System.out.println("X"); }
}
class Y extends X {
    void mensaje() { System.out.println("Y"); }
}
class Z extends Y { }

public class Test {
    public static void main(String[] args) {
        X ref1 = new Y();
        X ref2 = new Z();
        Y ref3 = new Z();

        ref1.mensaje();
        ref2.mensaje();
        ref3.mensaje();
    }
}
```

¿Qué imprime cada llamada?

Ejercicio 6: Encuentra el error — ClassCastException

```
Animal a = new Perro();
Gato g = (Gato) a;
g.maullar();
```


Suponiendo que Perro y Gato extienden Animal, y Gato tiene método maullar(), ¿qué ocurre en tiempo de ejecución? ¿Cómo lo arreglarías?

Ejercicio 7: Escribe este programa — la granja polimórfica

Crea una clase Animal con método hacerSonido(). Crea Vaca, Oveja y Gallina que lo sobrescriban. En main(), crea un ArrayList<Animal>, mete una vaca, una oveja y una gallina, recórrelo con un for-each llamando a hacerSonido().

```
// Salida esperada:  
// Muuuu  
// Beee  
// Cloc cloc
```

Referencias para seguir practicando

- CodeWars: [Is this a triangle?](#) (7 kyu)
- CodeWars: [Basic subclasses - Adam and Eve](#) (8 kyu)
- AceptaElReto.com: [120 - Número de pares y nones](#)
- AceptaElReto.com: [154 - El ascensor](#)

Boletín 6 – Resuelto: Herencia y Polimorfismo

Dificultad progresiva. De ★ a ★★★★★.

★ Ejercicio 1: La orquesta polimórfica

Crea una clase `Instrumento` con método `tocar()`. Crea 3 subclases: `Guitarra`, `Piano`, `Bateria` que sobrescriban `tocar()`. En `main()`, crea un array de `Instrumento` con una instancia de cada uno y recórrelo llamando a `tocar()`.

★ Ejercicio 2: Herencia de constructores — la cadena alimenticia

Crea: `Animal` (constructor con `String nombre`), `Mamifero` (constructor con `String nombre`, `boolean tienePelo`), `Perro` (constructor con `String nombre`, `boolean tienePelo`, `String raza`). Cada constructor debe llamar al de su padre. Demuestra que funciona.

★ Ejercicio 3: El problema del jarrón (Fragile Base Class)

Crea `Base` con métodos `a()` y `b()` donde `b()` llama internamente a `a()`. Crea `Derivada` que sobrescribe `a()`. Crea una instancia de `Derivada` y llama a `b()`. ¿Qué ocurre?

★ ★ Ejercicio 4: Batalla de personajes

Crea `Personaje` (abstracta) con `nombre`, `vida`, `atacar(Personaje p)` y `recibirDano(int dmg)`. Crea `Guerrero` (daño fijo 20), `Mago` (daño 30 pero solo cada 2 turnos) y `Arquero` (daño 15 pero siempre acierta). Simula una batalla.

★ ★ Ejercicio 5: La calculadora de figuras

Crea `Figura` con método abstracto `calcularArea()`. Implementa `Circulo`, `Rectangulo` y `Triangulo`. En `main()`, crea un `ArrayList<Figura>`, añade varias figuras y calcula el área total.

☆☆ Ejercicio 6: Downcasting seguro

Jerarquía `Empleado` → `Programador`, `Diseñador`. `Programador` tiene `escribirCodigo()`, `Diseñador` tiene `diseñar()`. Crea un `ArrayList<Empleado>` con varios empleados y recórrelo usando `instanceof` para llamar a los métodos específicos.

☆☆☆ Ejercicio 7 (ProgramaMe): El simulador de ecosistema

Diseña un simulador de ecosistema con la siguiente jerarquía:

```
SerVivo (abstracta)
├─ Animal (abstracta)
│   ├── Herbivoro
│   └─ Carnivoro
└─ Planta
```

Cada `SerVivo` tiene `int energia`. Los `Animal` pueden `mover()`, `comer(SerVivo objetivo)`. Los `Herbivoro` solo comen `Planta`. Los `Carnivoro` solo comen `Animal`. Cada `Planta` hace `fotosintesis()` que le da +10 de energía.

Simula un ecosistema con 3 plantas, 2 herbívoros y 1 carnívoro durante 10 turnos. Cada turno: las plantas hacen fotosíntesis, los herbívoros comen plantas si pueden, el carnívoro come herbívoros si puede. Si un ser vivo llega a 0 de energía, muere.

Al final, muestra quién sobrevivió.

☆☆☆ Ejercicio 8 (ProgramaMe): Vehículos con combustible

Crea una jerarquía de vehículos:

- `Vehiculo (abstracta)`: `String matricula`, `int combustible`, `abstract void mover()`

- Coche: gasta 5 de combustible por movimiento
- Moto: gasta 3 de combustible por movimiento
- Camion: gasta 10 de combustible por movimiento, pero puede llevar int carga

Cada vehículo tiene un método `mover()` que reduce el combustible. Si no hay suficiente, imprime “Sin combustible”.

En `main()`, crea un `ArrayList<Vehiculo>` con varios vehículos. Cada vehículo se mueve repetidamente hasta que se queda sin combustible. Lleva la cuenta de cuántos movimientos hizo cada uno.

Referencias para seguir practicando

- CodeWars: [Thinkful - Logic Drills: Traffic light](#) (7 kyu)
- CodeWars: [Fun with ES6 Classes #1 - People](#) (7 kyu)
- CodeWars: [Fight Club](#) (7 kyu)
- AceptaElReto.com: [364 - Spiderman](#)
- AceptaElReto.com: [462 - Tres dedos](#)
- AceptaElReto.com: [458 - El espejo](#)

Boletín 6 – Intermedio: Herencia, Polimorfismo e Interfaces

Ejercicios de dificultad progresiva. Los ★ son para calentar, ★★ para pensar, ★★★ para concursar. La herencia es como la familia: a veces heredas cosas buenas, a veces te toca la colección de sellos de tu tío abuelo. Pero con interfaces, al menos eliges qué implementar.

★ Ejercicio 1: Interfaz FiguraGeometrica

Crea una interfaz `FiguraGeometrica` con dos métodos:

- `double calcularArea()`
- `double calcularPerimetro()`

Implementa la interfaz en:

- `Circulo` (constructor con radio)
- `Rectangulo` (constructor con ancho y alto)
- `TrianguloRectangulo` (constructor con base y altura)

```

public interface FiguraGeometrica {
    double calcularArea();
    double calcularPerimetro();
}

public class Circulo implements FiguraGeometrica {
    private double radio;

    public Circulo(double radio) {
        this.radio = radio;
    }

    @Override
    public double calcularArea() {
        return Math.PI * radio * radio;
    }

    @Override
    public double calcularPerimetro() {
        return 2 * Math.PI * radio;
    }
}

// Implementa Rectangulo y TrianguloRectangulo

```

En el main, crea un `ArrayList<FiguraGeometrica>`, añade un círculo de radio 5, un rectángulo 4x3 y un triángulo rectángulo 3x4. Recórrelos imprimiendo área y perímetro de cada uno.

★ Ejercicio 2: Jerarquía de Empleados

Crea una clase base `Empleado` con:

- `String nombre`
- `double salarioBase`
- Constructor, getters
- Método `double calcularSalario()` que devuelva el `salarioBase`

Crea dos subclases:

- `Gerente`: tiene un `double bono extra`. `calcularSalario()` devuelve `salarioBase + bono`.

- Vendedor: tiene un double comision por venta y un int ventasRealizadas. calcularSalario() devuelve salarioBase + comision * ventasRealizadas.

```
public class Empleado {
    protected String nombre;
    protected double salarioBase;

    public Empleado(String nombre, double salarioBase) {
        this.nombre = nombre;
        this.salarioBase = salarioBase;
    }

    public double calcularSalario() {
        return salarioBase;
    }
}

public class Gerente extends Empleado {
    private double bono;
    // constructor y override de calcularSalario()
}

public class Vendedor extends Empleado {
    private double comision;
    private int ventasRealizadas;
    // constructor y override de calcularSalario()
}
```

En el main, crea un array de Empleado con un gerente y un vendedor, recórrelo polimórficamente y muestra el salario de cada uno.

★ ★ Ejercicio 3: Sistema de pagos con interfaz

Crea una interfaz Pagable con el método:

- boolean procesarPago(double cantidad)

Implementa la interfaz en:

- TarjetaCredito: tiene double limite y double saldoUsado. Puede pagar si cantidad + saldoUsado <= limite.

- PayPal: tiene double saldo. Puede pagar si cantidad <= saldo.
- TransferenciaBancaria: tiene double saldo. Puede pagar siempre que cantidad <= saldo, pero tiene un coste fijo de 1€ por transferencia.

```
public interface Pagable {  
    boolean procesarPago(double cantidad);  
}
```

Ejemplo:

```
Pagable tarjeta = new TarjetaCredito(1000, 0);  
tarjeta.procesarPago(500); // true  
tarjeta.procesarPago(600); // false (supera el límite)
```

★ ★ Ejercicio 4: Interfaces múltiples: Volador y Nadador

Crea dos interfaces:

- Volador: método void volar()
- Nadador: método void nadar()

Crea una clase Pato que implemente ambas interfaces, más una clase Avion que solo implemente Volador y una clase Pez que solo implemente Nadador.


```

public interface Volador {
    void volar();
}

public interface Nadador {
    void nadar();
}

public class Pato implements Volador, Nadador {
    @Override
    public void volar() {
        System.out.println("El pato vuela en formación en V");
    }

    @Override
    public void nadar() {
        System.out.println("El pato nada tranquilamente en el estanque");
    }
}

```

En el main, crea un `ArrayList<Volador>` con un Pato y un Avion, y recórrelo llamando a `volar()`. Luego haz lo mismo con un `ArrayList<Nadador>`.

Ejercicio 5: Sistema de notificaciones polimórfico

Crea una interfaz `Notificable` con:

- `void enviar(String mensaje)`
- `String getEstado()`

Implementa:

- `EmailNotificacion`: atributos `String direccion`, `boolean enviado`. Al enviar, imprime “Enviando email a [dirección]: [mensaje]”. Estado: “Enviado” o “Pendiente”.
- `SMSNotificacion`: atributos `String telefono`, `boolean enviado`. Al enviar, imprime “Enviando SMS a [teléfono]: [mensaje]”. Estado similar.
- `PushNotificacion`: atributos `String dispositivoId`, `boolean enviado`. Al enviar, imprime “Enviando push a [dispositivoid]: [mensaje]”.

```
public interface Notificable {  
    void enviar(String mensaje);  
    String getEstado();  
}
```

En el main, crea un `ArrayList<Notificable>` con los tres tipos. Añade también un método estático que recorra la lista y envíe todas las notificaciones:

```
public static void enviarTodas(List<Notificable> notificaciones, String mensaje) {  
    for (Notificable n : notificaciones) {  
        n.enviar(mensaje);  
    }  
}
```

☆☆☆ Ejercicio 6: CodeWars — Is this a triangle?

Resuelve la kata “Is this a triangle?” (7 kyu) en CodeWars.

Implementa una clase `Triangle` con un método estático `isTriangle(int a, int b, int c)` que devuelva `true` si con las tres longitudes se puede formar un triángulo (la suma de dos lados siempre es mayor que el tercero).

```
public class TriangleValidator {  
    public static boolean isTriangle(int a, int b, int c) {  
        // tu código aquí  
    }  
}
```

Ejemplo: `isTriangle(3, 4, 5) → true`, `isTriangle(1, 2, 3) → false` (1+2 no es mayor que 3).

Pista: Un triángulo existe si $a + b > c$ && $a + c > b$ && $b + c > a$. Es decir, la suma de dos lados siempre es mayor que el lado restante. Esta kata es de lógica pura, pero puedes aprovechar para practicar herencia creando una jerarquía de figuras si quieres ir más allá.



Ejercicio 7: AceptaElReto — 154 El ascensor

Resuelve el problema **154 — El ascensor** en [AceptaElReto.com](https://www.aceptaelreto.com).

El problema modela un ascensor que recorre varias plantas. Dada una secuencia de plantas a las que va, calcula cuántas veces cambia de dirección (sube → baja o baja → sube). El ascensor empieza en la planta 0.

Ejemplo: Plantas: 3, 5, 2, 7, 9, 1 → Direcciones: sube(3), sube(5), baja(2), sube(7), sube(9), baja(1) → Cambios: 3 (cuando baja a 2, cuando sube a 7, cuando baja a 1).

Pista: Lee la secuencia de plantas y lleva la cuenta de la dirección anterior. Cuando la dirección cambie, incrementa el contador. Si el ascensor se queda en la misma planta, no hay cambio de dirección. Piensa en cómo modelarías esto con objetos: una clase Ascensor con método moverA(int planta).



Referencias

Plataforma	Problema	Dificultad
CodeWars	Is this a triangle?	7 kyu
CodeWars	Thinkful - Logic Drills: Traffic light	7 kyu
AceptaElReto	154 — El ascensor	Medio
AceptaElReto	369 — Navegación en Google Maps	Difícil

Boletín 08 — Extras: Herencia, Polimorfismo e Interfaces

★ Extras — Conviértete en un programador de muy alto nivel

Estos ejercicios son completamente opcionales, pero si aspiras a ser un programador o programadora de élite, aquí tienes tu campo de entrenamiento. CodeWars y AceptaElReto son plataformas donde los mejores programadores del mundo se retan a diario. Cada problema que resuelvas te hará más fuerte, más rápido y más creativo. No los subestimes: son la diferencia entre saber programar y ser un verdadero profesional.

CodeWars:

- [Convert string to camel case](#) (6 kyu)

Pista: Divide el string por guiones o guiones bajos, capitaliza la primera letra de cada palabra excepto la primera.

- [Counting Duplicates](#) (6 kyu)

Pista: Convierte a minúsculas, cuenta frecuencias con un `Map<Character, Long>` y filtra las que aparecen más de una vez.

- [Human Readable Time](#) (5 kyu)

Pista: Usa división entera y módulo: `horas = segundos / 3600`, `minutos = (segundos % 3600) / 60`, `segundos = segundos % 60`. Formatea con `String.format("%02d:%02d:%02d")`.

AceptaElReto:

- [154 - El ascensor](#) (★ ★)

Pista: Simula planta por planta; cuenta solo las veces que cambia la dirección del ascensor.

- [369 - Navegación en Google Maps](#) (★ ★ ★)

Pista: Modela las intersecciones como nodos de un grafo y aplica Dijkstra con cola de prioridad.

Boletín 8 – Inicial Resuelto: Arrays y Colecciones

Con soluciones. A aprender.

Ejercicio 1: ¿Qué imprime? — Array básico

```
int[] nums = {1, 2, 3, 4, 5};
for (int i = 0; i < nums.length; i++) {
    nums[i] = nums[i] * 2;
}
System.out.println(nums[3]);
```

Solución: Imprime 8.

💡 **Explicación:** El array original es {1, 2, 3, 4, 5}. El bucle multiplica cada elemento por 2: {2, 4, 6, 8, 10}. `nums[3]` es el cuarto elemento (recuerda: los índices empiezan en 0). Así que `nums[3] = 8`.

Ejercicio 2: Encuentra el error — `ArrayIndexOutOfBoundsException`

```
String[] nombres = new String[3];
nombres[0] = "Ana";
nombres[1] = "Bob";
nombres[2] = "Carlos";
nombres[3] = "Diana";
```

Solución: La última línea lanza `ArrayIndexOutOfBoundsException`. El array tiene tamaño 3, los índices válidos son 0, 1 y 2. `nombres[3]` está fuera de los límites.

💡 **Explicación:** Los arrays en Java tienen tamaño fijo. Si declaras `new String[3]`, las plazas son 0, 1 y 2. Intentar acceder a la 3 es como intentar aparcar en una plaza que no existe: el parking solo tiene 3 plazas (índices 0, 1, 2) y tú buscas la 3. Crash asegurado.

Ejercicio 3: Completa el código — for-each

```
int[] numeros = {4, 7, 2, 9, 5};
int suma = 0;

for (int n : numeros) {
    suma += n;
}

System.out.println("Suma: " + suma);
```

Solución: `int n : numeros` y `suma += n`. La suma total es 27.

💡 **Explicación:** El for-each (o “enhanced for”) recorre cada elemento del array sin necesidad de índice. `int n` toma el valor de cada elemento en cada iteración. `suma += n` es equivalente a `suma = suma + n`. Es más limpio que el for tradicional cuando solo necesitas leer y no modificar.

Ejercicio 4: Escribe este programa — Array de 10 enteros al revés

```
public class ArrayInverso {
    public static void main(String[] args) {
        int[] numeros = new int[10];
        for (int i = 0; i < numeros.length; i++) {
            numeros[i] = i + 1;
        }
        for (int i = numeros.length - 1; i >= 0; i--) {
            System.out.print(numeros[i] + " ");
        }
    }
}
```

Salida: 10 9 8 7 6 5 4 3 2 1

💡 **Explicación:** Dos bucles: uno para rellenar (del 1 al 10) y otro para imprimir al revés (del índice 9 al 0). El truco está en `numeros.length - 1` como índice inicial (último elemento) y `i >= 0` como condición (incluir el primero). Es como leer una lista del final al principio.

Ejercicio 5: ¿Qué imprime? — ArrayList misterioso

```
import java.util.ArrayList;

public class Test {
    public static void main(String[] args) {
        ArrayList<Integer> lista = new ArrayList<>();
        lista.add(10);
        lista.add(20);
        lista.add(30);
        lista.add(1, 15);          // inserta 15 en índice 1
        lista.remove(Integer.valueOf(20)); // borra el objeto 20

        for (Integer n : lista) {
            System.out.print(n + " ");
        }
    }
}
```

Solución: Imprime 10 15 30.

💡 **Explicación:** Paso a paso: `add(10)` → [10], `add(20)` → [10, 20], `add(30)` → [10, 20, 30], `add(1, 15)` → [10, **15**, 20, 30] (inserta, no reemplaza), `remove(Integer.valueOf(20))` → [10, 15, 30] (borra la primera ocurrencia del valor 20). Fíjate que `remove(1)` (con int) borraría por índice, pero `remove(Integer.valueOf(1))` (con Integer) borra por valor.

Ejercicio 6: Encuentra el error — colección sin genérico

```
ArrayList lista = new ArrayList();
lista.add("Hola");
lista.add(42);
lista.add(3.14);

String texto = (String) lista.get(1); // 🌟 ClassCastException
```

Solución: `lista.get(1)` devuelve el Integer 42 (por el autoboxing), pero intentas castearlo a String. Como un Integer no es un String, salta `ClassCastException` en tiempo de ejecución.

💡 **Explicación:** Sin genéricos, ArrayList es una caja de caos donde todo es Object. El compilador no te avisa. Te enteras cuando el programa ya está ejecutándose y explota. Con genéricos: ArrayList<String> solo acepta Strings, y el error saltaría en compilación. Los genéricos convierten errores de runtime en errores de compilación, que es donde queremos que estén.

Ejercicio 7: Escribe este programa — HashSet sin duplicados

```
import java.util.HashSet;
import java.util.Scanner;

public class SinDuplicados {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        HashSet<String> palabras = new HashSet<>();

        System.out.println("Introduce 5 palabras:");
        for (int i = 0; i < 5; i++) {
            System.out.print("Palabra " + (i + 1) + ": ");
            String p = sc.nextLine();
            palabras.add(p);
        }

        System.out.println("Palabras distintas: " + palabras.size());
        System.out.println("Contenido: " + palabras);
        sc.close();
    }
}
```

💡 **Explicación:** HashSet no permite duplicados. Si el usuario introduce “hola” dos veces, la segunda llamada a add(“hola”) simplemente no hace nada y devuelve false. Al final, size() te dice cuántas palabras distintas hay. Es el mecanismo más sencillo para eliminar duplicados en Java: deja que HashSet haga el trabajo sucio.

Referencias para seguir practicando

- CodeWars: [Convert number to reversed array of digits](#) (8 kyu)
- CodeWars: [Find the smallest integer in the array](#) (7 kyu)
- AceptaElReto.com: [100 - Kaprekar](#)
- AceptaElReto.com: [152 - Suma pares e impares](#)

Boletín 8 – Inicial: Arrays y Colecciones

Sin soluciones. A programar.

Ejercicio 1: ¿Qué imprime? — Array de booleanos

```
boolean[] flags = new boolean[3];
flags[1] = true;
System.out.println(flags[0] + " " + flags[1] + " " + flags[2]);
```

¿Qué imprime? ¿Cuál es el valor por defecto de un boolean en un array?

Ejercicio 2: Encuentra el error — NullPointerException

```
String[] nombres = new String[3];
nombres[0] = "Ana";
nombres[1] = "Bob";
System.out.println(nombres[2].toUpperCase());
```

¿Qué ocurre al ejecutar este código? ¿Por qué?

Ejercicio 3: Completa el código — for básico para buscar el mayor

Completa el siguiente programa para que encuentre e imprima el número más grande del array:

```
int[] numeros = {12, 45, 7, 34, 89, 23};
int mayor = numeros[0];

for (int i = 1; i < _____; i++) {    // ¿hasta dónde llega el bucle?
    if (numeros[i] _____ mayor) {    // ¿qué operador?
        _____ = numeros[i];        // ¿qué asignamos?
    }
}

System.out.println("El mayor es: " + mayor);
```

Ejercicio 4: Escribe este programa — contar números pares

Crea un array de 10 enteros con valores que tú elijas. Recórrelo con un bucle for y cuenta cuántos de ellos son pares. Al final, imprime el total de pares y el array original.

Ejemplo de salida:

```
Array: [3, 8, 12, 5, 7, 10, 2, 9, 6, 1]
Pares: 5
```

Ejercicio 5: ¿Qué imprime? — ArrayList remove por índice vs valor

```
import java.util.ArrayList;

public class Test {
    public static void main(String[] args) {
        ArrayList<String> lista = new ArrayList<>();
        lista.add("A");
        lista.add("B");
        lista.add("C");
        lista.add("B");
        lista.add("D");

        lista.remove(1);           // remove por índice
        lista.remove("B");         // remove por objeto

        System.out.println(lista);
    }
}
```

¿Qué imprime? ¿Por qué el segundo remove("B") no borra el mismo que el primero?

Ejercicio 6: Encuentra el error — length vs length()

```
int[] numeros = {10, 20, 30};
String texto = "Hola";

System.out.println(numeros.length());
System.out.println(texto.length);
```

¿Qué líneas tienen error? Explica la diferencia entre length (sin paréntesis) y length() (con paréntesis).

Ejercicio 7: Escribe este programa — búsqueda lineal

Crea un array de enteros llamado edades con 8 valores. Pide al usuario un número por teclado (con Scanner) y busca si ese número está en el array. Imprime «Encontrado» o «No encontrado».

Introduce edad a buscar: 25

Encontrado en posición 3

Referencias para seguir practicando

- CodeWars: [Convert number to reversed array of digits](#) (8 kyu)
- CodeWars: [Find the smallest integer in the array](#) (7 kyu)
- AceptaElReto.com: [152 - Suma pares e impares](#)
- AceptaElReto.com: [100 - Kaprekar](#)

Boletín 8 – Resuelto: Arrays y Colecciones

Dificultad progresiva. De ★ a ★★★★★.

★ Ejercicio 1: El inversor de arrays

Escribe un método `invertirArray(int[] arr)` que devuelva un nuevo array con los elementos en orden inverso. Pruébalo en `main()`.

★ Ejercicio 2: ArrayList de números pares

Crea un programa que genere los primeros 20 números pares (2, 4, 6, ..., 40) y los guarde en un `ArrayList<Integer>`. Luego, recorre la lista y suma solo los que sean múltiplos de 5.

★ Ejercicio 3: Estadísticas con array

Pide al usuario las notas de 10 alumnos (usa `Scanner`), guárdalas en un array `double[]`, y calcula: nota media, nota máxima, nota mínima y cuántos aprobaron (≥ 5).

★ ★ Ejercicio 4: Buscaminas simplificado

Crea un array bidimensional `boolean[5][5]` que represente un campo de minas. Coloca 5 minas (`true`) en posiciones aleatorias. El usuario introduce coordenadas (fila, columna) y el programa dice si hay mina. Si acierta una mina, el juego termina.

★ ★ Ejercicio 5: Lista de la compra con ArrayList

Crea un programa que gestione una lista de la compra usando `ArrayList<String>`. Menú con opciones: 1) Añadir, 2) Eliminar, 3) Marcar como comprado (añadir “[OK]” al final), 4) Mostrar lista, 0) Salir.

☆☆ Ejercicio 6: HashSet y TreeSet — eliminando duplicados

Crea un programa que lea una frase del usuario, separe las palabras (con `split(" ")`), las guarde en un `TreeSet<String>` y luego las muestre ordenadas alfabéticamente y sin duplicados.

☆☆☆ Ejercicio 7 (ProgramaMe): El juego de la memoria

Crea un juego de memoria usando un array bidimensional `int[4][4]`. Coloca 8 pares de números (1-8) en posiciones aleatorias. El jugador destapa dos posiciones. Si son iguales, permanecen visibles. Si no, se ocultan de nuevo. El juego termina cuando todos los pares están descubiertos.

☆☆☆ Ejercicio 8 (ProgramaMe): Estadísticas de texto con colecciones

Crea un programa que analice un texto largo (puedes hardcodearlo) y muestre:

1. Número total de palabras
2. Las 5 palabras más frecuentes
3. La palabra más larga
4. Porcentaje de palabras únicas

Usa `HashMap<String, Integer>` para frecuencias, `TreeSet` para orden, y `ArrayList` para manipulación.

Referencias para seguir practicando

- CodeWars: [Convert number to reversed array of digits](#) (8 kyu)
- CodeWars: [Find the smallest integer in the array](#) (7 kyu)
- CodeWars: [Sorting arrays](#) (7 kyu)
- AceptaElReto.com: [100 - Kaprekar](#)

- AceptaElReto.com: [340 - Juegos de naipes](#)
- AceptaElReto.com: [101 - Cuadrado Sator](#)

Boletín 8 - Intermedio: Arrays y Colecciones

Ejercicios de dificultad progresiva. De ★ a ★★★★★.

★ Ejercicio 1: La fusión de arrays ordenados

Escribe un método `fusionarArrays(int[] a, int[] b)` que reciba dos arrays ordenados de menor a mayor y devuelva un **nuevo array** también ordenado con todos los elementos de ambos. No uses `Arrays.sort()` ni colecciones. Hazlo con el algoritmo de fusión (merge) tipo «dos punteros».

Pista: Avanza con dos índices, uno por array, comparando en cada paso cuál elemento es menor.

★ Ejercicio 2: Rotación circular a la derecha

Implementa un método `rotarDerecha(int[] arr, int k)` que desplace cada elemento del array k posiciones hacia la derecha. Los elementos que «salen» por el final vuelven a entrar por el principio. No uses estructuras auxiliares grandes (vale un array temporal del tamaño de k , pero no copies todo).

Ejemplo: $\{1, 2, 3, 4, 5\}$ con $k = 2 \rightarrow \{4, 5, 1, 2, 3\}$.

★ Ejercicio 3: Suma de diagonales (matriz cuadrada)

Crea un programa que genere una matriz cuadrada `int[N][N]` con valores aleatorios entre 1 y 100, y calcule:

1. Suma de la **diagonal principal** (de arriba-izquierda a abajo-derecha).
2. Suma de la **diagonal secundaria** (de arriba-derecha a abajo-izquierda).
3. Diferencia absoluta entre ambas sumas.

Usa $N = 5$ para las pruebas.

☆☆ Ejercicio 4: Cola del supermercado con LinkedList

Simula una cola de supermercado usando `LinkedList<String>`. El programa debe mostrar un menú:

1. **Llega cliente** → Añade un nombre al final de la cola.
2. **Atender cliente** → Elimina y muestra el primero de la cola.
3. **¿Quién sigue?** → Muestra el primero sin eliminarlo.
4. **Estado de la cola** → Muestra todos los clientes en orden.
5. **Salir**

Usa los métodos `addLast()`, `removeFirst()`, `getFirst()` de `LinkedList`.

☆☆ Ejercicio 5: Intersección y unión de conjuntos

Crea dos `HashSet<Integer>` con números aleatorios (entre 1 y 20, 8 elementos cada uno). Calcula y muestra:

- **Intersección:** elementos que están en ambos conjuntos.
- **Unión:** todos los elementos sin repetir.
- **Diferencia simétrica:** elementos que están en uno u otro, pero no en ambos.

Pista: Usa `retainAll()`, `addAll()`, `removeAll()` de la interfaz `Set`.

☆☆ Ejercicio 6: Eliminar duplicados manteniendo orden

Crea un `ArrayList<Integer>` con elementos repetidos (`[3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]`). Escribe un método que devuelva un nuevo `ArrayList<Integer>` **sin duplicados pero manteniendo el orden de primera aparición**. No puedes usar `HashSet` directamente porque pierdes el orden. La solución es usar un `LinkedHashSet` o recorrer manualmente.

☆☆☆ Ejercicio 7 (CodeWars): Find the odd int

Dado un array de enteros, encuentra el que aparece un número impar de veces.

```
public static int findIt(int[] arr) {  
    // tu código aquí  
}
```

Ejemplo: {1, 2, 2, 3, 3, 3, 4, 3, 3, 3, 2, 2, 1} → devuelve 4.

Pista: Puedes usar un `HashMap<Integer, Integer>` para contar frecuencias, o el truco del XOR (operador ^).

 **CodeWars:** [Find the odd int](#) (6 kyu)

☆☆☆ Ejercicio 8 (AceptaElReto): La lotería de la peña

En una peña de lotería, cada socio aporta una cantidad y compra participaciones. Implementa un programa que:

1. Lea un array con las aportaciones de cada socio (en euros).
2. Calcule el total recaudado.
3. Si toca un premio de X euros, calcule cuánto le corresponde a cada socio proporcionalmente a su aportación.
4. Muestre los resultados ordenados de mayor a menor ganancia.

Usa arrays para almacenar nombres y aportaciones. El premio se pide por teclado.

 **AceptaElReto.com:** [340 - Juegos de naipes](#)

Referencias

- **CodeWars:** [Find the odd int](#) (6 kyu)
- **CodeWars:** [Array.diff](#) (6 kyu)
- **CodeWars:** [Sort the odd](#) (6 kyu)
- **AceptaElReto.com:** [340 - Juegos de naipes](#)

- AceptaElReto.com: [101 - Cuadrado Sator](#)
- AceptaElReto.com: [241 - Buscando el nivel](#)

Boletín 09 — Extras: Arrays y Colecciones

★ Extras — Conviértete en un programador de muy alto nivel

Estos ejercicios son completamente opcionales, pero si aspiras a ser un programador o programadora de élite, aquí tienes tu campo de entrenamiento. CodeWars y AceptaElReto son plataformas donde los mejores programadores del mundo se retan a diario. Cada problema que resuelvas te hará más fuerte, más rápido y más creativo. No los subestimes: son la diferencia entre saber programar y ser un verdadero profesional.

CodeWars:

- [Mexican Wave](#) (6 kyu)

Pista: Itera índice a índice, construye cada string poniendo en mayúscula el carácter en la posición i.

- [Delete occurrences of an element](#) (6 kyu)

Pista: Usa un HashMap para llevar la cuenta de cada elemento; solo añádelo al resultado si su contador no ha superado el límite.

- [Roman Numerals Encoder](#) (6 kyu)

Pista: Recorre los valores en orden descendente (1000→1), resta el valor y añade el símbolo romano correspondiente.

AceptaElReto:

- [102 - Encriptación de mensajes](#) (★ ★)

Pista: Aplica un desplazamiento fijo (cifrado César) a cada carácter; vigila el wrap-around al final del alfabeto.

- [341 - Matriz identidad](#) (★ ★)

Pista: Comprueba que todos los elementos de la diagonal son 1 y el resto son 0.

Boletín 9 – Inicial Resuelto: Genéricos y Mapas

Con soluciones. A aprender.

Ejercicio 1: Completa el código — clase genérica

```
public class Caja<T> {  
    private T contenido;  
  
    public void guardar(T contenido) {  
        this.contenido = contenido;  
    }  
  
    public T sacar() {  
        return contenido;  
    }  
}
```

La declaración correcta es `public class Caja<T>`.

💡 **Explicación:** `<T>` declara un parámetro de tipo. `T` es una convención (de “Type”), pero podrías usar cualquier letra. A partir de ahí, puedes usar `T` como un tipo cualquiera dentro de la clase. Cuando alguien haga `Caja<String>`, todas las `T` se convierten en `String`. Es como una plantilla: dejas huecos (`T`) que se rellenan cuando alguien usa la clase.

Ejercicio 2: ¿Qué imprime? — HashMap básico

```
import java.util.HashMap;

public class Test {
    public static void main(String[] args) {
        HashMap<String, String> capitales = new HashMap<>();
        capitales.put("España", "Madrid");
        capitales.put("Francia", "París");
        capitales.put("Italia", "Roma");
        capitales.put("España", "Barcelona");

        System.out.println(capitales.get("España"));
    }
}
```

Solución: Imprime Barcelona.



Explicación: En un HashMap, las claves son únicas. Cuando haces `put("España", "Madrid")` y luego `put("España", "Barcelona")`, la segunda llamada SOBREScribe el valor anterior. “Madrid” se pierde para siempre. Es como apuntar un número en tu agenda y luego tacharlo para poner otro: solo queda el último. Si quisieras mantener ambos, tendrías que usar una estructura diferente (como `HashMap<String, List<String>>`).

Ejercicio 3: Encuentra el error — tipo primitivo en genérico

```
ArrayList<int> numeros = new ArrayList<>(); // ERROR
numeros.add(1);
numeros.add(2);
numeros.add(3);
```

Solución: No puedes usar tipos primitivos como parámetros de tipo en genéricos. `int` debe ser `Integer`.

```
ArrayList<Integer> numeros = new ArrayList<>();
numeros.add(1); // autoboxing: int → Integer
numeros.add(2);
numeros.add(3);
```

💡 **Explicación:** Los genéricos solo funcionan con tipos referencia (objetos). `int`, `double`, `boolean` son tipos primitivos y no pueden ser parámetros de tipo. Por eso existen las clases wrapper: `Integer`, `Double`, `Boolean`. El autoboxing de Java convierte automáticamente `int` a `Integer` al añadir y `Integer` a `int` al obtener, por lo que apenas notas la diferencia en el código.

Ejercicio 4: Escribe este programa — mini agenda HashMap

```
import java.util.HashMap;
import java.util.Scanner;

public class MiniAgenda {
    public static void main(String[] args) {
        HashMap<String, String> agenda = new HashMap<>();
        agenda.put("Ana", "612345678");
        agenda.put("Bob", "698765432");
        agenda.put("Carlos", "655111222");

        Scanner sc = new Scanner(System.in);
        System.out.print("Buscar teléfono de: ");
        String nombre = sc.nextLine();

        String telefono = agenda.get(nombre);
        if (telefono != null) {
            System.out.println(nombre + " → " + telefono);
        } else {
            System.out.println(nombre + " no está en la agenda.");
        }
        sc.close();
    }
}
```

💡 **Explicación:** `HashMap` es perfecto para asociar nombres con teléfonos. La clave es el nombre (único) y el valor es el teléfono. `get(nombre)` devuelve `null` si la clave no existe, así que comprobamos antes de usarlo. Es la estructura de datos perfecta para una agenda: búsqueda rápida por clave ($O(1)$).

Ejercicio 5: Completa el código — getOrDefault

```
HashMap<String, Integer> edades = new HashMap<>();
edades.put("Ana", 25);
edades.put("Bob", 30);

int edadAna = edades.get("Ana");           // 25
int edadCarlos = edades.get("Carlos");     // null → NullPointerException (en
realidad: error de compilación si es int, NPE si Integer)
int edadCarlosSeguro = edades.getOrDefault("Carlos", 0); // 0
```

Solución: edadAna = 25. edad.get("Carlos") devuelve null (la clave no existe). Si la variable es int, no compila o da error porque no puedes asignar null a un primitivo. getOrDefault("Carlos", 0) devuelve 0 (el valor por defecto).

💡 **Explicación:** getOrDefault() es el salvavidas de los HashMap. En lugar de hacer if (map.get(clave) != null), haces map.getOrDefault(clave, valorDefecto) y te ahorras líneas y posibles NullPointerException. Es como tener un plan B: “si no encuentras a Carlos, devuelve 0”. Siempre que trabajes con mapas, úsalo.

Ejercicio 6: ¿Qué imprime? — método genérico

```
public class Util {
    public static <T> void imprimir(T elemento) {
        System.out.println("Valor: " + elemento);
    }

    public static void main(String[] args) {
        Util.imprimir(42);
        Util.imprimir("Hola");
        Util.imprimir(3.14);
    }
}
```

Solución:

Valor: 42
Valor: Hola
Valor: 3.14

💡 **Explicación:** El método imprimir es genérico: <T> antes del tipo de retorno. El compilador INFIERE el tipo T a partir del argumento. En la primera llamada, T es Integer. En la segunda, T es String. En la tercera, T es Double. No necesitas especificarlo: Java lo deduce solo. Es como un profesor que se adapta al alumno: da la misma clase pero adaptada a cada uno.

Ejercicio 7: Encuentra el error — la clave mutable

```
HashMap<ArrayList<Integer>, String> mapa = new HashMap<>();  
ArrayList<Integer> lista = new ArrayList<>();  
lista.add(1);  
lista.add(2);  
mapa.put(lista, "valor");  
lista.add(3); // modificamos la clave después de usarla  
System.out.println(mapa.get(lista)); // posiblemente null
```

Solución: El problema es que las claves de un HashMap deben ser INMUTABLES. Al modificar lista después de usarla como clave, su hashCode() cambia. El HashMap busca en el bucket antiguo, pero la clave tiene un hash diferente ahora, así que get() puede devolver null aunque la clave esté en el mapa.

💡 **Explicación:** Las claves de un HashMap deben ser inmutables (como String o Integer). Si usas un objeto mutable y lo modificas, el HashMap se vuelve impredecible. Es como cambiar la cerradura de tu casa y esperar que tu llave vieja siga funcionando. Por eso String es la clave perfecta: es inmutable. Nunca uses ArrayList, arrays, o tus propias clases mutables como claves de un HashMap a menos que sepas muy bien lo que haces (spoiler: no lo sabes).

Referencias para seguir practicando

- CodeWars: [Grasshopper - Grade book](#) (7 kyu)
- CodeWars: [Word Count](#) (7 kyu)
- AceptaElReto.com: [416 - Casillas](#)
- AceptaElReto.com: [462 - Tres dedos](#)

Boletín 9 – Inicial: Genéricos y Mapas

Sin soluciones. A darle.

Ejercicio 1: Completa el código — clase con dos tipos genéricos

```
public class Par<_____, _____> {    // ¿qué dos tipos faltan?
    private T primero;
    private U segundo;

    public Par(T primero, U segundo) {
        this.primero = primero;
        this.segundo = segundo;
    }

    public T getPrimero() { return primero; }
    public U getSegundo() { return segundo; }
}
```

Completa la declaración para que Par acepte dos tipos genéricos distintos.

Ejercicio 2: ¿Qué imprime? — HashMap con merge

```
import java.util.HashMap;

public class Test {
    public static void main(String[] args) {
        HashMap<String, Integer> mapa = new HashMap<>();
        mapa.put("Ana", 10);
        mapa.put("Bob", 20);
        mapa.put("Ana", 30);

        System.out.println(mapa.get("Ana"));
        System.out.println(mapa.size());
    }
}
```

¿Qué imprime? ¿Por qué size() no es 3?

Ejercicio 3: Encuentra el error — TreeSet sin Comparable

```
import java.util.TreeSet;

public class Test {
    public static void main(String[] args) {
        TreeSet<Persona> conjunto = new TreeSet<>();
        conjunto.add(new Persona("Ana"));
        conjunto.add(new Persona("Bob"));
        System.out.println(conjunto);
    }
}

class Persona {
    String nombre;
    Persona(String nombre) { this.nombre = nombre; }
}
```

¿Qué ocurre al ejecutar? ¿Por qué TreeSet necesita que Persona implemente una interfaz concreta?

Ejercicio 4: Escribe este programa — contador de palabras con HashMap

Crea un programa que tenga un array de palabras (hardcodeado) como este:

```
String[] palabras = {"hola", "mundo", "hola", "java", "mundo", "hola", "adios"};
```

Usa un `HashMap<String, Integer>` para contar cuántas veces aparece cada palabra. Al final, recorre el mapa con un bucle `for-each` sobre `entrySet()` y muestra cada palabra con su cuenta.

Ejercicio 5: ¿Qué imprime? — método genérico con límite

```
public class Util {  
    public static <T extends Comparable<T>> T maximo(T a, T b) {  
        return a.compareTo(b) > 0 ? a : b;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(maximo(5, 8));  
        System.out.println(maximo("gato", "perro"));  
    }  
}
```

¿Qué imprime? ¿Qué pasaría si T no tuviera el límite Comparable<T>?

Ejercicio 6: Encuentra el error — clave duplicada el primero se pierde

```
HashMap<Integer, String> mapa = new HashMap<>();  
mapa.put(1, "uno");  
mapa.put(2, "dos");  
mapa.put(1, "Uno otra vez");  
  
System.out.println(mapa.get(1));
```

¿Qué imprime? ¿Se pierde el primer valor asociado a la clave 1?

Ejercicio 7: Escribe este programa — TreeMap con valores por defecto

Crea un `TreeMap<String, Integer>` para almacenar las edades de 5 personas. Rellénalo con nombres y edades. Luego, pide al usuario un nombre por teclado y muestra su edad. Si el nombre no existe, muestra un mensaje de error.

Usa `getOrDefault()` para evitar el `null`.

Referencias para seguir practicando

- CodeWars: [Grasshopper - Grade book](#) (7 kyu)
- CodeWars: [Word Count](#) (7 kyu)
- AceptaElReto.com: [416 - Casillas](#)
- AceptaElReto.com: [462 - Tres dedos](#)

Boletín 9 – Resuelto: Genéricos y Mapas

Dificultad progresiva. De ★ a ★★★★★.

★ Ejercicio 1: Clase genérica Pareja<T, U>

Crea una clase genérica `Pareja<T, U>` que almacene dos objetos de tipos posiblemente distintos. Incluye métodos `getPrimero()`, `getSegundo()`, `setPrimero(T)`, `setSegundo(U)` y un método `intercambiar()` que devuelva una nueva `Pareja<U, T>`.

★ Ejercicio 2: Contador de palabras con HashMap

Escribe un programa que lea un texto (hardcodeado o por `Scanner`) y cuente cuántas veces aparece cada palabra usando `HashMap<String, Integer>`.

★ Ejercicio 3: Método genérico maximo

Implementa un método genérico `public static <T extends Comparable<T>> T maximo(T[] array)` que devuelva el elemento más grande de un array usando el método `compareTo()`.

★ ★ Ejercicio 4: Agenda completa con menú

Implementa una agenda usando `HashMap<String, String>` con menú interactivo: añadir contacto, buscar por nombre, listar todos, borrar contacto y salir.

★ ★ Ejercicio 5: TreeMap — ordenando por clave

Crea un programa que lea 5 países y sus capitales, los guarde en un `TreeMap<String, String>` y los muestre ordenados alfabéticamente por país. Luego muestra el primer y último país alfabéticamente.

☆☆☆ Ejercicio 6 (ProgramaMe): Wildcards — el método de comparación

Crea un método `compararMedias(List<? extends Number> a, List<? extends Number> b)` que compare la media de dos listas de números. Devuelve -1, 0 o 1 según si la media de a es menor, igual o mayor que la de b. Usa wildcards para que acepte `List<Integer>`, `List<Double>`, etc.

☆☆☆ Ejercicio 7 (ProgramaMe): Sistema de votaciones

Crea un sistema de votaciones donde:

- Cada votante puede votar por un candidato (`String`)
- Usa un `HashMap<String, Integer>` para los votos
- Usa un `TreeMap<String, Integer>` para mostrar el ranking ordenado

Crea un método genérico `public static <T> T obtenerGanador(Map<T, Integer> votos)` que devuelva la clave con más votos.

☆☆☆ Ejercicio 8 (ProgramaMe): Cache simple con genéricos

Implementa una clase `Cache<K, V>` que almacene hasta `maxElementos` pares clave-valor usando un `LinkedHashMap<K, V>`. Cuando se añade un elemento y la cache está llena, elimina el elemento más antiguo (LRU - Least Recently Used). Incluye `get(K clave)` y `put(K clave, V valor)`.

Referencias para seguir practicando

- CodeWars: [Word Count](#) (7 kyu)
- CodeWars: [Sort arrays - 1](#) (6 kyu)
- CodeWars: [Counting duplicates](#) (6 kyu)
- AceptaElReto.com: [416 - Casillas](#)
- AceptaElReto.com: [417 - Binomiales](#)

- AceptaElReto.com: [462 - Tres dedos](#)

Boletín 9 – Intermedio: Genéricos y Mapas

Ejercicios de dificultad progresiva. De ★ a ★★★★★.

★ Ejercicio 1: Pila genérica <T>

Implementa una clase genérica `Pila<T>` que funcione como una pila (LIFO). Debe tener los métodos:

- `void push(T elemento)` — apila un elemento.
- `T pop()` — desapila y devuelve el elemento superior (lanza `EmptyStackException` si está vacía).
- `T peek()` — devuelve el elemento superior sin desapilarlo.
- `boolean isEmpty()` — indica si está vacía.
- `int size()` — número de elementos.

Internamente, usa un `ArrayList<T>` como almacenamiento. Pruébala con `Pila<Integer>`, `Pila<String>` y `Pila<Double>`.

★ Ejercicio 2: HashMap inverso

Escribe un método genérico estático:

```
public static <K, V> HashMap<V, K> invertirMapa(HashMap<K, V> original)
```

Que devuelva un nuevo `HashMap` intercambiando claves y valores. Si hay valores duplicados en el mapa original, el último encontrado sobrescribe al anterior.

Prueba con un mapa de `String` → `Integer` y otro de `String` → `String`.

★ Ejercicio 3: Método genérico — filtrar por condición

Implementa un método genérico:

```
public static <T> List<T> filtrar(List<T> lista, Predicate<T> condicion)
```

Que devuelva una nueva lista con los elementos que cumplen la condición. Usa la interfaz `Predicate<T>` de Java.

Pruébalo filtrando números pares de una `List<Integer>` y palabras que empiecen por «A» de una `List<String>`.

★ ★ Ejercicio 4: Caché LRU con LinkedHashMap

Crea una clase `CacheLRU<K, V>` que extienda o use internamente un `LinkedHashMap<K, V>` con capacidad máxima de 5 elementos. Cuando se añade un elemento y ya hay 5, se elimina el **menos recientemente usado** (acceso, no inserción).

Pista: `LinkedHashMap` tiene el constructor con `accessOrder=true` y el método `removeEldestEntry()`.

★ ★ Ejercicio 5: TreeMap — frecuencia de letras

Escribe un programa que lea un texto por teclado (o use uno hardcodeado) y cuente cuántas veces aparece cada **letra** (ignorando espacios, números y signos). Usa un `TreeMap<Character, Integer>` para que las letras se muestren automáticamente ordenadas alfabéticamente.

Ejemplo de salida para «Hola mundo»:

```
a: 1, d: 1, h: 1, l: 1, m: 1, n: 1, o: 2, u: 1
```

★ ★ Ejercicio 6 (Wildcards): Suma de números genérica

Implementa un método que sume todos los números de una lista, aceptando cualquier subtipo de `Number`:

```
public static double sumar(List<? extends Number> lista)
```

Pruébalo con `List<Integer>`, `List<Double>` y `List<Float>`. ¿Qué ocurre si intentas pasar una `List<String>`?

Crea también un segundo método que **mezcle** dos listas de números de tipos distintos en una sola `List<Double>`:

```
public static List<Double> mezclar(List<? extends Number> a, List<? extends Number> b)
```

☆☆☆ Ejercicio 7 (ProgramaMe): Sistema de inventario genérico

Crea un sistema de inventario genérico para una tienda:

1. Clase `Producto<T>` con `String` nombre, `double` precio, `T` categoria (el tipo de categoría puede ser `String` o un `Enum`).
2. Clase `Inventario<T>` que almacene `Producto<T>` en un `HashMap<String, Producto<T>>` (clave = nombre del producto).
3. Métodos: `agregar`, `eliminar`, `buscar`, `listar`, `valorTotal()`.
4. Crea un inventario de productos con categoría `String` y otro con categoría `enum`.

☆☆☆ Ejercicio 8 (CodeWars + AceptaElReto)

Resuelve los siguientes problemas aplicando mapas y genéricos:

CodeWars: [Counting Duplicates](#) (6 kyu) — Cuenta cuántos caracteres aparecen más de una vez en una cadena. Ideal para `HashMap<Character, Integer>`.

AceptaElReto: [416 - Casillas de corrección](#) — Gestiona casillas de corrección usando un mapa de errores por alumno.

Referencias

- CodeWars: [Counting Duplicates](#) (6 kyu)
- CodeWars: [Sort arrays - 1](#) (6 kyu)

- CodeWars: [Merging sorted integer arrays](#) (6 kyu)
- AceptaElReto.com: [416 - Casillas](#)
- AceptaElReto.com: [462 - Tres dedos](#)
- AceptaElReto.com: [417 - Binomiales](#)

Boletín 10 — Extras: Genéricos y Mapas

★ Extras — Conviértete en un programador de muy alto nivel

Estos ejercicios son completamente opcionales, pero si aspiras a ser un programador o programadora de élite, aquí tienes tu campo de entrenamiento. CodeWars y AceptaElReto son plataformas donde los mejores programadores del mundo se retan a diario. Cada problema que resuelvas te hará más fuerte, más rápido y más creativo. No los subestimes: son la diferencia entre saber programar y ser un verdadero profesional.

CodeWars:

- [Counting sheep](#) (8 kyu)

Pista: Filtra los valores null y cuenta los true con `stream().filter(b -> b).count()`.

- [Find the unique number](#) (6 kyu)

Pista: Divide los tres primeros números para saber cuál se repite; el que no coincide es el único. O usa XOR.

- [Array.diff](#) (6 kyu)

Pista: Convierte el segundo array en un `Set<Integer>` y filtra el primero con `removeIf` o un `stream`.

AceptaElReto:

- [302 - Cuadrado mágico](#) (★ ★)

Pista: Calcula la suma objetivo = total / n; verifica filas, columnas y diagonales contra ese valor.

- [432 - Número de veces que aparece](#) (★)

Pista: Usa un `HashMap<Character, Integer>` para contar frecuencias de cada carácter.

CodeWars:

- [Who likes it?](#) (6 kyu)

Pista: Usa un `Map<Integer, String>` con los patrones de respuesta. La clave es el tamaño del array, el valor es el template.

AceptaElReto:

- [217 - ¿Qué lado de la calle?](#) (★ ★)

Pista: Lee el número como String, no como int. Solo te interesa el último dígito para saber si es par o impar.

Boletín 10 – Inicial Resuelto: Consola y Ficheros

Con soluciones. A aprender.

Ejercicio 1: ¿Qué imprime? — printf()

```
String nombre = "Ana";
int edad = 20;
double nota = 9.5;

System.out.printf("Nombre: %s, Edad: %d, Nota: %.2f%n", nombre, edad, nota);
```

Solución: Imprime Nombre: Ana, Edad: 20, Nota: 9.50 y un salto de línea.

💡 **Explicación:** printf() no es println(): no añade salto de línea automático. Por eso se usa %n al final, que es el salto de línea independiente de plataforma. %s para String, %d para entero, %.2f para decimal con 2 dígitos. El %n es como un \n pero más educado: sabe si estás en Windows (salto con \r\n) o Linux (solo \n).

Ejercicio 2: Encuentra el error — nextInt() + nextLine()

```
Scanner sc = new Scanner(System.in);
System.out.print("Edad: ");
int edad = sc.nextInt();           // lee "25" y deja el "\n" sin consumir
System.out.print("Nombre: ");
String nombre = sc.nextLine();     // se traga el "\n" que quedó
System.out.println(nombre + " tiene " + edad + " años.");
sc.close();
```

Solución: nextInt() NO consume el salto de línea. Cuando el usuario escribe 25 y pulsa Enter, en el buffer queda 25\n. nextInt() consume 25 y deja \n. Luego nextLine() consume ese \n y devuelve una cadena vacía. El programa imprime tiene 25 años. (nombre vacío).

Para arreglarlo: añade un `sc.nextLine()` extra después del `nextInt()` para consumir el salto de línea.

```
int edad = sc.nextInt();
sc.nextLine(); // consume el \n pendiente
String nombre = sc.nextLine();
```

💡 **Explicación:** Es el error más famoso de Java con Scanner. Siempre que uses `nextInt()`, `nextDouble()` o `next()` y luego `nextLine()`, necesitas un `nextLine()` extra para limpiar el buffer. O mejor: usa siempre `nextLine()` y convierte con `Integer.parseInt()`, que es más seguro.

Ejercicio 3: Completa el código — leer archivo

```
import java.io.*;
import java.nio.file.*;
import java.util.List; // ¡este import falta!

public class Test {
    public static void main(String[] args) throws IOException {
        Path ruta = Paths.get("datos.txt");
        List<String> lineas = Files.readAllLines(ruta);
        for (String linea : lineas) {
            System.out.println(linea);
        }
    }
}
```

Falta línea en el `println` y también el `import java.util.List;`.

💡 **Explicación:** `Files.readAllLines()` devuelve un `List<String>`, que necesita `import`. El `for-each` recorre cada línea y `linea` es la variable que contiene cada línea. Sin `import java.util.List;`, el compilador no sabe qué es `List` y se queja.

Ejercicio 4: Escribe este programa — Hola mundo archivo

```

import java.io.*;

public class HolaArchivo {
    public static void main(String[] args) {
        // Escribir
        try {
            FileWriter writer = new FileWriter("saludo.txt");
            writer.write(";Hola, archivo!\n");
            writer.write("Esto es una segunda línea.\n");
            writer.close();
            System.out.println("Archivo escrito.");
        } catch (IOException e) {
            System.out.println("Error al escribir: " + e.getMessage());
        }

        // Leer
        try {
            BufferedReader reader = new BufferedReader(new FileReader("saludo.txt"));
            String linea = reader.readLine();
            while (linea != null) {
                System.out.println(linea);
                linea = reader.readLine();
            }
            reader.close();
        } catch (IOException e) {
            System.out.println("Error al leer: " + e.getMessage());
        }
    }
}

```

💡 **Explicación:** `FileWriter` escribe caracteres en un archivo. Si el archivo no existe, lo crea. Si existe, lo sobrescribe. `BufferedReader` envuelve a `FileReader` para leer por líneas (más rápido). El bucle `while (linea != null)` es el estándar para leer archivos línea por línea. Siempre cierra los recursos con `close()` para evitar pérdidas de datos.

Ejercicio 5: ¿Qué imprime? — el contador de líneas

```
import java.io.*;

public class Test {
    public static void main(String[] args) throws IOException {
        File f = new File("datos.txt");
        FileWriter w = new FileWriter(f);
        w.write("linea1\nlinea2\nlinea3\n");
        w.close();

        BufferedReader r = new BufferedReader(new FileReader(f));
        int contador = 0;
        while (r.readLine() != null) {
            contador++;
        }
        r.close();
        System.out.println(contador);
    }
}
```

Solución: Imprime 3.

💡 **Explicación:** El archivo tiene tres líneas: “linea1”, “linea2”, “linea3” (el último \n crea una línea adicional vacía, pero `readLine()` no la cuenta como línea porque devuelve `null` cuando no hay más). `readLine()` devuelve cada línea hasta que no quedan más, momento en que devuelve `null`. El contador se incrementa 3 veces. Nota: en realidad, si el archivo termina con \n, `readLine()` leería “linea3” y luego devolvería `null`, por lo que contaríamos 3 líneas. Si no hubiera \n al final, también serían 3.

Ejercicio 6: Encuentra el error — archivo no cerrado

```
FileWriter writer = new FileWriter("notas.txt");
writer.write("Esto es una nota importante.");
// falta: writer.close();
```

Solución: Falta `writer.close()`. Sin cerrar el archivo, los datos pueden no escribirse físicamente en el disco porque `FileWriter` usa un buffer interno.

💡 **Explicación:** `FileWriter` no escribe directamente en el disco. Usa un buffer. Cuando haces `write()`, los datos se almacenan en el buffer. Si no llamas a `close()` o `flush()`, parte de los datos pueden perderse cuando el programa termina. Es como echar una carta al buzón pero no cerrar la puerta: el cartero puede no recogerla. Además, el sistema operativo mantiene el archivo bloqueado hasta que se cierre. **Siempre cierra los archivos** o usa `try-with-resources` (Java 7+).

Ejercicio 7: Escribe este programa — Scanner que suma números

```
import java.util.Scanner;

public class SumaNumeros {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int suma = 0;
        int num;

        System.out.println("Introduce números enteros (0 para terminar):");
        do {
            System.out.print("Número: ");
            num = sc.nextInt();
            suma += num;
        } while (num != 0);

        System.out.printf("La suma total es: %d\n", suma);
        sc.close();
    }
}
```

💡 **Explicación:** El bucle `do-while` asegura que se pida al menos un número. Cuando el usuario introduce 0, el bucle termina. `suma += num` acumula todos los números, incluido el 0 final (que no afecta a la suma). `printf()` con `%d` formatea el entero. Es un programa simple pero que combina todas las piezas: `Scanner`, bucle, acumulación y salida formateada.

Referencias para seguir practicando

- CodeWars: [Get the Middle Character](#) (7 kyu)
- CodeWars: [String repeat](#) (8 kyu)
- AceptaElReto.com: [140 - Suma de dígitos](#)
- AceptaElReto.com: [149 - San Fermines](#)

Boletín 10 – Inicial: Consola, Ficheros y Regex

Sin soluciones. A darle al teclado.

Ejercicio 1: ¿Qué imprime? — printf con conversiones

```
int entero = 42;
double decimal = 3.1416;
String texto = "Java";

System.out.printf("%d %f %s %n", entero, decimal, texto);
```

¿Qué imprime? ¿Qué hace %n al final?

Ejercicio 2: Encuentra el error — IOException sin capturar

```
import java.io.*;

public class Test {
    public static void main(String[] args) {
        FileWriter writer = new FileWriter("salida.txt");
        writer.write("Hola mundo");
        writer.close();
    }
}
```

Este código no compila. ¿Por qué? ¿Qué dos formas hay de solucionarlo?

Ejercicio 3: Completa el código — try-with-resources

Completa el siguiente programa para que lea un archivo y muestre su contenido:

```
import java.io.*;
import java.nio.file.*;

public class Lector {
    public static void main(String[] args) {
        Path ruta = Paths.get("datos.txt");

        try (_____ reader = Files.newBufferedReader(ruta)) { // ¿qué tipo?
            String linea;
            while ((linea = reader.readLine()) != null) {
                System.out.println(_____); // ¿qué va aquí?
            }
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Ejercicio 4: Escribe este programa — guardar array en archivo

Crea un array de cadenas con 5 nombres de ciudades. Escribe cada nombre en una línea de un archivo llamado `ciudades.txt`, usando `BufferedWriter` y `try-with-resources`. No olvides el salto de línea.

Ejercicio 5: ¿Qué imprime? — Scanner next vs nextLine

```
import java.util.Scanner;

public class Test {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Palabra: ");
        String palabra = sc.next();
        System.out.print("Frase: ");
        String frase = sc.nextLine();
        System.out.println "[" + palabra + "] [" + frase + "]");
    }
}
```

Si el usuario introduce `Hola mundo` y pulsa `Enter`, luego `Esto es una frase` y pulsa `Enter`, ¿qué imprime exactamente? ¿Por qué `next()` y `nextLine()` se comportan distinto?

Ejercicio 6: Encuentra el error — `File.createNewFile` sin comprobar

```
File f = new File("documento.txt");
f.createNewFile();
FileWriter w = new FileWriter(f);
w.write("Contenido importante");
w.close();
```

¿Qué pasa si el archivo `documento.txt` ya existe? ¿Qué devuelve `createNewFile()`?

Ejercicio 7: Escribe este programa — menú con `Scanner` y `switch`

Crea un programa que muestre un menú con estas opciones:

1. **Saludar** → imprime «¡Hola, programador!»
2. **Despedirse** → imprime «¡Hasta luego!»
3. **Cuenta atrás** → pide un número y cuenta desde ese número hasta 0
4. **Salir**

El menú debe repetirse hasta que el usuario elija 0. Usa Scanner, switch y printf() para formatear la salida.

Referencias para seguir practicando

- CodeWars: [Get the Middle Character](#) (7 kyu)
- CodeWars: [String repeat](#) (8 kyu)
- AceptaElReto.com: [140 - Suma de dígitos](#)
- AceptaElReto.com: [149 - San Fermín](#)

Boletín 10 – Resuelto: Consola y Ficheros

Dificultad progresiva. De ★ a ★★★★★.

★ Ejercicio 1: La entrevista de trabajo

Crea un programa que haga una entrevista de trabajo simulada. Pregunta nombre, edad, años de experiencia y lenguaje favorito. Al final, muestra un resumen con `printf()` formateado en forma de tarjeta.

★ Ejercicio 2: Calculadora de propinas

Solicita el total de la cuenta y el porcentaje de propina. Calcula la propina y el total final. Muestra los resultados con 2 decimales usando `printf()`.

★ Ejercicio 3: El diario personal (append)

Crea un programa que escriba en un archivo `diario.txt` una línea con la fecha y el texto que el usuario introduzca. Cada ejecución debe añadir una nueva entrada al final sin borrar las anteriores. Usa `FileWriter` en modo `append`.

★ ★ Ejercicio 4: El contador de líneas, palabras y caracteres

Crea un programa que lea un archivo de texto y muestre cuántas líneas, palabras y caracteres tiene. Usa `BufferedReader` para leer.

★ ★ Ejercicio 5: El gestor de contactos (archivo)

Crea un programa con menú que permita: añadir un contacto (nombre, teléfono) a `contactos.txt`, listar todos los contactos, buscar un contacto por nombre. Usa `BufferedReader` y `PrintWriter`. Formatea con `printf()`.

★ ★ Ejercicio 6: Lectura con NIO (Files y Paths)

Reescribe el ejercicio del contador de líneas usando la API NIO (`Files.readAllLines()` y `Path`). Compara la diferencia.

★ ★ ★ Ejercicio 7 (ProgramaMe): Cifrado César con archivos

Crea un programa que lea un archivo `mensaje.txt`, desplace cada carácter 3 posiciones en el alfabeto (cifrado César) y escriba el resultado en `mensaje_cifrado.txt`. Luego, otro programa (o el mismo con una opción) que lo descifre. Usa `try-with-resources` y `BufferedReader/PrintWriter`.

★ ★ ★ Ejercicio 8 (ProgramaMe): Serialización de objetos

Crea una clase `Estudiante` que implemente `Serializable` con `String nombre`, `int edad`, `double notaMedia`. Crea un programa que guarde un `ArrayList<Estudiante>` en un archivo `estudiantes.dat` usando `ObjectOutputStream`. Luego, otro programa (o el mismo con opción) que lo lea con `ObjectInputStream` y muestre los datos formateados.

Referencias para seguir practicando

- CodeWars: [Get the Middle Character](#) (7 kyu)
- CodeWars: [String repeat](#) (8 kyu)
- CodeWars: [Exes and Ohs](#) (7 kyu)
- AceptaElReto.com: [140 - Suma de dígitos](#)
- AceptaElReto.com: [149 - San Fermín](#)
- AceptaElReto.com: [152 - Suma pares e impares](#)

Boletín 10 – Intermedio: Consola, Ficheros y Regex

Ejercicios de dificultad progresiva. De ★ a ★★ ★★.

★ Ejercicio 1: Formateador de ticket de compra

Crea un programa que simule un ticket de compra. Usa un array de productos (nombre, precio, cantidad) y muestra un ticket formateado con `printf()`:

```
=====
TICKET DE COMPRA
=====
Pan          2 x 1.20€ =  2.40€
Leche        3 x 0.95€ =  2.85€
Huevos       1 x 3.50€ =  3.50€
-----
TOTAL                =  8.75€
=====
```

Requisitos: alinear nombres a la izquierda, precios a la derecha, 2 decimales, ancho fijo de columnas.

★ Ejercicio 2: Buscador de archivos por extensión

Crea un programa que pida una ruta de directorio y una extensión (ej: `.txt`, `.java`) y liste **recursivamente** todos los archivos con esa extensión. Usa la clase `File` y su método `listFiles()`.

Pista: Si el archivo es un directorio, llama al método de nuevo (recursión).

★ Ejercicio 3: Lector de CSV con Scanner

Dado un archivo `datos.csv` con el siguiente formato (sin cabecera):

```
Ana;25;DAM
Bob;22;DAW
Carlos;30;DAM
```

Usa `Scanner` con `useDelimiter()` para leer el archivo y mostrar los datos en formato tabla alineada. Usa `printf()` para formatear.

Asegúrate de que el `Scanner` maneje correctamente tanto el delimitador `;` como el salto de línea.

☆☆ Ejercicio 4: Filtro de líneas por palabra clave

Crea un programa que lea un archivo de texto (`origen.txt`) y escriba en `destino.txt` solo las líneas que contienen una palabra clave (pedida al usuario). Usa `BufferedReader` y `PrintWriter`.

Muestra al final cuántas líneas coincidieron y cuántas se descartaron.

☆☆ Ejercicio 5: Separador de líneas pares e impares

Crea un programa que lea un archivo `entrada.txt` y genere dos archivos:

- `pares.txt` → contiene las líneas en posición par (0, 2, 4...).
- `impares.txt` → contiene las líneas en posición impar (1, 3, 5...).

Usa `try-with-resources` con **tres** recursos (un `BufferedReader` y dos `PrintWriter`).

☆☆ Ejercicio 6: Split con regex — analizador de frases

Escribe un programa que lea una frase del usuario y use `split()` con una expresión regular para:

1. Separar las palabras (ignorando espacios, comas, puntos, signos).

2. Mostrar cuántas palabras hay.
3. Mostrar la palabra más larga.
4. Mostrar las palabras que empiezan por vocal.

Ejemplo: "Hola, mundo. Esto es Java: ¿mola?" →

```
Palabras: 6
Más larga: "mundo"
Empiezan por vocal: ["Esto"]
```

☆☆☆ Ejercicio 7 (ProgramaMe): Validador de datos con regex

Crea un programa que lea un archivo `datos.txt` donde cada línea contiene un dato y su tipo (separados por ;):

```
ana@email.com;email
12345678Z;dni
+34 612345678;telefono
91 123 45 67;telefono
esto-no-es-email;email
```

Valida cada línea según el tipo usando expresiones regulares:

- **Email:** formato básico `xxx@xxx.xxx`
- **DNI:** 8 dígitos + letra mayúscula (la letra debe ser válida según el algoritmo)
- **Teléfono:** opcional +34 seguido de 9 dígitos, con o sin espacios

Muestra un resumen: cuántos válidos, cuántos inválidos, y lista los inválidos.

☆☆☆ Ejercicio 8 (CodeWars + AceptaElReto)

Resuelve estos problemas que combinan ficheros, regex y consola:

CodeWars: [Regex validate PIN code](#) (7 kyu) — Valida que un String sea un PIN de 4 o 6 dígitos exactos.

CodeWars: [Exes and Ohs](#) (7 kyu) — Cuenta si el número de X y O es el mismo en un String (puedes resolverlo con o sin regex).

AceptaElReto: [149 - San Fermine](#)s — Lectura de múltiples casos de prueba desde consola.



Referencias

- CodeWars: [Regex validate PIN code](#) (7 kyu)
- CodeWars: [Exes and Ohs](#) (7 kyu)
- CodeWars: [String repeat](#) (8 kyu)
- AceptaElReto.com: [149 - San Fermine](#)s
- AceptaElReto.com: [140 - Suma de dígitos](#)
- AceptaElReto.com: [152 - Suma pares e impares](#)

Boletín 11 — Extras: Consola, Ficheros y Expresiones Regulares

★ Extras — Conviértete en un programador de muy alto nivel

Estos ejercicios son completamente opcionales, pero si aspiras a ser un programador o programadora de élite, aquí tienes tu campo de entrenamiento. CodeWars y AceptaElReto son plataformas donde los mejores programadores del mundo se retan a diario. Cada problema que resuelvas te hará más fuerte, más rápido y más creativo. No los subestimes: son la diferencia entre saber programar y ser un verdadero profesional.

CodeWars:

- [Categorize New Member](#) (7 kyu)

Pista: Edad ≥ 55 y hándicap $> 7 \rightarrow$ “Senior”, si no $\rightarrow$ “Open”. Simple lógica booleana.

- [Two to One](#) (7 kyu)

Pista: Concatena los dos strings, conviértelos a `char[]`, ordénalos, y usa un `StringBuilder` para eliminar duplicados.

- [Primes in numbers](#) (5 kyu)

Pista: Divide por 2, luego por impares hasta $\sqrt{n}$; cuenta las repeticiones de cada divisor.

AceptaElReto:

- [108 - Hormigas](#) (★ ★)

Pista: Piensa en posiciones relativas: cuando dos hormigas se cruzan, es como si se ignoraran.

- [380 - ¡Qué grande es el cine!](#) (★ ★)

Pista: Simula llenado de butacas fila por fila; busca huecos consecutivos suficientes para cada grupo.

Boletín 14 – Inicial Resuelto: JDBC – Conexión y Consultas

💡 Los 5 pasos, bien explicados y con humor.

Ejercicio 1: Los 5 pasos

1. **Cargar el driver** → `Class.forName("com.mysql.cj.jdbc.Driver")` (opcional desde Java 6)
2. **Establecer conexión** → `DriverManager.getConnection(url, user, pass)` devuelve `Connection`
3. **Crear Statement** → `con.createStatement()` devuelve `Statement` o `con.prepareStatement(sql)` devuelve `PreparedStatement`
4. **Ejecutar consulta** → `stmt.executeQuery(sql)` para `SELECT` o `stmt.executeUpdate(sql)` para `INSERT/UPDATE/DELETE`
5. **Procesar resultados** → `rs.next() + rs.getXxx("columna")`
6. **(Bonus) Cerrar todo** → `con.close(), stmt.close(), rs.close()` (o `try-with-resources`)

💡 **Explicación:** El paso 1 es opcional desde Java 6 (los drivers JDBC 4.0 se cargan automáticamente si están en el classpath). Pero verás `Class.forName()` en mucho código legacy. No pasa nada si lo pones. El paso 6 es obligatorio: conexiones abiertas = servidor saturado.

Ejercicio 2: Completa la conexión

```
String url = "jdbc:mysql://localhost:3306/instituto";
String user = "root";
String pass = "admin123";

Connection con = DriverManager.getConnection(url, user, pass);
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM alumnos");

while (rs.next()) {
    System.out.println(rs.getString("nombre"));
}

// ¡Falta cerrar!
con.close();
stmt.close();
rs.close();
```

💡 **Explicación:** Connection, Statement, ResultSet. Al final hay que cerrar todo. Mejor con try-with-resources para que se cierre automáticamente. Si se te olvida cerrar, la conexión queda abierta hasta que el garbage collector decida visitarte (y no sabe cuándo venir).

Ejercicio 3: ¿Qué imprime?

```
try (Connection con = DriverManager.getConnection(url, user, pass);
    Statement stmt = con.createStatement();
    ResultSet rs = stmt.executeQuery("SELECT COUNT(*) FROM alumnos")) {

    if (rs.next()) {
        System.out.println(rs.getInt(1)); // número total de alumnos
    }
}
```

💡 **Explicación:** Imprime el número total de filas en la tabla alumnos. COUNT(*) devuelve una sola fila con una columna. Como no tiene nombre (o el nombre depende de la BD), accedemos por índice 1 (la primera columna). También funciona rs.getInt("COUNT(*)") pero es más feo. rs.getInt(1) es la forma estándar cuando hay una columna sin alias.

Ejercicio 4: Encuentra el error

```
public void buscarAlumno(String nombre) {  
    String sql = "SELECT * FROM alumnos WHERE nombre = '" + nombre + "'";  
    // ¡SQL INJECTION!  
    try (Statement stmt = con.createStatement();  
        ResultSet rs = stmt.executeQuery(sql)) { ... }  
}
```

💡 **Explicación:** ¡SQL Injection! Si nombre vale Luis'; DROP TABLE alumnos; --, la consulta se convierte en:

```
SELECT * FROM alumnos WHERE nombre = 'Luis'; DROP TABLE alumnos; --'
```

Adiós, tabla alumnos. Solución: usar PreparedStatement:

```
String sql = "SELECT * FROM alumnos WHERE nombre = ?";  
PreparedStatement pstmt = con.prepareStatement(sql);  
pstmt.setString(1, nombre);  
ResultSet rs = pstmt.executeQuery();
```

Nunca, jamás, concatenes SQL con datos de usuario. Es la regla de oro número 1 de JDBC.

Ejercicio 5: INSERT con PreparedStatement

```
String sql = "INSERT INTO alumnos (nombre, edad, curso) VALUES (?, ?, ?)";  
  
try (PreparedStatement pstmt = con.prepareStatement(sql)) {  
    pstmt.setString(1, "María");  
    pstmt.setInt(2, 22);  
    pstmt.setString(3, "DAM");  
    int filas = pstmt.executeUpdate();  
    System.out.println("Insertadas " + filas + " filas");  
}
```

💡 **Explicación:** setString, setInt, etc. reemplazan los ? en orden (empezando en 1). executeUpdate() devuelve el número de filas afectadas. Si es 1, todo bien. Si es 0, algo falló (o el INSERT tenía una condición WHERE que no coincidió — cosa rara en INSERT).

Ejercicio 6: ¿Qué hace executeUpdate()?

```
int filas = stmt.executeUpdate("DELETE FROM alumnos WHERE id = 10");
System.out.println(filas);
```

💡 **Explicación:** executeUpdate() devuelve el **número de filas afectadas**. Si el alumno con id=10 existe y se borra, imprime 1. Si no existe, imprime 0. No lanza excepción por no encontrar filas — solo devuelve 0. Siempre comprueba el valor devuelto para saber si la operación tuvo efecto.

Ejercicio 7: Diferencia entre executeQuery y executeUpdate

1. Mostrar todos los productos → executeQuery() (SELECT)
2. Borrar un cliente por ID → executeUpdate() (DELETE)
3. Actualizar el precio de un producto → executeUpdate() (UPDATE)
4. Contar cuántos usuarios hay → executeQuery() (SELECT COUNT)
5. Insertar un nuevo pedido → executeUpdate() (INSERT)

💡 **Explicación:** Regla mnemotécnica: si esperas datos de vuelta (filas con columnas), usa executeQuery(). Si solo quieres saber cuántas filas se modificaron, usa executeUpdate(). Mezclarlos da SQLException. Como meter un tenedor en el microondas.

🔗 **CodeWars:** [SQL with Street Fighter](#) (6kyu)

🔗 **AceptaElReto.com:** [200 - Aburrimiento en las aulas](#)

Boletín 14 – Inicial: Conexión a BD con JDBC

Sin soluciones. 7 pasos para no perder la conexión.

Ejercicio 1: ¿Qué necesitas para usar JDBC?

Responde brevemente:

1. ¿Qué archivo .jar necesitas incluir en tu proyecto para conectar con MySQL?
 2. ¿Qué clase de Java proporciona el método `getConnection()`?
 3. ¿Qué interfaz representa una conexión a la base de datos?
 4. ¿Qué excepción checked tienes que manejar siempre al trabajar con JDBC?
-

Ejercicio 2: Completa el código — try-with-resources

Completa los tipos que faltan:

```
String url = "jdbc:mysql://localhost:3306/instituto";
String user = "root";
String pass = "admin123";

String sql = "SELECT * FROM alumnos";

try (_____ con = DriverManager.getConnection(url, user, pass);
     _____ stmt = con.createStatement();
     _____ rs = stmt.executeQuery(sql)) {

    while (rs.next()) {
        System.out.println(rs.getString("nombre"));
    }

} catch (_____ e) { // ¿qué excepción?
    System.err.println("Error: " + e.getMessage());
}
```

Ejercicio 3: ¿Qué imprime? — ResultSet vacío

```
try (Connection con = DriverManager.getConnection(url, user, pass);
    Statement stmt = con.createStatement();
    ResultSet rs = stmt.executeQuery("SELECT * FROM alumnos WHERE id = 9999")) {

    if (rs.next()) {
        System.out.println("Encontrado: " + rs.getString("nombre"));
    } else {
        System.out.println("No encontrado");
    }
}
```

Si no hay ningún alumno con `id = 9999`, ¿qué imprime? ¿Qué devuelve `rs.next()` la primera vez que se llama?

Ejercicio 4: Encuentra el error — SQLException sin manejo

```
public class Test {
    public static void main(String[] args) {
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/instituto", "root", "admin123");
        Statement stmt = con.createStatement();
        ResultSet rs = stmt.executeQuery("SELECT * FROM alumnos");
        while (rs.next()) {
            System.out.println(rs.getString("nombre"));
        }
    }
}
```

Este código no compila. ¿Por qué? ¿Qué dos cosas faltan para que funcione?

Ejercicio 5: Escribe este programa — mostrar nombres de columnas

Escribe un programa que se conecte a la base de datos y, usando `ResultSetMetaData`, muestre:

1. El número de columnas de la tabla alumnos.
2. El nombre de cada columna.
3. El tipo de dato de cada columna (usando `getColumnTypeName()`).

Ejemplo de salida:

```
La tabla alumnos tiene 4 columnas:  
id (INT)  
nombre (VARCHAR)  
edad (INT)  
curso (VARCHAR)
```

Ejercicio 6: ¿Qué imprime? — `executeQuery` en `UPDATE`

```
Statement stmt = con.createStatement();  
ResultSet rs = stmt.executeQuery("UPDATE alumnos SET edad = 25 WHERE id = 1");
```

¿Qué ocurre al ejecutar esta línea? ¿Por qué no debes usar `executeQuery()` para un `UPDATE`?

Ejercicio 7: Encuentra el error — `PreparedStatement` con índices incorrectos

```
String sql = "INSERT INTO alumnos (nombre, edad, curso) VALUES (?, ?, ?)";  
PreparedStatement pstmt = con.prepareStatement(sql);  
pstmt.setString(0, "Ana");    // ¿índice correcto?  
pstmt.setInt(1, 22);          // ¿índice correcto?  
pstmt.setString(2, "DAM");    // ¿índice correcto?  
pstmt.executeUpdate();
```

¿Qué índices son correctos para los `?` en un `PreparedStatement`? ¿En qué número empiezan?

Ejercicio 8: Completa el código — cerrar recursos

```
public void listarAlumnos() {  
    // Escribe el código para:  
    // 1. Conectarte a la BD usando try-with-resources  
    // 2. Ejecutar un SELECT * FROM alumnos  
    // 3. Mostrar cada alumno con printf("%d - %s (%d)", id, nombre, edad)  
    // 4. Manejar SQLException  
}
```

Completa el método. Recuerda que try-with-resources cierra automáticamente Connection, Statement y ResultSet.

Referencias para seguir practicando

- CodeWars: [SQL with Street Fighter](#) (6 kyu)
- CodeWars: [SQL Basics: Simple JOIN](#) (6 kyu)
- AceptaElReto.com: [200 - Aburrimiento en las aulas](#)

Boletín 14 – Resuelto: JDBC – Conexión y Consultas

★ Ejercicio 2: INSERT con PreparedStatement

Crea un método que inserte un nuevo alumno. Los parámetros deben pasarse como argumentos. Usa PreparedStatement.

```
import java.sql.*;

public class InsertarAlumno {
    public static void insertar(String nombre, int edad, String curso) {
        String sql = "INSERT INTO alumnos (nombre, edad, curso) VALUES (?, ?, ?)";

        try (Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/instituto", "root", "admin123");
            PreparedStatement pstmt = con.prepareStatement(sql)) {

            pstmt.setString(1, nombre);
            pstmt.setInt(2, edad);
            pstmt.setString(3, curso);

            int filas = pstmt.executeUpdate();
            System.out.println("Insertado: " + filas + " fila(s)");

        } catch (SQLException e) {
            System.err.println("Error al insertar: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        insertar("Luis García", 22, "DAM");
    }
}
```

💡 **Explicación:** Los ? son placeholders. `setString(1, ...)` asigna al primer ?. El índice empieza en 1 (no en 0 como los arrays — trampa clásica). `executeUpdate()` devuelve 1 si todo fue bien. Si el nombre tiene una comilla simple como “Luis'O'Connell”, PreparedStatement la escapa automáticamente. Sin riesgo de SQL Injection.

★ Ejercicio 3: UPDATE con PreparedStatement

Actualiza la edad de un alumno buscándolo por nombre. Muestra cuántas filas se actualizaron.

```
import java.sql.*;

public class ActualizarAlumno {
    public static void main(String[] args) {
        String sql = "UPDATE alumnos SET edad = ? WHERE nombre = ?";

        try (Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/instituto", "root", "admin123");
            PreparedStatement pstmt = con.prepareStatement(sql)) {

            pstmt.setInt(1, 23);
            pstmt.setString(2, "Luis García");

            int filas = pstmt.executeUpdate();
            if (filas > 0) {
                System.out.println("Actualizados " + filas + " alumno(s)");
            } else {
                System.out.println("No se encontró al alumno");
            }

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

💡 **Explicación:** `executeUpdate()` devuelve filas afectadas. Si el `WHERE` no encuentra nada, devuelve 0 — no es error. Siempre comprueba el valor. Si olvidas el `WHERE`, actualizas TODOS los alumnos con la misma edad. No preguntes cómo lo sé.

★ Ejercicio 4: DELETE con PreparedStatement

Borra un alumno por ID. Pide confirmación antes de borrar.

```

import java.sql.*;
import java.util.Scanner;

public class EliminarAlumno {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("ID del alumno a borrar: ");
        int id = sc.nextInt();
        sc.nextLine();
        System.out.print("¿Seguro? (s/n): ");
        String conf = sc.nextLine();

        if (!conf.equalsIgnoreCase("s")) {
            System.out.println("Cancelado");
            return;
        }

        String sql = "DELETE FROM alumnos WHERE id = ?";

        try (Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/instituto", "root", "admin123");
            PreparedStatement pstmt = con.prepareStatement(sql)) {

            pstmt.setInt(1, id);
            int filas = pstmt.executeUpdate();
            System.out.println(filas > 0 ? "Eliminado" : "No encontrado");

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}

```

💡 **Explicación:** Confirmación antes de borrar: buena práctica. El DELETE sin WHERE es el beso de la muerte para tus datos. `executeUpdate()` devuelve filas afectadas. Si devuelve 0, el ID no existe. Y recuerda: en SQL no hay papelera de reciclaje.

★ ★ Ejercicio 5: Búsqueda por nombre con LIKE

Implementa una búsqueda de alumnos por nombre usando LIKE y PreparedStatement. El usuario escribe parte del nombre y se muestran todos los que coinciden.

```
import java.sql.*;

public class BuscarAlumnos {
    public static void buscar(String nombreParcial) {
        String sql = "SELECT * FROM alumnos WHERE nombre LIKE ?";

        try (Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/instituto", "root", "admin123");
            PreparedStatement pstmt = con.prepareStatement(sql)) {

            pstmt.setString(1, "%" + nombreParcial + "%");

            try (ResultSet rs = pstmt.executeQuery()) {
                boolean encontrado = false;
                while (rs.next()) {
                    System.out.printf("%d - %s (%d)%n",
                        rs.getInt("id"),
                        rs.getString("nombre"),
                        rs.getInt("edad"));
                    encontrado = true;
                }
                if (!encontrado) System.out.println("Sin resultados");
            }

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        buscar("Luis"); // Busca cualquier nombre que contenga "Luis"
    }
}
```

💡 **Explicación:** LIKE ? con `setString(1, "%" + nombre + "%")`. Los % son comodines SQL: %Luis% busca "Luis" en cualquier posición. La concatenación aquí es segura porque los % son parte del valor, no de la SQL. El ResultSet está anidado en otro try-with-resources para que se cierre automáticamente.

★ ★ Ejercicio 6: Contar alumnos por curso

Escribe un programa que muestre cuántos alumnos hay en cada curso usando GROUP BY.

```
import java.sql.*;

public class ContarPorCurso {
    public static void main(String[] args) {
        String sql = "SELECT curso, COUNT(*) AS total FROM alumnos GROUP BY curso ORDER BY total DESC";

        try (Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/instituto", "root", "admin123");
            Statement stmt = con.createStatement();
            ResultSet rs = stmt.executeQuery(sql)) {

            System.out.println("Alumnos por curso:");
            while (rs.next()) {
                System.out.printf("  %s: %d alumno(s)%n",
                    rs.getString("curso"),
                    rs.getInt("total"));
            }

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

💡 **Explicación:** GROUP BY agrupa por curso. COUNT(*) cuenta filas de cada grupo. AS total da nombre a la columna calculada. ORDER BY total DESC ordena de mayor a menor. Aquí usamos Statement porque no hay parámetros variables — la SQL es fija. PreparedStatement también valdría, pero no es necesario.

★ ★ Ejercicio 7: Transacciones básicas

Simula una transferencia de puntos entre dos alumnos. Usa transacciones: si alguna operación falla, todo se deshace.

```

import java.sql.*;

public class TransferenciaPuntos {
    public static void main(String[] args) {
        String sqlQuitar = "UPDATE alumnos SET puntos = puntos - ? WHERE id = ?";
        String sqlPoner = "UPDATE alumnos SET puntos = puntos + ? WHERE id = ?";

        try (Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/instituto", "root", "admin123")) {

            con.setAutoCommit(false);

            try (PreparedStatement q = con.prepareStatement(sqlQuitar);
                PreparedStatement p = con.prepareStatement(sqlPoner)) {

                // Quitar 10 puntos al alumno 1
                q.setInt(1, 10);
                q.setInt(2, 1);
                q.executeUpdate();

                // Poner 10 puntos al alumno 2
                p.setInt(1, 10);
                p.setInt(2, 2);
                p.executeUpdate();

                con.commit();
                System.out.println("Transferencia OK ✅");

            } catch (SQLException e) {
                con.rollback();
                System.err.println("Error, todo deshecho: " + e.getMessage());
            }

        } catch (SQLException e) {
            System.err.println("Error de conexión: " + e.getMessage());
        }
    }
}

```

💡 **Explicación:** `setAutoCommit(false)` desactiva el modo “cada sentencia es una transacción”. Ahora tú controlas: `commit()` confirma todo, `rollback()` deshace todo. Si falla

la segunda sentencia, la primera se deshace con `rollback()`. Las transacciones son para operaciones que deben ser atómicas: “todo o nada”.

☆☆☆ Ejercicio 8: CRUD completo con menú

Crea un programa con menú textual que permita: Listar, Insertar, Actualizar, Eliminar y Buscar alumnos. Cada operación en su propio método. Usa `PreparedStatement` siempre.

```

import java.sql.*;
import java.util.*;

public class CrudAlumnosCompleto {
    private static final String URL = "jdbc:mysql://localhost:3306/instituto";
    private static final String USER = "root";
    private static final String PASS = "admin123";

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int op;
        do {
            System.out.println("\n=== CRUD ALUMNOS ===");
            System.out.println("1. Listar  2. Buscar  3. Insertar");
            System.out.println("4. Actualizar  5. Eliminar  6. Salir");
            System.out.print("Opción: ");
            op = sc.nextInt(); sc.nextLine();

            switch (op) {
                case 1 -> listar();
                case 2 -> buscar(sc);
                case 3 -> insertar(sc);
                case 4 -> actualizar(sc);
                case 5 -> eliminar(sc);
            }
        } while (op != 6);
    }

    static void listar() {
        try (Connection c = DriverManager.getConnection(URL, USER, PASS);
            Statement st = c.createStatement();
            ResultSet rs = st.executeQuery("SELECT * FROM alumnos ORDER BY nombre")) {
            while (rs.next())
                System.out.printf("%d - %s (%d) %s\n",
                    rs.getInt("id"), rs.getString("nombre"),
                    rs.getInt("edad"), rs.getString("curso"));
        } catch (SQLException e) { System.err.println("Error: " + e.getMessage()); }
    }

    static void buscar(Scanner sc) {
        System.out.print("Nombre a buscar: ");
        try (Connection c = DriverManager.getConnection(URL, USER, PASS);

```



```

        PreparedStatement ps = c.prepareStatement("SELECT * FROM alumnos WHERE
nombre LIKE ?")) {
    ps.setString(1, "%" + sc.nextLine() + "%");
    ResultSet rs = ps.executeQuery();
    boolean ok = false;
    while (rs.next()) { ok = true;
        System.out.printf("%d - %s (%d)%n",
            rs.getInt("id"), rs.getString("nombre"), rs.getInt("edad")); }
    if (!ok) System.out.println("Sin resultados");
    rs.close();
} catch (SQLException e) { System.err.println("Error: " + e.getMessage()); }
}

static void insertar(Scanner sc) {
    System.out.print("Nombre: "); String n = sc.nextLine();
    System.out.print("Edad: "); int e = sc.nextInt(); sc.nextLine();
    System.out.print("Curso: "); String c = sc.nextLine();
    try (Connection con = DriverManager.getConnection(URL, USER, PASS);
        PreparedStatement ps = con.prepareStatement(
            "INSERT INTO alumnos (nombre, edad, curso) VALUES (?, ?, ?)")) {
        ps.setString(1, n); ps.setInt(2, e); ps.setString(3, c);
        System.out.println(ps.executeUpdate() > 0 ? "OK" : "Error");
    } catch (SQLException ex) { System.err.println("Error: " + ex.getMessage()); }
}

static void actualizar(Scanner sc) {
    System.out.print("ID: "); int id = sc.nextInt(); sc.nextLine();
    System.out.print("Nueva edad: "); int edad = sc.nextInt(); sc.nextLine();
    try (Connection con = DriverManager.getConnection(URL, USER, PASS);
        PreparedStatement ps = con.prepareStatement(
            "UPDATE alumnos SET edad = ? WHERE id = ?")) {
        ps.setInt(1, edad); ps.setInt(2, id);
        System.out.println(ps.executeUpdate() > 0 ? "Actualizado" : "No
encontrado");
    } catch (SQLException e) { System.err.println("Error: " + e.getMessage()); }
}

static void eliminar(Scanner sc) {
    System.out.print("ID: "); int id = sc.nextInt(); sc.nextLine();
    System.out.print("¿Seguro? (s/n): ");
    if (!sc.nextLine().equalsIgnoreCase("s")) { System.out.println("Cancelado");
return; }
    try (Connection con = DriverManager.getConnection(URL, USER, PASS);

```

```

        PreparedStatement ps = con.prepareStatement("DELETE FROM alumnos WHERE id
= ?")) {
    ps.setInt(1, id);
    System.out.println(ps.executeUpdate() > 0 ? "Eliminado" : "No encontrado");
} catch (SQLException e) { System.err.println("Error: " + e.getMessage()); }
    }
}

```

💡 **Explicación:** CRUD completo en ~130 líneas. Cada operación abre y cierra su propia conexión (no óptimo para producción, pero sí para aprender). PreparedStatement en todas las consultas con parámetros. Statement solo en listar (consulta fija sin parámetros). Fíjate que ResultSet también se cierra automáticamente cuando está dentro de try-with-resources.

Ejercicio 9: Paginación con LIMIT y OFFSET

Crea un método que muestre alumnos por páginas de 5 en 5. El usuario elige la página.

```

import java.sql.*;
import java.util.Scanner;

public class PaginacionAlumnos {
    private static final int TAM_PAGINA = 5;

    public static void mostrarPagina(int pagina) {
        String sql = "SELECT * FROM alumnos ORDER BY id LIMIT ? OFFSET ?";

        try (Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/instituto", "root", "admin123");
            PreparedStatement pstmt = con.prepareStatement(sql)) {

            int offset = (pagina - 1) * TAM_PAGINA;
            pstmt.setInt(1, TAM_PAGINA);
            pstmt.setInt(2, offset);

            try (ResultSet rs = pstmt.executeQuery()) {
                System.out.println("--- Página " + pagina + " ---");
                while (rs.next()) {
                    System.out.printf("%d - %s (%d) %s\n",
                        rs.getInt("id"), rs.getString("nombre"),
                        rs.getInt("edad"), rs.getString("curso"));
                }
            }

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int pag = 1;
        while (true) {
            mostrarPagina(pag);
            System.out.print("Siguiete (s) / Anterior (a) / Salir (x): ");
            String cmd = sc.nextLine();
            if (cmd.equals("s")) pag++;
            else if (cmd.equals("a") && pag > 1) pag--;
            else if (cmd.equals("x")) break;
        }
    }
}

```

```
}  
  
}
```

💡 **Explicación:** LIMIT ? OFFSET ?: LIMIT=5 (filas por página), OFFSET=(pag-1)*5 (desplazamiento). OFFSET permite navegar por páginas sin cargar todos los datos. Esto es eficiente para tablas grandes: la BD solo devuelve las 5 filas de la página. En PostgreSQL se usa LIMIT ? OFFSET ?, en SQL Server OFFSET ... FETCH NEXT.

☆☆☆ Ejercicio 10: Exportar a CSV con metadatos

Exporta todos los alumnos a un archivo CSV. Además de los datos, incluye una fila de metadatos: nombre de las columnas, número de filas exportadas y fecha de exportación.

```

import java.sql.*;
import java.io.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class ExportarCSV {
    public static void main(String[] args) {
        String csvFile = "alumnos_export.csv";

        try (Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/instituto", "root", "admin123");
            Statement stmt = con.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT * FROM alumnos ORDER BY id");
            BufferedWriter writer = new BufferedWriter(new FileWriter(csvFile))) {

            // Metadatos
            writer.write("# Exportación generada el " +
                LocalDateTime.now().format(DateTimeFormatter.ISO_LOCAL_DATE_TIME));
            writer.newLine();

            // Cabecera desde ResultSetMetaData
            ResultSetMetaData meta = rs.getMetaData();
            int numCols = meta.getColumnCount();
            for (int i = 1; i <= numCols; i++) {
                if (i > 1) writer.write(";");
                writer.write(meta.getColumnName(i));
            }
            writer.newLine();

            // Datos
            int filas = 0;
            while (rs.next()) {
                for (int i = 1; i <= numCols; i++) {
                    if (i > 1) writer.write(";");
                    String valor = rs.getString(i);
                    if (valor != null) writer.write(valor);
                }
                writer.newLine();
                filas++;
            }

            // Pie con total

```

```

        writer.write("# Total: " + filas + " filas exportadas");
        writer.newLine();

        System.out.println("Exportadas " + filas + " filas a " + csvFile);

    } catch (SQLException | IOException e) {
        System.err.println("Error: " + e.getMessage());
    }
}
}

```

💡 **Explicación:** ResultSetMetaData da información sobre las columnas: nombre, tipo, tamaño. getColumnCount() + getColumnName(i) genera la cabecera dinámicamente — funciona para cualquier consulta, no solo SELECT *. El CSV usa ; como separador (estilo español). La fecha de exportación y el total de filas se escriben como comentarios con #. Esto es exportación profesional.

🔗 **CodeWars:** [SQL with Street Fighter](#) (6kyu) • [SQL Basics: Simple JOIN](#) (6kyu)

🔗 **AceptaElReto.com:** [200 - Aburrimiento en las aulas](#) • [340 - Juegos de naipes](#) • [100 - Kaprekar](#)

Boletín 14 – Intermedio: Conexión a BD con JDBC

Ejercicios de dificultad progresiva. De ★ a ★★★★★.

★ Ejercicio 1: Conexión desde archivo de propiedades

Crea un archivo `db.properties` con los datos de conexión:

```
url=jdbc:mysql://localhost:3306/instituto
user=root
password=admin123
```

Escribe un programa que lea este archivo usando `Properties` y establezca la conexión. Si el archivo no existe o falta alguna propiedad, muestra un mensaje de error claro.

★ Ejercicio 2: INSERT con clave autogenerada

Inserta un nuevo alumno en la tabla `alumnos` y **recupera el ID** que la base de datos le ha asignado automáticamente. Usa `PreparedStatement` con la opción `Statement.RETURN_GENERATED_KEYS` y el método `getGeneratedKeys()`.

Pista:

```
PreparedStatement pstmt = con.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS);
```

★ Ejercicio 3: UPDATE condicional

Actualiza el curso de todos los alumnos que tengan una edad superior a un valor dado. Por ejemplo:

```
¿Edad mínima? 25
¿Nuevo curso? DAM2
```

Todos los alumnos mayores de 25 años pasan al curso «DAM2». Muestra cuántas filas se actualizaron.

★ ★ Ejercicio 4: INNER JOIN con PreparedStatement

Dada una tabla `matriculas` con `id_alumno`, `asignatura`, `nota`, escribe un programa que reciba un nombre de alumno y muestre todas sus asignaturas y notas. Usa un `INNER JOIN` entre `alumnos` y `matriculas`.

Ejemplo de salida:

```
Alumno: Ana García
Matemáticas: 8.5
Programación: 9.0
Bases de Datos: 7.5
```

★ ★ Ejercicio 5: Fechas en JDBC

Añade una columna `fecha_nacimiento` `DATE` a la tabla `alumnos` (asume que ya existe). Crea un programa que:

1. Pida nombre, edad, curso y fecha de nacimiento (formato `YYYY-MM-DD`).
2. Inserte el alumno usando `PreparedStatement` con `java.sql.Date.valueOf()`.
3. Liste todos los alumnos mostrando también su fecha de nacimiento formateada con `DateTimeFormatter`.

★ ★ Ejercicio 6: Batch INSERT — 100 alumnos de prueba

Crea un programa que inserte **100 alumnos de prueba** en la tabla `alumnos` usando lotes (batch). Los nombres pueden ser genéricos: `Alumno1`, `Alumno2`, etc.

Usa `addBatch()` y `executeBatch()` de `PreparedStatement`. Mide el tiempo que tarda con `System.currentTimeMillis()`.

Compara: ¿cuánto tardaría si hicieras 100 `executeUpdate()` individuales?

★ ★ ★ Ejercicio 7 (ProgramaMe): Patrón DAO

Implementa el patrón **Data Access Object (DAO)** para la tabla `alumnos`. Crea las siguientes clases:

1. `Alumno` — clase modelo con `id`, `nombre`, `edad`, `curso`.
 2. `AlumnoDAO` — interfaz con métodos:
 - `List<Alumno> listar()`
 - `Alumno buscarPorId(int id)`
 - `List<Alumno> buscarPorNombre(String nombre)`
 - `boolean insertar(Alumno a)`
 - `boolean actualizar(Alumno a)`
 - `boolean eliminar(int id)`
 3. `AlumnoDAOImpl` — implementación concreta con JDBC.
 4. `Main` — programa con menú que use el DAO.
-

★ ★ ★ Ejercicio 8 (CodeWars + AceptaElReto)

Resuelve estos problemas que refuerzan conceptos de Bases de Datos y SQL:

CodeWars: [SQL Basics: Simple JOIN](#) (6 kyu) — Practica JOINS en SQL.

CodeWars: [SQL with Street Fighter](#) (6 kyu) — Consultas con LIKE y ordenación.

AceptaElReto: [200 - Aburrimiento en las aulas](#) — Problema de estructura de datos que puedes resolver con JDBC.



Referencias

- CodeWars: [SQL Basics: Simple JOIN](#) (6 kyu)
- CodeWars: [SQL with Street Fighter](#) (6 kyu)
- CodeWars: [SQL Basics: Simple HAVING](#) (6 kyu)
- AceptaElReto.com: [200 - Aburrimiento en las aulas](#)
- AceptaElReto.com: [340 - Juegos de naipes](#)
- AceptaElReto.com: [100 - Kaprekar](#)

Boletín 12 — Extras: Conexión a Bases de Datos con JDBC

★ Extras — Conviértete en un programador de muy alto nivel

Estos ejercicios son completamente opcionales, pero si aspiras a ser un programador o programadora de élite, aquí tienes tu campo de entrenamiento. CodeWars y AceptaElReto son plataformas donde los mejores programadores del mundo se retan a diario. Cada problema que resuelvas te hará más fuerte, más rápido y más creativo. No los subestimes: son la diferencia entre saber programar y ser un verdadero profesional.

CodeWars:

- [SQL with Street Fighter](#) (6 kyu)

Pista: Usa `INNER JOIN` con `LIKE '%K%'` para filtrar por nombre y `ORDER BY` para ordenar.

- [SQL Basics: Simple JOIN](#) (6 kyu)

Pista: `INNER JOIN` sobre la clave foránea; selecciona las columnas que te pida el problema.

- [SQL Basics: Simple HAVING](#) (6 kyu)

Pista: `GROUP BY` la columna indicada y filtra con `HAVING COUNT(*) > algún valor`.

AceptaElReto:

- [245 - ¿Quién gana la partida?](#) (★ ★ ★)

Pista: Simula los turnos con una `Queue<Integer>`; cada ronda mueve al primero al final si no acierta.

- [424 - Billetes de autobús](#) (★ ★)

Pista: Algoritmo voraz: ordena por hora de salida, elige siempre la siguiente ruta que termine antes.

Boletín 11 – Inicial resuelto: Interfaz Web

Los mismos ejercicios con soluciones paso a paso.

Ejercicio 1: Hola Mundo Web

```
import com.sun.net.httpserver.HttpServer;
import java.io.OutputStream;
import java.net.InetSocketAddress;

public class HolaMundoWeb {
    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);
        server.createContext("/", e -> {
            String resp = "Hola Mundo";
            e.sendResponseHeaders(200, resp.getBytes().length);
            OutputStream os = e.getResponseBody();
            os.write(resp.getBytes());
            os.close();
        });
        server.setExecutor(null);
        server.start();
        System.out.println("Servidor en http://localhost:8080");
    }
}
```

Ejercicio 2: Hora del servidor

```
server.createContext("/hora", e -> {
    String resp = java.time.LocalDateTime.now().toString();
    e.sendResponseHeaders(200, resp.getBytes().length);
    e.getResponseBody().write(resp.getBytes());
    e.getResponseBody().close();
});
```

Ejercicio 3: HTML bonito

```

server.createContext("/", e -> {
    String html = ""
        <html><head><meta charset="UTF-8"><title>Mi Web</title>
        <style>body{background:#e0f7e0;font-family:sans-serif;padding:2em}</style>
        </head><body><h1>Bienvenido</h1><p>Esto es HTML servido desde Java.</p>
        </body></html>
        "";
    e.getResponseHeaders().set("Content-Type", "text/html; charset=UTF-8");
    e.sendResponseHeaders(200, html.getBytes().length);
    e.getResponseBody().write(html.getBytes());
    e.getResponseBody().close();
});

```

Ejercicio 4: Saludo personalizado

```

server.createContext("/saludo", e -> {
    String query = e.getRequestURI().getQuery();
    String nombre = "Mundo";
    if (query != null && query.startsWith("nombre="))
        nombre = java.net.URLDecoder.decode(query.split("=")[1], "UTF-8");
    String html = "<h1>Hola, " + nombre + "!</h1>";
    e.getResponseHeaders().set("Content-Type", "text/html; charset=UTF-8");
    e.sendResponseHeaders(200, html.getBytes().length);
    e.getResponseBody().write(html.getBytes());
    e.getResponseBody().close();
});

```

Ejercicio 5: Contador de visitas

```

public class ContadorVisitas {
    static int visitas = 0;
    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);
        server.createContext("/", e -> {
            visitas++;
            String resp = "Visita #" + visitas;
            e.sendResponseHeaders(200, resp.getBytes().length);
            e.getResponseBody().write(resp.getBytes());
            e.getResponseBody().close();
        });
        server.start();
        System.out.println("http://localhost:8080");
    }
}

```

Ejercicio 6: Tabla HTML

```

server.createContext("/tabla", e -> {
    StringBuilder sb = new StringBuilder("<table border=1>");
    sb.append("<tr><th>N</th><th>N²</th></tr>");
    for (int i = 1; i <= 10; i++)
        sb.append("<tr><td>").append(i).append("</td><td>").append(i*i).append("</td></tr>");
    sb.append("</table>");
    String html = "<html><body>" + sb + "</body></html>";
    e.getResponseHeaders().set("Content-Type", "text/html; charset=UTF-8");
    e.sendResponseHeaders(200, html.getBytes().length);
    e.getResponseBody().write(html.getBytes());
    e.getResponseBody().close();
});

```

Boletín 13 – Inicial: Servir y Consumir APIs con Web

Calienta motores con `HttpServer`. Prometo que ningún bit saldrá herido.

Ejercicio 1: Servidor de la hora

Crea un servidor HTTP en el puerto 8080 con una ruta `/hora` que devuelva la hora actual en texto plano con formato `HH:mm:ss`.

Cada vez que recargues el navegador, la hora cambia. Magia negra con `LocalTime.now()`.

```
// Pista:  
String hora = java.time.LocalTime.now().format(  
    java.time.format.DateTimeFormatter.ofPattern("HH:mm:ss")  
);
```

Ejercicio 2: Página que dice tu nombre

Añade una ruta `/saludo?nombre=Pepe` que devuelva una página HTML con `<h1>¡Hola, Pepe!
</h1>`.

Si no se pasa nombre, que diga `¡Hola, desconocido!`.

Pista: `e.getRequestURI().getQuery()` te da `nombre=Pepe`. Divide por `=` y listo.

Ejercicio 3: Contador de visitas global

Usa una variable `static int visitas = 0`. Cada vez que alguien visita `/`, incrementa el contador y devuelve:

```
Eres el visitante número 47
```

¿Qué pasa si dos personas recargan a la vez? (spoiler: problemas — pero de momento no te preocupes).

Ejercicio 4: Generador de excusas para entregas tarde

Crea rutas /excusa que devuelva una excusa generada aleatoriamente combinando elementos de tres arrays:

```
String[] sujetos = {"Mi perro", "GitHub", "El plugin de IntelliJ", "La conexión"};
String[] verbos = {"se comió", "borró", "corrompió", "perdió"};
String[] objetos = {"el examen", "la práctica", "los apuntes", "mi paciencia"};
```

Ejemplo: “*GitHub borró la práctica*”. Devuélvelo en HTML con buena letra.

Ejercicio 5: Tabla de multiplicar personalizada

Ruta /tabla?num=7 que genere una tabla HTML completa con la tabla de multiplicar del 7 (del 7×1 al 7×10).

Si no se pasa número, usa el 5 por defecto. Formatea la tabla con border=1 y colores alternos en las filas.

Ejercicio 6: Página de estado del servidor

Ruta /estado que devuelva un JSON con esta pinta:

```
{
  "servidor": "ok",
  "hora": "14:30:01",
  "visitas": 47,
  "version": "1.0",
  "autor": "Tu nombre aquí"
}
```

No olvides el Content-Type: application/json o el navegador se pondrá tonto.

Ejercicio 7: Conversor de euros a pesetas (sí, pesetas)

Ruta `/conversor?euros=50` que devuelva HTML con el resultado: “50 euros son 8319.3 pesetas”.

1 € = 166.386 pts. Muestra dos decimales.

Pista: `String.format("%.2f", valor)` para los decimales. Sí, pesetas. Si eres joven, pregúntale a un viejo qué era eso.



Referencias

- CodeWars: [Decode the Morse code](#) (6 kyu)
- AceptaElReto: [396 - ¿Cuántos días faltan?](#) (★ ★)
- Documentación Oracle: [HttpServer](#)

Boletín 11 – Resuelto: Interfaz Web

Ejercicios de nivel creciente. Sin soluciones, solo pistas.

★ Ejercicio 1: Servidor de archivos estáticos

Sirve cualquier fichero de la carpeta `web/` (HTML, CSS, JS, imágenes) usando `Files.probeContentType()` para el Content-Type.

Pista: usa `Files.readAllBytes(Path.of("web", ruta))` y captura `NoSuchFileException` para devolver 404.

★ Ejercicio 2: Formulario de contacto

Crea una ruta `/contacto` que sirva un formulario HTML (GET) y otra ruta `/enviar` que reciba los datos por POST y los muestre en una página de confirmación.

Pista: en el handler de `/enviar` comprueba `e.getRequestMethod().equals("POST")` y lee el cuerpo con `e.getRequestBody().readAllBytes()`.

★ ★ Ejercicio 3: API JSON de productos

Implementa una API REST:

- GET `/api/productos` → lista todos los productos
- POST `/api/productos` → añade uno (recibe JSON)
- DELETE `/api/productos/{id}` → borra uno

Usa un `ArrayList<Producto>` como almacén en memoria.

Pista: para parsear JSON a mano, usa `String.split()` o la clase `java.util.Scanner`. Para algo más serio, `org.json` o `Gson`.

★ ★ Ejercicio 4: Chat en memoria

Crea un chat simple donde los mensajes se guardan en un `ArrayList`. El frontend pregunta cada 2 segundos con `setInterval` si hay mensajes nuevos.

- GET `/api/mensajes?ultimoId=5` → mensajes nuevos desde el ID 5
- POST `/api/mensajes` → body: `{usuario:"Ana", texto:"Hola"}`

Pista: El frontend usa `fetch()` dentro de un `setInterval`. El backend filtra mensajes por ID mayor que `ultimoId`.

☆☆☆ Ejercicio 5: Mini framework MVC

Implementa un mini enrutador que permita definir rutas con esta sintaxis:

```
get("/usuarios", (req, res) -> res.html("<h1>Usuarios</h1>"));
post("/usuarios", (req, res) -> res.json(nuevoUsuario));
```

Pista: Crea una clase Router con mapas de `HashMap<String, Handler>` para GET y POST. Cada handler recibe `HttpExchange` y tiene métodos auxiliares `html()` y `json()`.

☆☆☆ Ejercicio 6: Subida de archivos

Permite al usuario seleccionar un archivo desde el navegador y subirlo al servidor. Guárdalo en `uploads/` y muestra el nombre y tamaño en una tabla.

- Frontend: `<input type="file">` + `FormData` + `fetch` POST
- Backend: leer `multipart/form-data` a mano o con `Scanner`

Pista: El Content-Type `multipart/form-data` incluye un `boundary`. Separa el cuerpo por ese `boundary` para extraer el nombre y contenido del archivo.

Boletín 13 – Intermedio: Servir y Consumir APIs con Web

Ejercicios para dominar el arte de servir JSON, procesar formularios y hacer que frontend y backend se hablen sin pelearse.

★ Ejercicio 1: API de frases motivacionales

Crea un endpoint GET `/api/frase` que devuelva un JSON con una frase aleatoria de un array precargado y su autor.

```
{"frase": "El código limpio es como un buen chiste: si tienes que explicarlo, es malo", "autor": "Alguien que sabe"}
```

El frontend es un HTML con un botón “Nueva frase” que al hacer clic hace `fetch('/api/frase')` y muestra la frase en pantalla.

Pista: usa `Math.random()` para elegir un índice aleatorio del array.

★ Ejercicio 2: Traductor chungo (pero funcional)

Implementa un endpoint POST `/api/traducir` que reciba:

```
{"texto": "hola", "idioma": "en"}
```

Y devuelva:

```
{"traduccion": "hello"}
```

Usa un `HashMap<String, HashMap<String, String>>` como diccionario. Mete al menos 10 palabras en español traducidas a inglés y francés.

Pista: Inicializa el diccionario con bloques static. `diccionario.get("hola").get("en")` te da “hello”.

☆☆ Ejercicio 3: API REST de tareas con prioridad

Implementa un CRUD completo de tareas donde cada tarea tiene: id, titulo, prioridad (“ALTA”, “MEDIA”, “BAJA”).

Método	Ruta	Descripción
GET	/api/tareas	Lista todas
POST	/api/tareas	Crea una (JSON: {"titulo": "...", "prioridad": "ALTA"})
PUT	/api/tareas/{id}	Cambia prioridad (JSON: {"prioridad": "BAJA"})
DELETE	/api/tareas/{id}	Borra una

Frontend: tabla con colores de fondo según prioridad (rojo ALTA, amarillo MEDIA, verde BAJA). Botones para crear, cambiar prioridad y borrar.

Pista: guarda las tareas en un `ConcurrentHashMap<Integer, Tarea>` con un `AtomicInteger` para IDs.

☆☆ Ejercicio 4: El tiempo que NO hace

Crea GET /api/clima?ciudad=Madrid que devuelva un JSON con datos meteorológicos **aleatorios** (generados cada vez):

```
{"ciudad": "Madrid", "temperatura": 28, "humedad": 45, "estado": "soleado"}
```

Estados posibles: “soleado”, “nublado”, “lluvia”, “tormenta”. Frontend con emojis y temperaturas de colores.

Pista: usa `String[] estados = {"soleado", "nublado", "lluvia", "tormenta"}` y elige aleatoriamente. La temperatura puede ser `random.nextInt(40) - 5`.

☆☆ Ejercicio 5: Catálogo de películas con filtros

Precarga un array de 10-15 películas (con titulo, genero, anyo, puntuacion). Implementa:

- GET /api/peliculas → lista todas
- GET /api/peliculas?genero=comedia → filtra por género
- GET /api/peliculas?genero=comedia&anyo=1994 → filtra por género y año
- GET /api/peliculas/3 → detalle de la película con ID 3

Frontend: selectores de género y año, que al cambiar actualizan la lista vía fetch.

Pista: para filtrar usa `stream().filter(p -> p.getGenero().equals(genero)).toList()`. Para el path param, parsea la URL.

☆☆☆ Ejercicio 6: Middleware de logging

Crea una clase `LoggerMiddleware` que envuelva cualquier `Handler` y registre en consola:

```
[2026-06-21 14:30:01] GET /api/peliculas → 200 (15ms)
[2026-06-21 14:30:05] POST /api/tareas → 201 (3ms)
```

Debe poder aplicarse a cualquier handler así:

```
server.createContext("/api", new LoggerMiddleware(new TareasHandler()));
```

Pista: guarda `System.currentTimeMillis()` antes y después de llamar al handler original. Usa `e.getRequestMethod()` y `e.getResponseCode()` (tras enviar cabeceras).

☆☆☆ Ejercicio 7: Server-Sent Events — El reloj del servidor

Implementa un endpoint GET /api/eventos que use Server-Sent Events (SSE). Cada 5 segundos, el servidor envía un evento con la hora actual:

```
data: {"hora": "14:30:05", "timestamp": 1718975405}
```

Frontend con `EventSource`:

```
const source = new EventSource('/api/eventos');
source.onmessage = (e) => {
    document.getElementById('reloj').textContent = JSON.parse(e.data).hora;
};
```

Pista: en el handler, pon `e.getResponseHeaders().set("Content-Type", "text/event-stream")` y NO cierres la conexión. Usa `e.getResponseBody().write()` en un bucle con `Thread.sleep(5000)`.

★ Ejercicio 8: Piedra, papel, tijera online

Endpoint POST `/api/jugar` que recibe:

```
{"jugada": "piedra"}
```

Y devuelve:

```
{"jugadaPC": "tijera", "resultado": "ganaste"}
```

Reglas clásicas: piedra > tijera, tijera > papel, papel > piedra.

Frontend: tres botones con emojis 🪨 📄 ✂️. Al hacer clic, envía la jugada y muestra el resultado. Lleva un contador de victorias/derrotas/empates.

Pista: la jugada del PC se elige con Random. Las reglas se pueden implementar con un `Map<String, String>` donde la clave vence al valor: `{"piedra": "tijera", "tijera": "papel", "papel": "piedra"}`.

Referencias

- CodeWars: [IP Validation](#) (6 kyu)
- CodeWars: [Simple URL parser](#) (6 kyu)
- AceptaElReto: [462 - Día de la semana](#) (★ ★)
- Documentación Oracle: [HttpExchange](#)
- MDN: [Server-Sent Events](#)

Boletín 13 — Extras: Servir y Consumir APIs con Web

★ Extras — Conviértete en un programador de muy alto nivel

Estos ejercicios son completamente opcionales, pero si aspiras a ser un programador o programadora de élite, aquí tienes tu campo de entrenamiento. CodeWars y AceptaElReto son plataformas donde los mejores programadores del mundo se retan a diario. Cada problema que resuelvas te hará más fuerte, más rápido y más creativo. No los subestimes: son la diferencia entre saber programar y ser un verdadero profesional.

CodeWars:

- [IP Validation](#) (6 kyu)

Pista: Divide por '.'; deben ser exactamente 4 partes, cada una entre 0 y 255, sin ceros a la izquierda.

- [Decode the Morse code](#) (6 kyu)

Pista: Crea un Map con cada símbolo Morse → letra; separa palabras por tres espacios y letras por uno.

- [Simple URL parser](#) (6 kyu)

Pista: Extrae primero el protocolo (antes de ://), luego el dominio (siguiente segmento) y el resto como ruta.

AceptaElReto:

- [396 - ¿Cuántos días faltan?](#) (★ ★)

Pista: Convierte cada fecha a día del año (número de días desde el 1 de enero) y resta.

- [462 - Día de la semana](#) (★ ★)

Pista: Usa la congruencia de Zeller o toma un día de referencia conocido para calcular el resto.